BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC CÔNG NGHỆ TP.HCM



Lập trình trên môi trường Windows

Biên soạn: ThS. Nguyễn Đình Ánh
ThS. Dương Thành Phết
ThS. Nguyễn Huy Cường

LẬP TRÌNH TRÊN MÔI TRƯỜNG WINDOWS

Ấn bản 2026

MỤC LỤC

MỤC LỤC I
HƯỚNG DẪN V
BÀI 1: TỔNG QUAN VỀ .NET VÀ NGÔN NGỮ C# 1
1.1 GIỚI THIỆU .NET 1
1.1.1 Tổng quan 1
1.1.2 Lịch sử hình thành 2
1.1.3 Quá trình phát triển 3
1.1.4 Cài đặt môi trường phát triển 5
1.2 ĐẶC ĐIỂM .NET 5
1.2.1 Kiến trúc .Net 6
1.2.2 Thư viện lớp .NET 7
1.2.3 Garbage collection - bộ thu dọn rác 9
1.3 NGÔN NGỮ LẬP TRÌNH C# 10
1.3.1 Giới thiệu ngôn ngữ lập trình C# 10
1.3.2 Tạo Console App đầu tiên 11
1.3.3 Nhập xuất dữ liệu với thư viện Console 13
1.3.4 Kiểu dữ liệu và biến 14
1.3.5 Toán tử và biểu thức 16
1.3.6 Cấu trúc điều khiển 18
1.3.7 Hàm 21
1.3.8 Mảng và Collection cơ bản 25
1.4 LINQ 29
1.4.1 LINQ là gì? 29
1.4.2 Cú pháp LINQ cơ bản 30
1.4.3 Các phương thức LINQ thường dùng 30
TÓM TẮT 33
CÂU HỎI ÔN TẬP 34
BÀI 2: HƯỚNG ĐỐI TƯỢNG TRONG C# 36
2.1 TỔNG QUAN VỀ HƯỚNG ĐỐI TƯỢNG C# 36
2.2 CLASS VÀ OBJECT 37
2.2.1 Khai báo Class, Constructor 37
2.2.2 Property 38
2.2.3 Tạo Object và sử dụng 40
2.2.4 Phương thức (Method) trong Class 40
2.2.5 Phạm vi truy cập (Access Modifier) 43
2.3 KẾ THỪA 44
2.3.1 Khái niệm và cú pháp kế thừa 44
2.3.2 Từ khóa base, override 45
2.4 ĐA HÌNH (POLYMORPHISM) 48

2.5 INTERFACE VÀ ABSTRACT CLASS 50
2.5.1 Abstract class 50
2.5.2 Interface 51
2.5.3 So sánh Abstract class và Interface 52
TÓM TẮT 53
CÂU HỎI ÔN TẬP 53
BÀI 3: LẬP TRÌNH GIAO DIỆN VỚI WINDOWS FORMS 56
3.1 CÁC KHÁI NIỆM 56
3.1.1 Windows Forms là gì, Lập trình hướng sự kiện 56
3.1.2 Tạo Project WinForms đầu tiên 57
3.1.3 Form — vòng đời 60
3.2 CÁC CONTROLS CƠ BẢN 62
3.2.1 Label, TextBox, Button 68
3.2.2 CheckBox, RadioButton 75
3.2.3 ComboBox và ListBox 78
3.2.4 DateTimePicker 81
3.2.5 PictureBox 82
3.2.6 DataGridView 83
3.2.7 Panel và GroupBox 87
TÓM TẮT 88
CÂU HOI ÔN TẬP 89
BÀI 4: ỨNG DỤNG NHIỀU FORM VÀ MDI 91
4.1 LÀM VIỆC VỚI NHIỀU FORM 91
4.1.1 MessageBox 91
4.1.2 Tạo và mở Form mới 94
4.1.3 Truyền dữ liệu giữa các Form 97
4.2 MDI (MULTIPLE DOCUMENT INTERFACE) 100
4.2.1 Khái niệm MDI — Form cha (Container) và Form con 100
4.2.2 Tạo MDI Parent và MDI Child 100
4.3 CÁC CONTROL HỖ TRỢ GIAO DIỆN ỨNG DỤNG 102
4.3.1 MenuStrip 102
4.3.2 ContextMenuStrip — menu chuột phải 105
4.3.3 ToolStrip 108
4.3.4 StatusStrip 110
4.3.5 OpenFileDialog, SaveFileDialog 111
4.3.6 TableLayoutPanel và TabControl 112
TÓM TẮT 115
CÂU HOI ÔN TẬP 115
BÀI 5: KẾT NỐI DỮ LIỆU VỚI ENTITY FRAMEWORK CORE 120
5.1 LẬP TRÌNH BẤT ĐỒNG BỘ 121
5.1.1 Đồng bộ và bất đồng bộ 121

5.1.2 Lập trình bất động bộ trong C# 121
5.2 TỪ ADO.NET ĐẾN EF CORE 123
5.2.1 ADO.NET — cách kết nối CSDL truyền thống 124
5.2.2 Hạn chế của ADO.NET 128
5.3 ENTITY FRAMEWORK CORE 130
5.3.1 DbContext và DbSet 132
5.3.2 Code First và Database First 134
5.3.3 Định nghĩa Entity (Model) 135
5.3.4 Viết DbContext — Connection String 137
5.3.5 Migration 138
5.3.6 CRUD cơ bản trên bảng 140
5.3.7 Áp dụng vào ứng dụng windows forms 143
TÓM TẮT 148
CÂU HỎI ÔN TẬP 148
BÀI 6: TRUY VẤN VÀ QUẢN LÝ DỮ LIỆU VỚI ENTITY FRAMEWORK CORE 150
6.1 QUAN HỆ GIỮA CÁC BẢNG TRONG EF CORE 150
6.1.1 Quan hệ 1-n (One-to-Many) 150
6.1.2 Quan hệ n-n (Many-to-Many) 154
6.2 TRUY VẤN DỮ LIỆU CÓ QUAN HỆ 155
6.2.1 Include 155
6.2.2 LINQ to Entities nâng cao — GroupBy, Join 156
6.3 TRANSACTION — ĐẢM BẢO TÍNH TOÀN VẸN DỮ LIỆU 158
6.3.1 Vấn đề: nhiều thao tác phải cùng thành công hoặc cùng thất bại 158
6.3.2 SaveChangesAsync() 159
6.3.3 Transaction khi cần nhiều SaveChangesAsync 160
6.4 BINDINGSOURCE 161
6.4.1 Vấn đề: DataGridView không tự cập nhật khi List thay đổi 161
6.4.2 BindingSource 161
6.4.3 Áp dụng BindingSource vào CRUD 163
6.5 VALIDATION DỮ LIỆU TRƯỚC KHI LƯU 164
6.5.1 ErrorProvider 164
6.5.2 Sự kiện Validating 166
6.5.3 Kết hợp validate với EF Core 167
TÓM TẮT 168
CÂU HỎI ÔN TẬP 168
BÀI 7: MÔ HÌNH TỔ CHỨC DỰ ÁN 171
7.1 ĐẶT VẤN ĐỀ 171
7.2 KIẾN TRÚC 3 TẦNG (3-LAYER ARCHITECTURE) 173
7.2.1 Presentation Layer — tầng giao diện 174
7.2.2 Business Logic Layer — tầng nghiệp vụ 174
7.2.3 Data Access Layer — tầng truy cập dữ liệu 175
7.2.4 Cách tổ chức thư mục trong project 175

7.3 TÁCH THÀNH NHIỀU PROJECT 176
7.3.1 Cách tổ chức thư mục trong project 178
TÓM TẮT 180
CÂU HỎI ÔN TẬP 180
BÀI 8: TÍCH HỢP THƯ VIỆN 183
8.1 TỔNG QUAN VỀ NUGET VÀ THƯ VIỆN BÊN NGOÀI 183
8.2 XUẤT DỮ LIỆU RA EXCEL — CLOSEDXML 185
8.3 TẠO PDF — QUESTPDF 186
8.4 VẼ BIỂU ĐỒ — SCOTTPLOT 187
8.5 MÃ QR — QRCODER 188
TÓM TẮT 189
CÂU HỎI ÔN TẬP 189
TÀI LIỆU THAM KHẢO 193

HƯỚNG DẪN

MÔ TẢ MÔN HỌC

Học phần **Lập trình trên môi trường Windows** cung cấp cho sinh viên những kiến thức và kỹ năng nền tảng về phát triển ứng dụng bằng nền tảng .NET và ngôn ngữ lập trình C#. Nội dung môn học bao gồm các kiến thức từ cơ bản đến nâng cao như cú pháp ngôn ngữ C#, lập trình hướng đối tượng, thiết kế giao diện với Windows Forms, kết nối và thao tác cơ sở dữ liệu bằng Entity Framework Core, lập trình bất đồng bộ, truy vấn dữ liệu với LINQ, cũng như tổ chức ứng dụng theo mô hình kiến trúc nhiều tầng (3-Layer Architecture).

Sau khi hoàn thành học phần, sinh viên có thể phân tích yêu cầu, thiết kế và phát triển các ứng dụng sử dụng .NET và C#, vận dụng Entity Framework Core để xây dựng các hệ thống quản lý dữ liệu, đồng thời áp dụng các nguyên tắc tổ chức phần mềm nhằm tạo ra các ứng dụng có tính mở rộng, dễ bảo trì và phù hợp với nhu cầu phát triển phần mềm hiện nay.

NỘI DUNG MÔN HỌC

- Bài 1. **Tổng quan về .NET và ngôn ngữ C#:** Giới thiệu nền tảng .NET, kiến trúc .NET, thư viện lớp, bộ thu dọn rác (Garbage Collection), các kiến thức cơ bản của ngôn ngữ C# như kiểu dữ liệu, biến, toán tử, cấu trúc điều khiển, hàm, mảng, collection và truy vấn dữ liệu với LINQ.
- Bài 2. **Hướng đối tượng trong C#:** Trình bày các nguyên lý lập trình hướng đối tượng, xây dựng lớp và đối tượng, constructor, property, phương thức, phạm vi truy cập, kế thừa, đa hình, abstract class và interface.
- Bài 3. **Lập trình giao diện với Windows Forms:** Hướng dẫn xây dựng giao diện ứng dụng Windows bằng Windows Forms, tìm hiểu mô hình lập trình hướng sự kiện (Event-driven Programming), sử dụng các điều khiển phổ biến như Label, TextBox, Button, CheckBox, RadioButton, ComboBox, ListBox, DateTimePicker, PictureBox và DataGridView.
- Bài 4. **Thiết kế ứng dụng nhiều Form (Multi-Form & MDI):** Xây dựng ứng dụng với nhiều cửa sổ, truyền dữ liệu giữa các Form, sử dụng MDI cùng các thành

phần giao diện như MenuStrip, ToolStrip, StatusStrip, ContextMenuStrip và các hộp thoại làm việc với tệp.

- Bài 5: **Kết nối dữ liệu với Entity Framework Core:** Giới thiệu lập trình bất đồng bộ, chuyển từ ADO.NET sang Entity Framework Core, xây dựng mô hình dữ liệu, cấu hình DbContext, Migration và thực hiện các thao tác CRUD trên cơ sở dữ liệu.
- Bài 6: **Truy vấn và quản lý dữ liệu với Entity Framework Core:** Khai thác các mối quan hệ dữ liệu trong Entity Framework Core, truy vấn dữ liệu với LINQ, sử dụng Transaction, BindingSource, ErrorProvider và Validation nhằm xây dựng các ứng dụng có tính ổn định và đảm bảo tính toàn vẹn dữ liệu.
- Bài 7: **Mô hình tổ chức dự án:** Giới thiệu mô hình kiến trúc 3 tầng (Presentation Layer, Business Logic Layer và Data Access Layer), phương pháp tổ chức thư mục, tách dự án thành nhiều project và xây dựng cấu trúc phần mềm dễ mở rộng, bảo trì.
- Bài 8: **Tích hợp thư viện:** Hướng dẫn sử dụng các thư viện phổ biến trong phát triển ứng dụng như ClosedXML để xuất Excel, QuestPDF để tạo báo cáo PDF, ScottPlot để vẽ biểu đồ và QRCoder để tạo mã QR phục vụ các yêu cầu thực tế.

KIẾN THỨC TIỀN ĐỀ

Để học tốt học phần **Lập trình C# trên Windows**, sinh viên cần có kiến thức nền tảng về **lập trình C** và **lập trình hướng đối tượng (OOP)**. Những kiến thức này là cơ sở để tiếp cận ngôn ngữ C#, xây dựng ứng dụng Windows và phát triển các hệ thống phần mềm sử dụng nền tảng .NET.

YÊU CẦU MÔN HỌC

Người học phải dự học đầy đủ các buổi lên lớp và làm bài tập đầy đủ ở nhà.

CÁCH TIẾP NHẬN NỘI DUNG MÔN HỌC

Để học tốt môn này, người học cần ôn tập các bài đã học, trả lời các câu hỏi và làm đầy đủ bài tập; đọc trước bài mới và tìm thêm các thông tin liên quan đến bài học.

Đối với mỗi bài học, người học đọc trước mục tiêu và tóm tắt bài học, sau đó đọc nội dung bài học. Kết thúc mỗi ý của bài học, người đọc trả lời câu hỏi ôn tập và kết thúc toàn bộ bài học, người đọc làm các bài tập.

PHƯƠNG PHÁP ĐÁNH GIÁ MÔN HỌC

Môn học được đánh giá gồm:

- Điểm quá trình (50%): Hình thức và cách đánh giá do giảng viên dạy lý thuyết quyết định được phê duyệt của bộ môn.
- Điểm thi cuối kỳ (50%): Hình thức làm đồ án môn học và chấm thi vấn đáp dựa trên nội dung đồ án môn học kết hợp với lý thuyết trong các bài học. Danh sách đồ án do giảng viên quyết định được bộ môn kiểm duyệt và cung cấp vào đầu khóa học.

BÀI 1: TỔNG QUAN VỀ .NET VÀ NGÔN NGỮ C#

Sau khi học xong bài này, học viên có thể:

- *Trình bày được kiến trúc tổng quan của .NET (CLR, BCL) và giải thích quá trình biên dịch từ source code → IL → native code*
- *So sánh được sự khác biệt giữa .NET Framework, .NET Core và .NET hiện đại (.NET 5+), nêu được một số điểm mới của .NET 10*
- *Cài đặt được môi trường phát triển (.NET SDK 10, Visual Studio) và tạo được một Console Project đầu tiên bằng cả CLI (dotnet new) và Visual Studio*
- *Viết được hàm (method) có tham số, giá trị trả về, và sử dụng được List<T>/Array để lưu trữ, xử lý một danh sách dữ liệu đơn giản. Sử dụng được các phương thức LINQ cơ bản (Where, Select, OrderBy) để truy vấn, lọc, sắp xếp dữ liệu trên List<T>.*

1.1 GIỚI THIỆU .NET

1.1.1 Tổng quan

Microsoft .NET gồm 2 phần chính: Platform (nền tảng/SDK) và Integrated Development Environment (IDE / môi trường phát triển).

- **Platform/SDK** cung cấp những gì cần thiết và căn bản nhất: runtime để thực thi ứng dụng (CLR), thư viện lớp cơ sở (BCL), và bộ công cụ biên dịch (SDK). Đây là "khung" theo đúng nghĩa — một hạ tầng theo quy ước chung để mọi ứng dụng .NET chạy nhất quán trên đó, bất kể viết bằng ngôn ngữ nào.
- **IDE** cung cấp môi trường giúp triển khai ứng dụng dễ dàng và nhanh chóng. Không có IDE, ta vẫn có thể dùng một trình soạn thảo văn bản bất kỳ (VS Code,

Notepad++...) kết hợp dòng lệnh (`dotnet build`, `dotnet run`) để biên dịch và thực thi — nhưng sẽ mất nhiều thời gian hơn. Dùng IDE (Visual Studio, VS Code với C# Dev Kit, Rider) vẫn là cách hiệu quả và phổ biến nhất hiện nay.

- Thành phần **Platform/SDK** là quan trọng nhất — đó là cốt lõi và tinh hoa của môi trường .NET; IDE chỉ là công cụ hỗ trợ phát triển dựa trên nền tảng đó. Trong .NET hiện đại, các ngôn ngữ **C#, F#, VB.NET** đều dùng chung một SDK, một runtime, và có thể dùng chung IDE để phát triển.

Tóm lại, Microsoft .NET là nền tảng **mã nguồn mở, đa nền tảng (cross-platform)** dùng để xây dựng và thực thi nhiều loại ứng dụng hiện đại — từ ứng dụng desktop, web, mobile, cloud, đến AI/ML — chạy được trên **Windows, Linux và macOS**.

1.1.2 Lịch sử hình thành

Cuối những năm 1990, Microsoft nhận thấy các công nghệ phát triển ứng dụng khi đó như Win32, COM và COM+ còn nhiều hạn chế: việc tái sử dụng mã nguồn khó khăn, các ngôn ngữ lập trình hoạt động rời rạc và chưa có một môi trường thực thi thống nhất. Vì vậy, Microsoft bắt đầu nghiên cứu một nền tảng phát triển ứng dụng mới với mục tiêu đơn giản hóa việc xây dựng phần mềm và cho phép nhiều ngôn ngữ lập trình cùng hoạt động trên một môi trường chung.

Dự án này được khởi động vào năm 1998 với tên gọi Next Generation Windows Services (NGWS). Sau đó, dự án được mở rộng và trở thành nền tảng .NET. Đến tháng 7/2000, Microsoft lần đầu giới thiệu .NET tại hội nghị Professional Developers Conference (PDC), và Beta 1 được phát hành vào tháng 11/2000.

Năm 2002, Microsoft chính thức phát hành .NET Framework 1.0 cùng với Visual Studio .NET, đánh dấu sự ra đời của một nền tảng phát triển ứng dụng hoàn toàn mới. Điểm nổi bật của .NET là cung cấp Common Language Runtime (CLR) để thực thi chương trình và Base Class Library (BCL) để chia sẻ các thư viện dùng chung, cho phép nhiều ngôn ngữ như C#, Visual Basic .NET và F# cùng hoạt động trên một nền tảng thống nhất.

Đến năm 2016, Microsoft giới thiệu .NET Core, phiên bản mã nguồn mở, đa nền tảng, có thể chạy trên Windows, Linux và macOS. Từ năm 2020, Microsoft hợp nhất

.NET Framework và .NET Core thành nền tảng .NET hiện đại (bắt đầu từ .NET 5). Hiện nay, các phiên bản như .NET 8 và .NET 9 tiếp tục được phát triển và là nền tảng chính để xây dựng các ứng dụng desktop, web, dịch vụ và điện toán đám mây.

1.1.3 Quá trình phát triển

Kể từ khi ra mắt vào năm 2002, .NET đã trải qua nhiều giai đoạn phát triển quan trọng. Ban đầu, Microsoft phát triển .NET Framework dành riêng cho hệ điều hành Windows. Sau đó, để đáp ứng nhu cầu phát triển ứng dụng đa nền tảng và điện toán đám mây, Microsoft giới thiệu .NET Core. Từ năm 2020, hai nền tảng này được hợp nhất thành .NET hiện đại.

Giai đoạn .NET Framework (2002 – 2014)

.NET Framework 1.0 (2002): Phiên bản đầu tiên, phát hành cùng Visual Studio .NET 2002, đặt nền móng cho nền tảng .NET với CLR và thư viện lớp cơ sở (BCL).

.NET Framework 2.0 (2005): Bổ sung nhiều tính năng quan trọng như Generics, hỗ trợ tốt hơn cho ứng dụng 64-bit và nâng cao hiệu năng.

.NET Framework 3.0 (2006): Giới thiệu các công nghệ mới như Windows Presentation Foundation (WPF), Windows Communication Foundation (WCF) và Windows Workflow Foundation (WF).

.NET Framework 3.5 (2007): Điểm nổi bật là sự xuất hiện của LINQ (Language Integrated Query), giúp truy vấn dữ liệu trực tiếp trong C#.

.NET Framework 4.x (2010 – 2019): Tiếp tục cải thiện hiệu năng, bổ sung thư viện mới và hỗ trợ tốt hơn cho lập trình bất đồng bộ với async/await.

Trong nhiều năm, .NET Framework là nền tảng chính để phát triển các ứng dụng Windows, đặc biệt là Windows Forms, WPF và ASP.NET.

Giai đoạn .NET Core (2016 – 2019)

Nhằm đáp ứng xu hướng mã nguồn mở, đa nền tảng và điện toán đám mây, Microsoft giới thiệu .NET Core vào năm 2016.

.NET Core 1.x (2016): Phiên bản đầu tiên, hỗ trợ Windows, Linux và macOS.

.NET Core 2.x (2017 – 2018): Cải thiện khả năng tương thích và hiệu năng, giới thiệu .NET Standard 2.0.

.NET Core 3.0 và 3.1 (2019): Bổ sung hỗ trợ Windows Forms và WPF, giúp các ứng dụng desktop cũng có thể chạy trên nền tảng mới. Trong đó, .NET Core 3.1 là phiên bản LTS (Long-Term Support) cuối cùng trước khi hợp nhất.

Kỷ nguyên .NET hiện đại (2020 đến nay)

Tháng 11 năm 2020, Microsoft phát hành .NET 5, chính thức hợp nhất .NET Framework, .NET Core và Mono thành một nền tảng duy nhất mang tên .NET.

Từ đó, Microsoft duy trì chu kỳ phát hành mỗi năm một phiên bản, trong đó:

Phiên bản số chẵn (6, 8, 10, ...) là LTS (Long-Term Support), được hỗ trợ dài hạn.

Phiên bản số lẻ (5, 7, 9, ...) là STS (Standard Term Support), được hỗ trợ ngắn hạn và bổ sung các công nghệ mới.

Một số phiên bản tiêu biểu:

.NET 6 (2021): Phiên bản LTS đầu tiên của nền tảng hợp nhất, giới thiệu .NET MAUI và cải thiện mạnh về hiệu năng.

.NET 7 (2022): Tập trung tối ưu hiệu năng và mở rộng Native AOT.

.NET 8 (2023): Phiên bản LTS với nhiều cải tiến về hiệu năng, ASP.NET Core và khả năng tích hợp AI.

.NET 9 (2024): Tiếp tục tối ưu runtime và công cụ phát triển.

.NET 10 (2025): Phiên bản LTS mới nhất, có hiệu năng cao, hỗ trợ đầy đủ desktop, web, cloud và AI. Đây cũng là phiên bản được sử dụng trong giáo trình này.

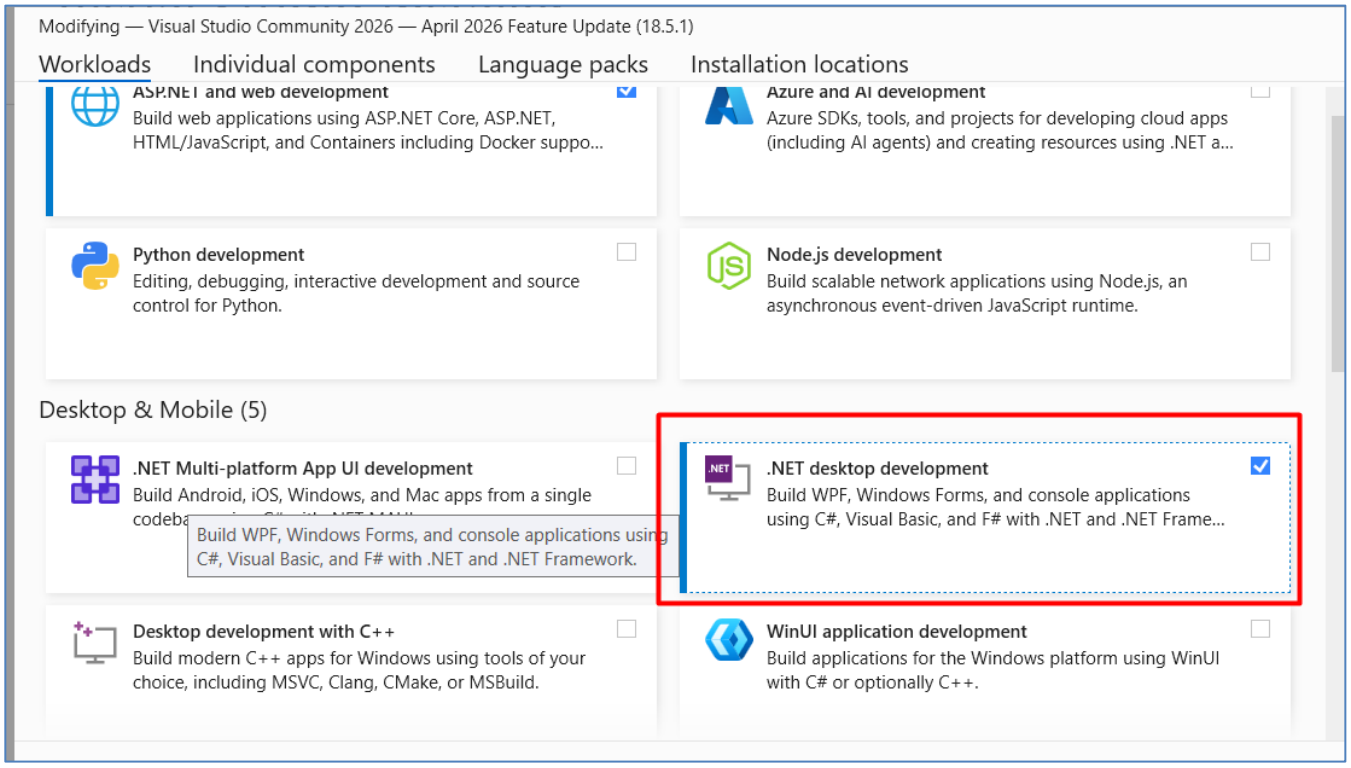


1.1.4 Cài đặt môi trường phát triển

Có hai cách thiết lập môi trường để lập trình C# và .NET:

Cách 1 — Cài Visual Studio (khuyến nghị cho môn học này): Visual Studio là IDE đầy đủ tính năng của Microsoft, tích hợp sẵn .NET SDK, trình biên dịch C#, và đặc biệt có WinForms Designer — công cụ kéo-thả để thiết kế giao diện trực quan.

Tải về tại: https://visualstudio.microsoft.com/ — chọn bản Community (miễn phí). Khi cài đặt, nhớ tick chọn workload .**NET desktop development** để có đầy đủ các công cụ cần thiết.



Cách 2 — Chỉ cài .NET SDK kết hợp VS Code: phù hợp khi chỉ cần viết code console hoặc web, không cần Designer kéo-thả. Vì môn học có phần thiết kế giao diện WinForms, cách này không được khuyến nghị.

1.2 ĐẶC ĐIỂM .NET

.NET hiện đại nổi bật với một loạt đặc điểm cốt lõi tạo nên sức hấp dẫn của nền tảng này.

- Mã nguồn mở — toàn bộ runtime, compiler và thư viện chuẩn được công bố công khai trên GitHub, cho phép cộng đồng theo dõi, báo cáo lỗi và đóng góp trực tiếp vào quá trình phát triển.
- Đa nền tảng — khác với .NET Framework chỉ chạy trên Windows, .NET hiện đại cho phép build và triển khai ứng dụng C# trên Windows, Linux và macOS mà hầu như không cần thay đổi mã nguồn, kể cả trong môi trường container như Docker.
- Nền tảng thống nhất — từ .NET 5 trở đi, mọi loại ứng dụng (desktop, web, cloud, IoT) đều chạy trên cùng một runtime và một SDK duy nhất, thay vì bị chia cắt giữa .NET Framework và .NET Core như trước đây.
- Đa ngôn ngữ — C#, F# và VB.NET cùng chạy trên một runtime chung (CLR), cho phép các thư viện viết bằng ngôn ngữ này được sử dụng trực tiếp từ project viết bằng ngôn ngữ khác.
- Hiệu năng cao — JIT compiler được tối ưu liên tục, Native AOT giúp ứng dụng khởi động gần như tức thì, Garbage Collector giải phóng lập trình viên khỏi việc quản lý bộ nhớ thủ công.
- An toàn và bảo mật — CLR kiểm tra kiểu dữ liệu nghiêm ngặt tại runtime, các cơ chế mã hóa, xác thực và quản lý quyền truy cập được tích hợp sẵn trong nền tảng.
- Hệ sinh thái phong phú — từ Visual Studio, VS Code đến hàng chục nghìn package trên NuGet, cùng với sự tích hợp ngày càng sâu với AI (ML.NET, OpenAI, Azure AI) từ .NET 8 trở đi.

1.2.1 Kiến trúc .Net

Kiến trúc .NET được xây dựng dựa trên ba thành phần cốt lõi: CLR, BCL, và quy trình biên dịch từ mã nguồn đến khi thực thi.

- CLR (Common Language Runtime) là môi trường thực thi chung cho mọi ứng dụng .NET, đóng vai trò trung gian giữa mã nguồn và hệ điều hành. CLR đảm nhiệm những công việc quan trọng mà lập trình viên không cần tự xử lý: quản lý bộ nhớ (Garbage Collector), kiểm tra an toàn kiểu dữ liệu, xử lý ngoại

lệ, quản lý luồng và nạp thư viện. Chính nhờ CLR đứng ở giữa, các ngôn ngữ như C#, F#, VB.NET đều có thể chạy trên cùng một runtime và gọi lẫn nhau dễ dàng.

- BCL (Base Class Library) là bộ công cụ khổng lồ đi kèm với CLR — tập hợp hàng nghìn class và method có sẵn cho những nhu cầu thường gặp: xử lý chuỗi, tập hợp dữ liệu, file, mạng, ngày giờ... Nhờ BCL, lập trình viên không cần "phát minh lại bánh xe" mỗi khi bắt đầu project mới.
- Quy trình biên dịch của .NET diễn ra theo hai bước. Khi nhấn build, trình biên dịch C# không tạo ra mã máy trực tiếp như C/C++, mà biên dịch ra IL (Intermediate Language) — mã trung gian độc lập với phần cứng, được đóng gói trong file .dll hoặc .exe. Đến khi chạy, CLR dùng JIT Compiler để biên dịch IL thành mã máy thực sự ngay tại thời điểm đó, chỉ biên dịch phần nào đang được gọi đến. Ngoài ra, từ các phiên bản .NET gần đây còn có thêm Native AOT — biên dịch thẳng ra mã máy ngay lúc build, bỏ qua JIT hoàn toàn, giúp ứng dụng khởi động nhanh hơn đáng kể.

1.2.2 Thư viện lớp .NET

Thư viện lớp .NET (BCL – Base Class Library, đôi khi gọi là FCL) là tập hợp sẵn hàng nghìn class, interface, struct và kiểu dữ liệu do Microsoft cung cấp, đóng vai trò như "bộ công cụ nền tảng" dùng chung cho mọi ứng dụng .NET. Nhờ đó, lập trình viên không cần tự xây dựng lại các chức năng cơ bản mà có thể sử dụng trực tiếp trong C#, F# hay VB.NET.

Nếu CLR là môi trường thực thi bên dưới, thì BCL chính là phần công cụ mà lập trình viên sử dụng hàng ngày thông qua từ khóa using trong mã nguồn.

Tổ chức theo Namespace

Thư viện lớp .NET được tổ chức theo hệ thống **namespace** (không gian tên), giúp nhóm các lớp có chức năng liên quan, tránh trùng lặp tên và hỗ trợ lập trình viên dễ dàng tìm kiếm, sử dụng các thành phần cần thiết trong quá trình phát triển ứng dụng.

Một số namespace quan trọng và thường gặp:

Namespace	Chức năng chính
System	Các kiểu dữ liệu cơ bản nhất: String, Int, DateTime, Console, Exception.
System.Collections.Generic	Các cấu trúc dữ liệu tổng quát: `List<T>`, `Dictionary<TKey, TValue>`, `Queue<T>`, `Stack<T>`
System.IO	Đọc/viết file, thư mục, luồng dữ liệu (stream)
System.Linq	Các phương thức truy vấn dữ liệu theo phong cách LINQ (`Where`, `Select`, `OrderBy`...)
System.Text	Xử lý chuỗi nâng cao, mã hóa ký tự (`StringBuilder`, `Encoding`)
System.Threading.Tasks	Lập trình bất đồng bộ (`Task`, `async/await`)
System.Windows.Forms	Làm việc với cơ sở dữ liệu qua EF Core (thuộc một package mở rộng riêng, không nằm trong BCL gốc)
Microsoft.EntityFrameworkCore	Làm việc với cơ sở dữ liệu qua EF Core (thuộc một package mở rộng riêng, không nằm trong BCL gốc)

Vai trò trong thực tế, Khi viết một dòng code đơn giản như:

```
List<string> students = new List<string>();
students.Add("Nguyễn Văn A");

foreach (var s in students.Where(x => x.StartsWith("Nguyễn")))
{
    Console.WriteLine(s);
}
```

Lập trình viên đang sử dụng đồng thời ba phần của Thư viện lớp .NET: `List<T>` (từ `System.Collections.Generic`), `Console.WriteLine` (từ `System`), và phương thức `Where` (từ `System.Linq`) — tất cả đều có sẵn, đã được kiểm thử kỹ lưỡng, và được tối ưu hiệu năng bởi chính đội ngũ phát triển .NET, mà lập trình viên không cần viết thêm một dòng code nào để "tự chế" những chức năng này. Biên dịch và MS Intermediate Language.

Mở rộng qua NuGet

Bên cạnh thư viện lõi của .NET, hệ sinh thái còn được mở rộng mạnh mẽ thông qua NuGet — kho quản lý package trung tâm của .NET. NuGet cung cấp hàng trăm nghìn thư viện do Microsoft và cộng đồng phát triển như Entity Framework Core, Newtonsoft.Json, Serilog hay AutoMapper.

Sự kết hợp giữa thư viện có sẵn và hệ sinh thái NuGet giúp .NET có năng suất phát triển cao, vì phần lớn các chức năng phổ biến đã được xây dựng sẵn, lập trình viên chỉ cần tập trung vào logic nghiệp vụ của ứng dụng.

1.2.3 Garbage collection - bộ thu dọn rác

Garbage Collection (GC) là cơ chế quản lý bộ nhớ tự động trong .NET, được thực hiện bởi Garbage Collector trong CLR. Thay vì phải giải phóng bộ nhớ thủ công như C/C++, lập trình viên chỉ cần sử dụng đối tượng, GC sẽ tự động thu hồi các đối tượng không còn được tham chiếu, giúp hạn chế rò rỉ bộ nhớ và tăng độ ổn định của ứng dụng.

GC hoạt động theo ba bước chính: Mark (đánh dấu đối tượng còn sử dụng), Sweep (giải phóng đối tượng không còn dùng) và Compact (dồn nén bộ nhớ để giảm phân mảnh).

Để tối ưu hiệu năng, GC chia đối tượng thành ba thế hệ: Gen 0 (đối tượng mới, được quét thường xuyên), Gen 1 (trung gian) và Gen 2 (đối tượng sống lâu, ít bị quét). Cách phân chia này giúp tối ưu hiệu suất vì phần lớn đối tượng trong ứng dụng có vòng đời ngắn.

1.3 Ngôn ngữ lập trình C#

1.3.1 Giới thiệu ngôn ngữ lập trình C#

C# là ngôn ngữ lập trình do Microsoft phát triển cùng với nền tảng .NET vào khoảng năm 2000, với mục tiêu trở thành ngôn ngữ tiêu biểu cho hệ sinh thái này. Ngôn ngữ này được thiết kế dưới sự dẫn dắt của Anders Hejlsberg — người từng tham gia phát triển Turbo Pascal và Delphi — vì vậy C# mang tính hiện đại, chặt chẽ và dễ tiếp cận.

C# là ngôn ngữ hướng đối tượng hoàn chỉnh, hỗ trợ đầy đủ bốn tính chất: đóng gói, kế thừa, đa hình và trừu tượng. Ngôn ngữ này có hệ thống kiểu dữ liệu an toàn, giúp phát hiện lỗi ngay từ giai đoạn biên dịch. Kết hợp với cơ chế Garbage Collection của .NET, C# giúp lập trình viên giảm thiểu việc quản lý bộ nhớ thủ công và tập trung vào logic ứng dụng.

C# được phát triển liên tục cùng .NET, mỗi phiên bản mới đều bổ sung thêm cú pháp và tính năng hiện đại như pattern matching, record, LINQ hay top-level statements, giúp mã nguồn ngắn gọn và dễ đọc hơn.

Ngoài ra, C# có tính đa năng cao, được sử dụng để phát triển nhiều loại ứng dụng như console, desktop (WinForms, WPF), web (ASP.NET Core, Blazor), cloud và cả game thông qua Unity. Nhờ đó, C# trở thành lựa chọn phổ biến trong giảng dạy và phát triển phần mềm thực tế, đặc biệt tại Việt Nam.

Trong môn học này, C# được sử dụng làm ngôn ngữ chính, bắt đầu từ Console App, sau đó đến WinForms và cuối cùng là làm việc với cơ sở dữ liệu qua Entity Framework Core.

Để hình dung cụ thể hơn về sự "gọn gàng" mà C# hiện đại mang lại, hãy cùng xem một chương trình nhỏ — in ra một lời chào đơn giản:

```
1  Console.WriteLine("Hello, World!");
2  Console.WriteLine("Xin chào, đây là chương trình C# đầu tiên!");
3  string ten = "Sinh viên";
4  Console.WriteLine($"Chào {ten}, chúc bạn học tốt môn .NET!");
```

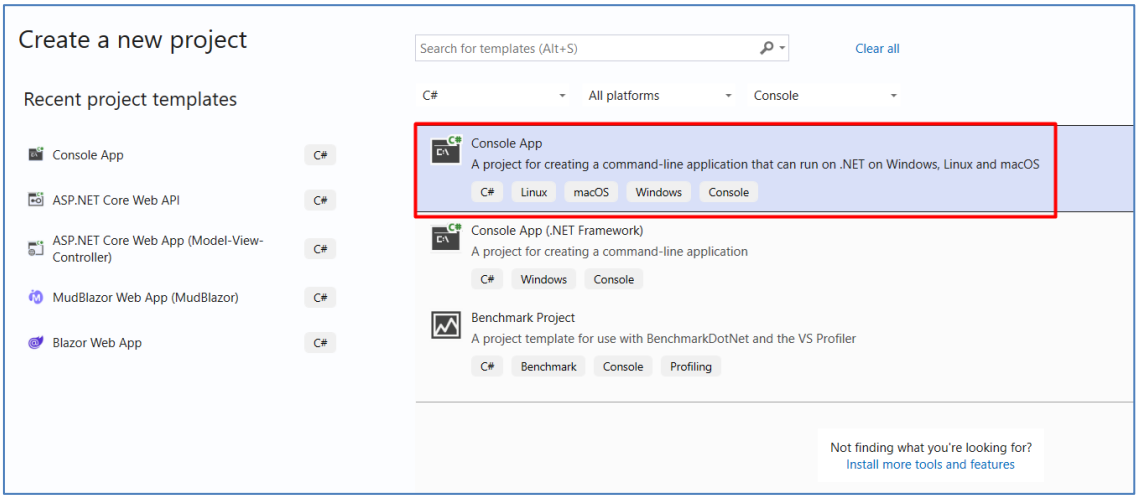
1.3.2 Tạo Console App đầu tiên

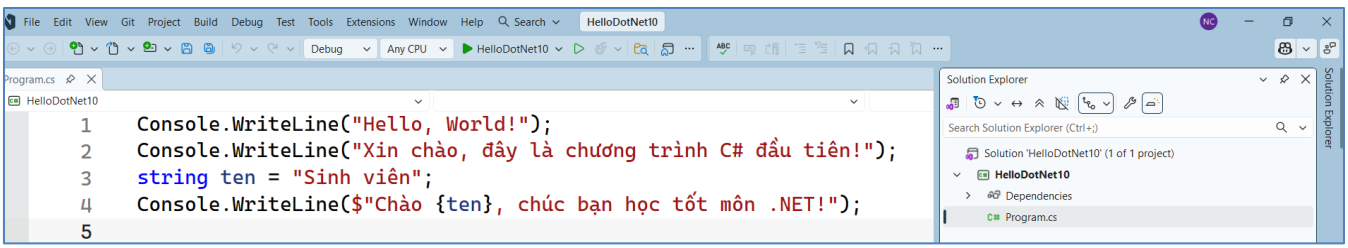
Sau khi đã làm quen với C# qua phần giới thiệu trước đó, bước tiếp theo là tạo chương trình C# đầu tiên. Có hai cách phổ biến để thực hiện: sử dụng giao diện trực quan của Visual Studio hoặc sử dụng dòng lệnh dotnet CLI được cung cấp bởi .NET SDK. Cả hai phương pháp đều cho cùng một kết quả và việc nắm vững cả hai sẽ giúp linh hoạt hơn trong quá trình phát triển sau này.

Cách 1: Tạo project bằng Visual Studio

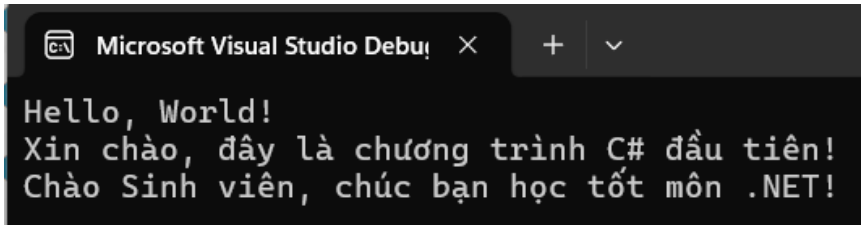
Đây là cách quen thuộc và trực quan nhất, đặc biệt phù hợp cho người mới bắt đầu. Người học mở Visual Studio và chọn **Create a new project** ở màn hình khởi động. Trong danh sách mẫu dự án, tìm và chọn **Console App** (C#).

Sau đó, đặt tên project (ví dụ: HelloDotNet10), chọn thư mục lưu trữ và xác nhận phiên bản **.NET 10.0**. Chỉ với vài thao tác đơn giản, Visual Studio sẽ tự động tạo cấu trúc dự án hoàn chỉnh và mở file Program.cs, nơi bắt đầu viết những dòng code C# đầu tiên.





và sẵn sàng để nhấn nút **Run** màu xanh ở phía trên để xem kết quả.



Cách 2: Tạo project bằng dòng lệnh (CLI)

Với những ai muốn hiểu rõ cách hoạt động "bên dưới" của Visual Studio, việc sử dụng dòng lệnh sẽ giúp quan sát rõ hơn quá trình tạo project trong .NET.

Mở Terminal hoặc Command Prompt và thực hiện các lệnh sau:

```
dotnet new console -n HelloDotNet10
cd HelloDotNet10
dotnet run
```

Lệnh đầu tiên dùng để tạo project Console App mới, lệnh thứ hai di chuyển vào thư mục project vừa tạo, và lệnh thứ ba dùng để biên dịch và chạy chương trình.

Khám phá cấu trúc project vừa tạo

Dù tạo project bằng Visual Studio hay CLI, kết quả đều là một thư mục có cấu trúc tương tự nhau. Việc hiểu rõ các thành phần bên trong giúp nắm được nền tảng của mọi dự án .NET sau này.

- File **.csproj** (ví dụ: HelloDotNet10.csproj)

Đây là file cấu hình của project, được viết dưới dạng XML. Tên file thường trùng với tên project và phần mở rộng .csproj là viết tắt của C# Project. File này chứa các thông tin quan trọng như phiên bản .NET sử dụng, các thư viện tham chiếu và cấu hình build của dự án. Nội dung của nó thường khá ngắn gọn và dễ đọc.

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

- File **Program.cs**

Đây là file chứa điểm bắt đầu của chương trình. Từ các phiên bản C# hiện đại, file này thường được viết theo kiểu top-level statements, nghĩa là không cần khai báo class Main như trước đây nhưng vẫn đảm bảo chương trình có thể chạy được.

- Thư mục **obj** và **bin**

Đây là các thư mục được hệ thống tạo tự động trong quá trình build. obj chứa các file trung gian khi biên dịch, còn bin chứa file thực thi cuối cùng (.exe hoặc .dll).

1.3.3 Nhập xuất dữ liệu với thư viện Console

Trong ứng dụng Console, toàn bộ việc giao tiếp với người dùng — hiển thị kết quả ra màn hình và đọc dữ liệu từ bàn phím — đều thực hiện thông qua lớp `Console` có sẵn trong thư viện .NET, không cần cài đặt thêm bất kỳ package nào.

Xuất dữ liệu ra màn hình

```
Console.Write("Xin chào!");      // Xuất và giữ nguyên vị trí con trỏ
Console.WriteLine("Xin chào!"); // Xuất và xuống dòng mới
Console.WriteLine();       // Xuống dòng trống (tạo khoảng cách)
```

Định dạng chuỗi với `$"..."` (string interpolation) — cách viết gọn và dễ đọc nhất để ghép biến vào chuỗi:

```
string hoTen = "Nguyễn Văn An";
int tuoi = 21;
double diem = 8.5;

Console.WriteLine($"Họ tên: {hoTen}");
Console.WriteLine($"Tuổi: {tuoi}");
Console.WriteLine($"Điểm: {diem:0.0}");    // Định dạng 1 chữ số thập phân
Console.WriteLine($"Điểm: {diem:0.00}");   // Định dạng 2 chữ số thập phân
```

Nhập dữ liệu từ bàn phím

```
// Đọc một dòng văn bản từ bàn phím
string? ten = Console.ReadLine();
// Đọc và chuyển đổi sang kiểu số
Console.Write("Nhập tuổi: ");
int tuoi = int.Parse(Console.ReadLine()!);
Console.Write("Nhập điểm: ");
double diem = double.Parse(Console.ReadLine()!);
```

`Console.ReadLine()` luôn trả về kiểu `string?` — chuỗi ký tự người dùng gõ vào cho đến khi nhấn **Enter**. Nếu cần giá trị số, phải chuyển đổi kiểu bằng `int.Parse()`, `double.Parse()` hoặc `Convert.ToInt32()`.

1.3.4 Kiểu dữ liệu và biến

Biến là gì?

Biến là một vùng nhớ dùng để lưu trữ dữ liệu trong quá trình chương trình chạy. Để khai báo một biến, cần xác định kiểu dữ liệu, sau đó là tên biến, và có thể gán luôn giá trị khởi tạo. Cú pháp cơ bản: kiểu_dữ_liệu tên_biến = giá_trị;

```
int tuoi = 25;
string hoTen = "Nguyễn Văn An";
double diemTrungBinh = 8.5;
bool daDangKy = true;
```

Hai nhóm kiểu dữ liệu: Value type và Reference type

Trong C#, kiểu dữ liệu được chia thành hai nhóm chính, ảnh hưởng trực tiếp đến cách lưu trữ và xử lý trong bộ nhớ.

- Value type: lưu trực tiếp giá trị trong Stack. Khi gán sang biến khác sẽ tạo bản sao độc lập, nên thay đổi một biến không ảnh hưởng biến còn lại. Ví dụ: int, double, char, bool.
- Reference type: lưu địa chỉ tham chiếu đến vùng nhớ Heap. Khi gán sang biến khác, cả hai cùng trỏ đến một đối tượng, nên thay đổi một biến sẽ ảnh hưởng biến còn lại. Ví dụ: string, class, List<T>

Các kiểu dữ liệu cơ bản thường dùng

Một số kiểu dữ liệu thường dùng:

Kiểu	Ý nghĩa	Ví dụ giá trị
`string`	Chuỗi ký tự (văn bản)	`"Nguyễn Văn An"`
`char`	Một ký tự duy nhất	`'A'`, `'5'`

`bool`	Giá trị đúng/sai (logic)	`true, false`
`int`	Số nguyên (32-bit)	`25, -10, 0`
`double`	Số thực (độ chính xác cao)	`8.5, 3.14159`
`decimal`	Số thực, độ chính xác cao nhất (thường dùng cho tiền tệ)	`1500000.50m`

Kiểu var — để trình biên dịch tự suy luận

Bên cạnh cách khai báo biến với kiểu dữ liệu rõ ràng, C# còn hỗ trợ từ khóa **`var`** để viết code ngắn gọn hơn:

```
var tuoi = 25;
var hoTen = "Nguyễn Văn An";
var diemTrungBinh = 8.5;
var daDangKy = true;
```

Cần lưu ý rằng var không có nghĩa là biến có thể thay đổi kiểu dữ liệu trong quá trình chạy như một số ngôn ngữ động (ví dụ JavaScript). Thực chất, trình biên dịch C# sẽ xác định kiểu dữ liệu ngay tại thời điểm khai báo dựa trên giá trị được gán, và kiểu đó sẽ cố định trong suốt vòng đời của biến.

Vì vậy, **var** chỉ là cách viết rút gọn giúp mã nguồn ngắn gọn và dễ đọc hơn trong những trường hợp kiểu dữ liệu đã rõ ràng, đặc biệt thường được sử dụng trong LINQ và các đoạn code xử lý dữ liệu sau này.

Kiểu dữ liệu Nullable

Trong C#, các kiểu value type như int, double, bool không thể nhận giá trị null. Tuy nhiên, trong thực tế có nhiều trường hợp cần biểu diễn trạng thái "chưa có dữ liệu".

C# giải quyết bằng cách thêm dấu **?** sau kiểu dữ liệu:

```
int? diem = null; // đại diện cho "chưa có điểm"
diem = 8;         // Sau đó có thể gán giá trị thực
```

Một ví dụ tổng hợp

Hãy xem một đoạn code nhỏ tổng hợp toàn bộ kiến thức vừa tìm hiểu, áp dụng vào project Console App đã tạo ở phần trước:

```
Console.OutputEncoding = System.Text.Encoding.UTF8;
string hoTen = "Nguyễn Văn An";
int tuoi = 20;
double diemTrungBinh = 8.5;
bool daTotNghiep = false;
int? diemThi = null;

Console.WriteLine($"Sinh viên: {hoTen}");
Console.WriteLine($"Tuổi: {tuoi}");
Console.WriteLine($"Điểm trung bình: {diemTrungBinh}");
Console.WriteLine($"Đã tốt nghiệp: {daTotNghiep}");
Console.WriteLine($"Điểm thi: {(diemThi == null ? "Chưa có điểm" : diemThi.ToString())}");
```

1.3.5 Toán tử và biểu thức

Một **biểu thức (expression)** trong C# là một tổ hợp gồm các giá trị, biến, và toán tử, được kết hợp theo quy tắc nhất định để cho ra một kết quả duy nhất. Hiểu rõ các nhóm toán tử dưới đây sẽ là nền tảng để viết được những điều kiện rẽ nhánh và vòng lặp ở phần tiếp theo của bài học.

Toán tử số học (Arithmetic Operators)

Đây là nhóm toán tử quen thuộc nhất, dùng để thực hiện các phép tính cơ bản trên những kiểu dữ liệu số như int, double: +, -, * , /

```
int a = 10, b = 3;
Console.WriteLine(a / b);     // Kết quả: 3
Console.WriteLine((double)a / b);  // Kết quả: 3.3333333333333335
```

Có một điểm cần đặc biệt lưu ý ở đây — phép chia giữa hai số `int` sẽ cho ra kết quả `int`, và phần dư bị **cắt bỏ hoàn toàn** chứ không được làm tròn. Ví dụ, `10 / 3` sẽ cho ra kết quả là `3`, không phải `3.33`. Nếu muốn có kết quả là số thực, cần đảm bảo ít nhất một trong hai số tham gia phép chia là `double`

Bên cạnh năm toán tử cơ bản trên, C# còn có toán tử tăng/giảm giá trị nhanh, thường dùng trong vòng lặp ở phần sau:

```
int x = 5;
x++;    // Tương đương x = x + 1
x--;    // Tương đương x = x - 1
```

Toán tử gán (Assignment Operators)

Toán tử `=` dùng để gán giá trị bên phải cho biến bên trái — chúng ta đã gặp rất nhiều lần ở phần trước. Bên cạnh đó, C# còn có các toán tử gán kết hợp, giúp viết gọn hơn khi muốn vừa tính toán vừa gán lại cho chính biến đó như: +=, -= , *=, /=

Toán tử so sánh (Comparison Operators)

Nhóm toán tử này dùng để so sánh hai giá trị, và luôn trả về một kết quả kiểu `bool` — `true` hoặc `false`. Đây là nhóm toán tử sẽ xuất hiện liên tục trong các cấu trúc điều kiện ở phần tiếp theo: ==, !=, >, <, >=, <=

Toán tử logic (Logical Operators)

Khi cần kết hợp nhiều điều kiện so sánh lại với nhau, ta dùng đến nhóm toán tử logic: &&, ||, !

Ví dụ, để kiểm tra một sinh viên có đủ điều kiện đạt môn hay không (điểm phải từ 5 đến 10):

```
1   double diem = 7.5;
2   bool dat = diem >= 5 && diem <= 10;
3   Console.WriteLine($"Sinh viên đạt môn: {dat}");
```

Toán tử hợp nhất null (Null-Coalescing Operators)

Đây là một nhóm toán tử khá đặc trưng của C# hiện đại, gắn liền với khái niệm kiểu dữ liệu Nullable:

```
1   int? diemThi = null;
2   int diemHienThi = diemThi ?? 0; // Nếu diemThi là null, lấy giá trị 0 thay thế
```

Toán tử `??` sẽ trả về giá trị bên trái nếu nó khác `null`, ngược lại trả về giá trị bên phải. Đây là một cách viết rất gọn để xử lý tình huống "nếu chưa có dữ liệu thì dùng giá trị mặc định".

Thứ tự ưu tiên của toán tử

Trong C#, khi một biểu thức có nhiều toán tử, chương trình sẽ thực hiện theo **thứ tự ưu tiên** tương tự quy tắc toán học: phép trong ngoặc `()` được tính trước, sau đó đến nhân chia, và cuối cùng là cộng trừ. Ví dụ

```
int ketQua = 5 + 3 * 2;         // Kết quả: 11 (nhân trước, cộng sau)
int ketQua2 = (5 + 3) * 2;      // Kết quả: 16 (ngoặc trước)
```

1.3.6 Cấu trúc điều khiển

Cấu trúc rẽ nhánh: if / else

Cấu trúc `if` là cấu trúc điều khiển cơ bản nhất: nó kiểm tra một điều kiện (luôn phải là một biểu thức trả về `bool`), và chỉ thực thi đoạn code bên trong nếu điều kiện đó đúng.

```
double diem = 7.5;
if (diem >= 5)
{
    Console.WriteLine("Sinh viên đạt học phần");
}
```

Để xử lý cả trường hợp ngược lại — khi điều kiện sai — ta thêm vào nhánh `else`:

```
double diem = 7.5;
if (diem >= 5)
{
    Console.WriteLine("Sinh viên đạt học phần");
}
else
{
    Console.WriteLine("Sinh viên không đạt học phần");
}
```

Và khi có nhiều hơn hai trường hợp cần phân biệt, các nhánh `else if` được nối tiếp nhau, được kiểm tra tuần tự từ trên xuống — ngay khi gặp điều kiện đầu tiên đúng, các nhánh còn lại sẽ bị bỏ qua hoàn toàn

```
double diem = 7.5;
if (diem >= 8.5)
{
    Console.WriteLine("Xuất sắc");
}
else if (diem >= 7)
{
    Console.WriteLine("Khá");
}
else if (diem >= 5)
{
    Console.WriteLine("Trung bình");
}
else
{
    Console.WriteLine("Yếu");
}
```

Cấu trúc switch

Khi cần so sánh một biến với nhiều giá trị cụ thể khác nhau (không phải khoảng giá trị như ví dụ trên), `switch` thường giúp code trở nên rõ ràng hơn so với việc viết một chuỗi dài `else if`:

```
int thuTrongTuan = 2;
string tenThu;
switch (thuTrongTuan)
{
    case 1:
        tenThu = "Chủ nhật";
        break;
    case 2:
        tenThu = "Thứ hai";
        break;
    case 3:
        tenThu = "Thứ ba";
        break;
    default:
        tenThu = "Không xác định";
        break;
}
Console.WriteLine(tenThu);
```

Mỗi nhánh case cần kết thúc bằng từ khóa break để dừng việc kiểm tra tiếp các nhánh sau — nếu thiếu break, trình biên dịch C# (khác với một số ngôn ngữ như C/C++) sẽ báo lỗi ngay, chứ không "rơi" tiếp xuống nhánh dưới.

C# hiện đại còn cung cấp một cách viết gọn hơn nhiều, gọi là **switch expression**, đặc biệt hữu ích khi mục đích chỉ là gán một giá trị dựa trên điều kiện:

```
int thuTrongTuan = 2;
string tenThu = thuTrongTuan switch
{
    1 => "Chủ nhật",
    2 => "Thứ hai",
    3 => "Thứ ba",
    _ => "Không xác định"
};
Console.WriteLine(tenThu);
```

Vòng lặp for

Khi cần thực hiện một công việc lặp lại với một số lần xác định trước, vòng lặp `for` là lựa chọn phù hợp nhất. Cấu trúc của nó gồm ba phần, ngăn cách bởi dấu chấm phẩy: giá trị khởi tạo, điều kiện lặp, và bước thay đổi sau mỗi lần lặp.

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine($"Lần lặp thứ {i}");
}
```

Vòng lặp while và do-while

Khác với `for` — thường dùng khi đã biết trước số lần lặp — `while` phù hợp hơn với những tình huống số lần lặp không biết trước, mà phụ thuộc vào một điều kiện nào đó còn đúng hay không:

```
int soDu = 100;
while (soDu > 0)
{
    Console.WriteLine($"Số dư còn lại: {soDu}");
    soDu -= 20;
}
```

Vòng lặp `while` kiểm tra điều kiện **trước**, rồi mới thực thi — nghĩa là nếu điều kiện sai ngay từ đầu, đoạn code bên trong sẽ không được chạy lấy một lần nào. Trong khi đó, biến thể `do-while` đảm bảo đoạn code bên trong luôn được thực thi **ít nhất một lần**, vì điều kiện chỉ được kiểm tra sau khi đã chạy xong:

```
int lanThu = 0;
do
{
    lanThu++;
    Console.WriteLine($"Đang thử lần {lanThu}");
} while (lanThu < 6);
```

Vòng lặp foreach

Đây là vòng lặp được thiết kế chuyên biệt để duyệt qua từng phần tử trong một tập hợp dữ liệu (như `List<T>` hay `Array`). So với `for`, `foreach` không cần quan tâm đến chỉ số (index) của từng phần tử, mà chỉ đơn giản lấy ra từng phần tử một để xử lý:

```
string[] danhSachSinhVien = { "An", "Bình", "Chi" };
foreach (string ten in danhSachSinhVien)
{
    Console.WriteLine($"Sinh viên: {ten}");
}
```

Một ví dụ tổng hợp

```
double[] danhSachDiem = { 9.0, 4.5, 7.0, 5.5, 3.0 };
int soDat = 0;
foreach (double diem in danhSachDiem)
{
    string xepLoai = diem switch
    {
        >= 8.5 => "Xuất sắc",
        >= 7 => "Khá",
        >= 5 => "Trung bình",
        _ => "Yếu"
    };
    Console.WriteLine($"Điểm {diem}: {xepLoai}");
    if (diem >= 5)
    {
        soDat++;
    }
}
Console.WriteLine($"Tổng số sinh viên đạt: {soDat}/{danhSachDiem.Length}");
```

1.3.7 Hàm

Khi chương trình ngày càng lớn, việc viết toàn bộ logic trong một khối code sẽ trở nên khó quản lý và dễ rối. Vì vậy, C# sử dụng **hàm (method)** để tách chương trình thành các phần nhỏ, mỗi phần đảm nhận một chức năng riêng và có thể tái sử dụng nhiều lần.

Hàm là gì, vì sao cần dùng?

Hàm giúp **tái sử dụng code**, tức là một đoạn logic chỉ cần viết một lần nhưng có thể gọi lại ở nhiều nơi. Đồng thời, nó giúp chương trình **dễ đọc, dễ bảo trì và dễ sửa lỗi** hơn, vì mỗi hàm chỉ thực hiện một nhiệm vụ cụ thể thay vì gom toàn bộ vào một khối code lớn.

Cấu trúc cơ bản của một hàm

Một hàm trong C# bao gồm bốn thành phần chính: **kiểu dữ liệu trả về**, **tên hàm**, **danh sách tham số** (đặt trong dấu ngoặc tròn), và **phần thân hàm** (đặt trong dấu ngoặc nhọn) — nơi chứa logic xử lý thực sự.

```
static double TinhDiemTrungBinh(double diem1, double diem2, double diem3)
{
    double trungBinh = (diem1 + diem2 + diem3) / 3;
    return trungBinh;
}
```

Để sử dụng hàm này, ta chỉ cần **gọi** nó bằng tên, kèm theo các giá trị cụ thể cho từng tham số:

```
double ketQua = TinhDiemTrungBinh(8.0, 7.5, 9.0);
Console.WriteLine($"Điểm trung bình: {ketQua}");
```

Hàm không trả về giá trị — kiểu void

Không phải mọi hàm đều cần trả về kết quả. Có những hàm chỉ thực hiện một nhiệm vụ nhất định như in dữ liệu, hiển thị thông báo… mà không cần trả về giá trị cho nơi gọi.

Trong trường hợp này, C# sử dụng từ khóa **void** để biểu thị rằng hàm không trả về dữ liệu nào:

```
static void InThongTinSinhVien(string hoTen, int tuoi)
{
    Console.WriteLine($"Họ tên: {hoTen}");
    Console.WriteLine($"Tuổi: {tuoi}");
}
```

Khi gọi một hàm `void`, ta chỉ đơn giản gọi tên hàm mà không cần (và không thể) gán kết quả của nó cho một biến nào, vì bản chất hàm này không trả về gì cả:

```
InThongTinSinhVien("Nguyễn Văn An", 20);
```

Tham số mặc định

C# cho phép gán giá trị mặc định cho tham số ngay trong khai báo hàm. Khi gọi hàm, nếu không truyền giá trị cho tham số đó, hệ thống sẽ tự động sử dụng giá trị mặc định.

```
static void ChaoSinhVien(string hoTen, string loiChao = "Xin chào")
{
    Console.WriteLine($"{loiChao}, {hoTen}!");
}
ChaoSinhVien("An");                          // Xin chào, An!
ChaoSinhVien("Bình", "Chúc mừng");           // Chúc mừng, Bình!
```

Tính năng này giúp giảm việc viết nhiều hàm gần giống nhau chỉ khác nhau ở một vài tham số.

Truyền tham số theo tham chiếu — ref và out

Mặc định trong C#, tham số được truyền vào hàm theo kiểu giá trị (pass by value), tức là hàm chỉ làm việc với một bản sao của dữ liệu, nên thay đổi bên trong hàm không ảnh hưởng đến biến gốc.

Khi cần thay đổi trực tiếp giá trị của biến bên ngoài, C# cung cấp từ khóa **ref**:

```
static void TangDiem(ref double diem, double mucTang)
{
    diem += mucTang;
}
double diemHienTai = 7.0;
TangDiem(ref diemHienTai, 1.5);
Console.WriteLine(diemHienTai);   // Kết quả: 8.5 – biến gốc đã thay đổi
```

Ngoài ra, từ khóa `out` cũng truyền tham chiếu tương tự `ref`, nhưng thường dùng khi muốn trả về nhiều giá trị từ một hàm hoặc khi biến chưa cần khởi tạo trước khi truyền vào:

```
1 reference
static bool ThuChiaSo(int a, int b, out int ketQua)
{
    if (b == 0)
    {
        ketQua = 0;
        return false;
    }
    ketQua = a / b;
    return true;
}
if (ThuChiaSo(10, 2, out int kq))
{
    Console.WriteLine($"Kết quả chia: {kq}");
}
```

Tham số params — số lượng tham số không cố định

Có những tình huống ta không biết trước chính xác sẽ truyền vào bao nhiêu giá trị. Từ khóa `params` cho phép một hàm nhận vào một số lượng tham số tùy ý, cùng kiểu dữ liệu:

```
static double TinhTong(params double[] cacSo)
{
    double tong = 0;
    foreach (double so in cacSo)
        tong += so;
    return tong;
}
Console.WriteLine(TinhTong(1, 2, 3));          // 6
Console.WriteLine(TinhTong(5, 10, 15, 20));    // 50
```

Phạm vi (scope) của biến trong hàm

Trong C#, biến được khai báo bên trong một hàm chỉ tồn tại và sử dụng được trong chính hàm đó. Phạm vi này được gọi là scope của biến.

Khi ra khỏi hàm, biến sẽ không còn tồn tại và không thể truy cập được nữa

```
static void ViDuPhamVi()
{
    int soTrongHam = 100;
    Console.WriteLine(soTrongHam);   // Hợp lệ
}
Console.WriteLine(soTrongHam);   // Lỗi! Biến này không tồn tại ở đây
```

Một ví dụ tổng hợp

Hãy mở rộng đoạn code quản lý điểm số ở phần trước, lần này tách logic xếp loại thành một hàm riêng để code trở nên rõ ràng và dễ tái sử dụng hơn:

```
static string XepLoai(double diem)
{
    return diem switch
    {
        >= 8.5 => "Xuất sắc",
        >= 7 => "Khá",
        >= 5 => "Trung bình",
        _ => "Yếu"
    };
}
3 references
static void InKetQuaSinhVien(string hoTen, double diem)
{
    string xepLoai = XepLoai(diem);
    Console.WriteLine($"{hoTen} - Điểm {diem} - Xếp loại: {xepLoai}");
}
InKetQuaSinhVien("An", 9.0);
InKetQuaSinhVien("Bình", 6.5);
InKetQuaSinhVien("Chi", 4.0);
```

1.3.8 Mảng và Collection cơ bản

Trong các ví dụ trước, chúng ta mới chỉ làm việc với từng biến riêng lẻ như một điểm số hay một tên. Tuy nhiên, trong thực tế, dữ liệu thường tồn tại dưới dạng tập hợp như danh sách sinh viên, danh sách điểm số,...

Nếu khai báo từng biến riêng lẻ sẽ rất khó mở rộng. Vì vậy, C# cung cấp mảng (Array) và Collection để lưu trữ nhiều giá trị cùng kiểu dữ liệu dưới một biến duy nhất.

Mảng (Array)

Mảng là cấu trúc dữ liệu cơ bản dùng để lưu trữ một tập hợp các giá trị **cùng kiểu dữ liệu**, được sắp xếp liên tiếp trong bộ nhớ. Mỗi phần tử trong mảng được truy cập thông qua **chỉ số (index)**, bắt đầu từ 0 thay vì 1.

```
string[] danhSachSinhVien = { "An", "Bình", "Chi", "Dung" };
Console.WriteLine(danhSachSinhVien[0]);   // An
Console.WriteLine(danhSachSinhVien[1]);   // Bình
Console.WriteLine(danhSachSinhVien.Length); // 4 - số lượng phần tử
```

Có thể truy cập hoặc thay đổi giá trị tại một vị trí cụ thể thông qua chỉ số đó:

```
5   danhSachSinhVien[2] = "Châu";    // Thay "Chi" thành "Châu"
```

Một đặc điểm quan trọng của mảng là kích thước cố định ngay khi khai báo, không thể thêm hoặc xóa phần tử sau đó.

Trong trường hợp cần tạo mảng với số lượng phần tử xác định nhưng chưa gán giá trị cụ thể, có thể khai báo như sau:

```
6   double[] diemKiemTra = new double[5];   // Mảng gồm 5 phần tử double, giá trị mặc định là 0
```

Việc truy cập vào chỉ số nằm ngoài phạm vi của mảng (ví dụ `danhSachSinhVien[10]` trong khi mảng chỉ có 4 phần tử) sẽ gây ra lỗi khi chạy chương trình (**runtime error**) với ngoại lệ `IndexOutOfRangeException`. Đây là lỗi khá phổ biến đối với người mới học, thường xảy ra do nhầm lẫn giữa **c**hỉ số bắt đầu từ 0 và số lượng phần tử thực tế của mảng**.**

Hạn chế của mảng — và lý do cần đến List<T>

Hạn chế lớn nhất của mảng là kích thước cố định, không thể thêm hoặc xóa phần tử sau khi đã khai báo. Điều này gây khó khăn trong thực tế vì dữ liệu thường thay đổi theo thời gian, ví dụ như số lượng sinh viên trong một lớp.

Để giải quyết vấn đề này, C# cung cấp List<T> — một collection linh hoạt hơn mảng, có khả năng tăng giảm kích thước động, cho phép thêm, xóa và chỉnh sửa phần tử trong quá trình chạy chương trình.

List<T> — collection linh hoạt nhất

Để sử dụng List<T>, cần khai báo namespace:

```
using System.Collections.Generic;
```

(Trong các project hiện đại, dòng này thường được tự động thêm nhờ Implicit Usings)

T trong List<T> là viết tắt của Generic type, đại diện cho kiểu dữ liệu mà danh sách sẽ chứa. Nhờ Generic, List<T> có thể dùng cho nhiều kiểu dữ liệu khác nhau nhưng vẫn đảm bảo an toàn kiểu dữ liệu khi biên dịch.

```
1   List<string> danhSachSinhVien = new List<string>();
2   danhSachSinhVien.Add("An");
3   danhSachSinhVien.Add("Bình");
4   danhSachSinhVien.Add("Chi");
5   Console.WriteLine(danhSachSinhVien.Count);   // 3 - số lượng phần tử hiện tại
```

Cũng có thể khởi tạo `List<T>` với sẵn một số giá trị ban đầu, tương tự cách khai báo mảng:

```
List<double> danhSachDiem = new List<double> { 8.5, 7.0, 9.5 };
```

Một số phương thức thường dùng nhất khi làm việc với `List<T>`:

Phương thức	Chức năng	Ví dụ
Add	Thêm một phần tử vào cuối list	`danhSach.Add("An");`
Remove	Xóa phần tử đầu tiên khớp với giá trị	`danhSach.Remove("An");`
RemoveAt	Xóa phần tử tại vị trí chỉ số	`danhSach.RemoveAt(0);`
Contains	Kiểm tra list có chứa giá trị đó không	`danhSach.Contains("An");`
Count	Số lượng phần tử hiện tại (thuộc tính, không phải hàm)	`danhSach.Count();`
Clear	Xóa toàn bộ phần tử trong list	`danhSach.Clear();`

Cũng giống mảng, các phần tử trong `List<T>` vẫn có thể truy cập theo chỉ số (bắt đầu từ `0`), và vẫn dùng `foreach` để duyệt qua từng phần tử một cách tự nhiên:

```
foreach (string ten in danhSach)
{
    Console.WriteLine(ten);
}
```

Nên dùng Array hay List<T>?

Trong thực tế lập trình hiện đại, List<T> được sử dụng phổ biến hơn mảng vì tính linh hoạt: có thể thêm, xóa và thay đổi kích thước trong quá trình chạy. Đây cũng là cấu trúc sẽ được dùng nhiều trong môn học, đặc biệt khi làm việc với LINQ hoặc binding dữ liệu trong Windows Forms.

Trong khi đó, mảng (Array) vẫn có vai trò riêng trong các trường hợp đã biết trước kích thước dữ liệu và cần tối ưu hiệu năng ở mức cao nhất.

Dictionary<TKey, TValue> — lưu trữ theo cặp khóa-giá trị

Bên cạnh List<T>, một collection rất quan trọng khác là Dictionary<TKey, TValue>. Đây là cấu trúc dữ liệu dùng để lưu trữ theo dạng cặp khóa (key) và giá trị (value), tương tự như một "từ điển", trong đó ta dùng từ khóa để tra cứu nội dung tương ứng.

```
Dictionary<string, double> diemSinhVien = new Dictionary<string, double>();
diemSinhVien.Add("An", 8.5);
diemSinhVien.Add("Bình", 7.0);
diemSinhVien["Chi"] = 9.5;   // Cũng có thể thêm/sửa theo cách này
Console.WriteLine(diemSinhVien["An"]);   // Truy xuất giá trị qua khóa: 8.5
```

Khác với List<T> phải truy cập theo chỉ số hoặc duyệt tuần tự, Dictionary cho phép tra cứu trực tiếp theo key, giúp thao tác nhanh và rõ ràng hơn. Vì vậy, nó rất phù hợp cho các tình huống cần ánh xạ dữ liệu, ví dụ như tên sinh viên và điểm số.

Một ví dụ tổng hợp

Kết hợp List<T> với kiến thức về hàm và cấu trúc điều khiển đã học, ta có thể xây dựng một chương trình quản lý điểm số đơn giản:

```
List<string> danhSachTen = new List<string> { "An", "Bình", "Chi", "Dung" };
List<double> danhSachDiem = new List<double> { 9.0, 4.5, 7.5, 5.5 };
int soDat = 0;
for (int i = 0; i < danhSachTen.Count; i++)
{
    string ten = danhSachTen[i];
    double diem = danhSachDiem[i];
    string xepLoai = diem switch
    {
        >= 8.5 => "Xuất sắc",
        >= 7 => "Khá",
        >= 5 => "Trung bình",
        _ => "Yếu"
    };
    Console.WriteLine($"{ten} - Điểm {diem} - Xếp loại: {xepLoai}");
    if (diem >= 5)
    {
        soDat++;
    }
}
Console.WriteLine($"Tổng số sinh viên đạt: {soDat}/{danhSachTen.Count}");
```

Đoạn code thường sử dụng vòng lặp for để duyệt đồng thời hai danh sách song song (một danh sách tên và một danh sách điểm tương ứng). Đây là cách làm phổ biến khi dữ liệu được lưu tách rời.

Tuy nhiên, cách dùng hai List song song chưa tối ưu. Trong phần Hướng đối tượng ở bài sau, chúng ta sẽ gom tên và điểm vào một lớp Student, từ đó sử dụng List<Student> để quản lý dữ liệu rõ ràng và hiệu quả hơn.

1.4 LINQ

1.4.1 LINQ là gì?

Xét một yêu cầu quen thuộc: từ danh sách điểm số, lấy ra những điểm đạt (>= 5) và sắp xếp từ cao xuống thấp. Cách làm tự nhiên nhất với những gì đã học — vòng lặp `for` và câu lệnh `if` — trông như sau:

```
List<double> ketQua = new List<double>();
 for (int i = 0; i<diem.Count; i++)
 {
     if (diem[i] >= 5)
     {
         ketQua.Add(diem[i]);
     }
 }
 // Sắp xếp giảm dần (bubble sort thủ công)
 for (int i = 0; i < ketQua.Count - 1; i++)
 {
     for (int j = i + 1; j < ketQua.Count; j++)
     {
         if (ketQua[j] > ketQua[i])
         {
             double tam = ketQua[i];
             ketQua[i] = ketQua[j];
             ketQua[j] = tam;
         }
     }
 }
```

Đoạn code trên hoàn toàn đúng — nhưng cần đến 15 dòng để thực hiện một yêu cầu có thể diễn đạt bằng một câu tiếng Việt rất ngắn: *"lấy điểm đạt, sắp xếp giảm dần"*. Phần lớn những dòng đó không nói lên ý định của người viết — chúng chỉ là chi tiết kỹ thuật của vòng lặp: khởi tạo biến đếm, so sánh chỉ số, hoán đổi giá trị. Logic thực sự của bài toán — lọc và sắp xếp — bị chìm khuất trong đó.

LINQ (Language Integrated Query) ra đời để giải quyết đúng vấn đề này: thay vì mô tả *cách làm* (duyệt từng phần tử, so sánh, hoán đổi...), LINQ cho phép diễn đạt trực tiếp *điều muốn làm* — gần với ngôn ngữ tự nhiên hơn, và ngắn gọn hơn đáng kể. Đoạn code 15 dòng ở trên thu gọn lại thành:

```
List<double> ketQua = diem.Where(d => d >= 5)
                          .OrderByDescending(d => d)
                          .ToList();
```

Hai cách cho cùng một kết quả — nhưng cách thứ hai đọc gần như giống hệt yêu cầu ban đầu: lấy từ `diem`, những phần tử thỏa `d >= 5`, sắp xếp giảm dần, chuyển về `List`. Đây chính là giá trị cốt lõi mà LINQ mang lại: code ngắn hơn, dễ đọc hơn, và ý định của người viết hiện ra rõ ràng hơn — không bị che khuất bởi các chi tiết kỹ thuật của vòng lặp.

1.4.2 Cú pháp LINQ cơ bản

LINQ trong C# có hai cách viết: Method Syntax (gọi các phương thức nối tiếp nhau bằng dấu chấm) và Query Syntax (viết gần giống câu lệnh SQL, dùng from, where, select). Môn học này tập trung hoàn toàn vào Method Syntax — cách viết phổ biến hơn trong thực tế, và nhất quán với LINQ to Entities sẽ dùng khi làm việc với EF Core ở các bài sau.

Cấu trúc chung của Method Syntax luôn có dạng:

Nguồn_dữ_liệu.PhươngThức(***điều_kiện***)

Trong đó điều kiện thường được viết dưới dạng một lambda expression — một hàm rút gọn, không cần đặt tên, với cú pháp: **tham_số => biểu_thức**

Ví dụ, x => x.Diem >= 5 có thể đọc là: "với mỗi phần tử x, kiểm tra xem x.Diem có lớn hơn hoặc bằng 5 không". Đây chính là "ngôn ngữ" mà hầu hết các phương thức LINQ dùng để diễn đạt điều kiện xử lý — nắm vững cú pháp này là nền tảng để sử dụng toàn bộ các phương thức LINQ.

1.4.3 Các phương thức LINQ thường dùng

Danh sách phân loại các phương thức truy vấn của LINQ được liệt kê trong bảng dưới đây:

Loại	Phương thức	Mô tả
Lọc	*Where()*	Lọc phần tử theo điều kiện
Sắp xếp	*OrderBy()*	Sắp xếp tăng dần
	OrderByDescending()	Sắp xếp giảm dần
	ThenBy()	Sắp xếp phụ tăng dần (sau OrderBy)
	ThenByDescending()	Sắp xếp phụ giảm dần

Chiếu	*Select()*	Chọn/biến đổi từng phần tử thành dạng mới
Tổng hợp	*Count()*	Đếm số phần tử
	Sum()	Tính tổng
	Average()	Tính trung bình
	Min() / Max()	Giá trị nhỏ nhất / lớn nhất
Phân trang	*Skip()*	Bỏ qua N phần tử đầu
	Take()	Lấy N phần tử đầu
Tìm kiếm	*First() / FirstOrDefault()*	Lấy phần tử đầu tiên (hoặc null nếu không có)
	Any()	Kiểm tra có phần tử nào thỏa điều kiện không
	All()	Kiểm tra tất cả phần tử có thỏa điều kiện không
	Contains()	Kiểm tra có chứa phần tử cụ thể không
	Single() / SingleOrDefault()	Lấy đúng một phần tử thỏa điều kiện
Nhóm	*GroupBy()*	Nhóm phần tử theo một tiêu chí
Chuyển đổi	*ToList()*	Chuyển sang `List<T>`
	ToArray()	Chuyển sang mảng
	ToDictionary()	Chuyển sang `Dictionary<K,V>`
Kết hợp	*Join()*	Kết hợp hai tập hợp theo khóa chung (tương tự `INNER JOIN`)
	GroupJoin()	Kết hợp có nhóm (tương tự `LEFT JOIN`)
	Concat()	Nối đuôi hai danh sách (giữ nguyên trùng lặp)
Tổng hợp nâng cao (Từ NET9)	*CountBy()*	Đếm số phần tử theo từng nhóm khóa, trả về `IEnumerable<KeyValuePair<K, int>>` — gọn hơn `GroupBy().Count()`
	AggregateBy()	Gộp dữ liệu theo nhóm khóa với một phép tính tùy chỉnh — tổng quát hơn `CountBy()`
	Index()	Duyệt phần tử kèm theo chỉ số, trả về `(index, element)` — thay thế cách dùng `Select((x, i) => ...)`

Toàn bộ ví dụ trong mục này dùng chung hai danh sách:

```
List<string> hoTen = new List<string>
{
    "Nguyễn Văn An", "Trần Thị Bình", "Lê Văn Chi",
    "Phạm Thị Dung", "Hoàng Văn Em", "Ngô Thị Phương"
};
List<double> diem = new List<double> { 8.5, 6.0, 9.2, 4.5, 7.8, 5.5 };
```

Lọc và sắp xếp

Ví dụ 1: Lấy danh sách điểm đạt (>= 5), sắp xếp từ cao xuống thấp.

```
List<double> ketQua = diem
                    .Where(d => d >= 5)
                    .OrderByDescending(d => d)
                    .ToList();
// Kết quả: [9.2, 8.5, 7.8, 6.0, 5.5]
```

Ví dụ 2: Lấy danh sách tên có chứa chữ "Văn", sắp xếp theo alphabet.

```
List<string> ketQua = hoTen
     .Where(ten => ten.Contains("Văn"))
     .OrderBy(ten => ten)
     .ToList();
 // Kết quả: ["Hoàng Văn Em", "Lê Văn Chi", "Nguyễn Văn An"]
```

Chiếu dữ liệu — Select

Ví dụ 3: Từ danh sách điểm, tạo ra danh sách xếp loại tương ứng ("Giỏi" / "Đạt" / "Yếu").

```
List<string> xepLoai = diem
  .Select(d => d >= 8 ? "Giỏi" : d >= 5 ? "Đạt" : "Yếu")
  .ToList();
 // Kết quả: ["Giỏi", "Đạt", "Giỏi", "Yếu", "Đạt", "Đạt"]
```

Tổng hợp

Ví dụ 4: Tính các thống kê cơ bản của danh sách điểm: tổng số, số đạt, điểm trung bình, điểm cao nhất, điểm thấp nhất.

```
int tongSo = diem.Count();
int soDat = diem.Count(d => d >= 5);
double diemTB = diem.Average();
double diemCao = diem.Max();
double diemThap = diem.Min();
```

Phân trang — Skip và Take

Ví dụ 5: Hiển thị danh sách tên theo từng trang, mỗi trang 2 tên.

```
int soTrenMoiTrang = 2;

 // Trang 1
 List<string> trang1 = hoTen.Take(2).ToList();
 // Kết quả: ["Nguyễn Văn An", "Trần Thị Bình"]

 // Trang 2
 List<string> trang2 = hoTen.Skip(2).Take(2).ToList();
 // Kết quả: ["Lê Văn Chi", "Phạm Thị Dung"]

 // Trang 3
 List<string> trang3 = hoTen.Skip(4).Take(2).ToList();
 // Kết quả: ["Hoàng Văn Em", "Ngô Thị Phương"]
```

Tìm kiếm

Ví dụ 5: Kiểm tra danh sách điểm có sinh viên nào không đạt không, và tất cả sinh viên có đạt không?

```
bool coKhongDat = diem.Any(d => d < 5);  // True
bool tatCaDat = diem.All(d => d >= 5); // False
```

Ví dụ 6: Lấy điểm cao nhất trong danh sách.

```
double diemCaoNhat = diem.Max();
```

Kết hợp nhiều phương thức

Ví dụ 7: Tính điểm trung bình của riêng những sinh viên đạt (>= 5).

```
double diemTBDat = diem .Where(d => d >= 5) .Average(); // Kết quả: 7.4
```

Ví dụ 8: Lấy top 3 điểm cao nhất trong số những sinh viên đạt.

```
List<double> top3 = diem.Where(d => d >= 5)
                        .OrderByDescending(d => d).Take(3).ToList();
// Kết quả: [9.2, 8.5, 7.8]
```

TÓM TẮT

Trong bài này, học viên làm quen với nền tảng của hệ sinh thái .NET và ngôn ngữ lập trình C#, bao gồm:

Tổng quan về .NET: lịch sử hình thành, quá trình phát triển qua các phiên bản, và cách cài đặt môi trường phát triển (Visual Studio, .NET SDK)

Đặc điểm kiến trúc .NET: mô hình CLR, thư viện lớp .NET (Base Class Library), và cơ chế Garbage Collection tự động quản lý bộ nhớ

Ngôn ngữ lập trình C#: kiểu dữ liệu và biến, toán tử và biểu thức, cấu trúc điều khiển (if, switch, for, while), hàm (method), mảng và Collection cơ bản

LINQ (Language Integrated Query): khái niệm, cú pháp Method Syntax, và các phương thức thường dùng (Where, Select, OrderBy, GroupBy...)

CÂU HỎI ÔN TẬP

Câu 1: Nêu ít nhất 4 đặc điểm nổi bật của .NET hiện đại. Theo bạn, đặc điểm nào tạo ra sự khác biệt lớn nhất so với .NET Framework trước đây? Giải thích vì sao.

Câu 2: Phân biệt **Value type** và **Reference type** trong C#. Cho ví dụ minh họa bằng đoạn code thể hiện rõ sự khác nhau khi gán một biến value type so với một biến reference type cho biến khác.

Câu 3: Giải thích sự khác nhau giữa vòng lặp `while` và `do-while`. Viết một đoạn code ví dụ mà nếu dùng `while` sẽ cho kết quả khác với khi dùng `do-while`.

Câu 4: Từ khóa `ref` và `out` trong khai báo hàm dùng để làm gì? Cho một ví dụ cụ thể thể hiện rõ lý do tại sao không thể giải quyết bài toán đó chỉ bằng `return` thông thường.

Câu 5: So sánh ưu và nhược điểm giữa **Array** và **List<T>**. Trong trường hợp nào nên ưu tiên dùng Array, và trường hợp nào nên dùng List<T>?

Câu 6: LINQ là gì? Viết câu lệnh LINQ (Method syntax) để từ một `List<double>` chứa điểm số, lọc ra các điểm đạt (>= 5), sắp xếp giảm dần, và lấy ra 3 điểm cao nhất.

Câu 7: Cho đoạn code sau xử lý danh sách điểm bằng vòng lặp truyền thống:

```
var diemList = new double[] { 8.5, 5.6, 4.9, 5.8, 6.8 };
double tong = 0;
int dem = 0;
foreach (double d in diemList)
{
    if (d >= 5)
    {
        tong += d;
        dem++;
    }
}
double tb = dem > 0 ? tong / dem : 0;
Console.WriteLine($"Tổng: {tong}, Số lượng đạt: {dem}, Trung bình: {tb}");
```

Hãy viết lại đoạn code trên bằng LINQ sao cho ngắn gọn hơn.

Câu 8: Viết chương trình C# yêu cầu người dùng nhập vào N số nguyên (N do người dùng nhập trước), lưu vào một List<int>. Sau đó, tìm và hiển thị ra tất cả các cặp số

(x, y) trong danh sách sao cho x + y = 100 (x và y là hai phần tử khác vị trí nhau trong danh sách, không tính một số tự cộng với chính nó hai lần).Yêu cầu cụ thể:

a) Viết chương trình bằng cách dùng vòng lặp lồng nhau (for/foreach) để tìm các cặp số thỏa điều kiện

b) Viết lại chương trình trên bằng LINQ, sử dụng tổ hợp phù hợp (gợi ý: kết hợp Where với chỉ số phần tử, hoặc dùng SelectMany)

c) Đếm tổng số cặp số thỏa điều kiện tìm được.

BÀI 2: HƯỚNG ĐỐI TƯỢNG TRONG C#

Sau khi học xong bài này, học viên có thể:

- *Trình bày được khái niệm lập trình hướng đối tượng (OOP) và giải thích vì sao cần dùng OOP thay vì lập trình thủ tục thuần túy*
- *Phân biệt và trình bày được bốn tính chất cơ bản của OOP: tính đóng gói, tính kế thừa, tính đa hình, tính trừu tượng*
- *Khai báo được một Class với đầy đủ thuộc tính (field/property), constructor, và phương thức; tạo object từ class đó và sử dụng trong chương trình*
- *Áp dụng được tính đóng gói (Encapsulation) thông qua Property (get/set), kiểm soát được việc truy cập và thay đổi dữ liệu của object một cách an toàn*
- *Xây dựng được mối quan hệ kế thừa giữa các class (sử dụng :, base, override), và giải thích được lợi ích của việc tái sử dụng code thông qua kế thừa*

2.1 TỔNG QUAN VỀ HƯỚNG ĐỐI TƯỢNG C#

Học phần Hướng đối tượng (Java) đã trang bị đầy đủ tư duy nền tảng: class, object, bốn tính chất OOP (đóng gói, kế thừa, đa hình, trừu tượng). Bài học này **không lặp lại lý thuyết đó**, mà tập trung vào cách C# triển khai các khái niệm này — phần nào giống Java, phần nào khác — để có thể áp dụng ngay vào việc thiết kế các class phục vụ Windows Forms ở các bài sau.

Những điểm khác biệt cần chú ý khi chuyển từ Java sang C#

Khái niệm	**Java**	**C#**
Kế thừa	`class Student extends Person`	`class Student : Person`

Gọi constructor lớp cha	`super(...)`	`base(...)`
Override	`@Override`	`override` (bắt buộc khai báo `virtual` ở lớp cha)
Getter/Setter	viết tay `getTen()`, `setTen()`	dùng **Property** (`{ get; set; }`) — gọn hơn nhiều
Interface	`implements`	`:` (dùng chung cú pháp với kế thừa class)
Không cho kế thừa thêm	`final class`	`sealed class`
Trừu tượng	`abstract class`, `interface` — tương tự	tương tự, nhưng C# có thêm cú pháp gọn cho property trong interface

2.2 CLASS VÀ OBJECT

2.2.1 Khai báo Class, Constructor

Cú pháp khai báo class trong C# không khác biệt nhiều so với Java — cùng dùng `class`, cùng có constructor cùng tên với class, cùng dùng **`new`** để tạo object. Dưới đây là class `Student` cơ bản, viết theo đúng cách mà SV đã quen ở Java:

```
internal class Student
{
    public string Ten;
    public double Diem;
    // Constructor
    1 reference
    public Student(string ten, double diem)
    {
        Ten = ten;
        Diem = diem;
    }
    1 reference
    public string XepLoai()
    {
        return Diem switch
        {
            >= 8.5 => "Xuất sắc",
            >= 7 => "Khá",
            >= 5 => "Trung bình",
            _ => "Yếu"
        };
    }
}
```

Tạo và sử dụng object:

```
Student sv1 = new Student("An", 9.0);
Console.WriteLine($"{sv1.Ten} - {sv1.XepLoai()}");
```

Đoạn code trên dùng `public string Ten;` — đây là một **field** (trường dữ liệu) công khai, truy cập trực tiếp. Cách viết này hoạt động được, nhưng C# có một cách viết tốt hơn hẳn cho việc này, và đây chính là điểm khác biệt lớn nhất so với Java.

2.2.2 Property

Ở Java, để bảo vệ dữ liệu (đóng gói đúng chuẩn), ta phải khai báo field là `private`, rồi viết tay từng cặp hàm `getTen()` / `setTen()`:

C# giải quyết đúng vấn đề này bằng **Property** — một cú pháp tích hợp sẵn, gọn hơn rất nhiều, nhưng vẫn cho hiệu quả tương đương (vẫn kiểm soát được việc đọc/ghi dữ liệu):

```
public class Student
{
    0 references
    public string Ten { get; set; }
    0 references
    public double Diem { get; set; }
}
```

Chỉ với `{ get; set; }`, C# đã tự động sinh ra cơ chế tương đương getter/setter của Java ở phía sau — không cần viết tay từng hàm. Đây gọi là **auto-implemented property**.

Khi cần kiểm soát logic — Property với get/set tùy chỉnh

Khi cần validate dữ liệu (giống lúc viết `setDiem()` có kiểm tra ở Java), Property trong C# vẫn cho phép viết logic riêng:

```
public class Student
{
    0 references
    public string Ten { get; set; }

    private double diem;
    0 references
    public double Diem
    {
        get { return diem; }
        set
        {
            if (value < 0 || value > 10)
                throw new ArgumentException("Điểm phải từ 0 đến 10");
            diem = value;
        }
    }
}
```

`value` ở đây là từ khóa đặc biệt, đại diện cho giá trị được gán vào — tương đương tham số trong `setDiem(double diem)` ở Java, nhưng không cần khai báo tham số tường minh.

Property chỉ đọc (read-only) — phù hợp khi field không nên đổi sau khi tạo

```
public class Student
{
    1 reference
    public string MaSinhVien { get; }   // chỉ gán được trong constructor, không set lại sau
    0 references
    public Student(string maSinhVien)
    {
        MaSinhVien = maSinhVien;
    }
}
```

2.2.3 Tạo Object và sử dụng

Với class `Student` viết lại bằng Property, cách tạo và dùng object không đổi:

```
Student sv1 = new Student(){  Ten = "An", Diem = 9.0 }; // Object initializer
Console.WriteLine($"{sv1.Ten} - {sv1.Diem}");
sv1.Diem = 7.5;   // Gán lại qua property, vẫn chạy logic validate nếu có
```

Cú pháp `new Student { Ten = "An", Diem = 9.0 }` gọi là **Object Initializer** — một tiện ích chỉ có ở C#, không có ở Java, cho phép gán giá trị cho property ngay khi tạo object mà không cần viết constructor riêng cho từng tổ hợp tham số.

2.2.4 Phương thức (Method) trong Class

Phương thức (method) là nơi định nghĩa **hành vi** của một object — những việc mà object đó có thể thực hiện, dựa trên dữ liệu (property) mà nó đang mang. Một class hoàn chỉnh thường không chỉ có dữ liệu, mà còn có các method xử lý logic gắn liền với chính dữ liệu đó.

- Phương thức tĩnh (khai báo kèm với từ khóa static) hoạt động như phương thức toàn cục với tất cả các đối tượng của lớp. Khi cài đặt không được phép sử dụng bất kỳ thành phần dữ liệu, hay phương thức non static của lớp.
- Khi gọi non static method, ta gọi thong qua thể hiện của lớp, còn khi ta truy cập phương thức static mà không cần phải tạo bất cứ thể hiện của lớp, mà truy cập thông qua tên lớp. Ví dụ:

```
public class Student
{
    1 reference
    public string Ten { get; set; }
    5 references
    public double Diem { get; set; }
    1 reference
    public string XepLoai()
    {
        return Diem switch
        {
            >= 8.5 => "Xuất sắc",
            >= 7 => "Khá",
            >= 5 => "Trung bình",
            _ => "Yếu"
        };
    }
    1 reference
    public void TangDiem(double mucTang)
    {
        Diem += mucTang;
    }
    1 reference
    public bool DaDat()
    {
        return Diem >= 5;
    }
}
```

Cách gọi method thông qua object đã tạo:

```
Student sv = new Student { Ten = "An", Diem = 7.0 };
Console.WriteLine(sv.XepLoai());      // Khá
sv.TangDiem(1.5);
Console.WriteLine(sv.Diem);           // 8.5
Console.WriteLine(sv.DaDat());        // True
```

Method tham số mặc định và overload — áp dụng vào class

Tham số mặc định giúp một method có thể được gọi với ít tham số hơn khai báo gốc, miễn là các tham số bị thiếu đó đã được gán giá trị mặc định sẵn. Điều này hữu ích khi phần lớn trường hợp sử dụng chỉ cần giá trị "thông thường", và chỉ một số trường hợp hiếm cần tùy chỉnh thêm.

```
// Tham số mặc định
0 references
public void In(string tienTo = "Sinh viên")
{
    Console.WriteLine($"{tienTo}: {Ten} - {Diem}");
}
```

Method overload cho phép khai báo nhiều method cùng tên, miễn là khác nhau về số lượng hoặc kiểu dữ liệu tham số. Trình biên dịch sẽ tự xác định chính xác phiên bản nào cần gọi dựa vào tham số truyền vào lúc gọi method. Đây là một cách rất tự nhiên để diễn đạt "cùng một hành động, nhưng có vài biến thể khác nhau về cách thực hiện" — mà không cần đặt tên kiểu TangDiemMucDon, TangDiemCoGioiHan gây rối.

```
// Overload - cùng tên TangDiem, khác tham số
1 reference
public void TangDiem(double mucTang)
{
    Diem += mucTang;
}
0 references
public void TangDiem(double mucTang, double diemToiDa)
{
    Diem = Math.Min(Diem + mucTang, diemToiDa);
}
```

Override ToString() — hiển thị object theo ý mình

Mọi class trong C# kế thừa ngầm từ `object`, trong đó có sẵn method `ToString()`. Mặc định, in trực tiếp một object ra màn hình sẽ chỉ hiện tên class (không có thông tin gì hữu ích) — override lại `ToString()` để kiểm soát nội dung hiển thị:

```
public class Student
{
    2 references
    public string Ten { get; set; }
    3 references
    public double Diem { get; set; }

    0 references
    public override string ToString()
    {
        return $"{Ten} - {Diem} - {XepLoai()}";
    }
    1 reference
    public string XepLoai()
    {
        return Diem >= 5 ? "Đạt" : "Không đạt";
    }
}
```

Đây là một thói quen nên hình thành từ sớm: mỗi class quan trọng nên có một `ToString()` được override hợp lý, vì nó giúp việc debug và hiển thị dữ liệu (kể cả khi

binding lên `ComboBox`/`ListBox` trong WinForms ở các bài sau) trở nên dễ dàng hơn nhiều, mà không cần phải tự viết hàm `In()` riêng cho mục đích hiển thị.

```
Student sv = new Student { Ten = "An", Diem = 9.0 };
Console.WriteLine(sv);   // Tự động gọi ToString() -> "An - 9 - Đạt"
```

2.2.5 Phạm vi truy cập (Access Modifier)

Mỗi class, field, property và method trong C# đều có thể khai báo kèm **access modifier** để quy định phạm vi truy cập. Đây là cơ chế giúp thực hiện **tính đóng gói (Encapsulation)**, cho phép kiểm soát thành phần nào được truy cập từ bên ngoài và thành phần nào chỉ sử dụng trong nội bộ class.

Các mức phạm vi thường dùng

Từ khóa	Ý nghĩa
`public`	Truy cập được từ **bất kỳ đâu** — class khác, project khác
`private`	Chỉ truy cập được **trong chính class đó** — mặc định nếu không ghi gì
`protected`	Truy cập được trong chính class, và trong các **class con kế thừa** từ nó
`internal`	Truy cập được trong cùng **project/assembly**, không lộ ra ngoài

Áp dụng vào class Student

```
public class Student
{
    1 reference
    public string Ten { get; set; }       // Public - bên ngoài đọc/ghi tự do
    1 reference
    public double Diem { get; private set; }  // Get public, set private - đã thấy ở mục Property

    private string maSinhVienNoiBo;       // Private - chỉ dùng trong class, không ai bên ngoài thấy
    1 reference
    public void TangDiem(double mucTang)
    {
        Diem += mucTang;
        GhiLog($"Tăng điểm: +{mucTang}");   // Gọi method private nội bộ
    }
    1 reference
    private void GhiLog(string noiDung)   // Private method - chi tiết nội bộ, không cần lộ ra
    {
        Console.WriteLine($"[LOG] {noiDung}");
    }
}
```

Vì sao nên hạn chế `public`

Chỉ nên khai báo **`public`** cho những thành phần thực sự cần sử dụng từ bên ngoài, còn các chi tiết xử lý nội bộ nên để **`private`**. Nhờ đó, có thể thay đổi cách cài đặt bên trong mà không ảnh hưởng đến các đoạn code khác. Đây chính là mục tiêu của Encapsulation**:** che giấu chi tiết cài đặt và chỉ công khai những gì cần thiết**.**

2.3 KẾ THỪA

2.3.1 Khái niệm và cú pháp kế thừa

Trong nhiều trường hợp, các đối tượng khác nhau có những đặc điểm chung. Thay vì khai báo lặp lại các thuộc tính và phương thức ở nhiều class, **kế thừa (Inheritance)** cho phép một class sử dụng lại thành viên của class khác và chỉ bổ sung những đặc điểm riêng. Class được kế thừa gọi là **lớp cha (base class)**, còn class kế thừa gọi là **lớp con (derived class)**.

Cú pháp kế thừa trong C# dùng dấu hai chấm :

```
public class Person
{
    1 reference
    public string? Ten { get; set; }
    1 reference
    public int Tuoi { get; set; }
}
2 references
public class Student : Person
{
    1 reference
    public double Diem { get; set; }
}
```

Class Student giờ đây tự động có sẵn cả Ten và Tuoi (kế thừa từ Person), mà không cần viết lại:

```
2   Student sv = new Student();
3   sv.Ten = "An";       // Có sẵn từ Person
4   sv.Tuoi = 20;         // Có sẵn từ Person
5   sv.Diem = 8.5;        // Riêng của Student
```

Một class chỉ được kế thừa từ **một** class cha duy nhất trong C# (không hỗ trợ đa kế thừa như một số ngôn ngữ khác) — nhưng có thể kế thừa từ một class đồng thời triển khai (implement) nhiều interface, sẽ tìm hiểu sau.

2.3.2 Từ khóa base, override

`base` — gọi đến constructor hoặc thành phần của lớp cha

Khi lớp cha có constructor riêng (không phải constructor mặc định), lớp con cần dùng từ khóa `base` để gọi đến constructor đó, đảm bảo phần dữ liệu thuộc lớp cha được khởi tạo đúng cách:

```
public class Person
{
    2 references
    public string Ten { get; set; }
    2 references
    public int Tuoi { get; set; }
    1 reference
    public Person(string ten, int tuoi)
    {
        Ten = ten;
        Tuoi = tuoi;
    }
}
3 references
public class Student : Person
{
    2 references
    public double Diem { get; set; }

    1 reference
    public Student(string ten, int tuoi, double diem) : base(ten, tuoi)
    {
        Diem = diem;
    }
}
```

Cú pháp `: base(ten, tuoi)` nghĩa là: trước khi thực thi phần thân constructor của `Student`, hãy gọi constructor của `Person` với hai giá trị `ten`, `tuoi` để khởi tạo phần dữ liệu kế thừa trước.

```
Student sv = new Student("An", 20, 8.5);
```

`virtual` và `override` — cho phép lớp con thay đổi hành vi lớp cha

Để một method ở lớp cha có thể được lớp con **thay đổi cách thực hiện**, lớp cha phải khai báo method đó với từ khóa `virtual`. Lớp con khi đó dùng `override` để viết lại logic riêng:

```
public class Person
{
    4 references
    public string? Ten { get; set; }
    3 references
    public virtual string GioiThieu()
    {
        return $"Tôi là {Ten}";
    }
}
2 references
public class Student : Person
{
    2 references
    public double Diem { get; set; }
    3 references
    public override string GioiThieu()
    {
        return $"Tôi là sinh viên {Ten}, điểm {Diem}";
    }
}
```

```
Person p = new Person { Ten = "An" };
Console.WriteLine(p.GioiThieu());          // Tôi là An

Student sv = new Student { Ten = "Bình", Diem = 9.0 };
Console.WriteLine(sv.GioiThieu());          // Tôi là sinh viên Bình, điểm 9
```

Nếu cần, lớp con vẫn có thể gọi lại phiên bản gốc của lớp cha bên trong bản override của mình, thông qua `base.TênMethod()`:

```
public override string GioiThieu()
{
    return base.GioiThieu() + $" (sinh viên, điểm {Diem})";
}
```

Ví dụ thực tế: Student kế thừa Person

Ghép toàn bộ lại thành một ví dụ đầy đủ, mở rộng thêm class `Teacher` cũng kế thừa từ `Person` — minh họa rõ giá trị thực sự của kế thừa khi có **nhiều** lớp con cùng dùng chung một lớp cha:

```
public class Person
{
    2 references
    public string Ten { get; set; }
    2 references
    public int Tuoi { get; set; }
    2 references
    public Person(string ten, int tuoi)
    {
        Ten = ten;
        Tuoi = tuoi;
    }
    6 references
    public virtual string GioiThieu()
    {
        return $"Tôi là {Ten}, {Tuoi} tuổi";
    }
}

public class Student : Person
{
    3 references
    public double Diem { get; set; }

    2 references
    public Student(string ten, int tuoi, double diem) : base(ten, tuoi)
    {
        Diem = diem;
    }
    5 references
    public override string GioiThieu()
    {
        return base.GioiThieu() + $" - là sinh viên, điểm {Diem}";
    }
    0 references
    public string XepLoai()
    {
        return Diem >= 5 ? "Đạt" : "Không đạt";
    }
}
```

```
public class Teacher : Person
{
    2 references
    public string MonDay { get; set; }
    1 reference
    public Teacher(string ten, int tuoi, string monDay) : base(ten, tuoi)
    {
        MonDay = monDay;
    }
    5 references
    public override string GioiThieu()
    {
        return base.GioiThieu() + $" - là giảng viên, dạy môn {MonDay}";
    }
}
```

Student và **Teacher** kế thừa các thuộc tính, constructor và phương thức chung từ **Person**, giúp tránh lặp lại code. Đồng thời, mỗi lớp con có thể **ghi đè (`override`)** phương thức `GioiThieu()` để bổ sung hành vi riêng, vừa **tái sử dụng mã nguồn** vừa **tùy biến chức năng**.

```
Student sv = new Student("An", 20, 8.5);
Teacher gv = new Teacher("Cường", 35, "Lập trình C#");
Console.WriteLine(sv.GioiThieu());
// Tôi là An, 20 tuổi - là sinh viên, điểm 8.5
Console.WriteLine(gv.GioiThieu());
// Tôi là Cường, 35 tuổi - là giảng viên, dạy môn Lập trình C#
```

2.4 Đa hình (Polymorphism)

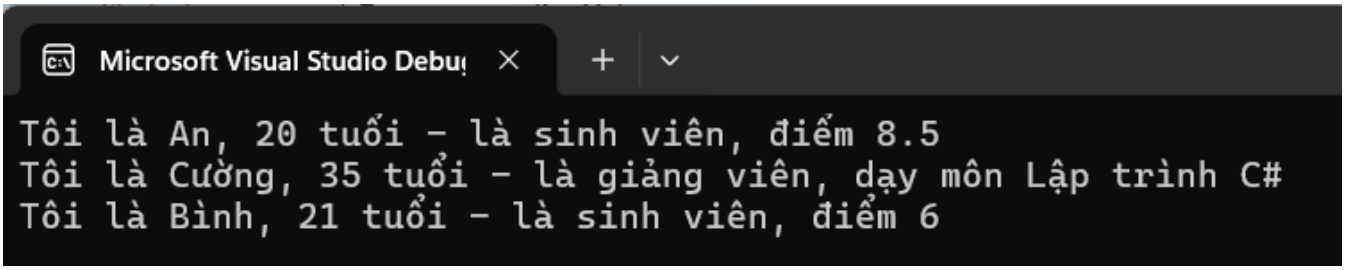
Đa hình (Polymorphism) cho phép cùng một lời gọi phương thức nhưng thực hiện hành vi khác nhau tùy vào đối tượng cụ thể. Tính chất này phát huy hiệu quả khi kết hợp với **kế thừa**.

Đa hình qua Override (Runtime Polymorphism)

Đa hình thể hiện rõ khi nhiều đối tượng thuộc các lớp con được quản lý thông qua kiểu của **lớp cha**, nhưng khi gọi cùng một phương thức, mỗi đối tượng sẽ thực hiện phiên bản `override` của riêng mình.

```
List<Person> danhSach = new List<Person>
{
    new Student("An", 20, 8.5),
    new Teacher("Cường", 35, "Lập trình C#"),
    new Student("Bình", 21, 6.0)
};
foreach (Person p in danhSach)
{
    Console.WriteLine(p.GioiThieu());
}
```

Kết quả



Dù các đối tượng được lưu trong `List<Person>` và đều gọi `p.GioiThieu()`, mỗi đối tượng sẽ thực thi phiên bản `override` tương ứng của mình. Việc lựa chọn phương thức được **CLR** quyết định tại **runtime**, thể hiện đúng bản chất của **đa hình**: **cùng một lời gọi nhưng hành vi phụ thuộc vào đối tượng thực tế**.

Đa hình qua Overload (Compile-time Polymorphism)

Đa hình qua **`overload`** (Compile-time Polymorphism) là việc nhiều phương thức cùng tên nhưng khác danh sách tham số. Trình biên dịch sẽ lựa chọn phương thức phù hợp **ngay tại thời điểm biên dịch (compile-time)** dựa trên số lượng và kiểu dữ liệu của các tham số truyền vào.

```
public class MayTinh
{
    1 reference
    public int Cong(int a, int b) => a + b;
    1 reference
    public double Cong(double a, double b) => a + b;
    1 reference
    public int Cong(int a, int b, int c) => a + b + c;
}
```

```
MayTinh mt = new MayTinh();
Console.WriteLine(mt.Cong(2, 3));          // Gọi bản int
Console.WriteLine(mt.Cong(2.5, 3.5));      // Gọi bản double
Console.WriteLine(mt.Cong(1, 2, 3));       // Gọi bản 3 tham số
```

Giá trị thực tế của đa hình

Lợi ích lớn nhất thể hiện rõ khi viết một hàm xử lý chung cho nhiều loại object khác nhau, mà không cần biết trước (và không cần quan tâm) object đó cụ thể là gì:

```
static void InDanhSach(List<Person> ds)
{
    foreach (Person p in ds)
    {
        Console.WriteLine(p.GioiThieu());   // Không cần if/else kiểm tra là Student hay Teacher
    }
}
```

Nếu sau này có thêm class `Staff : Person` mới, hàm `InDanhSach` ở trên **không cần sửa gì cả** — chỉ cần `Staff` cũng override `GioiThieu()` đúng cách, đa hình sẽ tự động xử lý đúng khi object đó được đưa vào danh sách. Đây là lý do đa hình được xem là một trong những tính chất giúp code dễ mở rộng nhất trong OOP — thêm loại đối tượng mới mà không phải sửa lại những đoạn code đã viết từ trước.

Đa hình qua Interface/Abstract class

Ngoài đa hình qua override (đã thấy với `Person`/`Student`/`Teacher`), C# còn cho phép đạt được đa hình thông qua **Interface** hoặc **Abstract class** — hai cách thiết kế giúp định nghĩa "hợp đồng" về hành vi mà nhiều class khác nhau, dù không có quan hệ kế thừa trực tiếp, đều phải tuân theo.

2.5 INTERFACE VÀ ABSTRACT CLASS

2.5.1 Abstract class

- **Abstract class** (lớp trừu tượng) là một class đặc biệt — không thể tạo object trực tiếp từ nó, mà chỉ tồn tại để làm "lớp cha" cho các class con kế thừa. Nó dùng khi muốn vừa **chia sẻ code chung** cho các lớp con, vừa **buộc** mỗi lớp con phải tự định nghĩa riêng một số hành vi.

```
public abstract class Person
{
    1 reference
    public string? Ten { get; set; }
    // Method thường - có sẵn logic, lớp con dùng được ngay, không cần viết lại
    0 references
    public void ChaoHoi()
    {
        Console.WriteLine($"Xin chào, tôi là {Ten}");
    }
    // Method abstract - không có thân, buộc lớp con phải override
    8 references
    public abstract string GioiThieu();
}


// Person p = new Person();   // LỖI biên dịch! Không thể tạo object từ abstract class
4 references
public class Student : Person
{
    1 reference
    public double Diem { get; set; }

    5 references
    public override string GioiThieu()   // Bắt buộc phải có, nếu thiếu sẽ báo lỗi biên dịch
    {
        return $"Tôi là sinh viên {Ten}, điểm {Diem}";
    }
}
```

```
Student sv = new Student { Ten = "An", Diem = 8.5 };
sv.ChaoHoi();                // Dùng trực tiếp method có sẵn từ Person
Console.WriteLine(sv.GioiThieu());
```

2.5.2 Interface

Interface là một hợp đồng quy định các phương thức mà một class phải cài đặt. Interface chỉ khai báo tên, tham số và kiểu trả về của phương thức, **không chứa phần thân thực thi**. Mọi class triển khai (`implement`) interface đều **bắt buộc** phải định nghĩa đầy đủ các phương thức đó.

```
public interface ITinhDiem
{
    0 references
    string XepLoai();
}
```

```
public class Student : Person, ITinhDiem
{
    2 references
    public double Diem { get; set; }
    3 references
    public override string GioiThieu() => $"Sinh viên {Ten}";
    1 reference
    public string XepLoai()   // Buộc phải triển khai vì đã implement ITinhDiem
    {
        return Diem >= 5 ? "Đạt" : "Không đạt";
    }
}
```

Cú pháp `class Student : Person, ITinhDiem` cho thấy một class có thể **vừa kế thừa một class cha, vừa triển khai nhiều interface cùng lúc** — khác với kế thừa class (chỉ được một lớp cha duy nhất), một class có thể implement bao nhiêu interface tùy ý:

2.5.3 So sánh Abstract class và Interface

Abstract class và **Interface** đều dùng để định nghĩa hành vi cho các lớp, nhưng có mục đích sử dụng khác nhau.

- Abstract class cho phép chứa thuộc tính, phương thức đã cài đặt sẵn để các lớp con tái sử dụng.
- Interface chỉ khai báo các thành viên, lớp triển khai phải tự cài đặt toàn bộ.

Ngoài ra, một class chỉ kế thừa một abstract class, nhưng có thể triển khai nhiều interface, giúp thiết kế linh hoạt hơn khi một đối tượng cần nhiều vai trò.

Tổng hợp lại các điểm khác biệt then chốt:

Tiêu chí	**Abstract class**	**Interface**
Có method với sẵn logic?	Có	Không (thuần khai báo)
Có thể có field/property với giá trị?	Có	Không (chỉ khai báo, không lưu trữ trạng thái)
Một class kế thừa/triển khai được bao nhiêu?	Chỉ 1 abstract class	Nhiều interface cùng lúc

Có constructor?	Có	Không
Phù hợp khi nào	Các class có quan hệ "họ hàng" rõ ràng, cần chia sẻ code chung (`Person` → `Student`, `Teacher`)	Các class không liên quan trực tiếp, chỉ cần chung một "khả năng" (`ITinhDiem`, `ISaveable`, `IComparable`)

TÓM TẮT

Bài học đã giới thiệu các khái niệm cốt lõi của lập trình hướng đối tượng (OOP) trong C#, gồm Class, Object, Property và Method, giúp tổ chức dữ liệu và chức năng một cách khoa học, dễ bảo trì và mở rộng.

Người học cũng đã tìm hiểu bốn tính chất quan trọng của OOP: Encapsulation (đóng gói), Inheritance (kế thừa), Polymorphism (đa hình) và cách áp dụng Override, Overload. Bên cạnh đó, bài học trình bày Abstract class và Interface, đồng thời phân biệt trường hợp sử dụng của từng cơ chế.

Đây là những kiến thức nền tảng để phát triển các ứng dụng C# theo hướng đối tượng và là cơ sở cho các nội dung tiếp theo như Entity Framework Core, kiến trúc nhiều tầng (3-Layer Architecture) và các mẫu thiết kế phần mềm (Design Patterns) trong .NET.

CÂU HỎI ÔN TẬP

Câu 1: Trình bày vì sao cách lưu trữ dữ liệu sinh viên bằng nhiều List<T> tiềm ẩn rủi ro, và giải thích cách lập trình hướng đối tượng giải quyết vấn đề đó. ?

Câu 2: Phân biệt field và Property trong C#. Viết một class Product có property SoLuong với phần set kiểm tra không cho gán giá trị âm.

Câu 3: Phân biệt từ khóa virtual và override. Nếu một method ở lớp cha không khai báo virtual, điều gì sẽ xảy ra khi lớp con cố gắng override nó?

Câu 4: Phân biệt đa hình qua Override và đa hình qua Overload. Đa hình nào được xác định lúc biên dịch (compile-time), đa hình nào được xác định lúc chạy (runtime)?

Câu 5: Trình bày sự khác biệt giữa Abstract class và Interface theo 2 tiêu chí: (a) khả năng chứa method có sẵn logic, (b) số lượng có thể kế thừa/triển khai. Cho một ví dụ thực tế nên dùng Interface thay vì Abstract class.

Câu 6: Tại sao không thể viết Person p = new Person(); nếu Person là một abstract class? Class Person đó có vai trò gì trong thiết kế hệ thống nếu không thể tạo object trực tiếp từ nó?

Câu 7: Cho một `List<Person>` chứa hỗn hợp `Student` và `Teacher`. Viết đoạn code sử dụng LINQ (gợi ý: tìm hiểu thêm phương thức `OfType<T>()`) để lọc ra **chỉ** các object kiểu `Student`, sau đó tính điểm trung bình của riêng các sinh viên đó ?

Câu 8: Hệ thống quản lý "Học liệu số" (Digital Learning Resource)

Một trường đại học đang triển khai chuyển đổi số, số hóa toàn bộ học liệu (tài liệu học tập) thành nhiều dạng khác nhau: Video bài giảng, Tài liệu PDF, và Bài kiểm tra trực tuyến. Hãy xây dựng chương trình Console quản lý các học liệu số này.

Yêu cầu:

1. Tạo abstract class HocLieuSo chứa thuộc tính chung: TenHocLieu, NgayDangTai (kiểu DateTime), LuotXem (số nguyên). Có method abstract HienThiThongTin().
2. Tạo 3 class kế thừa từ HocLieuSo:

VideoBaiGiang: thêm ThoiLuong (phút), override HienThiThongTin()

TaiLieuPDF: thêm SoTrang, override HienThiThongTin()

BaiKiemTraTrucTuyen: thêm SoCauHoi, ThoiGianLamBai (phút), override HienThiThongTin()

3. Tạo interface ICoTheDanhGia với method TinhDoPhoBien() (trả về string: "Phổ biến" nếu LuotXem >= 1000, ngược lại "Chưa phổ biến"). Cho cả 3 class trên triển khai interface này.

4. Tạo một List<HocLieuSo> chứa hỗn hợp cả 3 loại học liệu, mô phỏng dữ liệu thực tế của một khoa đang chuyển đổi số (ít nhất 6 học liệu, lượt xem đa dạng).

5. Dùng foreach in ra HienThiThongTin() và TinhDoPhoBien() của từng học liệu (minh họa đa hình qua cả abstract class và interface).

6. Dùng LINQ:

   - Lọc ra danh sách học liệu "Phổ biến" (lượt xem ≥ 1000), sắp xếp giảm dần theo lượt xem
   - Tính tổng lượt xem của toàn bộ học liệu trong hệ thống
   - Đếm xem có bao nhiêu VideoBaiGiang riêng trong danh sách

BÀI 3: LẬP TRÌNH GIAO DIỆN VỚI WINDOWS FORMS

Sau khi học xong bài này, học viên có thể nắm được:

- *Hiểu được khái niệm Windows Forms (WinForms) và mô hình lập trình Event-driven Programming.*
- *Tạo và cấu hình một dự án Windows Forms bằng Visual Studio.*
- *Hiểu được vòng đời của Form và các sự kiện thường gặp như Load, Shown, Closing và Closed.*
- *Sử dụng các điều khiển cơ bản như Label, TextBox và Button để xây dựng giao diện.*
- *Sử dụng CheckBox và RadioButton để xử lý các lựa chọn của người dùng.*
- *Sử dụng ComboBox, ListBox để hiển thị và lựa chọn dữ liệu.*
- *Sử dụng DateTimePicker để nhập dữ liệu ngày, tháng, năm.*
- *Hiển thị hình ảnh bằng điều khiển PictureBox.*
- *Hiển thị dữ liệu dạng bảng bằng DataGridView.*
- *Xây dựng các ứng dụng Windows Forms đơn giản và xử lý các sự kiện của người dùng.*

3.1 CÁC KHÁI NIỆM

3.1.1 Windows Forms là gì, Lập trình hướng sự kiện

Hầu hết các phần mềm quen thuộc trên máy tính Windows đã từng dùng — từ phần mềm quản lý sinh viên của nhà trường, phần mềm kế toán, đến các công cụ đơn giản như **Notepad**, **Calculator**, hay hộp thoại **Find and Replace** trong Word — đều

được xây dựng theo đúng mô hình này: một cửa sổ với các nút bấm, ô nhập liệu, danh sách lựa chọn, và chương trình chỉ phản hồi khi người dùng thực hiện một thao tác nào đó.

Trong lập trình Console, chương trình chạy tuyến tính — dòng lệnh này xong mới đến dòng lệnh kế tiếp, luồng thực thi do lập trình viên quyết định hoàn toàn từ đầu đến cuối.

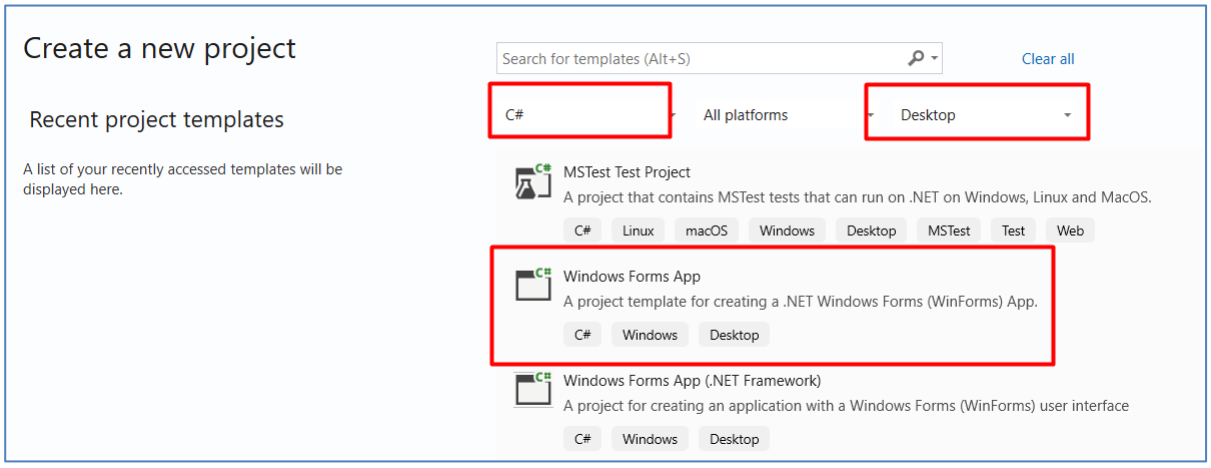
Windows Forms (WinForms) là thư viện thuộc .NET, cho phép xây dựng ứng dụng desktop có giao diện đồ họa (GUI) trên Windows với các thành phần trực quan như nút bấm, ô nhập liệu, danh sách — gọi chung là Control. Nhưng điều quan trọng hơn là WinForms đòi hỏi một cách tư duy hoàn toàn khác về luồng chạy của chương trình.

Với ứng dụng có giao diện, người dùng toàn quyền quyết định điều gì xảy ra tiếp theo — click nút bất cứ lúc nào, nhập liệu theo thứ tự tùy ý, hay đóng cửa sổ giữa chừng. Không có trình tự cố định nào lập trình viên có thể viết sẵn để bao quát hết mọi tình huống.

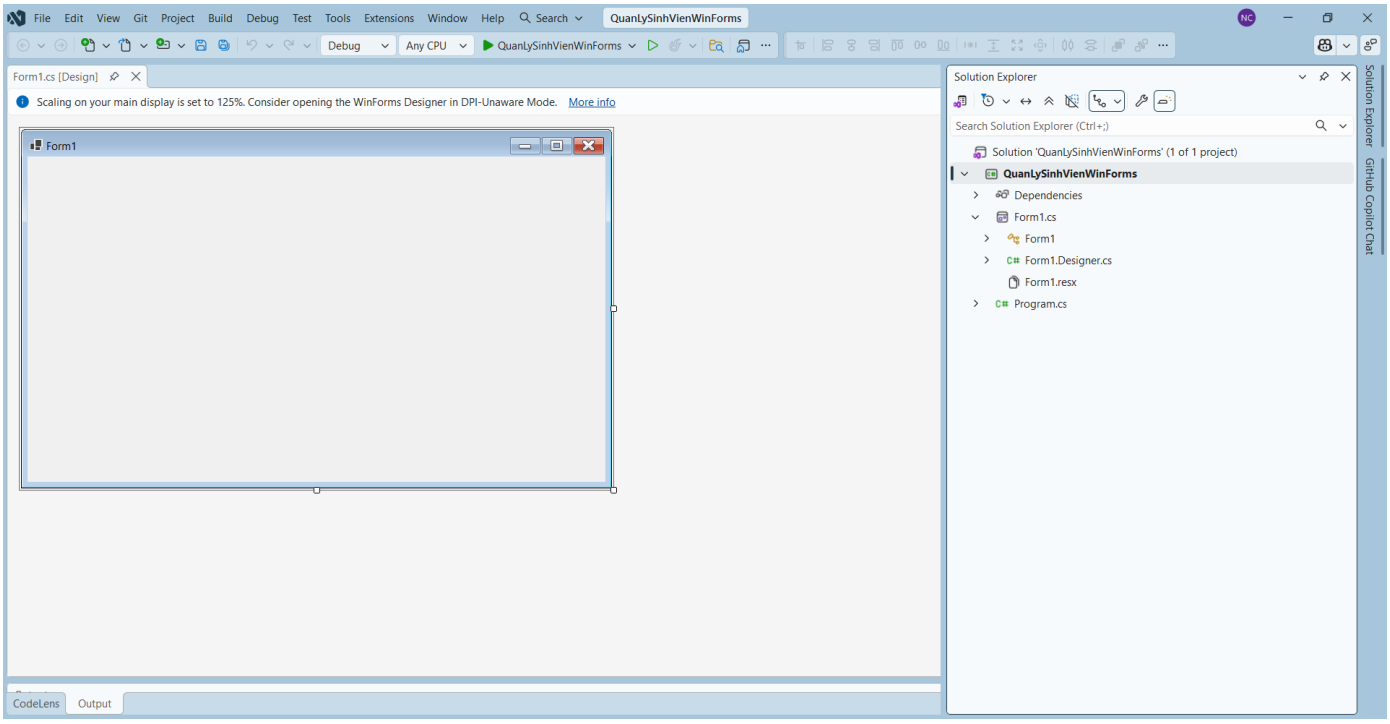
Đây là lý do WinForms được xây dựng theo mô hình lập trình hướng sự kiện (Event-driven Programming): thay vì chạy tuần tự, chương trình "ngồi chờ" trong một vòng lặp xử lý sự kiện (message loop), và chỉ thực thi đoạn code tương ứng — gọi là Event Handler — khi người dùng thực hiện một thao tác nào đó (click, gõ phím, chọn mục...).

3.1.2 Tạo Project WinForms đầu tiên

Mở Visual Studio, chọn **Create a new project**. Trong danh sách mẫu project, tìm và chọn Windows Forms App với ngôn ngữ **C#**. Đặt tên project (ví dụ QuanLySinhVienWinForms), chọn thư mục lưu, và xác nhận Framework là **.NET 10.0**. Nhấn **Create** để Visual Studio tạo project và mở sẵn một Form trống trong chế độ Designer.



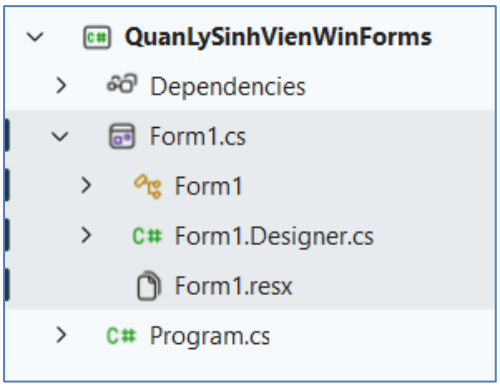
Sau khi project được tạo, Visual Studio mở ra môi trường làm việc hoàn toàn khác so với Console App: ở trung tâm là **Form Designer** hiển thị một Form trống (`Form1`) — đây là nơi thiết kế giao diện bằng cách kéo-thả trực tiếp; bên trái là **Toolbox** chứa danh sách các Control có sẵn (`Button`, `TextBox`, `Label`...); bên phải là **Solution Explorer** quản lý các file trong project và **Properties Window** cho phép chỉnh thuộc tính của Control đang chọn.



Hình 3.1 Tạo project Windows Forms đầu tiên

Nhấn nút **Run** (hoặc F5), Visual Studio sẽ build và chạy chương trình — một cửa sổ trống hiện ra trên màn hình, đây chính là Form đầu tiên đã chạy được, dù chưa có bất kỳ Control hay xử lý sự kiện gì.

Cấu trúc project WinForms gồm ba file chính đáng chú ý:



- **Form1.cs** — file chứa mã nguồn C# của Form, nơi viết toàn bộ code xử lý sự kiện. Đây là file làm việc chính trong suốt quá trình phát triển ứng dụng.
- **Form1.Designer.cs** — file do Visual Studio **tự động sinh ra**, chứa code khai báo và cấu hình các Control đã kéo-thả trên Designer (vị trí, kích thước, tên...). Mỗi Control trên Form, dù được tạo bằng kéo-thả trực quan, thực chất vẫn là một object được khởi tạo bằng code — Visual Studio đã viết sẵn phần đó thay lập trình viên. ***Không nên chỉnh sửa file này thủ công***.
- **Program.cs** — chứa lệnh khởi chạy ứng dụng WinForms (Application.Run(new Form1())), thay vì logic chương trình trực tiếp như Console App.

```
1   namespace QuanLySinhVienWinForms
2   {
        0 references
3       internal static class Program
4       {
5           /// <summary>
6           ///  The main entry point for the application.
7           /// </summary>
8           [STAThread]
            0 references
9           static void Main()
10          {
11              // To customize application configuration such as set high DPI settings or default font,
12              // see https://aka.ms/applicationconfiguration.
13              ApplicationConfiguration.Initialize();
14              Application.Run(new Form1());
15          }
16      }
17  }
```

`Application.Run(new Form1())` thực hiện hai việc cùng lúc: tạo object `Form1` và khởi động vòng lặp xử lý sự kiện (message loop) — từ đây chương trình "ngồi chờ" mọi tương tác của người dùng trên Form, đúng với tinh thần Event-driven Programming.

3.1.3 Form — vòng đời

Một Form, từ lúc được tạo ra đến lúc đóng lại, trải qua nhiều giai đoạn khác nhau — gọi là vòng đời (lifecycle) của Form. Mỗi giai đoạn tương ứng với một sự kiện cụ thể mà lập trình viên có thể "lắng nghe" và viết code xử lý riêng.

- **Hai sự kiện quan trọng và thường dùng nhất:**

  - `Load` — xảy ra ngay sau khi Form được tạo ra và sắp hiển thị lên màn hình, nhưng trước khi người dùng có thể nhìn thấy và tương tác. Đây là nơi lý tưởng để thực hiện các công việc khởi tạo — ví dụ: nạp dữ liệu ban đầu vào danh sách, đặt giá trị mặc định cho các ô nhập liệu, hay kết nối cơ sở dữ liệu.

```
public partial class Form1 : Form
{
    1 reference
    public Form1()
    {
        InitializeComponent();
    }

    1 reference
    private void Form1_Load(object sender, EventArgs e)
    {
        Console.WriteLine("Form đã load - chuẩn bị hiển thị");
    }
}
```

  - `FormClosing` — xảy ra ngay khi người dùng thực hiện hành động đóng Form (click nút X hoặc gọi this.Close() trong code), nhưng trước khi Form thực sự bị đóng hoàn toàn. Đây là nơi phù hợp để xác nhận với người dùng trước khi đóng, hoặc thực hiện các công việc dọn dẹp cuối cùng:

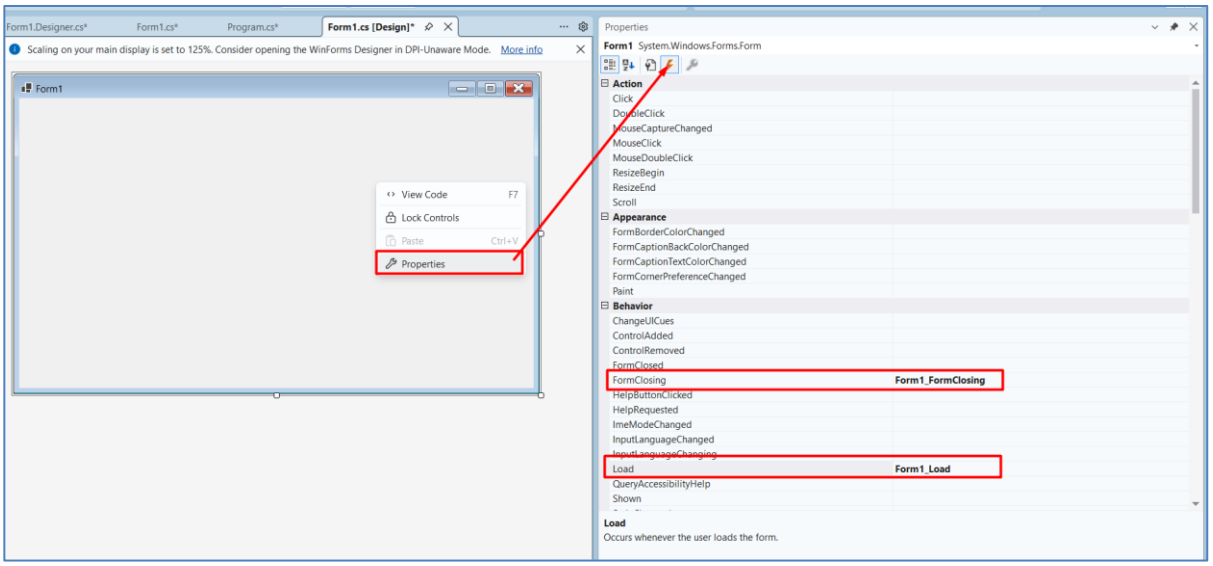
```
private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{
    DialogResult ketQua = MessageBox.Show(
            "Bạn có chắc muốn đóng ứng dụng?",
            "Xác nhận",
            MessageBoxButtons.YesNo
        );
    if (ketQua == DialogResult.No)
    {
            e.Cancel = true;    // Hủy việc đóng Form, Form vẫn tiếp tục mở
    }
}
```

Điểm đặc biệt của `FormClosing` so với các sự kiện thông thường: tham số `e` (kiểu `FormClosingEventArgs`) có thuộc tính `Cancel` — gán `e.Cancel = true` sẽ hủy bỏ việc đóng Form ngay lập tức, Form vẫn tiếp tục hiển thị như chưa có gì xảy ra. Đây là ví dụ điển hình cho thấy lập trình viên không chỉ "lắng nghe" sự kiện, mà còn có thể **can thiệp vào kết quả** của chính sự kiện đó.

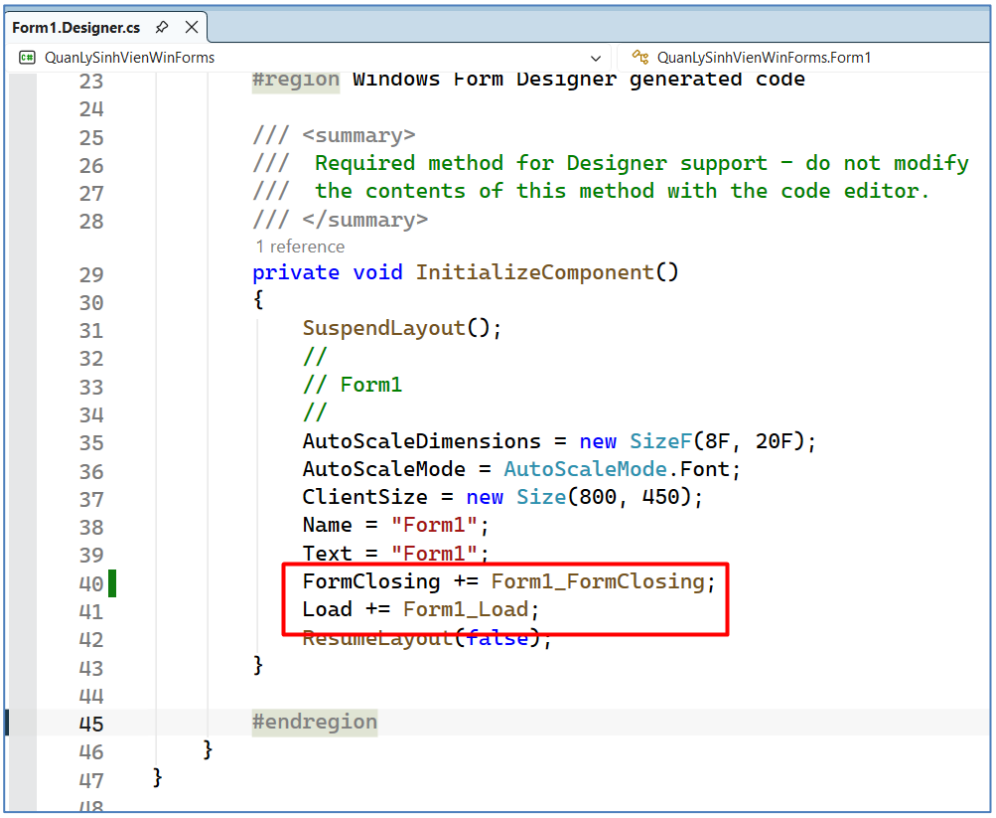
- **Cách gắn sự kiện vào Form**

  Có hai cách để gắn method xử lý sự kiện vào Form:

  Cách 1 — Qua Designer (trực quan): chọn Form trên Designer → mở cửa sổ Properties (F4) → chuyển sang tab sự kiện (biểu tượng ⚡) → tìm đúng sự kiện (Load, FormClosing...) → double-click để Visual Studio tự sinh sẵn khung method, hoặc gõ tên method xử lý vào ô tương ứng.



Cách 2 — Bằng code: gắn trực tiếp trong constructor của Form bằng toán tử +=:



`Load += Form1_Load` sử dụng toán tử `+=` để "đăng ký" method `Form1_Load` vào sự kiện `Load` — khi sự kiện xảy ra, .NET tự động gọi đến method đó. Đây là cơ chế nền tảng của toàn bộ lập trình hướng sự kiện trong WinForms, và được áp dụng nhất quán với mọi Control khác (`Button`, `TextBox`...) trong suốt quá trình phát triển ứng dụng.

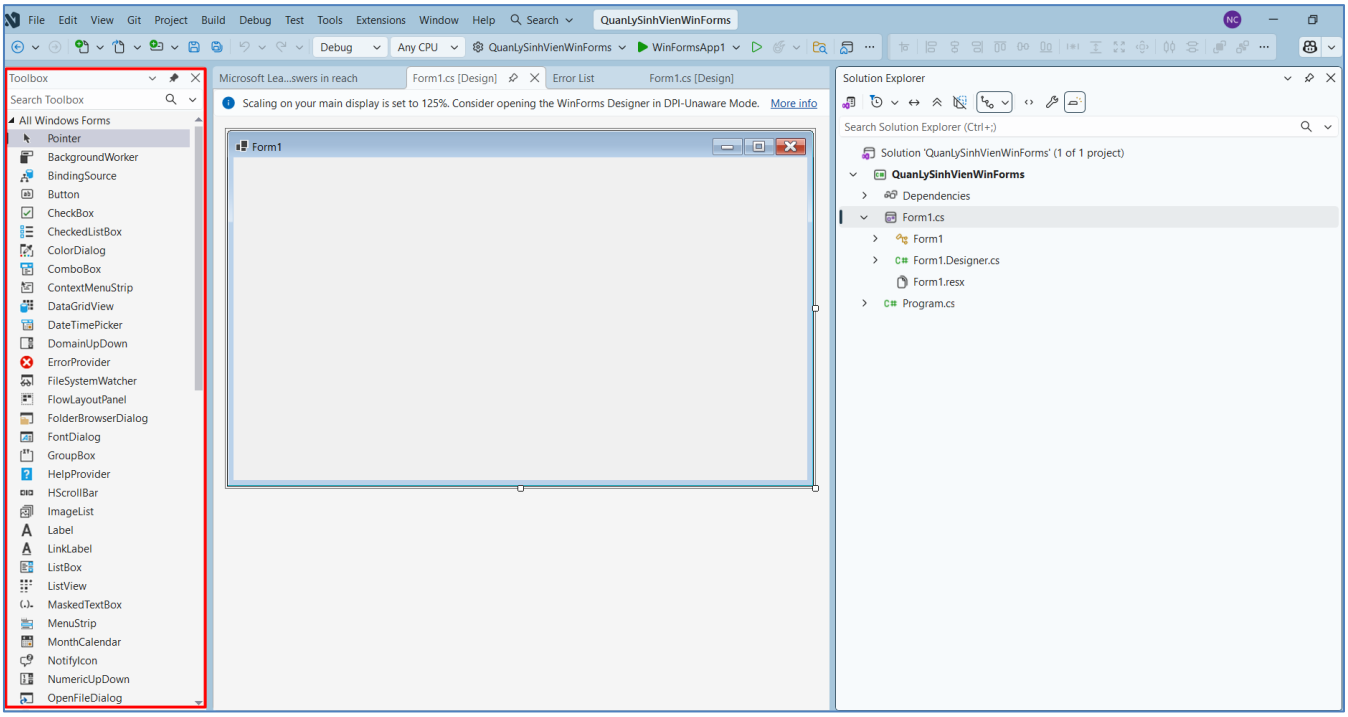
Cách 1 phù hợp khi mới làm quen — Visual Studio tự xử lý phần code gắn sự kiện. Cách 2 linh hoạt hơn, thường dùng khi cần gắn/gỡ sự kiện động trong quá trình chạy chương trình.

3.2 Các controls cơ bản

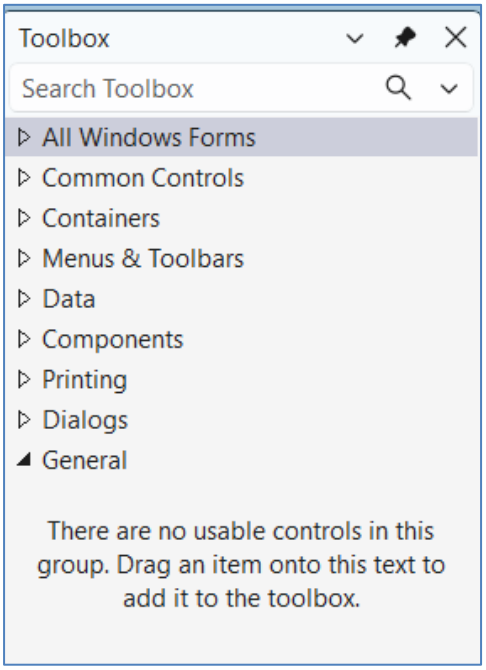
Toolbox — Hộp công cụ chứa các Control

Toolbox là cửa sổ trong Visual Studio chứa danh sách toàn bộ các Control có thể dùng để thiết kế giao diện WinForms. Mỗi Control là một class đã được Microsoft xây dựng sẵn — khi kéo-thả lên Form, Visual Studio tự động tạo object tương ứng và sinh code khởi tạo vào `Form1.Designer.cs`.

Cách mở: vào menu **View** → **Toolbox** (hoặc nhấn Ctrl + W + X). Toolbox thường nằm ở cạnh trái màn hình, có thể cố định hoặc tự ẩn (auto-hide) khi không dùng đến.



Cấu trúc: các Control được nhóm theo danh mục, thu/mở bằng cách click vào tiêu đề nhóm. Các nhóm thường dùng nhất:



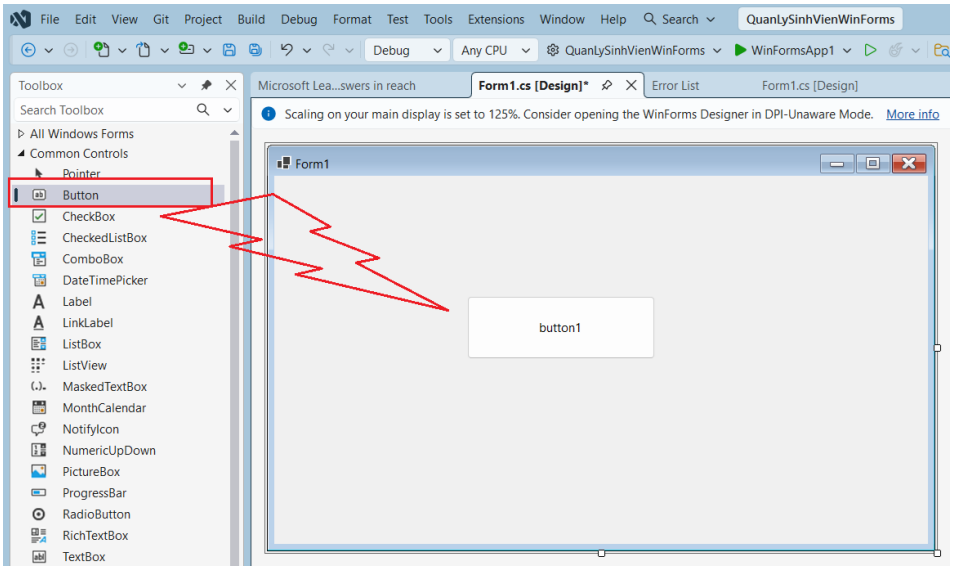
Hình 3.2 Cấu trúc công cụ toolbox trong ứng dụng

Nhóm	Một số Control điển hình
Common Controls	Button, Label, TextBox, CheckBox, RadioButton, ComboBox, ListBox
Containers	Panel, GroupBox, TabControl, SplitContainer
Menus & Toolbars	MenuStrip, ToolStrip, StatusStrip
Data	DataGridView, BindingSource, BindingNavigator
Dialogs	OpenFileDialog, SaveFileDialog, ColorDialog

Trong bài học này, trọng tâm sẽ nằm ở nhóm Common Controls — những Control cơ bản và dùng nhiều nhất, sẽ tìm hiểu lần lượt ở các mục tiếp theo.

Cách sử dụng: chỉ cần kéo (drag) Control mong muốn từ Toolbox, thả (drop) vào vị trí cần đặt trên Form Designer. Visual Studio sẽ tự động:

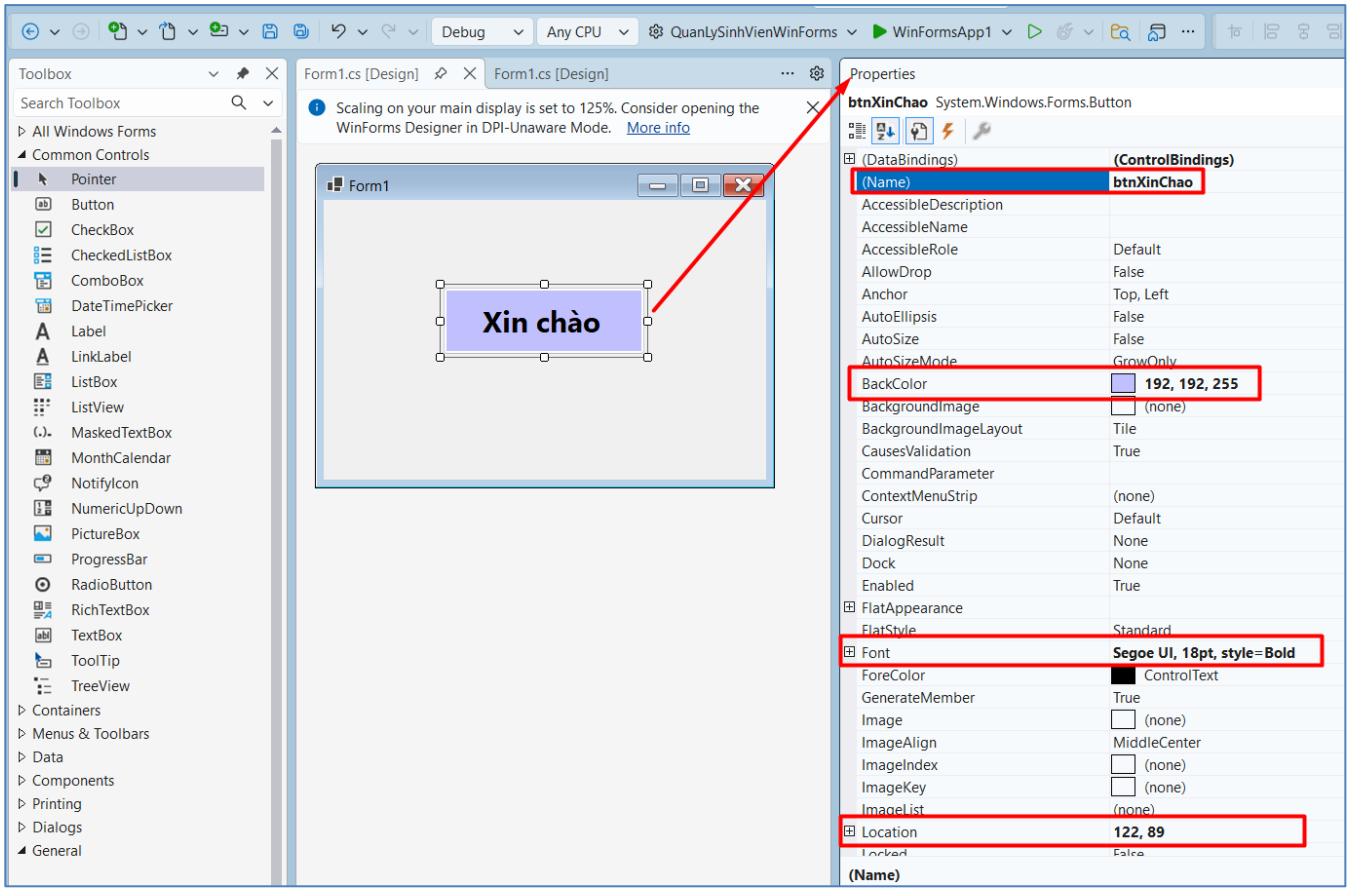
1. Tạo object tương ứng (ví dụ một Button mới) với tên mặc định (button1, button2...)
2. Sinh sẵn đoạn code khai báo object đó trong Form1.Designer.cs
3. Hiển thị Control đó ngay trên Form, có thể kéo để đổi vị trí, kéo góc để đổi kích thước



Hình 3.3 Kéo, Thả Button từ Toolbox

Sau khi kéo một Control lên Form, người dùng chỉ cần chọn Control đó và mở cửa sổ Properties (chọn View → Properties Window hoặc nhấn F4) để xem và thay đổi các thuộc tính của nó một cách trực quan.

Tại đây, có thể dễ dàng chỉnh sửa các thuộc tính phổ biến như **Name** (tên đối tượng dùng trong chương trình), **Text** (nội dung hiển thị), **Size** (kích thước), **Location** (vị trí), **Font** (phông chữ), **BackColor** (màu nền), cùng nhiều thuộc tính khác.



Hình 3.4 công cụ giúp thao tác với Property

Cửa sổ Properties chính là công cụ giúp thao tác với Property của đối tượng theo đúng tư duy lập trình hướng đối tượng (OOP) mà không cần phải viết mã lệnh. Mỗi thay đổi được thực hiện trong cửa sổ này sẽ được Visual Studio tự động chuyển thành mã nguồn tương ứng và lưu trong tệp Form1.Designer.cs, giúp việc thiết kế giao diện trở nên nhanh chóng, trực quan và thuận tiện.

Thuộc tính của các Control

Trước khi tìm hiểu từng Control cụ thể, cần hiểu rằng hầu hết các Control trong Windows Forms như Label, TextBox, Button, CheckBox, ComboBox,... đều được kế thừa từ lớp cơ sở (base class) **Control**. Nhờ cơ chế kế thừa (Inheritance) trong lập trình hướng đối tượng (OOP), các Control này đều sở hữu một nhóm thuộc tính, phương thức và sự kiện chung. Bảng dưới đây trình bày một số thuộc tính chung thường gặp của các Control.

Thuộc tính	Ý nghĩa
`Name`	Tên định danh của Control, được sử dụng để truy cập và thao tác với Control trong mã nguồn. Nên đặt tên theo quy tắc CamelCase: [Tiền tố viết tắt] + [Mô tả chức năng] (ví dụ: **txtUsername, btnSubmit, chkRememberMe)**
`Text`	Nội dung văn bản hiển thị (ý nghĩa cụ thể khác nhau tùy Control — với Button là chữ trên nút, với TextBox là nội dung đang nhập)
`Enabled`	`true/false` — quyết định Control có thể **tương tác được hay không**. Nếu `false`, Control hiển thị mờ đi, người dùng không click/nhập được
`Visible`	`true/false` — quyết định Control có **hiển thị trên Form hay không**. Nếu `false`, Control biến mất hoàn toàn khỏi giao diện
`Location`	Vị trí của Control trên Form (tính bằng toạ độ X, Y)
`Size`	Kích thước Control (chiều rộng, chiều cao)
`Font`	Phông chữ, cỡ chữ, kiểu chữ
`BackColor, ForeColor`	Màu nền, màu chữ

`TabIndex`	Thứ tự khi người dùng nhấn phím `Tab` để chuyển qua các Control

Phân biệt `Enabled` và `Visible`

- **Enabled**: Dùng để bật hoặc tắt khả năng tương tác của Control. Khi Enabled = False, Control vẫn hiển thị nhưng bị làm mờ và người dùng không thể thao tác.

  Ví dụ: Tạm khóa nút Lưu cho đến khi người dùng nhập đầy đủ thông tin.

- **Visible**: Dùng để hiển thị hoặc ẩn Control trên Form. Khi Visible = False, Control sẽ biến mất hoàn toàn khỏi giao diện.

  Ví dụ: Ẩn nhóm ô nhập liệu khi Form đang ở chế độ Chỉ xem.

```
btnLuu.Enabled = false;   // Nút vẫn hiện, nhưng bị mờ, click không có tác dụng
btnLuu.Visible = false;   // Nút biến mất hoàn toàn khỏi Form
```

Sự kiện chung của các Control

Tương tự như các thuộc tính, hầu hết các Control trong Windows Forms đều kế thừa từ lớp Control, nên chúng cũng sở hữu một số sự kiện chung. Những sự kiện này có thể được sử dụng với hầu hết các Control, bên cạnh các sự kiện riêng của từng loại (ví dụ TextChanged thường được sử dụng với TextBox).

Việc nắm vững các sự kiện chung sẽ giúp người học dễ dàng xử lý các thao tác phổ biến của người dùng trên nhiều loại Control khác nhau.

Sự kiện	Khi nào xảy ra
`Click`	Xảy ra khi người dùng nhấp chuột vào Control. Hầu hết các Control như **Button**, **Label**, **PictureBox**,... đều hỗ trợ sự kiện này.
`MouseEnter` `MouseLeave`	Xảy ra khi con trỏ chuột di chuyển vào hoặc rời khỏi vùng hiển thị của Control.
`MouseDown` `MouseUp`	Xảy ra khi người dùng nhấn xuống hoặc thả nút chuột trên Control. Hai sự kiện này cho biết chính xác thời điểm bắt đầu và kết thúc thao tác nhấn chuột.

`GotFocus` `LostFocus`	Xảy ra khi Control nhận hoặc mất **Focus** (trạng thái sẵn sàng nhận dữ liệu hoặc tương tác từ người dùng).

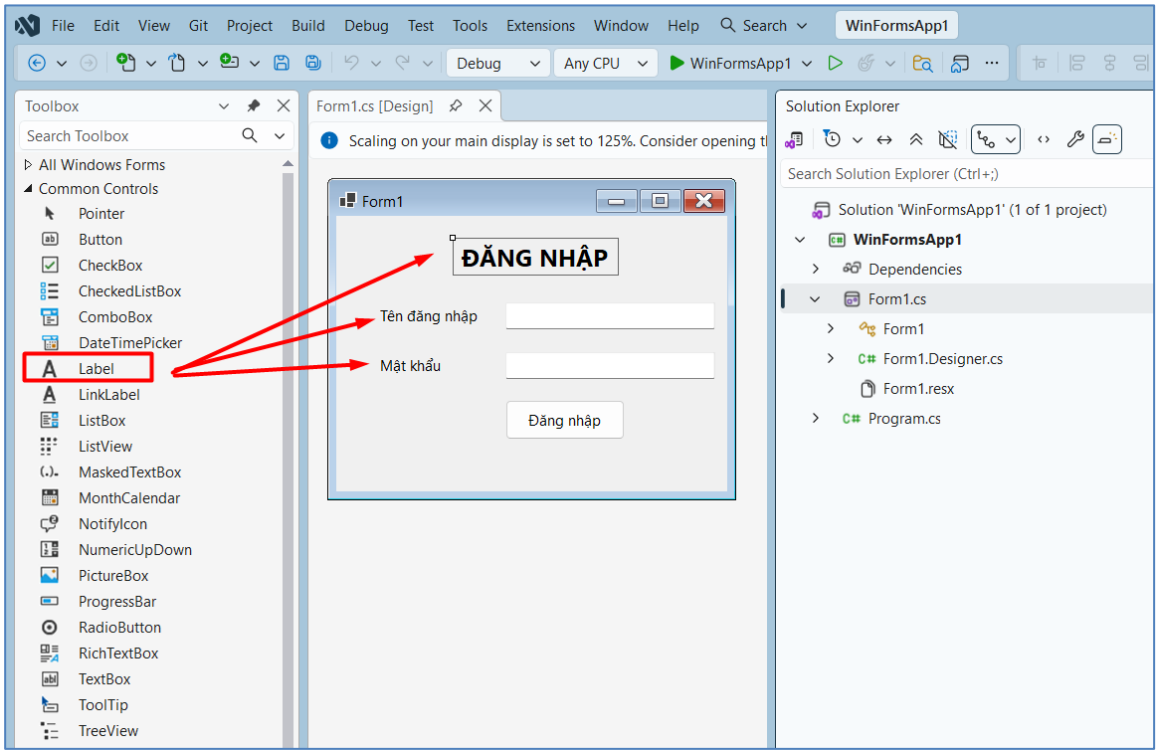
`KeyDown`
`KeyPress`
`KeyUp`
	Xảy ra khi người dùng nhấn, giữ hoặc thả phím trên bàn phím, với điều kiện Control đang có **Focus**.

3.2.1 Label, TextBox, Button

Label, **TextBox** và **Button** là ba Control cơ bản, được sử dụng thường xuyên trong hầu hết các ứng dụng Windows Forms. Đây là những Control đầu tiên mà người học cần nắm vững, bởi chúng xuất hiện trong gần như mọi giao diện nhập liệu và tương tác với người dùng. Mỗi Control đảm nhận một vai trò khác nhau:

Label dùng để hiển thị thông tin, **TextBox** dùng để nhập dữ liệu và **Button** dùng để thực hiện một thao tác khi người dùng nhấn vào.

Ví dụ:Thiết kế giao diện đăng nhập gồm



Trong ví dụ trên:

- Label: Hiển thị các nội dung "Tên đăng nhập:", "Mật khẩu:" và tiêu đề "Đăng nhập hệ thống".
- TextBox: Cho phép người dùng nhập tên đăng nhập và mật khẩu.
- Button: Khi người dùng nhấn "Đăng nhập", chương trình sẽ xử lý việc kiểm tra thông tin.

- **Label** dùng để hiển thị văn bản **tĩnh** trên Form — người dùng không thể chỉnh sửa trực tiếp nội dung của Label (khác với TextBox sẽ tìm hiểu ngay sau đây). Label thường dùng làm tiêu đề, chú thích, hoặc hiển thị kết quả/thông báo ra màn hình.

Label - *Thuộc tính thường dùng*	
Font	Phông chữ, kích thước, kiểu chữ (đậm, nghiêng)
Text	Nội dung văn bản hiển thị
TextAlign	Vị trí căn chữ (trái, giữa, phải) bên trong khung Label
ForeColor	Màu chữ
AutoSize	Tự động co giãn kích thước Label theo độ dài nội dung `Text`

- **TextBox** là ô nhập liệu, cho phép người dùng **gõ và chỉnh sửa** văn bản trực tiếp — đây là Control chủ yếu để nhận dữ liệu đầu vào từ người dùng trong WinForms, tương đương với `Console.ReadLine()` ở Console App, nhưng trực quan và linh hoạt hơn.

TextBox	
Thuộc tính thường dùng	
PlaceholderText	Chữ gợi ý mờ, hiện khi ô đang trống (chưa nhập gì)
Multiline	Cho phép nhập nhiều dòng (kết hợp `ScrollBars` để có thanh cuộn)
PasswordChar	Ký tự thay thế hiển thị (ví dụ `*`) — dùng cho ô nhập mật khẩu
ReadOnly	Nếu `true`, người dùng chỉ xem được, không sửa được nội

	dung
MaxLength	Số ký tự tối đa được phép nhập
Event thường dùng	
TextChanged	Mỗi khi nội dung trong ô thay đổi (gõ thêm/xóa ký tự)

Lấy dữ liệu người dùng đã nhập trong code, luôn thông qua thuộc tính **Text**:

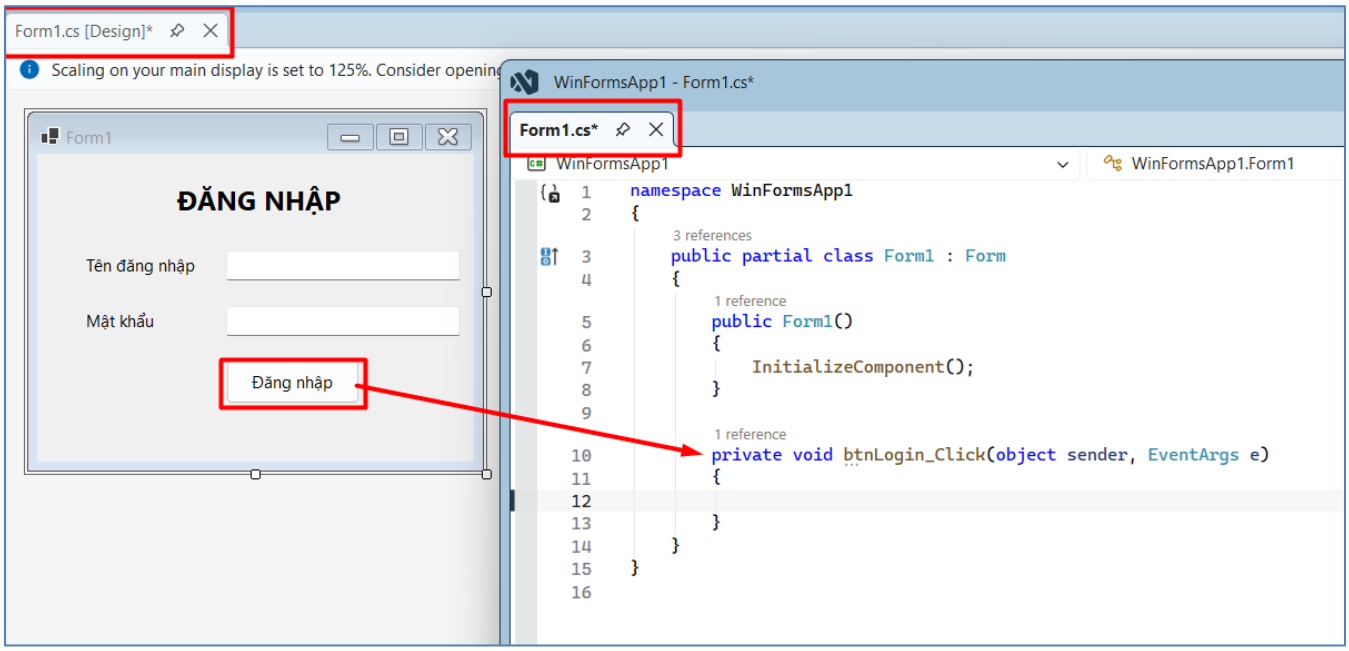
```
string hoTen = txtHoTen.Text;
```

- **Button**: Là Control dùng để tạo nút bấm trên giao diện. Khi người dùng nhấn vào Button, Control sẽ phát sinh **sự kiện Click** để chương trình thực hiện một chức năng hoặc thao tác đã được lập trình sẵn, chẳng hạn như **Đăng nhập**, **Lưu**, **Xóa**, **Tìm kiếm** hoặc **Thoát**.

Button	
Thuộc tính thường dùng	
Text	Chuỗi hiển thị trên bề mặt button
DialogResult	Giá trị trả về khi nút được nhấn trên một Dialog (sẽ gặp ở các bài sau)
Image	Ảnh hiển thị trên nút (kết hợp với `Text` hoặc thay thế hoàn toàn)
Event thường dùng	
Click	Kích hoạt khi user kích vào button, khai báo mặc định khi người lập trình kích đúp vào button trong màn hình Design View của Form.

Gắn sự kiện Click cho Button

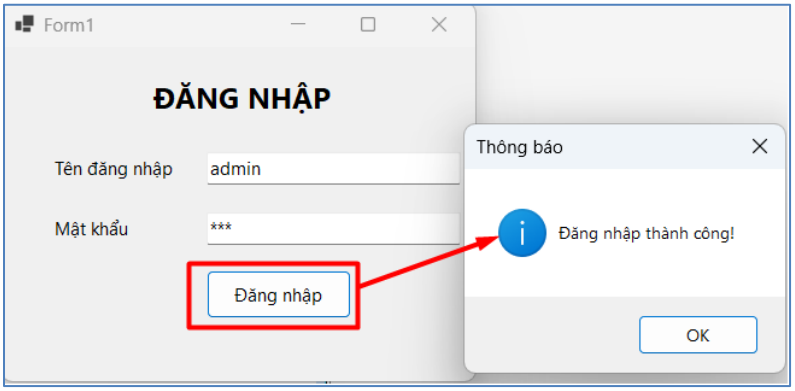
Cách nhanh nhất để gắn sự kiện cho Button: Trên Form Designer, chỉ cần nhấp đúp (**double-click**) vào **Button**. Visual Studio sẽ tự động tạo phương thức xử lý sự kiện Click tương ứng và liên kết Button với phương thức đó trong tệp Form1.cs, giúp người lập trình có thể viết ngay phần mã xử lý khi người dùng nhấn nút.



Sau khi **double-click** vào Button trên Form Designer, Visual Studio sẽ tự động tạo phương thức xử lý sự kiện Click:

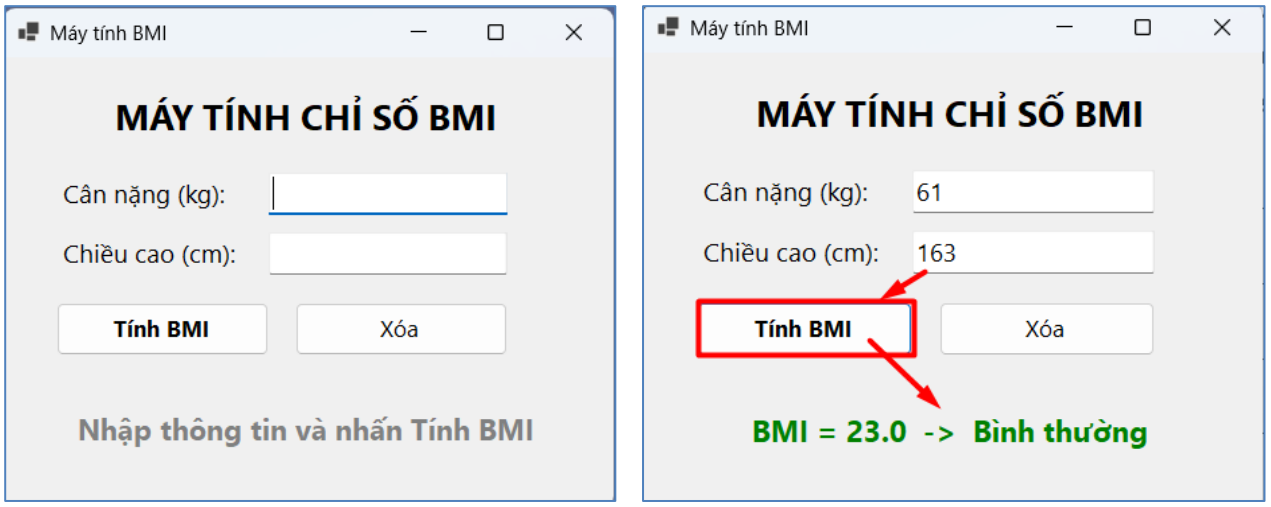
```
1 reference
private void btnLogin_Click(object sender, EventArgs e)
{
    if (txtUser.Text == "admin" && txtPassword.Text == "123")
    {
        MessageBox.Show("Đăng nhập thành công!", "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    else
    {
        MessageBox.Show("Tên đăng nhập hoặc mật khẩu không đúng!", "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}
```

Khi người dùng nhấn nút **Đăng nhập**, chương trình sẽ kiểm tra nội dung của hai ô nhập liệu. Nếu tên đăng nhập là **admin** và mật khẩu là **123**, một thông báo **"Đăng nhập thành công!"** sẽ xuất hiện. Ngược lại, chương trình sẽ hiển thị thông báo **"Tên đăng nhập hoặc mật khẩu không đúng!"**.



Ví dụ trên có sử dụng **MessageBox** là hộp thoại được Windows Forms cung cấp sẵn để hiển thị thông báo, cảnh báo, lỗi hoặc yêu cầu người dùng xác nhận một thao tác. Đây là một trong những thành phần được sử dụng phổ biến nhất trong các ứng dụng WinForms vì đơn giản, dễ sử dụng và không cần tự thiết kế giao diện.

Ví dụ vận dụng — Máy tính BMI (Chỉ số khối cơ thể)



Vận dụng kiến thức về Control cơ bản (Label, TextBox, Button), xử lý sự kiện (Event), Xây dựng ứng dụng WinForms cho phép người dùng nhập **cân nặng (kg)** và **chiều cao (cm)**, sau đó

Tính chỉ số BMI = cân nặng / (chiều cao * chiều cao)

Xếp loại theo bảng chuẩn sau:

BMI	**Xếp loại**
< 18.5	Thiếu cân
18.5 – < 25	Bình thường
25 – < 30	Thừa cân
≥ 30	Béo phì

HƯỚNG DẪN THỰC HIỆN

Bước 1: Tạo Project

1. Mở Visual Studio → Create a new project → chọn Windows Forms App (C#)

2. Đặt tên project, ví dụ MayTinhBMI, chọn Framework: .NET 10.0

Bước 2: Thiết kế giao diện (Form Designer)

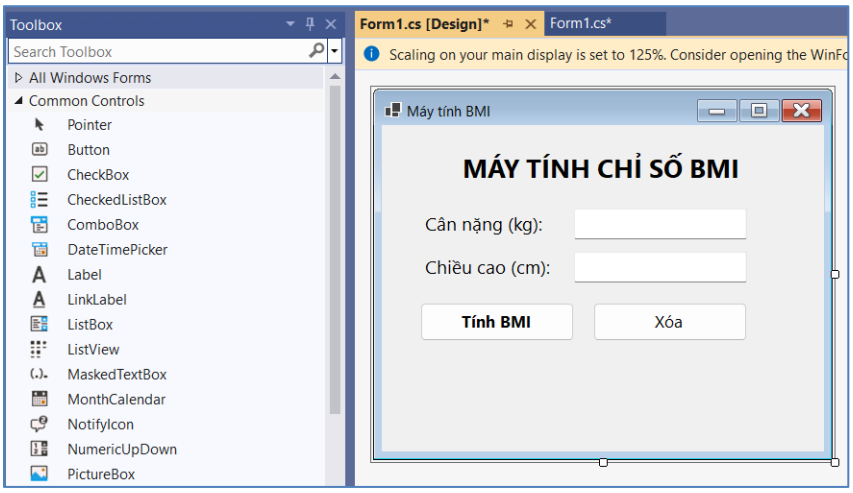
Từ **Toolbox**, kéo-thả các Control lên Form theo đúng vị trí, sau đó đổi `Name` và `Text` cho từng Control trong cửa sổ **Properties** (F4):

Thứ tự kéo	Control	Đổi `Name` thành	Đổi `Text` thành
1	Label	`lblTieuDe`	"MÁY TÍNH CHỈ SỐ BMI"
2	Label	`lblCanNang`	"Cân nặng (kg):"
3	TextBox	`txtCanNang`	(để trống)
4	Label	`lblChieuCao`	"Chiều cao (cm):"
5	TextBox	`txtChieuCao`	(để trống)
6	Button	`btnTinhBMI`	"Tính BMI"
7	Button	`btnXoa`	"Xóa"

Gợi ý chỉnh giao diện:

lblTieuDe: chọn Font lớn hơn (ví dụ 16pt, Bold) qua thuộc tính Font

lblKetQua: chọn Font lớn (14pt, Bold), TextAlign = MiddleCenter, AutoSize = False, kéo rộng khung label ra để chứa đủ chữ



Bước 3: Gắn sự kiện cho các nút

- Double-click vào Form (vùng nền trống) → Visual Studio tự tạo sẵn method Form1_Load
- Double-click vào btnTinhBMI → tự tạo **btnTinhBMI_Click**
- Double-click vào btnXoa → tự tạo **btnXoa_Click**

Bước 4: Viết logic xử lý — gợi ý từng bước

`Form1_Load` **— chạy khi Form vừa mở**

```
private void Form1_Load(object sender, EventArgs e)
{
    lblKetQua.Text = "Nhập thông tin và nhấn Tính BMI";
    lblKetQua.ForeColor = Color.Gray;
}
```

`btnTinhBMI_Click` **— xử lý khi nhấn "Tính BMI"**

```
private void btnTinhBMI_Click(object sender, EventArgs e)
{
    // Validate cân nặng
    if (!double.TryParse(txtCanNang.Text, out double canNang) || canNang <= 0)
    {
        lblKetQua.Text = "⚠ Vui lòng nhập cân nặng hợp lệ!";
        lblKetQua.ForeColor = Color.Red;
        txtCanNang.Focus();
        return;
    }

    // Validate chiều cao
    if (!double.TryParse(txtChieuCao.Text, out double chieuCaoCm) || chieuCaoCm <= 0)
    {
        lblKetQua.Text = "⚠ Vui lòng nhập chiều cao hợp lệ!";
        lblKetQua.ForeColor = Color.Red;
        txtChieuCao.Focus();
        return;
    }

    // Tính BMI
    double chieuCaoM = chieuCaoCm / 100;
    double bmi = canNang / (chieuCaoM * chieuCaoM);

    // Xếp loại
    string xepLoai = bmi switch
    {
        < 18.5 => "Thiếu cân",
        < 25 => "Bình thường",
        < 30 => "Thừa cân",
        _ => "Béo phì"
    };

    Color mauChu = bmi switch
    {
```

```
        < 18.5 => Color.Orange,
        < 25 => Color.Green,
        < 30 => Color.DarkOrange,
        _ => Color.Red
    };

    lblKetQua.Text = $"BMI = {bmi:0.0}  ->  {xepLoai}";
    lblKetQua.ForeColor = mauChu;
}
```

`btnXoa_Click` **— xử lý khi nhấn "Xóa"**

```
private void btnXoa_Click(object sender, EventArgs e)
{
    txtCanNang.Clear();
    txtChieuCao.Clear();
    lblKetQua.Text = "Nhập thông tin và nhấn Tính BMI";
    lblKetQua.ForeColor = Color.Gray;
    txtCanNang.Focus();
}
```

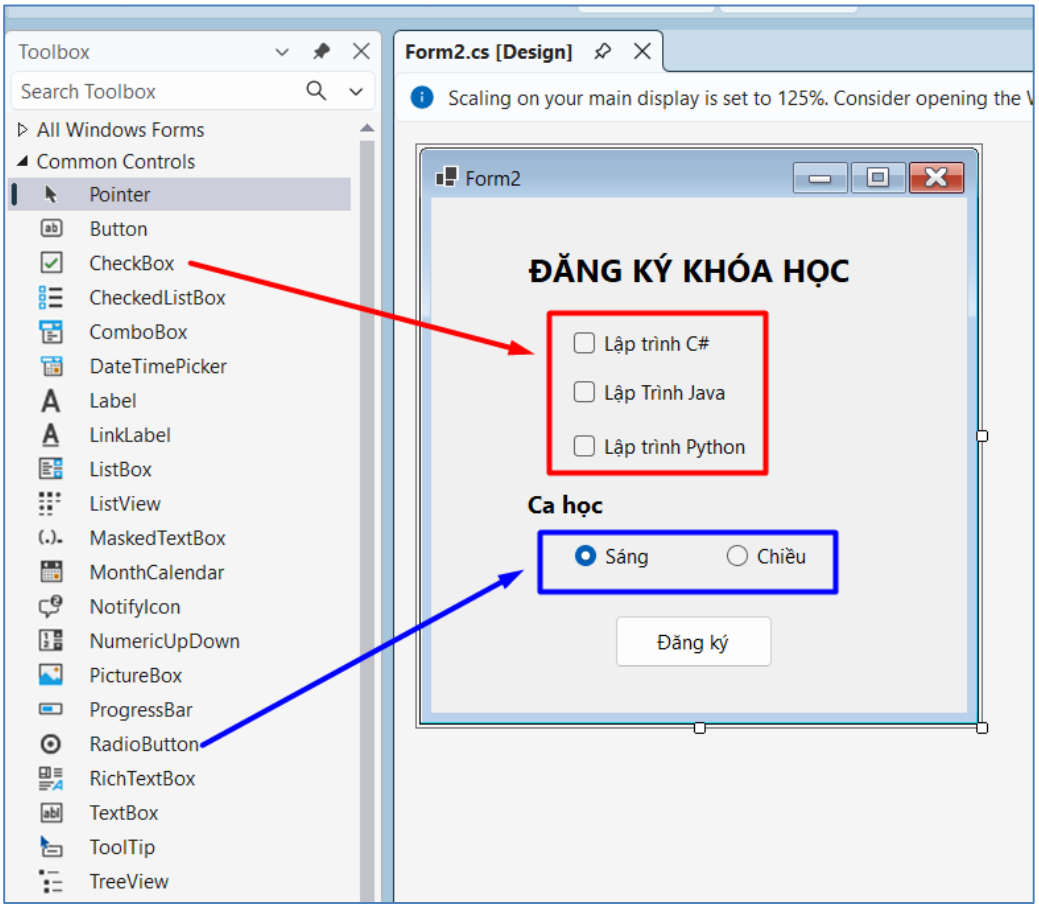
Bước 5: Chạy thử và kiểm tra (F5)

Kiểm tra đủ 3 trường hợp theo đúng yêu cầu

- Hợp lệ: nhập cân nặng 60, chiều cao 170 → kỳ vọng BMI = 20.8 - Bình thường (chữ xanh)
- Sai định dạng: nhập cân nặng abc → kỳ vọng thông báo lỗi, không crash, con trỏ quay lại đúng ô
- Số âm/bằng 0: nhập chiều cao 0 hoặc -5 → kỳ vọng thông báo lỗi tương ứng

3.2.2 CheckBox, RadioButton

CheckBox và RadioButton chỉ cho phép người dùng chọn giữa các tùy chọn đã được định nghĩa sẵn từ trước



CheckBox

CheckBox là Control dùng để biểu diễn một lựa chọn bật/tắt hoặc có/không. Người dùng có thể đánh dấu (✔) hoặc bỏ đánh dấu một CheckBox, và việc thay đổi trạng thái của một CheckBox không ảnh hưởng đến các CheckBox khác trên cùng Form.

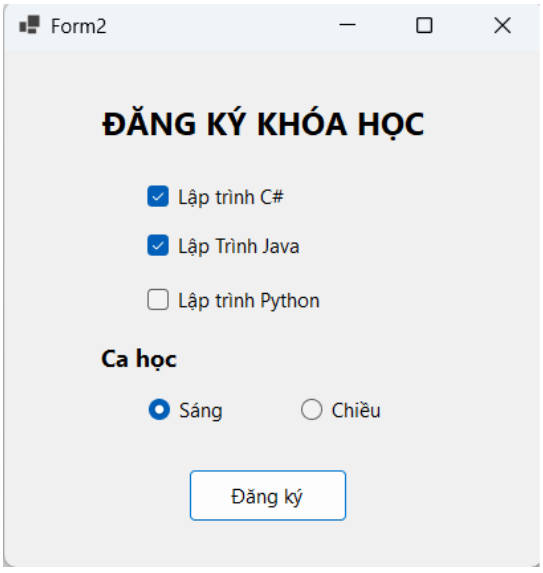
CheckBox thường được sử dụng khi người dùng có thể chọn nhiều tùy chọn cùng lúc hoặc trả lời các câu hỏi độc lập, chẳng hạn như "Đã thanh toán?", "Đăng ký nhận email?", hoặc chọn các môn học muốn đăng ký.

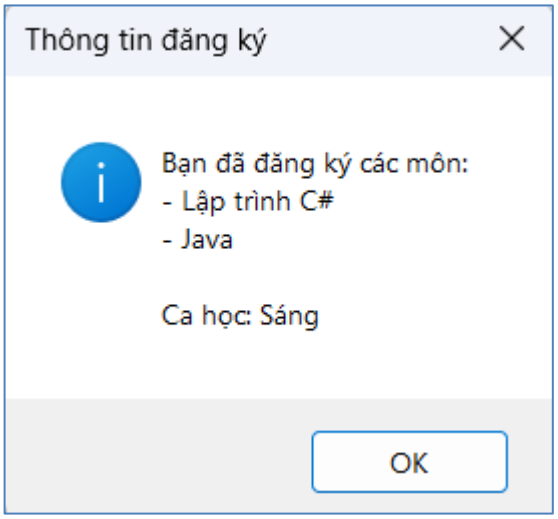
CheckBox	
Thuộc tính thường dùng	
`Checked`	`true/false` — trạng thái hiện tại có đang được tick hay không
`CheckState`	Chi tiết hơn `Checked`, có thêm trạng thái `Indeterminate` (chưa xác định — ít dùng ở mức cơ bản)
`ThreeState`	Nếu `true`, cho phép CheckBox có 3 trạng thái (`Checked`,

	`Unchecked`, `Indeterminate`) khi người dùng click liên tục
Event thường dùng	
CheckedChanged	Mỗi khi trạng thái `Checked` thay đổi (từ tick sang bỏ tick, hoặc ngược lại)

Để kiểm tra một CheckBox có đang được chọn hay không, sử dụng thuộc tính Checked. Thuộc tính này có kiểu dữ liệu bool (**true** hoặc **false**), vì vậy có thể sử dụng trực tiếp trong câu lệnh if để xử lý các tình huống khác nhau.

```
private void btnDangKy_Click(object sender, EventArgs e)
 {
     string monHoc = "";
     if (chkCSharp.Checked) monHoc += "- Lập trình C#\n";
     if (chkJava.Checked) monHoc += "- Java\n";
     if (chkPython.Checked) monHoc += "- Python\n";
     string caHoc = ""; if (radSang.Checked) caHoc = "Sáng";
     else if (radChieu.Checked) caHoc = "Chiều";
     MessageBox.Show($"Bạn đã đăng ký các môn:\n{monHoc}\nCa học: {caHoc}",
         "Thông tin đăng ký",
         MessageBoxButtons.OK,
         MessageBoxIcon.Information);
 }
```



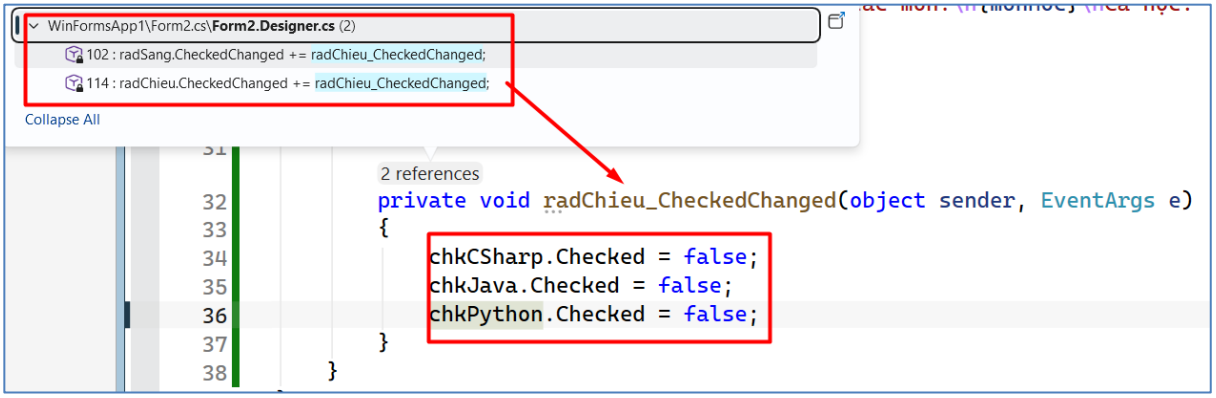


RadioButton

RadioButton là Control dùng để biểu diễn các lựa chọn loại trừ lẫn nhau. Trong cùng một nhóm, người dùng chỉ có thể chọn một RadioButton tại một thời điểm. Khi chọn một RadioButton, các RadioButton còn lại trong nhóm sẽ tự động bị bỏ chọn. RadioButton thường được sử dụng khi người dùng cần chọn một và chỉ một

phương án trong nhiều lựa chọn, chẳng hạn như giới tính, ca học, hình thức thanh toán hoặc phương thức vận chuyển.

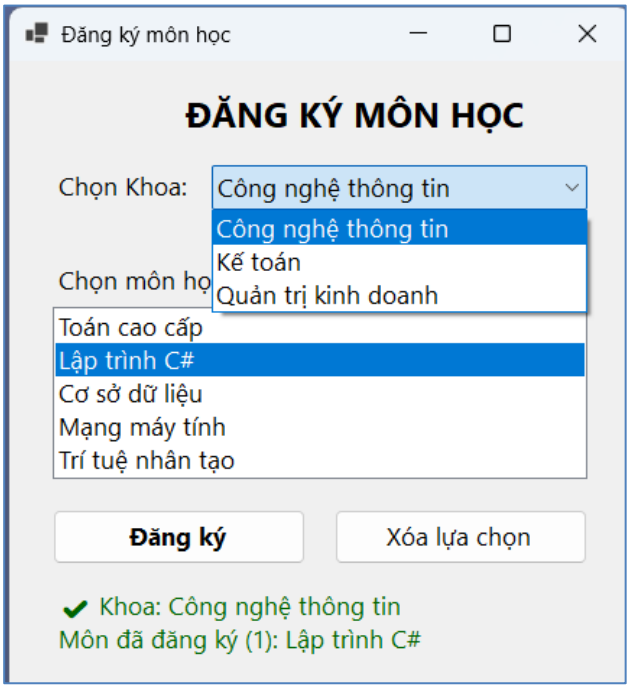
Sự kiện thường dùng: `CheckedChanged` – được kích hoạt khi người dùng chọn một RadioButton hoặc chuyển sang chọn RadioButton khác trong cùng nhóm. Ví dụ: Khi người dùng chuyển sang **ca học** khác, chương trình sẽ tự động bỏ chọn tất cả các môn học đã đăng ký trước đó.



3.2.3 ComboBox và ListBox

ComboBox và **ListBox** đều là các Control dùng để hiển thị danh sách các lựa chọn cho người dùng. Hai Control này có nhiều điểm tương đồng, cùng hỗ trợ các thuộc tính quan trọng như Items, SelectedItem và SelectedIndex để quản lý và lấy giá trị được chọn.

ComboBox — danh sách thả xuống



ComboBox hiển thị danh sách dưới dạng thu gọn — chỉ một dòng hiện tại được nhìn thấy, toàn bộ danh sách chỉ xuất hiện khi người dùng click vào để mở ra. Phù hợp khi danh sách lựa chọn dài, mà không muốn chiếm nhiều diện tích trên Form.

ListBox — danh sách hiển thị sẵn

ListBox hiển thị toàn bộ hoặc một phần danh sách ngay trên Form, không cần click để mở ra — người dùng nhìn thấy nhiều lựa chọn cùng lúc (kèm thanh cuộn nếu danh sách dài hơn khung hiển thị). Phù hợp khi muốn người dùng dễ dàng nhìn lướt qua toàn bộ các lựa chọn, hoặc khi cần cho phép chọn nhiều mục cùng lúc.

ComboBox, ListBox	
Thuộc tính thường dùng	
`Items`	Danh sách các lựa chọn
`SelectedItem`	Giá trị đang được chọn (kiểu `object`)
`SelectedIndex`	Vị trí (chỉ số) của lựa chọn đang chọn, bắt đầu từ `0`; bằng `-1` nếu chưa chọn gì
`SelectionMode`	Cho phép chọn **một** (`One`) hay **nhiều** (`MultiSimple`/`MultiExtended`) mục cùng lúc — ComboBox không có khả năng này
`SelectedItems`	Danh sách **tất cả** các mục đang được chọn (chỉ có ý nghĩa khi `SelectionMode` cho phép chọn nhiều)
Event thường dùng	
SelectedIndexChanged	Mỗi khi lựa chọn (theo vị trí) thay đổi — sự kiện dùng nhiều nhất, đã gặp ở các mục trước
SelectedValueChanged	Tương tự `SelectedIndexChanged`, nhưng theo dõi sự thay đổi của `SelectedValue` (thường dùng khi binding dữ liệu — sẽ gặp ở các bài sau)

Thêm dữ liệu — cách làm giống nhau cho cả hai, thường đặt trong `Form1_Load:`

```
private void Form1_Load(object sender, EventArgs e)
{
    // Nạp dữ liệu cho ComboBox – chọn Khoa
    cboKhoa.Items.Add("Công nghệ thông tin");
    cboKhoa.Items.Add("Kế toán");
    cboKhoa.Items.Add("Quản trị kinh doanh");
    cboKhoa.SelectedIndex = 0;

    // Nạp dữ liệu cho ListBox – chọn nhiều môn học
    lstMonHoc.Items.Add("Toán cao cấp");
    lstMonHoc.Items.Add("Lập trình C#");
    lstMonHoc.Items.Add("Cơ sở dữ liệu");
    lstMonHoc.Items.Add("Mạng máy tính");
    lstMonHoc.Items.Add("Trí tuệ nhân tạo");
    lstMonHoc.SelectionMode = SelectionMode.MultiExtended;

    lblKetQua.Text = "Chọn khoa và môn học, sau đó nhấn Đăng ký";
}
```

Sự kiện — cả hai cùng dùng `SelectedIndexChanged`:

```
private void lstMonHoc_SelectedIndexChanged(object sender, EventArgs e)
{
    lblThongBao.Text = $"Bạn vừa chọn: {lstMonHoc.SelectedItem}";
}
```

Thuộc tính riêng — chỉ có ở ListBox

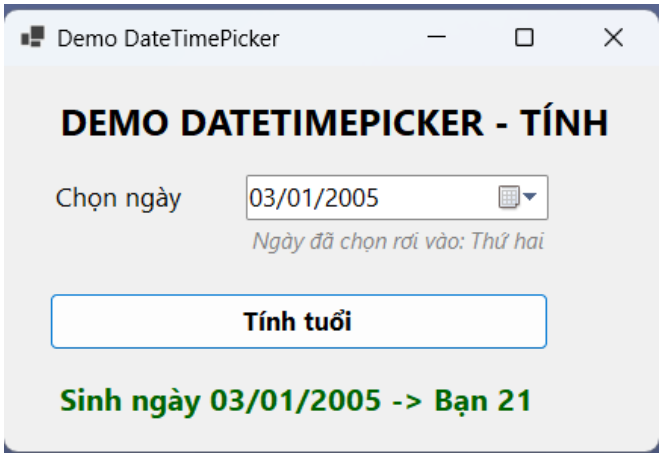
SelectionMode, SelectedItems là điểm khác biệt quan trọng nhất giữa hai Control: ComboBox chỉ cho chọn **một** giá trị duy nhất, còn ListBox (khi đặt `SelectionMode = MultiExtended` hoặc `MultiSimple`) cho phép chọn **nhiều** giá trị cùng lúc — một khả năng ComboBox hoàn toàn không có.

```
private void btnDangKy_Click(object sender, EventArgs e)
{
    List<string> monDaChon = new List<string>();
    foreach (object item in lstMonHoc.SelectedItems)
    {
        monDaChon.Add(item.ToString());
    }
    string ketQua = monDaChon.Count > 0
        ? string.Join(", ", monDaChon)
        : "(chưa chọn môn nào)";

    lblKetQua.Text = $"Các môn đã đăng ký: {ketQua}";
}
```

3.2.4 DateTimePicker

DateTimePicker là Control chuyên dùng để chọn **ngày, giờ**, hiển thị dưới dạng một ô có lịch thả xuống trực quan. Đây là lựa chọn tốt hơn hẳn so với việc dùng TextBox để người dùng tự gõ ngày tháng — vì cách gõ tự do rất dễ sai định dạng (`30/13/2026`, `2026-2-30`...) hoặc nhập ngày không tồn tại, còn DateTimePicker đảm bảo giá trị trả về **luôn luôn hợp lệ**, không cần validate thêm như khi xử lý TextBox.



Thuộc tính, Events thường dùng

DateTimePicker	
Thuộc tính thường dùng	
`Value`	Giá trị ngày/giờ đang được chọn (kiểu `DateTime`) — quan trọng nhất
`Format`	Kiểu hiển thị: `Long` (đầy đủ, ví dụ "Sunday, June 28, 2026"), `Short` (ngắn gọn, ví dụ "28/06/2026"), `Time` (chỉ hiện giờ), `Custom` (tự định nghĩa)
`CustomFormat`	Chuỗi định dạng tùy chỉnh, chỉ có tác dụng khi `Format = Custom` (ví dụ `"dd/MM/yyyy"`)
`MinDate`, `MaxDate`	Giới hạn khoảng ngày được phép chọn
`ShowUpDown`	Nếu `true`, hiển thị nút tăng/giảm thay cho lịch thả xuống (phù hợp khi chỉ cần chọn giờ)
Event thường dùng	
ValueChanged	Mỗi khi giá trị `Value` thay đổi (người dùng chọn lại ngày/giờ khác)

Làm việc với kiểu DateTime

Vì `Value` luôn trả về đúng kiểu `DateTime` có sẵn của .NET, có thể tận dụng ngay các thuộc tính/phương thức xử lý ngày tháng quen thuộc:

```
DateTime ngaySinh = dtpNgaySinh.Value;

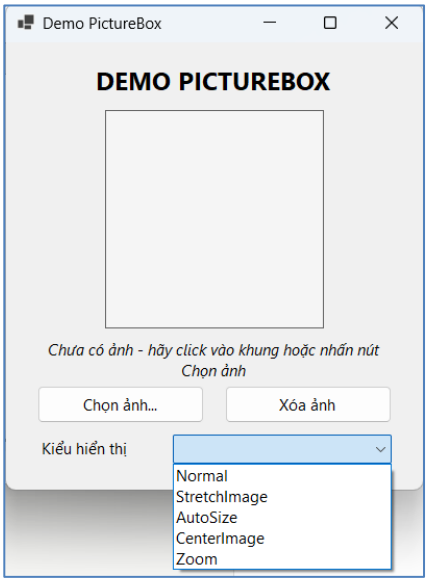
int nam = ngaySinh.Year;
int thang = ngaySinh.Month;
int ngay = ngaySinh.Day;
DayOfWeek thuTrongTuan = ngaySinh.DayOfWeek;

// Tính tuổi chính xác hơn (có xét đến đã qua sinh nhật trong năm hay chưa)
int tuoi = DateTime.Today.Year - ngaySinh.Year;
if (DateTime.Today.Month < ngaySinh.Month ||
    (DateTime.Today.Month == ngaySinh.Month && DateTime.Today.Day < ngaySinh.Day))
{
    tuoi--;
}
```

Đoạn tính tuổi phía trên minh họa một lỗi logic khá phổ biến nếu chỉ tính đơn giản `Năm hiện tại - Năm sinh`: nếu chưa qua sinh nhật trong năm nay, công thức đơn giản sẽ tính dư ra 1 tuổi — có thể kiểm tra thêm tháng và ngày để có kết quả chính xác.

3.2.5 PictureBox

PictureBox là Control dùng để hiển thị **hình ảnh** trên Form — có thể là ảnh tĩnh gán sẵn (logo, ảnh minh họa), hoặc ảnh được nạp động trong lúc chương trình đang chạy (ảnh đại diện người dùng, ảnh sản phẩm tải từ file...).



PictureBox	
Thuộc tính thường dùng	
`Image`	Ảnh đang hiển thị (kiểu `Image`) — quan trọng nhất
`SizeMode`	Cách ảnh hiển thị trong khung Control
`BorderStyle`	Kiểu viền xung quanh khung ảnh (None, FixedSingle, Fixed3D)

Các giá trị của `SizeMode`

Normal — Giữ nguyên kích thước gốc của ảnh, có thể bị cắt nếu ảnh lớn hơn khung

StretchImage — Kéo giãn ảnh lấp đầy toàn bộ khung — có thể làm méo ảnh nếu tỉ lệ khung khác tỉ lệ ảnh gốc

AutoSize — Khung PictureBox tự co giãn theo đúng kích thước ảnh

CenterImage — Giữ nguyên kích thước gốc, căn giữa ảnh trong khung

Zoom — Phóng to/thu nhỏ ảnh theo đúng tỉ lệ gốc (không méo), vừa khít với khung — thường dùng nhất trong thực tế

Gán ảnh từ file có sẵn

Cách đơn giản nhất — nạp ảnh từ một đường dẫn file cụ thể đã biết trước:

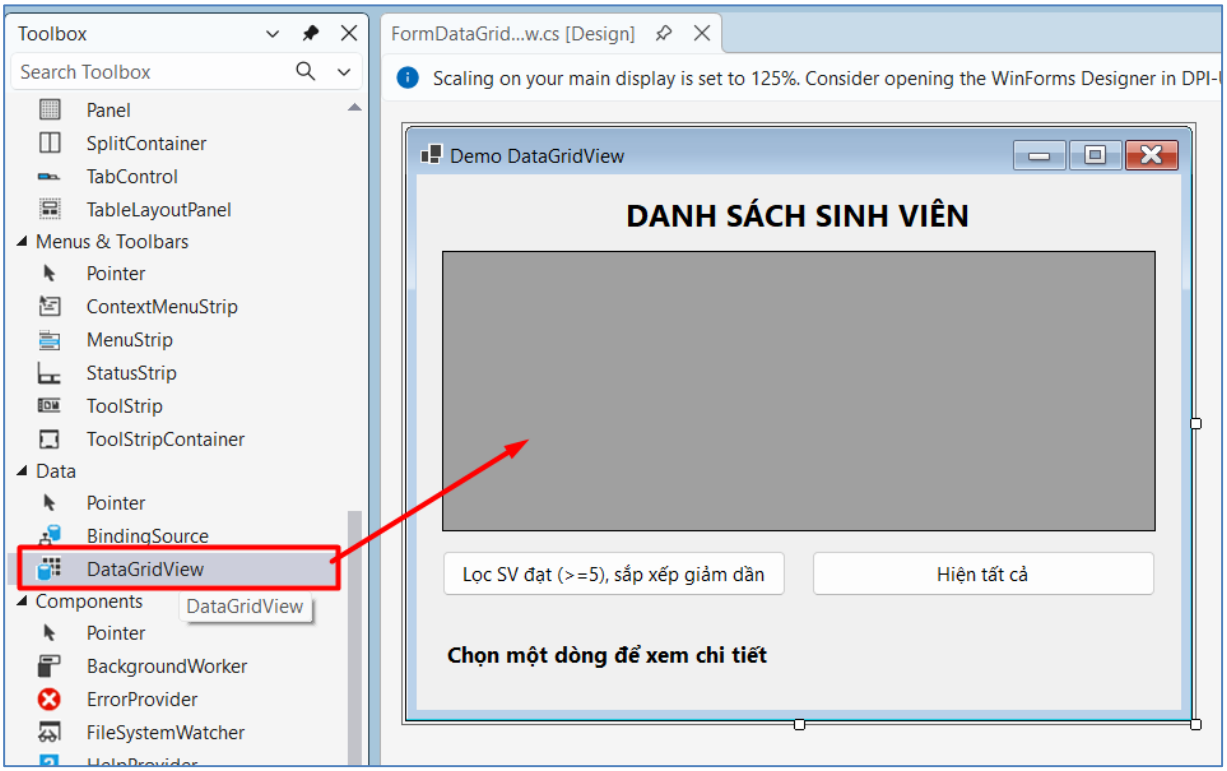
```
picAvatar.Image = Image.FromFile(@"D:\Anh\avatar.jpg");
```

Lưu ý: đường dẫn cố định như trên chỉ phù hợp khi chạy thử trên máy của chính người viết code — khi đóng gói ứng dụng để chạy trên máy khác, đường dẫn này gần như chắc chắn không còn đúng. Trong thực tế, ảnh thường được đặt trong thư mục con của project (ví dụ thư mục **`Images`**), và truy cập qua đường dẫn tương đối, hoặc cho người dùng tự chọn ảnh (xem mục tiếp theo).

3.2.6 DataGridView

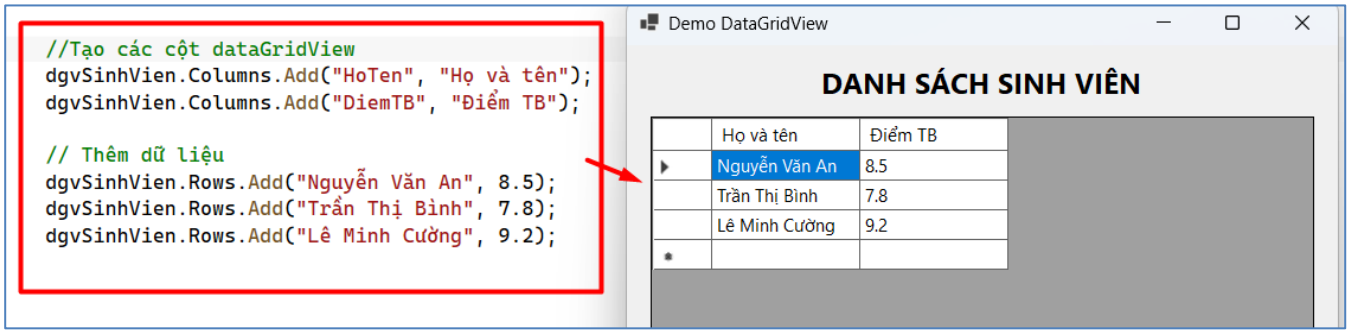
DataGridView là Control hiển thị dữ liệu dưới dạng **bảng** (gồm nhiều dòng, nhiều cột) — tương tự một bảng tính Excel thu nhỏ ngay trên Form. Đây là Control quan trọng nhất khi cần hiển thị **danh sách dữ liệu** (nhiều bản ghi cùng lúc), thay vì chỉ xem từng mục riêng lẻ như ComboBox/ListBox đã học — rất phù hợp để hiển thị danh

sách sinh viên, danh sách sản phẩm, hay bất kỳ tập hợp dữ liệu có cấu trúc dạng bảng.



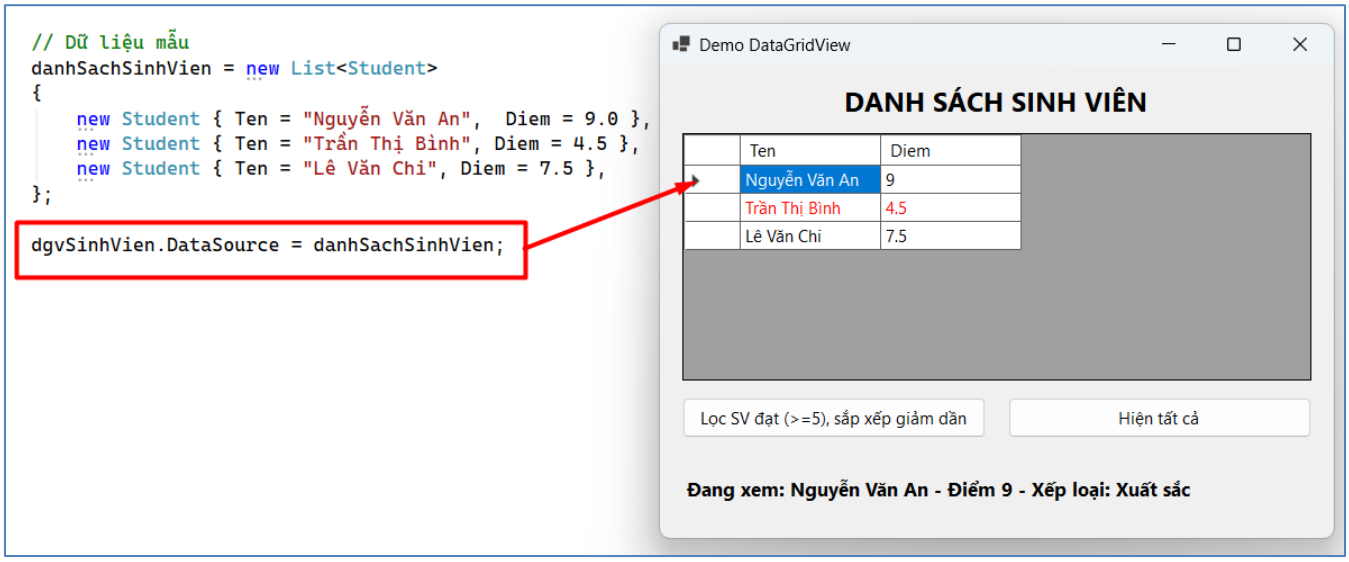
Hai cách đưa dữ liệu vào DataGridView

Cách 1 – Thêm dữ liệu thủ công bằng `Rows.Add()`: Cách này phù hợp để thử nghiệm nhanh, nhưng không thực tế khi dữ liệu đến từ một danh sách object.



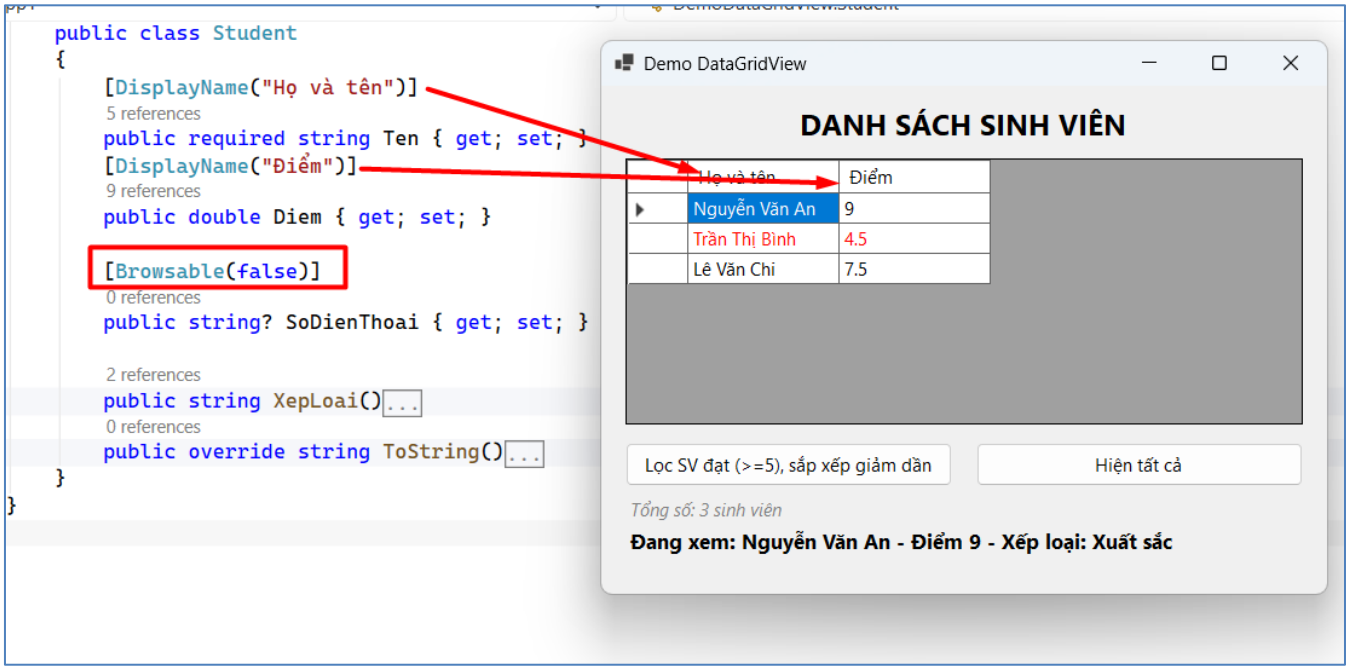
Cách 2 — Gán trực tiếp qua DataSource (cách dùng chính, kết hợp OOP đã học)

DataGridView có khả năng tự động hiển thị toàn bộ một `List<T>` thông qua thuộc tính `DataSource` — mỗi property của class `T` sẽ tự động trở thành một cột trong bảng:



Chỉ với một dòng `dgvSinhVien.DataSource = danhSachSinhVien;` toàn bộ danh sách **`Student`** hiển thị ngay lên bảng — mỗi property **`Ten`, `Diem`** tự động trở thành một cột, mỗi object trong list tự động trở thành một dòng.

C# còn hỗ trợ **Attribute** để bổ sung thông tin cho lớp, thuộc tính hoặc phương thức. Attribute không lưu trữ dữ liệu mà dùng để cung cấp thông tin cho chương trình hoặc các thư viện trong quá trình xử lý.



Attribute	Ý nghĩa

`DisplayName`	Đặt tên hiển thị cho Property thay vì sử dụng tên gốc. Ví dụ, `Ten` sẽ được hiển thị thành **Họ và tên**.
`Browsable(false)`	Ẩn Property khỏi các công cụ hỗ trợ thiết kế hoặc sinh cột tự động

Thuộc tính và sự kiện thường dùng

DataGridView	
Thuộc tính thường dùng	
`DataSource`	Nguồn dữ liệu gán vào bảng (thường là `List<T>`)
`Columns`	Danh sách các cột — có thể truy cập để đổi tên hiển thị, ẩn cột, đổi độ rộng
`Rows`	Danh sách các dòng dữ liệu hiện có
`SelectedRows`	Danh sách (các) dòng đang được chọn
`AutoSizeColumnsMode`	Cách các cột tự co giãn độ rộng (ví dụ `Fill` — lấp đầy toàn bộ chiều ngang bảng)
Sự kiện thường dùng	
`CellClick`	Khi click vào một ô (cell) cụ thể trong bảng
`CellDoubleClick`	Khi **double-click** vào một ô
`SelectionChanged`	Khi dòng/ô đang được chọn thay đổi

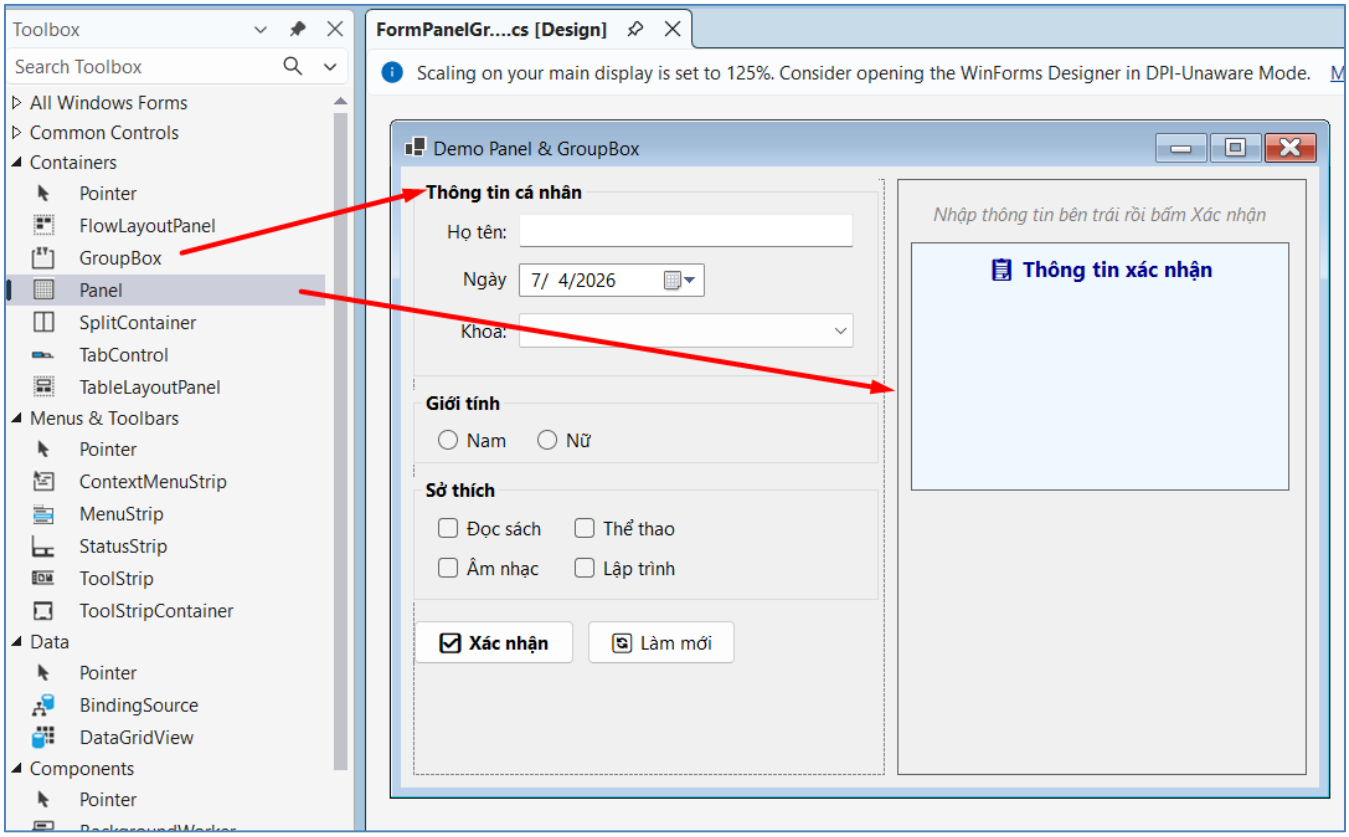
Cập nhật lại bảng khi dữ liệu thay đổi

Một điểm cần lưu ý: nếu `List<T>` gán vào `DataSource` có thay đổi sau đó (thêm/xóa phần tử), DataGridView **không tự động cập nhật lại hiển thị** ngay — cần gọi lại việc gán `DataSource`, hoặc dùng `BindingSource` để đồng bộ tự động:

```
danhSachSinhVien.Add(new Student { Ten = "Dung", Diem = 7.0 });
// Cách đơn giản để cập nhật lại hiển thị
dgvSinhVien.DataSource = null;
dgvSinhVien.DataSource = danhSachSinhVien;
```

3.2.7 Panel và GroupBox

Khi Form có nhiều Control, việc đặt tất cả lên cùng một mặt phẳng mà không có sự phân nhóm nào sẽ khiến giao diện trở nên rối và khó bảo trì. Panel và GroupBox giải quyết vấn đề này bằng cách đóng vai trò container — gom nhóm các Control liên quan vào một vùng riêng, giúp giao diện có cấu trúc rõ ràng hơn.



Panel

Panel là một vùng chứa trong suốt, không có tiêu đề — chỉ đơn giản là một khung để gom nhóm các Control lại với nhau. Thường dùng để chia Form thành các vùng chức năng (ví dụ: vùng nhập liệu bên trái, vùng danh sách bên phải) hoặc để kiểm soát hiển thị (Visible = false) cả một nhóm Control cùng lúc.

```
// Ẩn/hiện cả nhóm Control bên trong Panel chỉ bằng 1 dòng
 panelNhapLieu.Visible = false;
```

Thuộc tính hay dùng:

- BorderStyle: (Viền: None, FixedSingle, Fixed3D)

- Dock: Ghim vào cạnh Form: Left, Right, Fill...
- AutoScroll: Tự động thêm thanh cuộn khi nội dung tràn

GroupBox

GroupBox tương tự Panel nhưng có thêm tiêu đề hiển thị ở góc trên bên trái và đường viền bao quanh — giúp người dùng nhận biết rõ nhóm chức năng của các Control bên trong. Thường dùng để nhóm RadioButton (vì các RadioButton trong cùng một GroupBox tự động loại trừ lẫn nhau) hoặc nhóm các ô nhập liệu cùng chủ đề.

Thuộc tính hay dùng:

- Text: Tiêu đề hiển thị ở góc trên
- Font: Kiểu chữ của tiêu đề
- Enabled: Vô hiệu hóa toàn bộ Control bên trong

TÓM TẮT

Trong bài này, học viên làm quen với lập trình giao diện ứng dụng desktop sử dụng Windows Forms, bao gồm:

Tổng quan về WinForms: kiến trúc Event-driven Programming, cách tạo project WinForms đầu tiên, vòng đời của Form (Load, FormClosing) và cơ chế gắn sự kiện qua Designer hoặc toán tử +=

Các Control nhập liệu: TextBox, ComboBox, ListBox, CheckBox, RadioButton, DateTimePicker

Các Control hiển thị: Label, PictureBox, DataGridView — bao gồm gắn DataSource, định nghĩa cột và xử lý sự kiện SelectionChanged

Các Control bố cục: Panel (chia vùng giao diện, ẩn/hiện nhóm Control) và GroupBox (nhóm các Control liên quan với tiêu đề rõ ràng)

Thuộc tính chung của Control (Name, Text, Visible, Enabled, Font, BackColor...) và cách chỉnh sửa qua Properties Window

CÂU HỎI ÔN TẬP

Câu 1: Trình bày khái niệm Event-driven Programming. So sánh với mô hình lập trình tuần tự đã học ở Bài 1-2, chỉ ra sự khác biệt về cách luồng thực thi được quyết định.

Câu 2: Trình bày vòng đời cơ bản của một Form, tập trung vào hai sự kiện Load và FormClosing. Viết đoạn code minh họa cách hủy việc đóng Form khi người dùng chưa xác nhận.

Câu 3: Giải thích vì sao mọi Control trong WinForms đều có sẵn các thuộc tính Name, Enabled, Visible mà không cần lập trình viên tự định nghĩa riêng cho từng loại. Khái niệm OOP nào ở Bài 2 giải thích được điều này?

Câu 4: Phân biệt CheckBox và RadioButton về mặt: (a) số lượng được chọn cùng lúc, (b) vai trò của GroupBox khi có nhiều nhóm RadioButton trên cùng một Form.

Câu 5: So sánh ComboBox và ListBox. Trong tình huống cần cho phép người dùng chọn nhiều môn học cùng lúc, nên dùng Control nào, và cần cấu hình thêm thuộc tính gì?

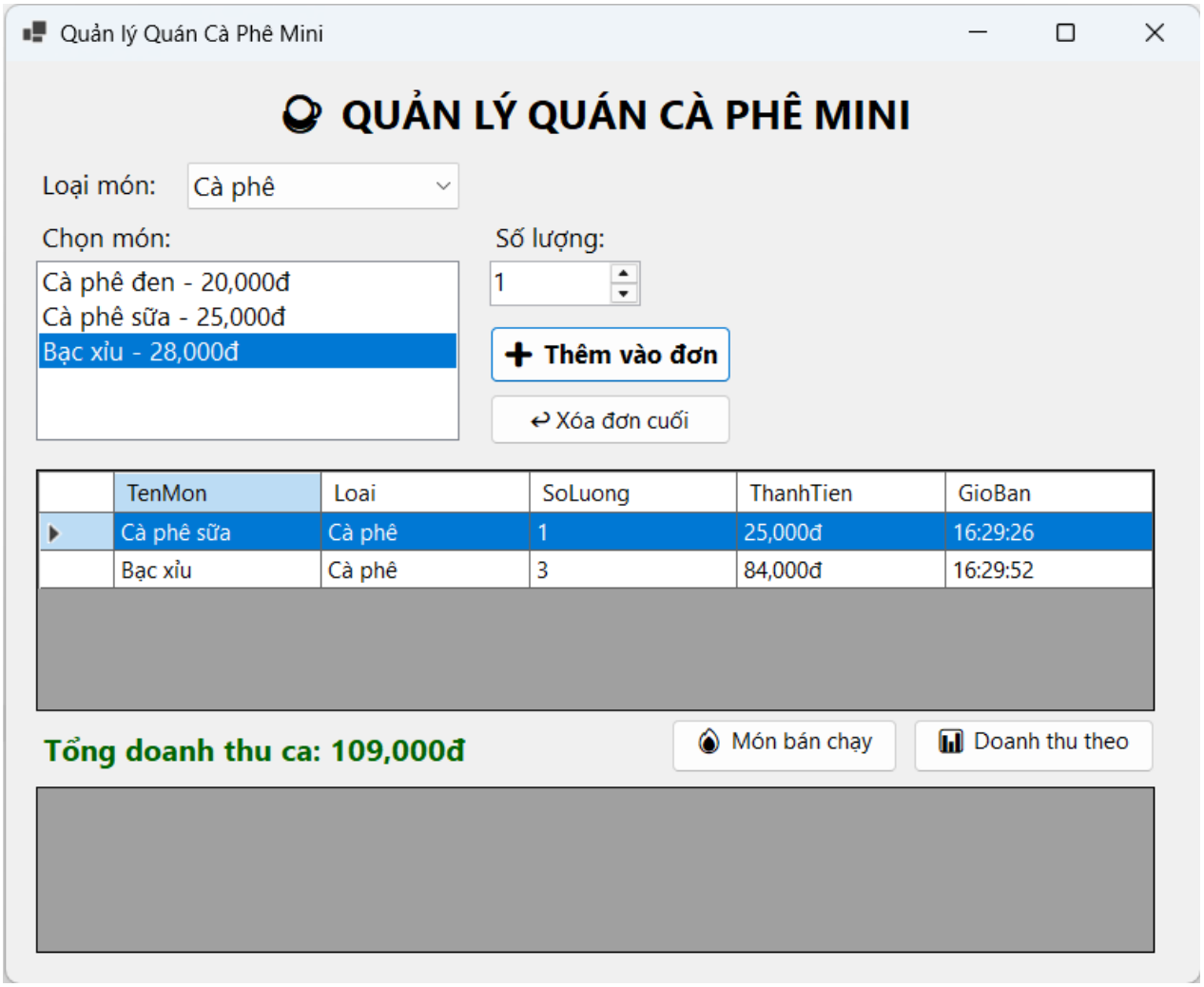
Câu 6: Vì sao nên ưu tiên dùng DateTimePicker thay cho TextBox khi cần người dùng nhập ngày sinh? Nêu rõ lợi ích về mặt dữ liệu đầu vào.

Câu 7: Trình bày sự khác nhau giữa các giá trị SizeMode của PictureBox: StretchImage, Zoom, CenterImage. Giá trị nào nên dùng để tránh ảnh bị méo khi hiển thị?

Câu 8: Cho class Student đã có ở Bài 2 (thuộc tính Ten, Diem, method XepLoai()). Viết đoạn code gán một List<Student> vào DataGridView, sau đó viết sự kiện SelectionChanged để hiển thị xếp loại của sinh viên đang được chọn lên một Label.

Câu 9: Cho một DataGridView đang hiển thị List<Student>. Viết đoạn code dùng LINQ để lọc ra các sinh viên có điểm từ 8 trở lên, sắp xếp tăng dần theo tên, rồi cập nhật lại dữ liệu hiển thị trên DataGridView đó.

Câu 10: Tạo ứng dụng dạng Windows Forms Ứng dụng "Quản lý Quán Cà Phê Mini"



Một quán cà phê nhỏ cần một ứng dụng đơn giản để **chọn món, tính tiền, và xem báo cáo bán hàng** trong ca làm việc.

BÀI 4: ỨNG DỤNG NHIỀU FORM VÀ MDI

Sau khi học xong bài này, học viên có thể nắm được:

- *Hiểu được vai trò và lợi ích của **ứng dụng nhiều Form (Multi-Form)** trong Windows Forms.*
- *Tạo, mở và quản lý nhiều Form trong cùng một ứng dụng.*
- *Sử dụng MessageBox để hiển thị thông báo, cảnh báo và hộp thoại xác nhận.*
- *Truyền dữ liệu giữa các **Form** bằng các phương pháp phù hợp.*
- *Xây dựng ứng dụng theo mô hình MDI, quản lý nhiều cửa sổ làm việc trong cùng một Form.*
- *Thiết kế giao diện ứng dụng bằng các điều khiển MenuStrip, ContextMenuStrip, ToolStrip và StatusStrip.*
- *Xây dựng các ứng dụng Windows Forms có giao diện trực quan, dễ sử dụng và tổ chức hợp lý.*

4.1 LÀM VIỆC VỚI NHIỀU FORM

4.1.1 MessageBox

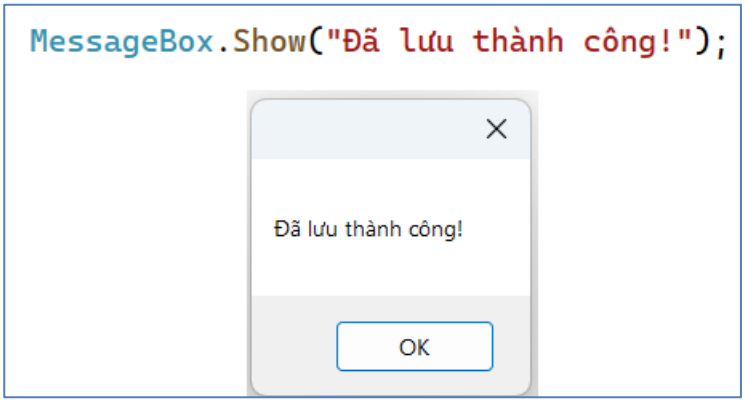
MessageBox là lớp có sẵn trong Windows Forms, dùng để hiển thị thông báo, cảnh báo hoặc hộp thoại xác nhận mà không cần tự thiết kế giao diện. Hộp thoại hoạt động theo cơ chế modal: khi xuất hiện, Form hiện tại sẽ bị khóa cho đến khi người dùng nhấn nút để đóng hộp thoại.

Có thể xem MessageBox như một Form được Microsoft thiết kế sẵn, chỉ cần gọi bằng một dòng lệnh để sử dụng.

Cú pháp cơ bản

Ở dạng đơn giản nhất, `MessageBox.Show()` chỉ cần một tham số là nội dung thông báo:

```
MessageBox.Show("Đã lưu thành công!");
```

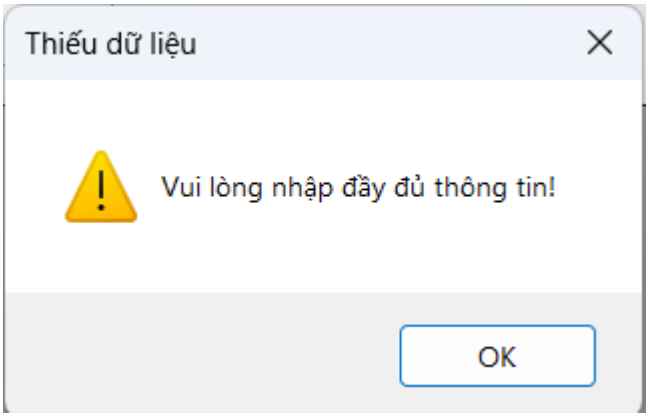


Cách gọi này hiển thị một hộp thoại chỉ gồm đoạn văn bản và một nút "OK" duy nhất để đóng lại.

Cú pháp đầy đủ với các tham số bổ sung

Trong phần lớn ứng dụng thực tế, cần kiểm soát thêm tiêu đề hộp thoại, icon minh họa mức độ thông báo, và bộ nút hiển thị:

```
MessageBox.Show(
    "Vui lòng nhập đầy đủ thông tin!",    // Nội dung thông báo
    "Thiếu dữ liệu",                       // Tiêu đề cửa sổ
    MessageBoxButtons.OK,                  // Bộ nút hiển thị
    MessageBoxIcon.Warning                 // Icon đi kèm
);
```



Các giá trị của MessageBoxButtons

- OK — Chỉ một nút "OK"
- OKCancel — "OK" và "Cancel"
- YesNo — "Yes" và "No"
- YesNoCancel — "Yes", "No", và "Cancel"
- RetryCancel — "Retry" và "Cancel"

Các giá trị của MessageBoxIcon

- Information — Biểu tượng thông tin
- Warning — Biểu tượng cảnh báo
- Error — Biểu tượng lỗi
- Question — Biểu tượng dấu hỏi, thường đi kèm YesNo khi cần hỏi xác nhận

Giá trị trả về — DialogResult

Một điểm quan trọng cần nắm: `MessageBox.Show()` **trả về một giá trị kiểu** `DialogResult`, cho biết chính xác người dùng đã bấm vào nút nào trong số các nút đã hiển thị. Nhờ giá trị trả về này, `MessageBox` không chỉ dùng để hiển thị thông tin, mà còn có thể dùng để **điều khiển luồng xử lý** tiếp theo trong code:

```
private void btnXoaDonCuoi_Click(object sender, EventArgs e)
{
    DialogResult ketQua = MessageBox.Show(
            "Bạn có chắc chắn muốn xóa đơn hàng này?",
            "Xác nhận xóa",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question
        );

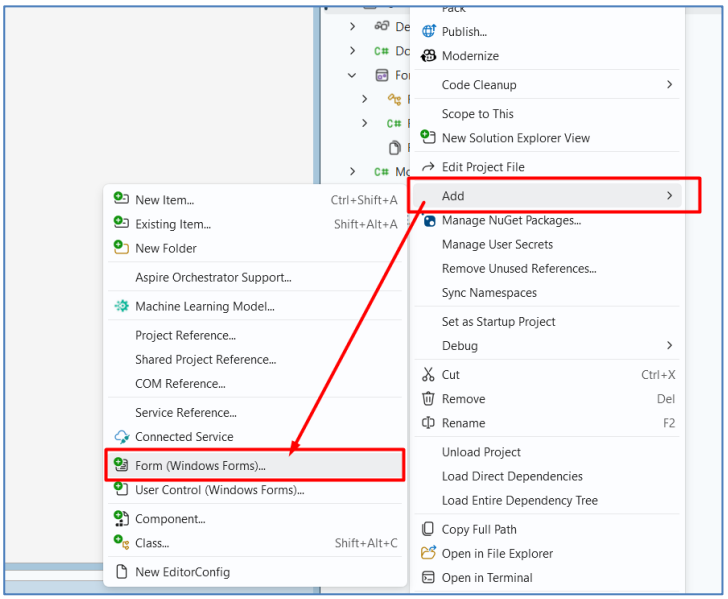
    if (ketQua == DialogResult.Yes)
    {
        if (danhSachDonHang.Count == 0)
        {
            MessageBox.Show("Chưa có đơn hàng nào để xóa!", "Thông báo",
                MessageBoxButtons.OK, MessageBoxIcon.Information);
            return;
        }
        danhSachDonHang.RemoveAt(danhSachDonHang.Count - 1);
        CapNhatBangDonHang();
        CapNhatTongDoanhThu();
    }
}
```

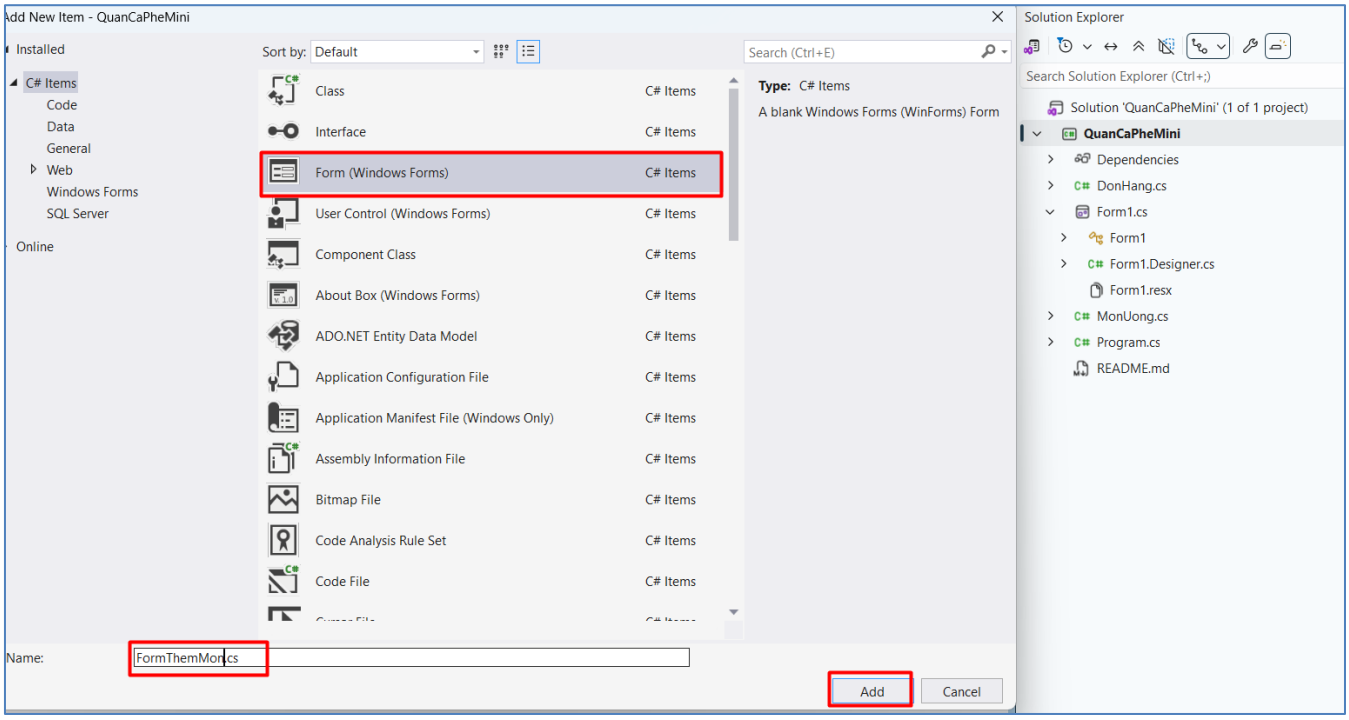
4.1.2 Tạo và mở Form mới

Ở các bài trước, ứng dụng chỉ sử dụng một Form duy nhất (Form1) nên phù hợp với các chương trình nhỏ. Tuy nhiên, khi ứng dụng có nhiều chức năng như thêm món, cập nhật dữ liệu hoặc xem báo cáo, việc đưa tất cả lên cùng một Form sẽ khiến giao diện rối và mã nguồn khó bảo trì.

Vì vậy, mỗi chức năng nên được tách thành một Form riêng, chẳng hạn Form bán hàng, Form quản lý món và Form báo cáo. Đây là cách tổ chức phổ biến giúp ứng dụng dễ phát triển, dễ bảo trì và mở rộng.

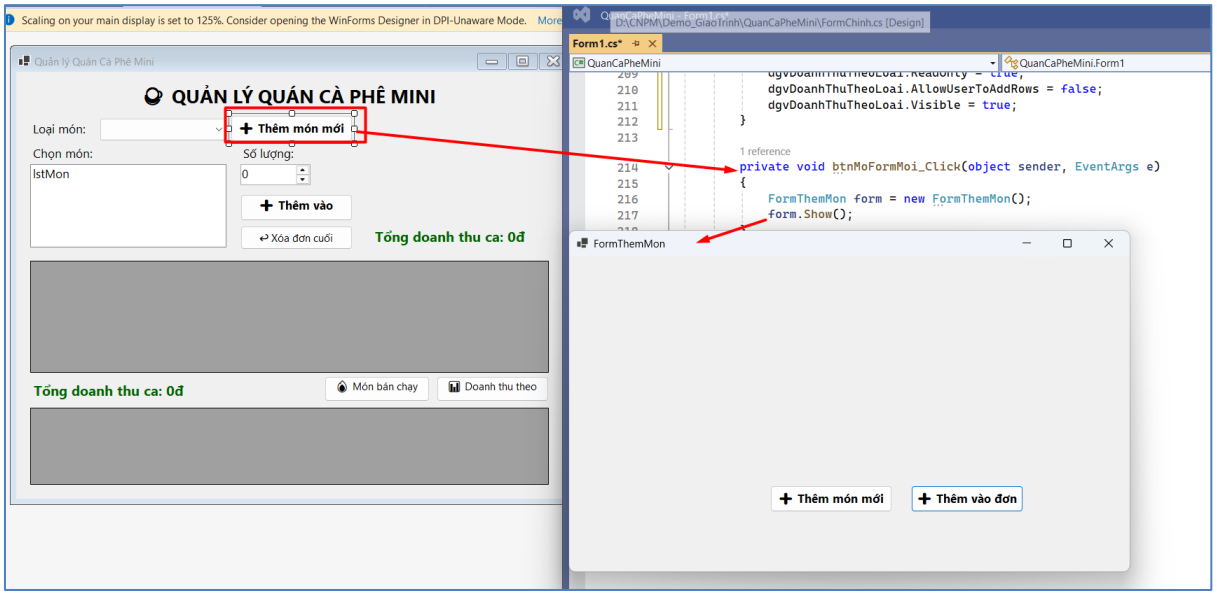
Để bắt đầu, cần thêm một Form mới vào project — việc này được thực hiện ngay trên Visual Studio: chuột phải vào tên project trong Solution Explorer, chọn **Add** → **Windows Form...**, đặt một tên có ý nghĩa (ví dụ `FormThemMon`), rồi nhấn **Add**.





Sau khi tạo, **Visual Studio** sẽ sinh một **Form mới** với file **Designer** riêng. Mỗi **Form** thực chất chỉ là **một class**, có constructor, property và method như các class đã học. Vì vậy, muốn hiển thị Form, trước hết phải **khởi tạo một object** của Form đó. Vì **Form là một class**, muốn mở một Form từ Form khác, trước tiên cần **khởi tạo object** của Form đó, sau đó gọi phương thức `Show()` hoặc `ShowDialog()` để hiển thị lên màn hình

- **Show**() mở Form mới theo kiểu không khóa (non-modal).



Có thể hình dung điều này giống như mở thêm một tab mới trong trình duyệt web — Form gốc (Form1) và Form mới mở ra đều cùng tồn tại, người dùng có thể tự do chuyển qua chuyển lại giữa hai cửa sổ này, không bị ràng buộc phải xử lý xong Form này rồi mới được động đến Form kia.

- **ShowDialog** - dùng nhiều hơn trong thực tế

```
private void btnMoFormMoi_Click(object sender, EventArgs e)
{
    FormThemMon form = new FormThemMon();
    DialogResult ketQua = form.ShowDialog();
}
```

ShowDialog() mở Form theo chế độ modal, nghĩa là khóa Form hiện tại cho đến khi Form mới được đóng. Cơ chế này phù hợp với các Form yêu cầu người dùng hoàn thành hoặc xác nhận một thao tác, như nhập dữ liệu hoặc xác nhận xóa.

Ngược lại, Show() mở Form không khóa (modeless), thích hợp cho các Form chỉ hiển thị thông tin hoặc hỗ trợ người dùng.

Ngoài ra, ShowDialog() còn cho phép nhận giá trị trả về (**DialogResult**), giúp Form gọi biết người dùng đã chọn Lưu, Hủy hay chỉ đóng cửa sổ, từ đó xử lý phù hợp.

```
// Trên Form con (FormThemMon), nút "Lưu"
1 reference
private void btnLuu_Click(object sender, EventArgs e)
{
    this.DialogResult = DialogResult.OK;
}
// Trên Form con, nút "Hủy"
1 reference
private void btnHuy_Click(object sender, EventArgs e)
{
    this.DialogResult = DialogResult.Cancel;
}
```

Khi gán this.DialogResult một giá trị khác None, Form sẽ tự động đóng mà không cần gọi Close(). Đây là cơ chế đặc biệt của DialogResult.

Sau khi ShowDialog() kết thúc, Form gọi có thể kiểm tra giá trị DialogResult để biết người dùng đã chọn OK, Cancel hoặc thao tác khác, từ đó thực hiện xử lý phù hợp.

```
private void btnMoFormMoi_Click(object sender, EventArgs e)
{
    FormThemMon form = new FormThemMon();
    if (form.ShowDialog() == DialogResult.OK)
    {
        lblThongBao.Text = "Đã lưu thành công!";
    }
    else
    {
        lblThongBao.Text = "Đã hủy.";
    }
}
```

Lệnh `form.ShowDialog()` chỉ **trả về khi Form con đã đóng**, nên câu lệnh phía sau mới được thực thi. Nhờ đó, Form gọi có thể **kiểm tra** `DialogResult` và tiếp tục **lấy dữ liệu** mà Form con đã nhập hoặc xử lý, chẳng hạn một đối tượng `MonUong` mới.

4.1.3 Truyền dữ liệu giữa các Form

Trong ứng dụng Windows Forms, các Form thường cần trao đổi dữ liệu với nhau. Một số cách thực hiện phổ biến là:

- Từ Form gốc đến Form con: truyền dữ liệu qua constructor có tham số hoặc property của Form con trước khi hiển thị.
- Từ Form con về Form gốc: lưu kết quả vào property công khai, sau đó Form gốc đọc dữ liệu sau khi ShowDialog() kết thúc.
- Trao đổi dữ liệu khi nhiều Form cùng mở: sử dụng event, delegate hoặc truyền tham chiếu giữa các Form.

Các cách trên chủ yếu phục vụ việc **trao đổi dữ liệu giữa các Form**. Trong nhiều trường hợp, dữ liệu còn cần được **liên kết trực tiếp với các điều khiển trên giao diện** để tự động hiển thị và cập nhật. Kỹ thuật này được gọi là **Data Binding**, sẽ được giới thiệu ở sau.

Ví dụ hướng dẫn — Truyền dữ liệu giữa hai Form

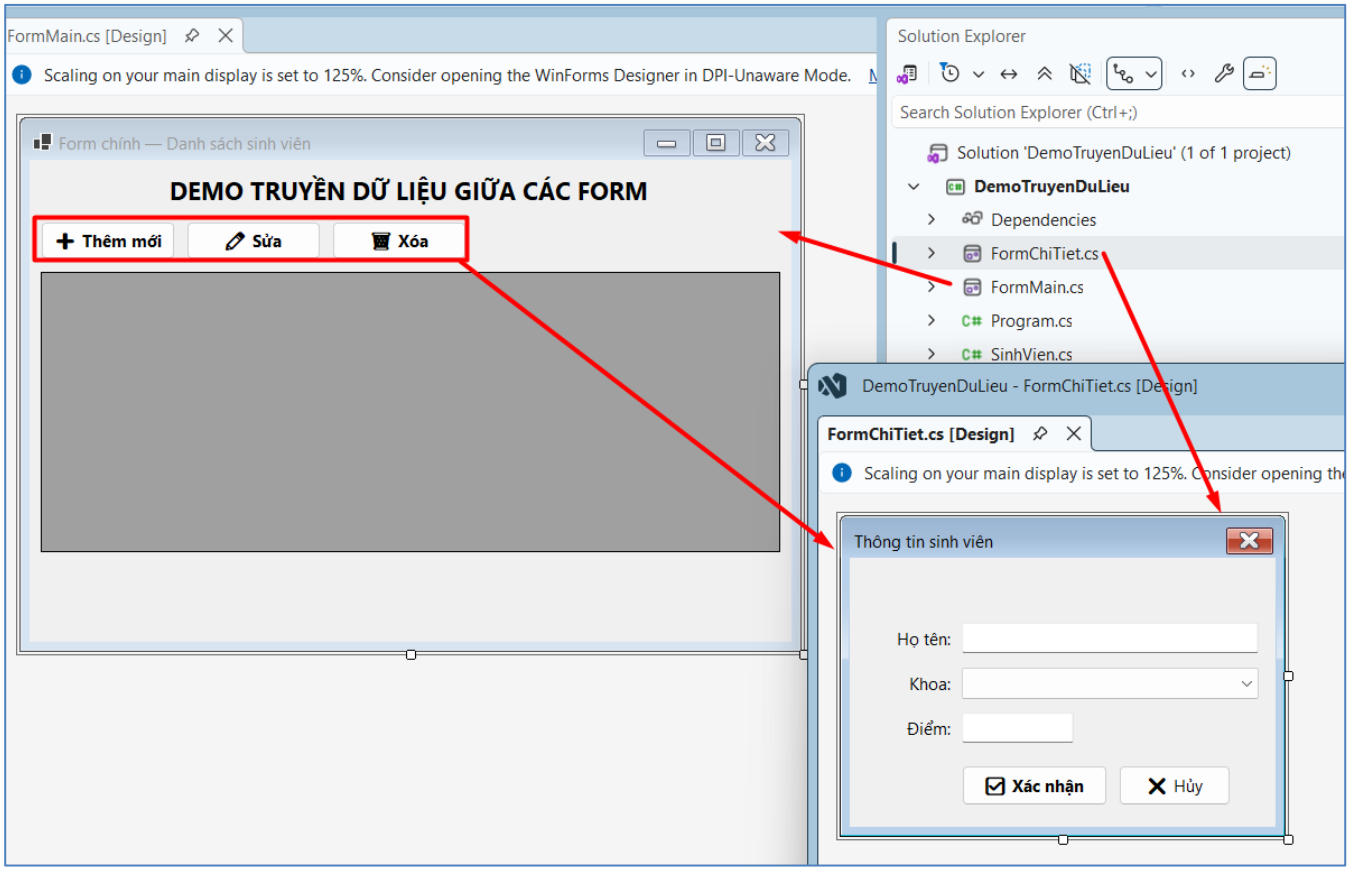
Xây dựng ứng dụng gồm hai Form:

FormMain — Form chính hiển thị danh sách sinh viên dạng bảng (DataGridView), có các nút Thêm mới, Sửa và Xóa

FormChiTiet — Form con dạng hộp thoại (ShowDialog) dùng để nhập hoặc chỉnh sửa thông tin một sinh viên

Khi bấm Thêm mới, FormChiTiet mở với các ô nhập liệu trống — người dùng nhập thông tin, bấm Xác nhận, dữ liệu được trả về FormMain và thêm vào danh sách.

Khi bấm Sửa, FormMain truyền thông tin sinh viên đang chọn sang FormChiTiet qua constructor — Form con tự điền sẵn dữ liệu vào các ô, người dùng chỉnh sửa, bấm Xác nhận, FormMain nhận kết quả và cập nhật lại danh sách.



Giải thích kịch bản thêm mới — Truyền dữ liệu giữa các Form

Bước 1: Người dùng bấm **Thêm mới** trên `FormMain`:

```
FormChiTiet formChiTiet = new FormChiTiet();  // Không truyền gì
formChiTiet.ShowDialog();       // Mở Form con, chờ đóng
```

Bước 2: Người dùng nhập thông tin trong `FormChiTiet`, bấm **Xác nhận**:

```
private void btnXacNhan_Click(object sender, EventArgs e)
```

```
{
      // Validate trước…

    // Lưu kết quả vào property – Form gốc sẽ đọc sau khi ShowDialog()
trả về
    KetQua = new SinhVien
    {
        HoTen = txtHoTen.Text.Trim(),
        Khoa  = cboKhoa.Text,
        Diem  = diem
    };

    DialogResult = DialogResult.OK;
    Close();
}
```

Bước 3: `ShowDialog()` trả về, `FormMain` đọc kết quả:

```
if (formChiTiet.ShowDialog() == DialogResult.OK)
{
    // Đọc kết quả từ property công khai của Form con
    SinhVien svMoi = formChiTiet.KetQua!;
    svMoi.Id = ++_idCounter;
    _danhSach.Add(svMoi);
    TaiDanhSach();
    lblTrangThai.Text = $"Đã thêm: {svMoi.HoTen}";
    lblTrangThai.ForeColor = Color.DarkGreen;

}
```

Giải thích kịch bản sửa— Truyền dữ liệu giữa các Form

FormMain	FormChiTiet

`// ===== SỬA: mở FormChiTiet với dữ liệu truyền qua constructor =====`
`private void btnSua_Click(object sender, EventArgs e)`
`{`
`SinhVien sv = (SinhVien)dgvSinhVien.CurrentRow.DataBoundItem;`
`// Truyền dữ liệu sang Form con qua constructor`
`FormChiTiet formChiTiet = new FormChiTiet(sv);`
 	
`if (formChiTiet.ShowDialog() == DialogResult.OK)`
`{`
`SinhVien ketQua = formChiTiet.KetQua!;`
`sv.HoTen = ketQua.HoTen;`
`sv.Khoa = ketQua.Khoa;`
`sv.Diem = ketQua.Diem;`
`TaiDanhSach();`
`}`
`}`
	`// ===== CONSTRUCTOR CÓ THAM SỐ: dùng khi Sửa – nhận dữ liệu từ Form gốc =====`
	`public FormChiTiet(SinhVien sv) : this()`
	`{`
	`// Điền dữ liệu nhận được vào các ô nhập liệu`
	`txtHoTen.Text = sv.HoTen;`
	`cboKhoa.Text = sv.Khoa;`
	`txtDiem.Text = sv.Diem.ToString();`
	
	`}`

4.2 MDI (MULTIPLE DOCUMENT INTERFACE)

4.2.1 Khái niệm MDI — Form cha (Container) và Form con

Thông thường, các Form mở bằng **Show**() hoặc **ShowDialog**() là những cửa sổ độc lập, có thể di chuyển tự do trên màn hình. Cách này phù hợp với các Form nhập liệu hoặc hộp thoại đơn giản.

Đối với các ứng dụng quản lý có nhiều chức năng hoạt động đồng thời, MDI (Multiple Document Interface) giúp tổ chức giao diện hiệu quả hơn. Trong mô hình này, Form cha (MDI Parent) đóng vai trò là cửa sổ chính, còn các Form con (MDI Child) chỉ hiển thị bên trong Form cha, tạo nên một không gian làm việc thống nhất.

MDI thường được sử dụng trong các phần mềm quản lý như kế toán, bán hàng, quản lý kho, nơi người dùng cần mở nhiều màn hình làm việc cùng lúc nhưng vẫn dễ theo dõi và quản lý.

So sánh nhanh giữa cách tổ chức Form thông thường và MDI:

Tiêu chí	**Form độc lập (`Show()` / `ShowDialog()`)**	**MDI**
Vị trí hiển thị	Tự do trên toàn màn hình	Giới hạn trong vùng Form cha
Số Form con mở cùng lúc	Không giới hạn, nhưng rời rạc	Nhiều Form con, được quản lý tập trung
Phù hợp khi	Form tạm thời (nhập liệu, xác nhận)	Nhiều khu vực làm việc tồn tại lâu dài trong cùng phiên sử dụng

4.2.2 Tạo MDI Parent và MDI Child

Biến một Form thành MDI Parent

Bất kỳ Form nào cũng có thể trở thành MDI Parent — Form chứa — chỉ bằng cách đặt thuộc tính `IsMdiContainer` thành `true`:

```
public partial class FormChinh : Form
{
    1 reference
    public FormChinh()
    {
        InitializeComponent();
        this.IsMdiContainer = true;
    }

}
```

Khi `IsMdiContainer = true`, vùng làm việc của Form sẽ đổi màu nền (thường tối hơn mặc định), đánh dấu đây là khu vực sẽ chứa các Form con — tương tự cách Word/Excel hiển thị vùng nền xám phía sau các cửa sổ tài liệu.

Mở một Form con bên trong MDI Parent

Để một Form trở thành MDI Child khi mở ra, cần gán thuộc tính `MdiParent` của nó, trỏ đến đúng Form cha, **trước khi** gọi `Show()`:

```
private void btnMoFormBanHang_Click(object sender, EventArgs e)
{
    FormBanHang formCon = new FormBanHang();
    formCon.MdiParent = this;   // "this" ở đây là Form cha đang gọi
    formCon.Show();
}
```

Một điểm quan trọng cần lưu ý: MDI Child **chỉ dùng được với `Show()`**, không dùng được với `ShowDialog()` — vì bản chất MDI Child cần tồn tại song song, không khóa tương tác với Form cha hay các Form con khác đang mở.

Sắp xếp các Form con

Khi có nhiều Form con cùng mở, .NET cung cấp sẵn phương thức `LayoutMdi()` để tự động sắp xếp chúng theo các kiểu bố cục có sẵn:

```
private void btnSapXepDoc_Click(object sender, EventArgs e)
{
    this.LayoutMdi(MdiLayout.TileVertical);   // Xếp các Form con theo chiều dọc
}

1 reference
private void btnSapXepNgang_Click(object sender, EventArgs e)
{
    this.LayoutMdi(MdiLayout.TileHorizontal);   // Xếp theo chiều ngang
}

1 reference
private void btnSapXepTangBac_Click(object sender, EventArgs e)
{
    this.LayoutMdi(MdiLayout.Cascade);   // Xếp chồng theo kiểu so le (tầng bậc)
}
```

Menu "Window" — tổng hợp các Form con đang mở

Một tính năng kinh điển của ứng dụng MDI là menu "Window", tự động liệt kê toàn bộ Form con đang mở, cho phép click để chuyển nhanh đến từng Form. .NET hỗ trợ tính năng này gần như hoàn toàn tự động, chỉ cần đặt đúng thuộc tính MdiWindowListItem trên MenuStrip, trỏ đến mục menu mong muốn đóng vai trò danh sách đó — cấu hình trực tiếp qua Designer (chọn MenuStrip → thuộc tính MdiWindowListItem → chọn đúng menu item).

4.3 Các Control hỗ trợ giao diện ứng dụng

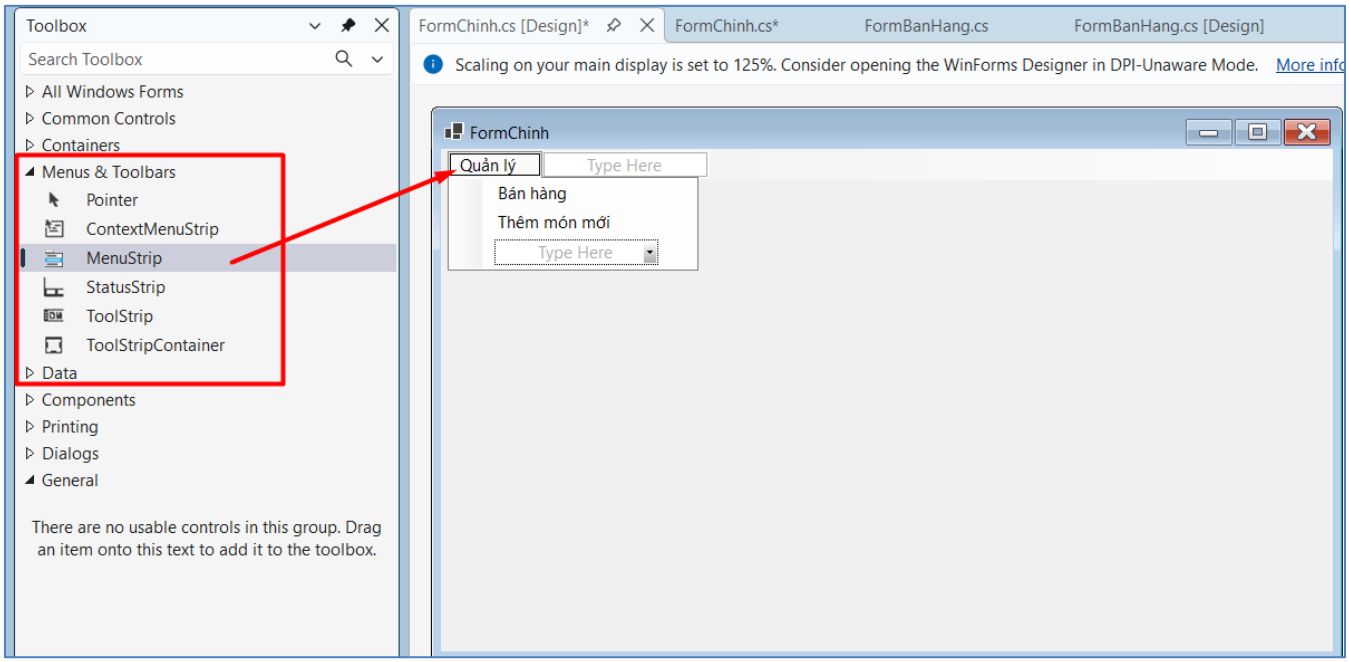
Một ứng dụng MDI hoàn chỉnh thường không chỉ có Form cha và Form con — mà còn cần thêm các Control hỗ trợ điều hướng và tương tác ở cấp độ toàn ứng dụng: thanh menu, thanh công cụ nhanh, thanh trạng thái, và các hộp thoại chọn/lưu file.

4.3.1 MenuStrip

Thêm MenuStrip vào Form và tạo các mục menu

Từ Toolbox, nhóm Menus & Toolbars, kéo MenuStrip thả vào Form — control này sẽ tự động "dính" lên cạnh trên cùng của Form, hiển thị một thanh trống với dòng chữ mờ "Type Here".Click trực tiếp vào dòng "Type Here" đó và gõ tên menu cấp 1 (ví dụ "Quản lý"), nhấn Enter để xác nhận. Ngay sau đó, một ô "Type Here" mới sẽ xuất

hiện bên dưới mục vừa gõ — đây là nơi để gõ tiếp các mục con thuộc menu "Quản lý" (ví dụ "Bán hàng", "Thêm món mới"). Cùng lúc, một ô "Type Here" khác xuất hiện bên phải mục "Quản lý" — đây là nơi để gõ tiếp menu cấp 1 tiếp theo (ví dụ "Báo cáo", "Thoát"). Cứ tiếp tục theo cách đó, gõ lần lượt từng mục, nhấn Tab/Enter để di chuyển, đến khi có đủ cấu trúc menu mong muốn:



Đổi tên (Name) cho từng mục menu

Mỗi mục menu vừa tạo, dù đã có Text hiển thị, vẫn cần đổi lại thuộc tính Name (qua cửa sổ Properties — F4, giống cách đặt tên cho Button/TextBox đã làm) thành tên có ý nghĩa, dùng trong code. Quy ước thường gặp: tiền tố mnu + tên mô tả, ví dụ mnuBanHang, mnuThemMonMoi, mnuThoat.

Lưu ý: nếu không đổi Name, Visual Studio sẽ tự đặt tên mặc định kiểu bánHàngToolStripMenuItem — vẫn dùng được, nhưng dài và khó nhớ khi viết code sau này.

Gắn sự kiện Click cho mục menu

Giống hoàn toàn cách làm với Button: double-click trực tiếp vào mục menu trên Designer, Visual Studio sẽ tự tạo sẵn method xử lý sự kiện `Click` tương ứng trong `Form1.cs`, chỉ cần viết code xử lý vào đó:

```
private void mnuBanHang_Click(object sender, EventArgs e)
{
    FormBanHang formCon = new FormBanHang();
    formCon.MdiParent = this;
    formCon.Show();
}
```

Thêm phím tắt (Shortcut Keys)

Sau khi đã có mục menu, chọn mục đó trên Designer, mở Properties, tìm thuộc tính `ShortcutKeys` — một cửa sổ nhỏ hiện ra cho phép tick chọn tổ hợp phím mong muốn (Ctrl, Shift, Alt kết hợp với một phím khác). Hoặc gán trực tiếp bằng code, ví dụ trong constructor của Form:

```
mnuBanHang.ShortcutKeys = Keys.Control | Keys.B; // Ctrl+B
```

Ngay sau khi gán, mục menu đó sẽ tự động hiển thị thêm dòng chữ "Ctrl+B" ở cạnh phải của nó trên giao diện — không cần làm thêm gì khác để thông báo cho người dùng biết về phím tắt này.

Thêm dấu phân cách (Separator)

Khi số lượng mục trong một menu ngày càng nhiều, việc nhóm các chức năng có liên quan lại với nhau bằng một đường kẻ ngang sẽ giúp người dùng định hướng nhanh hơn. Để thêm, chuột phải vào ngay vị trí muốn chèn trong menu (trên Designer), chọn **Separator** từ danh sách hiện ra — không cần viết bất kỳ dòng code nào. Một cách sắp xếp hợp lý, chẳng hạn, là tách riêng nhóm các chức năng nghiệp vụ (Bán hàng, Thêm món mới) ra khỏi nhóm các chức năng mang tính hệ thống (Thoát ứng dụng), bằng một đường kẻ ở giữa hai nhóm đó:

Menu dạng tick chọn (Checked)

Một dạng mục menu đặc biệt khác hoạt động giống một CheckBox, chỉ là được trình bày dưới hình thức một mục trong menu. Để có được hành vi này, chọn mục menu đó trên Designer, đặt thuộc tính `CheckOnClick` thành `true`. Từ đó, mỗi lần người dùng click vào, trạng thái `Checked` của nó sẽ tự động đảo ngược (kèm theo một dấu tick nhỏ hiện/ẩn ngay trên giao diện), và có thể đọc lại giá trị đó trong code để quyết định hành động tương ứng:

```
private void mnuHienThiStatusBar_Click(object sender, EventArgs e)
{
    statusStrip1.Visible = mnuHienThiStatusBar.Checked;
}
```

Mẫu hình này rất phù hợp cho những tùy chọn mang tính "bật/tắt" của giao diện — chẳng hạn cho phép người dùng tự quyết định có muốn hiển thị thanh trạng thái hay không, tùy theo sở thích cá nhân trong quá trình sử dụng ứng dụng.

4.3.2 ContextMenuStrip — menu chuột phải

Bên cạnh `MenuStrip` (luôn hiển thị cố định trên cùng Form), .NET còn cung cấp **`ContextMenuStrip`** — một menu chỉ xuất hiện khi người dùng **click chuột phải** vào đúng vị trí đã gán cho nó. Đây là kiểu menu rất quen thuộc khi dùng Windows: chuột phải vào một file để hiện "Copy, Delete, Rename"; áp dụng tương tự, chuột phải vào một dòng trên **`DataGridView`** để hiện nhanh các hành động "Sửa", "Xóa" cho đúng dòng đó.

Tạo ContextMenuStrip

Cách tạo mục menu hoàn toàn giống MenuStrip— kéo ContextMenuStrip từ Toolbox vào Form. Khác với MenuStrip (dính lên cạnh trên Form), ContextMenuStrip sẽ nằm ở khu vực "tray" phía dưới Designer, không hiển thị trực tiếp trên Form, vì bản chất là menu ẩn, chỉ bật ra khi cần.

Click vào nó trong khu vực tray để mở chế độ chỉnh sửa, rồi gõ các mục menu, đổi Name, gắn sự kiện Click — đúng quy trình đã làm với MenuStrip:

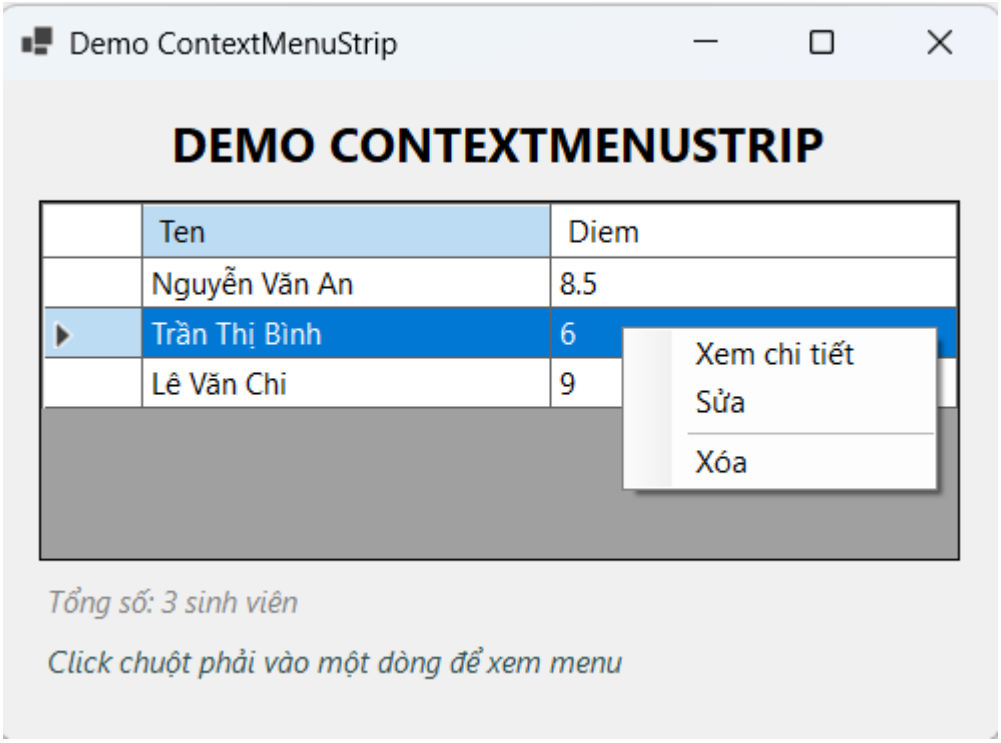
Gắn ContextMenuStrip vào một Control cụ thể

Để menu chuột phải này thực sự hoạt động, cần gắn nó vào đúng Control mong muốn (ví dụ `DataGridView`) — chọn Control đó trên Designer, vào Properties, tìm thuộc tính `ContextMenuStrip`, chọn đúng tên đã tạo. Hoặc gán bằng code:

```
dgvDonHang.ContextMenuStrip = contextMenuStrip1;
```

Từ đây, mỗi khi người dùng chuột phải vào dgvDonHang, menu vừa tạo sẽ bật ra ngay tại vị trí con trỏ.

Ví dụ thực tế — xóa một dòng trên DataGridView qua menu chuột phải




```
namespace DemoContextMenu
{
    public class SinhVien
    {
        public string Ten { get; set; }
        public double Diem { get; set; }
    }

    public partial class Form1 : Form
    {
        private List<SinhVien> danhSachSinhVien;

        public Form1()
        {
            InitializeComponent();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            danhSachSinhVien = new List<SinhVien>
            {
                new SinhVien { Ten = "Nguyễn Văn An", Diem = 8.5 },
                new SinhVien { Ten = "Trần Thị Bình", Diem = 6.0 },
                new SinhVien { Ten = "Lê Văn Chi", Diem = 9.0 },
            };

            dgvSinhVien.ReadOnly = true;
            dgvSinhVien.AllowUserToAddRows = false;
            dgvSinhVien.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
```

```
            dgvSinhVien.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            dgvSinhVien.MultiSelect = false;

            // Gắn ContextMenuStrip vào DataGridView bằng code
            dgvSinhVien.ContextMenuStrip = contextMenuStrip1;

            CapNhatBang();
        }
        private void CapNhatBang()
        {
            dgvSinhVien.DataSource = null;
            dgvSinhVien.DataSource = danhSachSinhVien;
            lblTongSo.Text = $"Tổng số: {danhSachSinhVien.Count} sinh viên";
        }

        // Trước khi mở menu - chọn đúng dòng dưới con trỏ chuột khi click phải
        private void dgvSinhVien_MouseDown(object sender, MouseEventArgs e)
        {
            if (e.Button == MouseButtons.Right)
            {
                var hit = dgvSinhVien.HitTest(e.X, e.Y);
                if (hit.RowIndex >= 0)
                {
                    dgvSinhVien.ClearSelection();
                    dgvSinhVien.Rows[hit.RowIndex].Selected = true;
                    dgvSinhVien.CurrentCell = dgvSinhVien.Rows[hit.RowIndex].Cells[0];
                }
            }
        }
        // Menu chuột phải - Sửa
        private void mnuSua_Click(object sender, EventArgs e)
        {
            if (dgvSinhVien.CurrentRow?.DataBoundItem is SinhVien sv)
            {
                using (var formNhap = new Form())
                {
                    formNhap.Text = "Sửa tên";
                    formNhap.Width = 300;
                    formNhap.Height = 130;
                    formNhap.StartPosition = FormStartPosition.CenterParent;

                    var txt = new TextBox { Text = sv.Ten, Left = 15, Top = 15, Width =
250 };
                    var btnOk = new Button { Text = "OK", Left = 95, Top = 50, Width = 80,
DialogResult = DialogResult.OK };

                    formNhap.Controls.Add(txt);
                    formNhap.Controls.Add(btnOk);
                    formNhap.AcceptButton = btnOk;

                    if (formNhap.ShowDialog() == DialogResult.OK &&
!string.IsNullOrWhiteSpace(txt.Text))
                    {
                        sv.Ten = txt.Text;
                        CapNhatBang();
                    }
                }
            }
        }

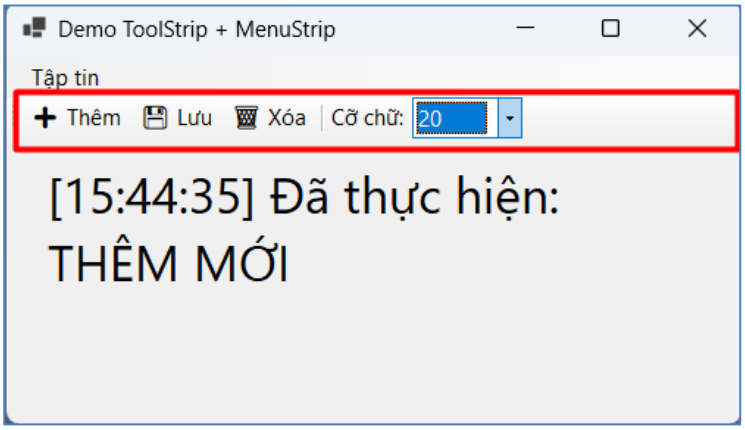
        // Menu chuột phải - Xóa
        private void mnuXoa_Click(object sender, EventArgs e)
```

```
        {
            if (dgvSinhVien.CurrentRow?.DataBoundItem is SinhVien sv)
            {
                DialogResult ketQua = MessageBox.Show(
                    $"Bạn có chắc muốn xóa sinh viên \"{sv.Ten}\"?",
                    "Xác nhận xóa",
                    MessageBoxButtons.YesNo,
                    MessageBoxIcon.Question
                );

                if (ketQua == DialogResult.Yes)
                {
                    danhSachSinhVien.Remove(sv);
                    CapNhatBang();
                }
            }
        }
        // Menu chuột phải - Xem chi tiết
        private void mnuXemChiTiet_Click(object sender, EventArgs e)
        {
            if (dgvSinhVien.CurrentRow?.DataBoundItem is SinhVien sv)
            {
                MessageBox.Show(
                    $"Họ tên: {sv.Ten}\nĐiểm: {sv.Diem}",
                    "Chi tiết sinh viên",
                    MessageBoxButtons.OK,
                    MessageBoxIcon.Information
                );
            }
        }
    }
```

4.3.3 ToolStrip

ToolStrip là một thanh chứa các nút bấm dạng icon, thường đặt ngay dưới `MenuStrip`, cho phép truy cập nhanh đến những chức năng dùng thường xuyên nhất mà không cần mở menu — tương tự thanh công cụ "New, Open, Save, Print" quen thuộc trong Word/Excel.



Thêm ToolStrip và các nút bấm

Từ Toolbox, nhóm Menus & Toolbars, kéo ToolStrip thả vào Form — control này tự động "dính" lên cạnh trên Form, thường nằm ngay dưới MenuStrip nếu Form đã có sẵn menu.

Trên ToolStrip vừa thả vào, click vào nút mũi tên nhỏ thả xuống để thêm các phần tử — phổ biến nhất là ToolStripButton (nút bấm dạng icon). Mỗi ToolStripButton thêm vào nên đổi Name (quy ước tiền tố tsb, ví dụ tsbThemMonMoi), và gán Image (icon) qua thuộc tính Image trong Properties — chọn ảnh từ máy hoặc dùng bộ icon có sẵn của Visual Studio.

Thuộc tính **DisplayStyle** quyết định ToolStripButton hiển thị icon, chữ, hay cả hai:

Gắn sự kiện cho ToolStripButton

Giống các Control đã học, double-click trực tiếp vào `ToolStripButton` trên Designer để Visual Studio tự tạo sẵn method xử lý sự kiện `Click`:

```
private void tsbThemMonMoi_Click(object sender, EventArgs e)
{
    MoFormConDuyNhat<FormThemMon>();
}
```

Một thực hành quan trọng cần lưu ý: nếu chức năng "Thêm món mới" đã có sẵn trên `MenuStrip`, thì nút `ToolStripButton` tương ứng trên `ToolStrip` nên **gọi lại đúng cùng một method** đã viết cho menu đó — tránh viết trùng logic chỉ vì có hai cách kích hoạt khác nhau (qua menu hoặc qua nút công cụ) cho cùng một chức năng nghiệp vụ. Đây cũng là lý do nên tổ chức các xử lý nghiệp vụ thành những method riêng (như `MoFormConDuyNhat<T>()`), để nhiều nơi khác nhau trên giao diện đều có thể gọi đến chung một nơi.

Một số phần tử khác trên ToolStrip

Ngoài ToolStripButton, ToolStrip còn hỗ trợ thêm các phần tử khác, tùy nhu cầu:

- ToolStripSeparator — đường kẻ phân cách dọc, dùng để nhóm các nút có liên quan, tương tự Separator đã học ở MenuStrip

- ToolStripLabel — hiển thị một đoạn chữ tĩnh ngay trên thanh công cụ
- ToolStripComboBox — một ComboBox thu nhỏ, đặt trực tiếp trên thanh công cụ (ví dụ: chọn nhanh bộ lọc, không cần mở thêm Form riêng)

Ví dụ tổng hợp

Ba `ToolStripButton` trên đều gọi lại cùng method **`MoFormConDuyNhat<T>()`** đã viết khi học MDI — cho thấy `ToolStrip`, dù là một Control hoàn toàn mới, không đòi hỏi viết thêm logic nghiệp vụ nào khác; nó chỉ đơn giản là một **lối tắt bổ sung** dẫn đến đúng những chức năng đã có sẵn từ menu, giúp người dùng thao tác nhanh hơn trong quá trình sử dụng ứng dụng hàng ngày.

```
private void tsbBanHang_Click(object sender, EventArgs e)
{
    MoFormConDuyNhat<FormBanHang>();
}
private void tsbThemMonMoi_Click(object sender, EventArgs e)
{
 MoFormConDuyNhat<FormThemMon>();
}
private void tsbBaoCao_Click(object sender, EventArgs e)
{
MoFormConDuyNhat<FormBaoCao>();
}
```

4.3.4 StatusStrip

StatusStrip là một Control luôn nằm cố định ở cạnh **dưới cùng** của Form, dùng để hiển thị các thông tin trạng thái ngắn gọn trong suốt quá trình sử dụng ứng dụng — ví dụ: tên người dùng đang đăng nhập, thời gian hiện tại, số dòng đang được chọn trên một bảng dữ liệu, hay trạng thái xử lý gần nhất ("Đã lưu", "Đang tải dữ liệu"...). Đây là vị trí quen thuộc mà người dùng Windows thường nhìn vào để nắm bắt nhanh

tình trạng hiện tại của ứng dụng, mà không cần phải tìm kiếm thông tin đó ở giữa màn hình.

Thêm StatusStrip vào Form

Từ Toolbox, nhóm Menus & Toolbars, kéo StatusStrip thả vào Form — control này sẽ tự động "dính" xuống cạnh dưới cùng của Form, bất kể được thả vào vị trí nào trên Designer.Tương tự ToolStrip, click vào nút mũi tên nhỏ thả xuống trên StatusStrip để thêm các phần tử — phổ biến nhất là ToolStripStatusLabel, dùng để hiển thị một đoạn văn bản trạng thái. Nên đổi Name cho mỗi label theo nội dung nó đảm nhiệm (ví dụ tsslNguoiDung, tsslThoiGian), để dễ phân biệt khi viết code.

4.3.5 OpenFileDialog, SaveFileDialog

Trong các ứng dụng Windows, người dùng thường có nhu cầu mở tệp dữ liệu để đọc hoặc lưu dữ liệu ra tệp. OpenFileDialog và SaveFileDialog là hai hộp thoại chuẩn của Windows Forms, giúp người dùng lựa chọn tệp một cách trực quan mà không cần nhập đường dẫn thủ công.

- OpenFileDialog: Hiển thị hộp thoại để chọn một tệp đã tồn tại trên máy tính.
- SaveFileDialog: Hiển thị hộp thoại để người dùng chọn vị trí và tên tệp cần lưu.

Việc sử dụng các hộp thoại này giúp ứng dụng thân thiện hơn, giảm sai sót khi nhập đường dẫn và tuân theo giao diện chuẩn.

OpenFileDialog

Lớp `OpenFileDialog` thuộc không gian tên: using System.Windows.Forms;

Một số thuộc tính thường dùng

Ví dụ: Chọn một tệp văn bản

```
private void btnOpen_Click(object sender, EventArgs e)
{
    OpenFileDialog openFileDialog = new OpenFileDialog();

    openFileDialog.Title = "Chọn tệp văn bản";
    openFileDialog.Filter = "Text Files (*.txt)|*.txt|All Files (*.*)|*.*";

    if (openFileDialog.ShowDialog() == DialogResult.OK)
    {
```

```
            txtFilePath.Text = openFileDialog.FileName;
        }
    }
```

Giải thích:

- ShowDialog() hiển thị hộp thoại.

- Nếu người dùng nhấn Open, phương thức trả về DialogResult.OK. Đường dẫn của tệp được lấy thông qua thuộc tính FileName.

SaveFileDialog

`SaveFileDialog` được sử dụng khi người dùng muốn lưu dữ liệu ra tệp.

Một số thuộc tính thường dùng

Ví dụ: Lưu nội dung TextBox xuống tệp

```
private void btnSave_Click(object sender, EventArgs e)
{
    SaveFileDialog saveFileDialog = new SaveFileDialog();

    saveFileDialog.Title = "Lưu tệp";
    saveFileDialog.Filter = "Text Files (*.txt)|*.txt";
    saveFileDialog.DefaultExt = "txt";
    saveFileDialog.FileName = "Document";

    if (saveFileDialog.ShowDialog() == DialogResult.OK)
    {
        File.WriteAllText(saveFileDialog.FileName,
txtContent.Text);

        MessageBox.Show("Lưu tệp thành công!");
    }
}
```

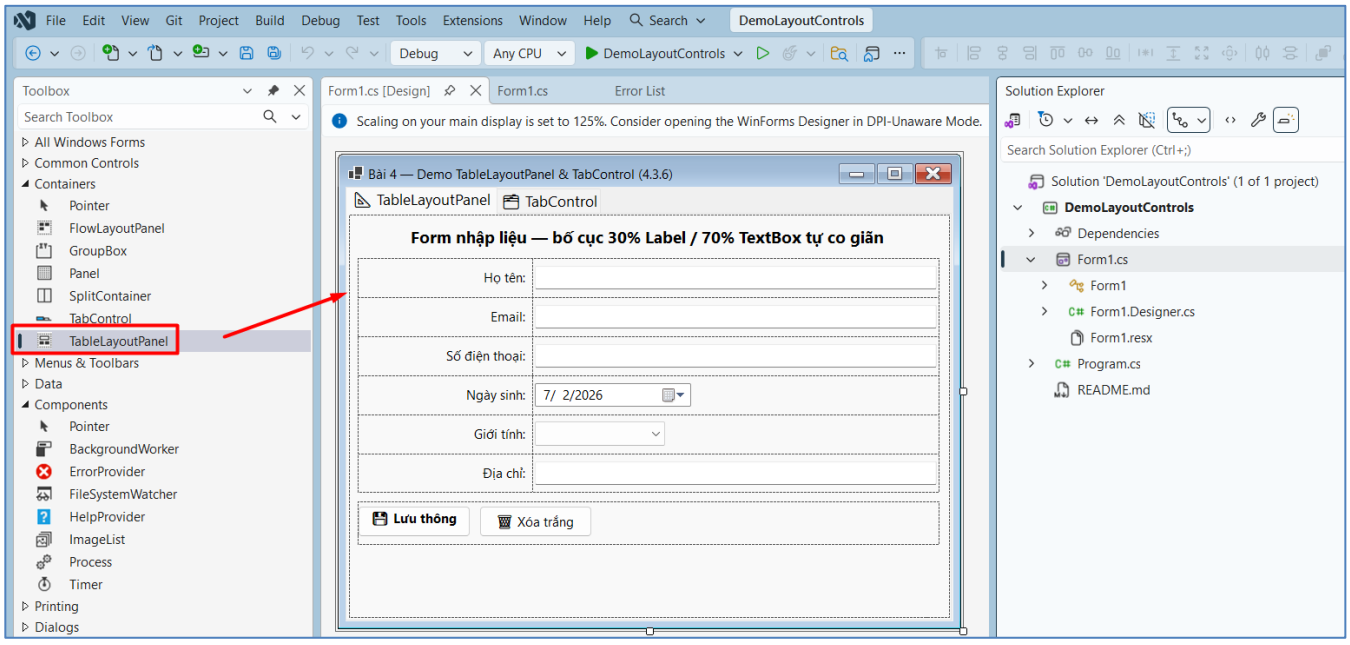
Lưu ý

- Nên sử dụng Filter để giới hạn loại tệp nhằm tránh người dùng chọn nhầm định dạng.
- Khi đọc hoặc ghi tệp, nên xử lý ngoại lệ (try...catch) để tránh lỗi như không có quyền truy cập hoặc tệp đang được sử dụng bởi chương trình khác.

4.3.6 TableLayoutPanel và TabControl

4.3.6.1 TableLayoutPanel — Bố cục dạng lưới

TableLayoutPanel là một Control dùng để sắp xếp các Control con theo dạng lưới hàng/cột, mỗi ô trong lưới chứa đúng một Control. Khác với cách đặt Control tự do bằng tọa độ cố định, TableLayoutPanel tự động co giãn và căn chỉnh lại các Control bên trong khi người dùng thay đổi kích thước Form — giúp giao diện trông gọn gàng và chuyên nghiệp hơn trên nhiều độ phân giải màn hình khác nhau.



- **Thêm TableLayoutPanel vào Form**

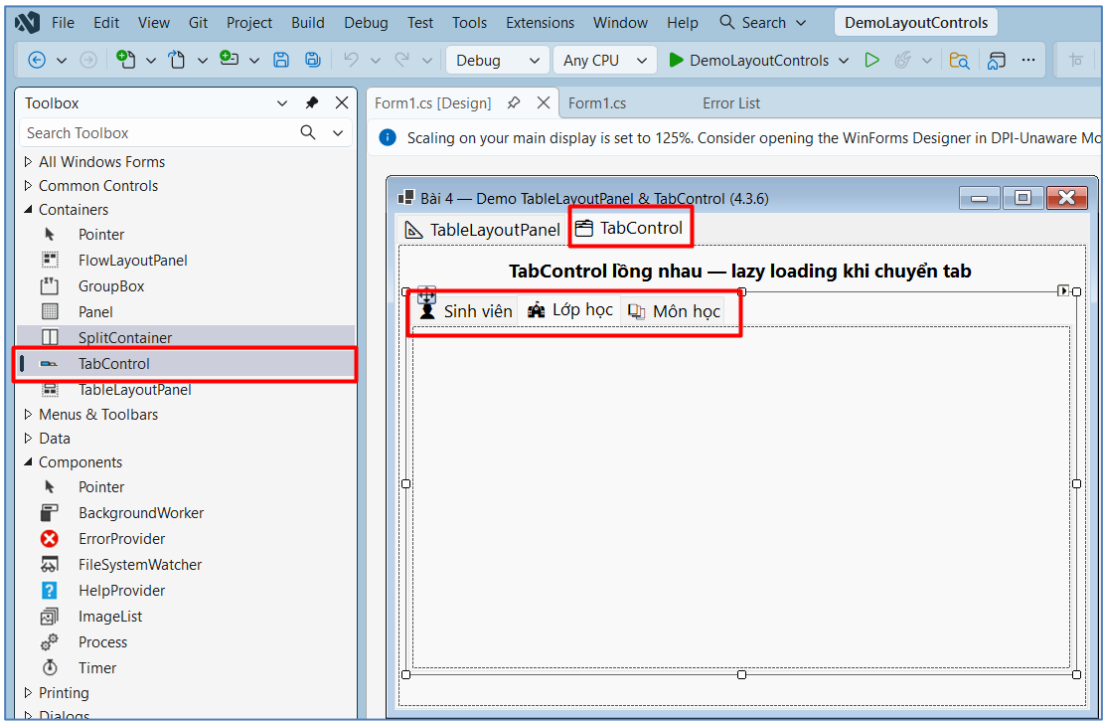
Từ Toolbox, nhóm Containers, kéo TableLayoutPanel vào Form. Mặc định, control này tạo ra một lưới 2 cột × 2 hàng. Để điều chỉnh số hàng/cột, chuột phải vào TableLayoutPanel → Edit Rows and Columns — một cửa sổ nhỏ hiện ra cho phép thêm/xóa hàng và cột, đặt kích thước theo pixel cố định, tỉ lệ phần trăm, hoặc để tự co giãn (AutoSize). Sau khi đã có lưới, kéo các Control khác (Label, TextBox, Button...) thả vào từng ô trong lưới — mỗi ô tự động căn chỉnh Control bên trong theo kích thước ô đó.

- **Các chế độ kích thước cột/hàng**

4.3.6.2 TabControl — Tổ chức giao diện nhiều tab

TabControl là Control cho phép tổ chức giao diện thành nhiều tab (thanh tab), mỗi tab chứa một nhóm nội dung riêng — giống giao diện nhiều tab của trình duyệt web, hoặc hộp thoại Options trong Visual Studio. Đây là cách tổ chức hiệu quả khi một

Form cần hiển thị nhiều nhóm chức năng khác nhau nhưng không muốn mở thêm Form con.



Thêm TabControl vào Form

Từ Toolbox, nhóm Containers, kéo TabControl vào Form. Mặc định, TabControl tạo sẵn hai tab (tabPage1, tabPage2). Để thêm tab, chuột phải vào TabControl → Add Tab; để xóa, chuột phải vào tab cần xóa → Remove Tab.

Mỗi tab thực chất là một TabPage — một container hoàn chỉnh, có thể kéo thả bất kỳ Control nào vào bên trong, hoàn toàn độc lập với các tab khác.

Đặt tên và tiêu đề cho từng tab

Chọn từng TabPage trong Designer, vào Properties để chỉnh:

- Name — tên dùng trong code (ví dụ tabSinhVien, tabLopHoc)
- Text — tiêu đề hiển thị trên thanh tab (ví dụ "Sinh viên", "Lớp học")

Chuyển tab bằng code

```
// Chuyển đến tab theo chỉ số (0 = tab đầu tiên)
tabControl1.SelectedIndex = 1;

// Chuyển đến tab theo tên TabPage
tabControl1.SelectedTab = tabSinhVien;
```

Xử lý sự kiện khi người dùng chuyển tab

```
private void tabControl1_SelectedIndexChanged(object sender, EventArgs e)
 {
     if (tabControl1.SelectedTab == tabSinhVien)
     {
         // Tải dữ liệu sinh viên khi người dùng chuyển sang tab này
         TaiDanhSachSinhVien();
     }
 }
```

Sự kiện `SelectedIndexChanged` kích hoạt mỗi khi người dùng bấm sang tab khác — đây là nơi phù hợp để tải dữ liệu lười (lazy loading): chỉ tải dữ liệu cho tab nào khi người dùng thực sự chuyển sang tab đó, thay vì tải toàn bộ ngay khi Form mở.

TÓM TẮT

Trong bài học này, người học đã biết cách xây dựng ứng dụng Windows Forms nhiều Form, sử dụng MessageBox, mở Form bằng Show() và ShowDialog(), đồng thời truyền dữ liệu giữa các Form để tổ chức chương trình rõ ràng và dễ mở rộng.

Bài học cũng giới thiệu mô hình MDI (Multiple Document Interface), giúp quản lý nhiều Form con trong một Form cha, phù hợp với các ứng dụng quản lý có nhiều màn hình làm việc.

Ngoài ra, người học đã làm quen với các thành phần giao diện như MenuStrip, ContextMenuStrip, ToolStrip, StatusStrip, cùng các hộp thoại OpenFileDialog và SaveFileDialog, làm nền tảng để xây dựng các ứng dụng Windows Forms hoàn chỉnh và chuyên nghiệp hơn.

CÂU HỎI ÔN TẬP

Câu 1: Trình bày cách tạo và hiển thị một Form mới trong Windows Forms. Phân biệt phương thức Show() và ShowDialog().

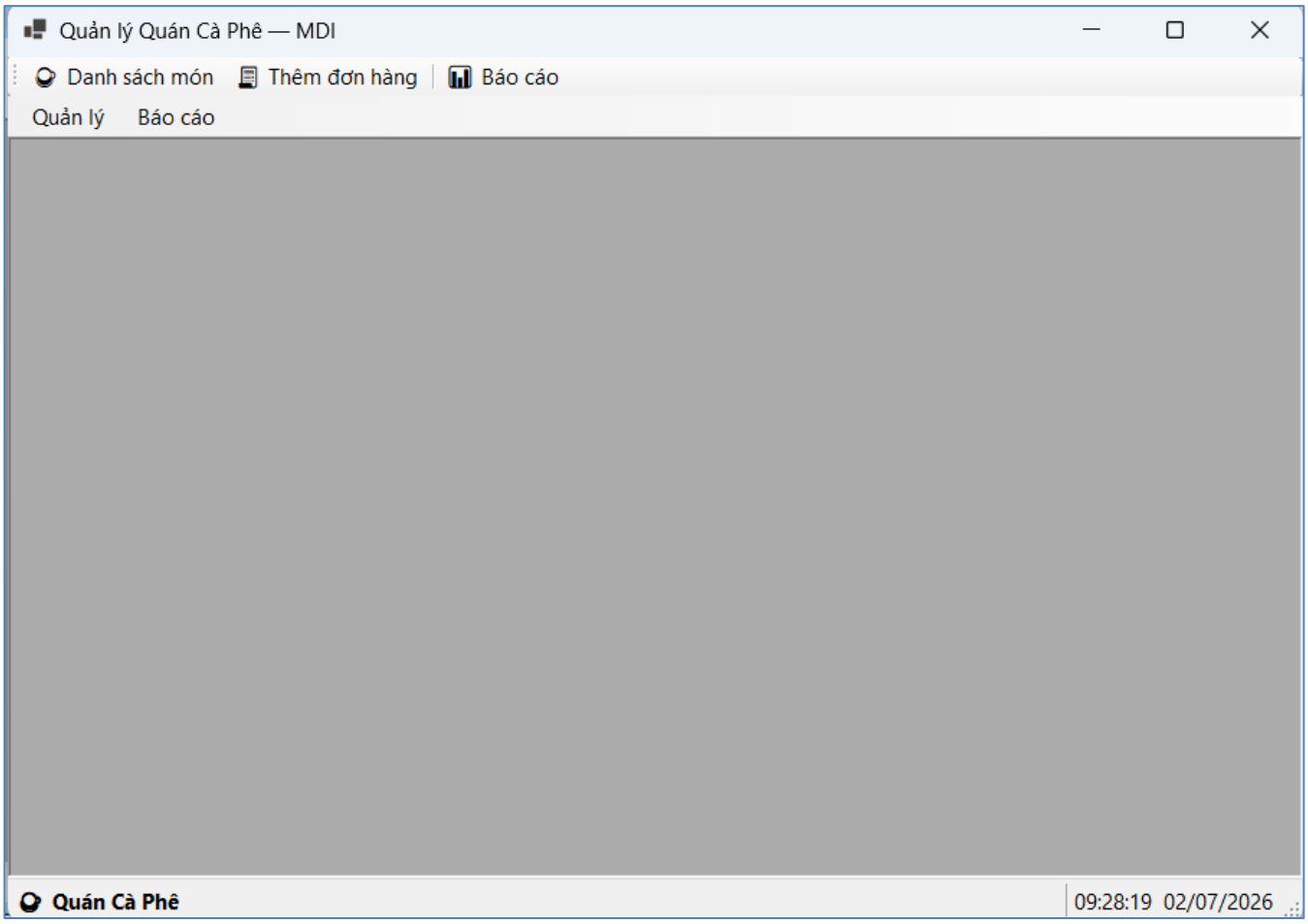
Câu 2: Phân biệt Form chính (Main Form) và Form phụ (Child Form). Trong trường hợp nào nên sử dụng nhiều Form trong một ứng dụng?

Câu 3: Trình bày các cách truyền dữ liệu giữa hai Form trong Windows Forms.

Câu 4: MDI (Multiple Document Interface) là gì? Phân biệt MDI Parent và MDI Child.

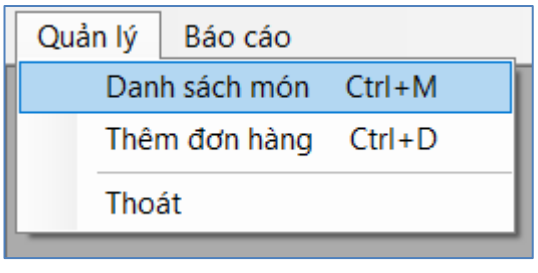
Câu 5: Thiết kế một ứng dụng có **MenuStrip** gồm các menu **Hệ thống**, **Quản lý** và **Trợ giúp**. Khi người dùng chọn **Thoát**, chương trình hiển thị hộp thoại xác nhận trước khi đóng ứng dụng.

Câu 6: Xây dựng ứng dụng WinForms theo mô hình **MDI (Multiple Document Interface)** để quản lý quán cà phê



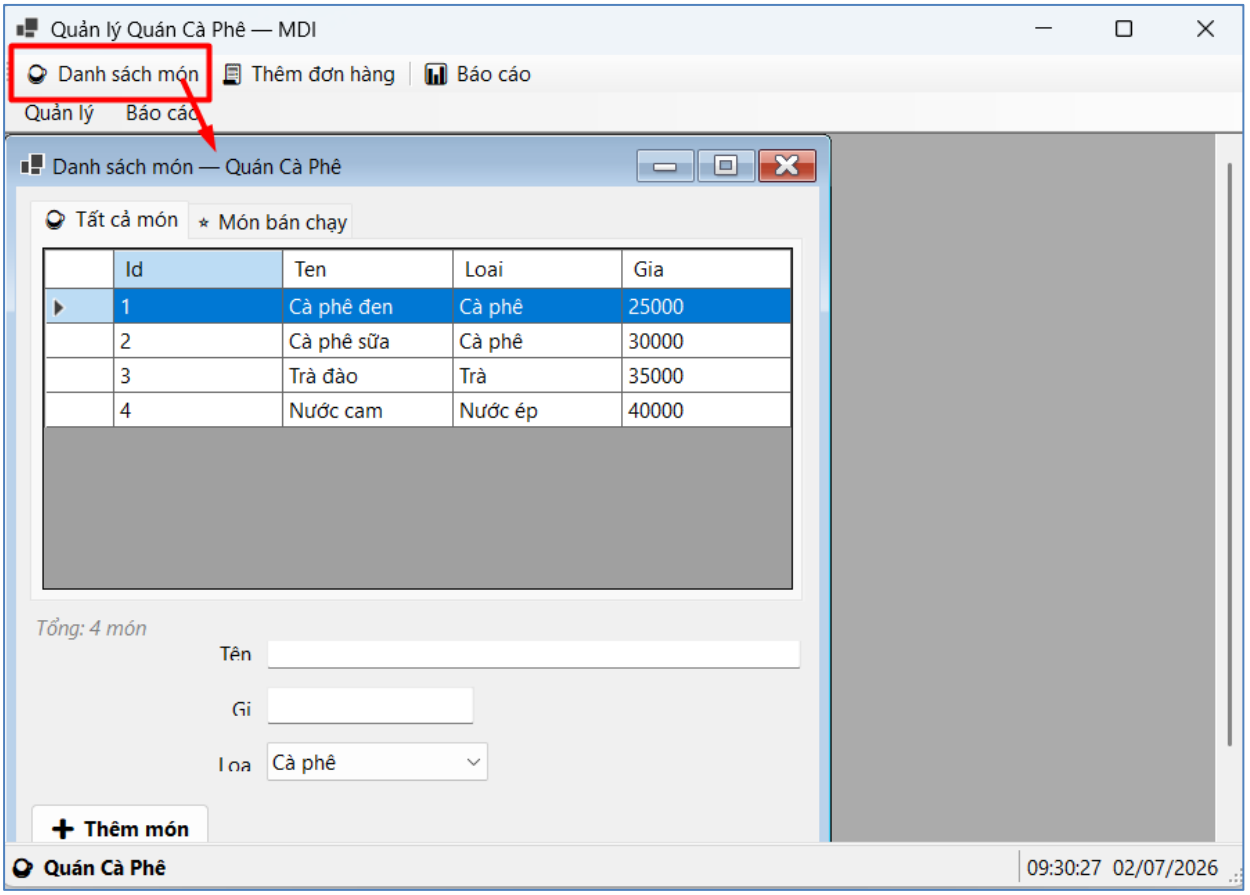
Form chính (MDI Parent)

- Có MenuStrip với các mục: Quản lý → Danh sách món / Thêm đơn hàng; Báo cáo → Doanh thu; Thoát



- Có ToolStrip với các nút tương ứng (icon/text) cho các chức năng trên
- Có StatusStrip hiển thị: tên quán bên trái, đồng hồ chạy thời gian thực bên phải
- Đảm bảo không mở trùng Form con cùng loại (kiểm tra MdiChildren)

Form con 1 — Danh sách món (FormDanhSachMon)



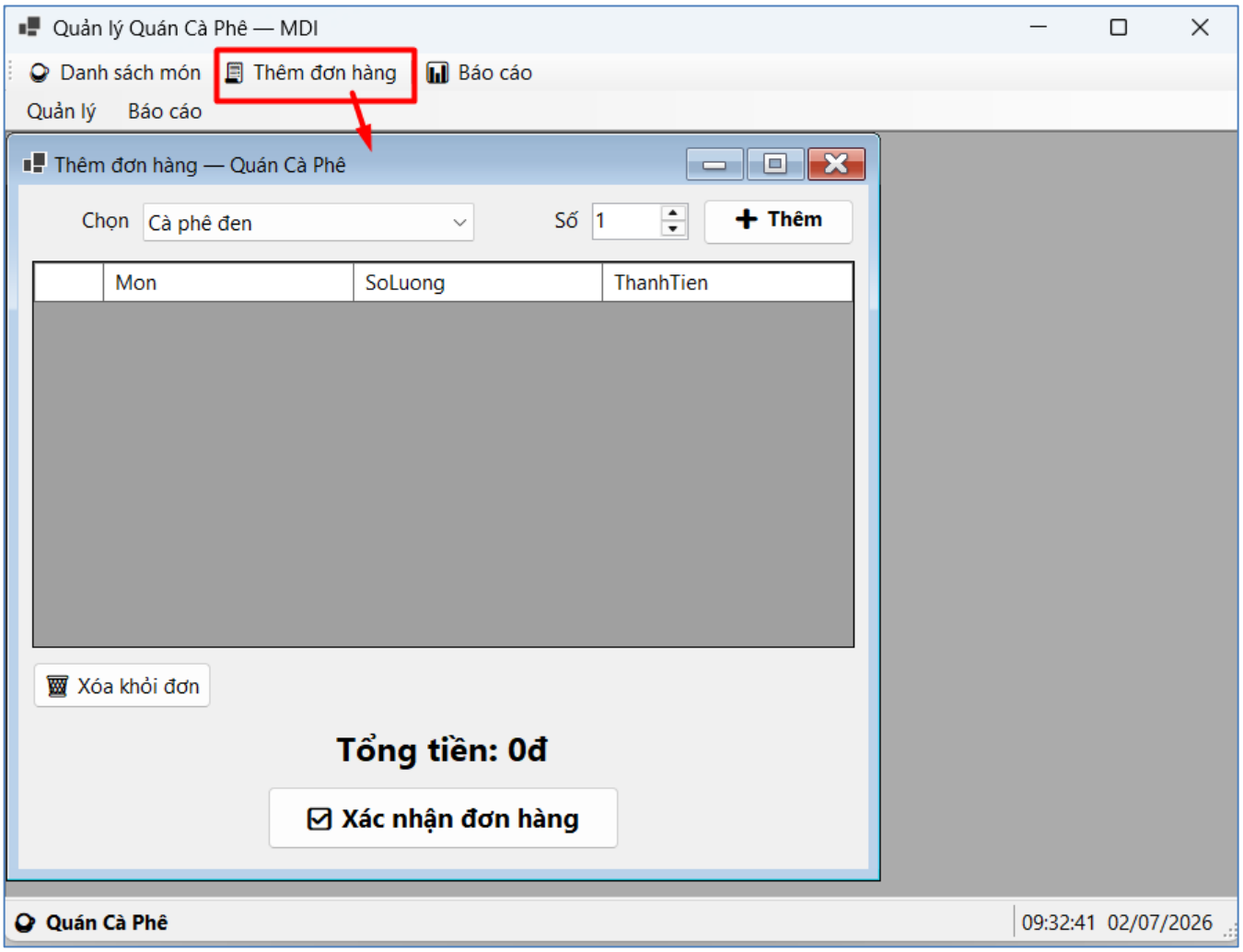
Dùng TabControl với 2 tab: "Tất cả món" và "Món bán chạy"

- Tab "Tất cả món": DataGridView hiển thị danh sách món (Tên, Loại, Giá), có ContextMenuStrip chuột phải để Sửa / Xóa
- Tab "Món bán chạy": lọc bằng LINQ, hiển thị top 3 món có giá cao nhất
- Dùng TableLayoutPanel để bố cục phần nhập liệu thêm món mới

Form con 2 — Thêm đơn hàng (FormThemDonHang)

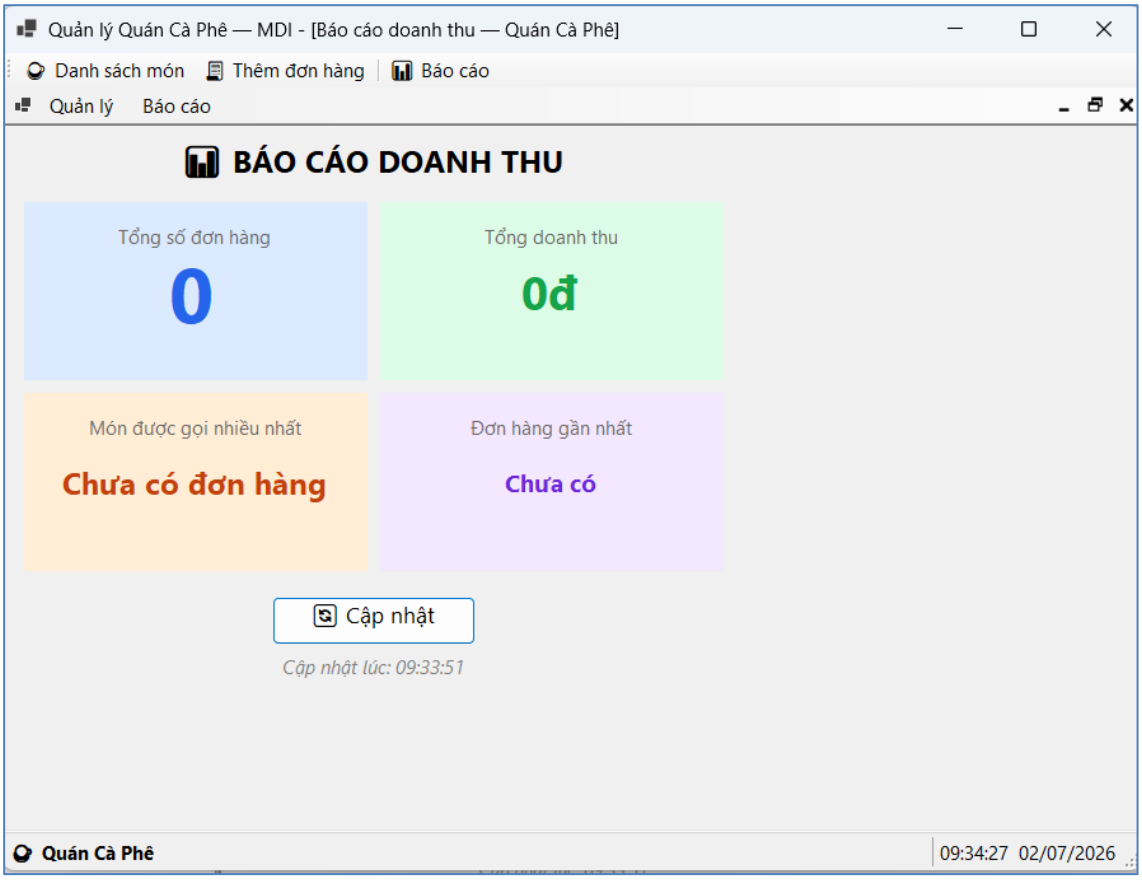
- ComboBox chọn món, NumericUpDown chọn số lượng, Button thêm vào đơn
- DataGridView hiển thị các món trong đơn hàng hiện tại
- Label hiển thị tổng tiền, cập nhật tự động mỗi khi thêm/xóa món

- Nút "Xác nhận đơn" → hiện MessageBox tóm tắt đơn hàng, hỏi xác nhận YesNo



Form con 3 — Báo cáo doanh thu (FormBaoCao)

- Hiển thị thống kê: tổng số đơn hàng, tổng doanh thu, món được gọi nhiều nhất
- Dùng TableLayoutPanel để bố cục các ô thống kê


Quản lý Quán Cà Phê — MDI - [Báo cáo doanh thu — Quán Cà Phê]
Danh sách món
Thêm đơn hàng
Báo cáo
Quản lý
Báo cáo
BÁO CÁO DOANH THU
Tổng số đơn hàng
0
Tổng doanh thu
0đ
Món được gọi nhiều nhất
Chưa có đơn hàng
Đơn hàng gần nhất
Chưa có
Cập nhật
Cập nhật lúc: 09:33:51
Quán Cà Phê
09:34:27 02/07/2026

BÀI 5: KẾT NỐI DỮ LIỆU VỚI ENTITY FRAMEWORK CORE

Sau khi học xong bài này, học viên có thể:

- *Hiểu được lý do cần sử dụng cơ sở dữ liệu thay vì lưu dữ liệu tạm thời bằng các cấu trúc trong bộ nhớ như List<T>.*
- *Hiểu khái niệm ORM (Object Relational Mapping) và lợi ích của Entity Framework Core.*
- *Nhận biết các thành phần cơ bản của Entity Framework Core: Entity, DbContext và DbSet.*
- *Xây dựng mô hình dữ liệu (Entity) và lớp DbContext.*
- *Sử dụng Migration để tạo và cập nhật cơ sở dữ liệu từ mã nguồn (Code First).*
- *Thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên cơ sở dữ liệu bằng Entity Framework Core.*
- *Xây dựng ứng dụng Windows Forms kết nối với SQL Server thông qua Entity Framework Core.*
- *Lập trình bất đồng bộ với async/await — nền tảng bắt buộc khi làm việc với EF Core trong WinForms.*

5.1 Lập trình bất đồng bộ

5.1.1 Đồng bộ và bất đồng bộ

Trong lập trình đồng bộ (Synchronous Programming), các câu lệnh được thực thi tuần tự: lệnh trước phải hoàn thành thì lệnh sau mới được thực hiện. Cách này phù hợp với các thao tác xử lý nhanh như tính toán hoặc xử lý dữ liệu trong bộ nhớ.

Tuy nhiên, với các thao tác mất nhiều thời gian như đọc tệp, truy cập cơ sở dữ liệu hoặc gọi dịch vụ trên Internet, việc chờ hoàn thành sẽ làm chương trình tạm dừng xử lý.

Lập trình bất đồng bộ (Asynchronous Programming) cho phép khởi động các tác vụ tốn thời gian mà không chặn luồng hiện tại. Trong khi chờ tác vụ hoàn thành, chương trình vẫn có thể tiếp tục thực hiện các công việc khác. Khi tác vụ kết thúc, kết quả sẽ được xử lý tiếp.

Nói ngắn gọn, lập trình đồng bộ buộc chương trình phải chờ, còn lập trình bất đồng bộ cho phép chương trình tiếp tục làm việc trong khi chờ, giúp ứng dụng phản hồi tốt hơn và sử dụng tài nguyên hiệu quả hơn

5.1.2 Lập trình bất động bộ trong C#

Trong Windows Forms, toàn bộ giao diện được quản lý bởi một UI Thread duy nhất, chịu trách nhiệm vẽ giao diện và xử lý các sự kiện như Click hay KeyDown.

Nếu UI Thread thực hiện một tác vụ đồng bộ mất nhiều thời gian, giao diện sẽ không phản hồi (UI Freeze) cho đến khi tác vụ hoàn thành. Đây là nguyên nhân xuất hiện trạng thái "Not Responding".

Để khắc phục, lập trình bất đồng bộ cho phép UI Thread tiếp tục xử lý giao diện trong khi chờ tác vụ hoàn thành, giúp ứng dụng luôn phản hồi tốt. Vì vậy, các thao tác như đọc tệp, truy cập cơ sở dữ liệu hoặc gọi API thường được thực hiện theo cơ chế bất đồng bộ.

Trong C#, lập trình bất đồng bộ được hỗ trợ thông qua hai từ khóa **async** và **await**:

- async: khai báo phương thức bất đồng bộ.

- await: tạm dừng phương thức để chờ tác vụ hoàn thành mà không chặn UI Thread.




```
        // ===== CÁCH 1: ĐỒNG BỘ - Form sẽ bị "đứng hình" =====
        private void btnDongBo_Click(object sender, EventArgs e)
        {
            lblTrangThai.Text = "Đang tải (đồng bộ)...";
            lblTrangThai.Refresh(); // ép vẽ lại ngay - nếu không dòng chữ này sẽ không kịp hiện ra

            Thread.Sleep(3000); // Đứng yên 3 giây - khóa hoàn toàn UI Thread

            lblTrangThai.Text = "Đã tải xong (đồng bộ)!";
        }

        // ===== CÁCH 2: BẤT ĐỒNG BỘ - Form vẫn phản hồi bình thường =====
        private async void btnBatDongBo_Click(object sender, EventArgs e)
        {
            lblTrangThai.Text = "Đang tải (bất đồng bộ)...";

            await Task.Delay(3000); // "Chờ" 3 giây nhưng không khóa UI Thread

            lblTrangThai.Text = "Đã tải xong (bất đồng bộ)!";
        }
```

Kết quả Khi chạy thử project demo (2 nút "Tải dữ liệu Đồng bộ" / "Tải dữ liệu Bất đồng bộ", cùng 1 nút "Click thử" để kiểm tra phản hồi), kết quả quan sát được như sau:

- Khi bấm nút Đồng bộ (Thread.Sleep(3000)):

Giao diện bị đóng băng (UI Freeze), dòng "Đang tải..." không hiển thị ngay.

Các thao tác như bấm nút, kéo hoặc thay đổi kích thước Form đều không phản hồi.

Sau khoảng 3 giây, Form hoạt động trở lại và hiển thị "Đã tải xong!".

- Khi bấm nút Bất đồng bộ (await Task.Delay(3000)):

Dòng "Đang tải..." hiển thị ngay lập tức.

Trong thời gian chờ, Form vẫn phản hồi bình thường, người dùng có thể tiếp tục thao tác.

Sau khoảng 3 giây, giao diện tự động cập nhật thành "Đã tải xong!".

Kết luận: Thread.Sleep() chặn UI Thread, gây treo giao diện, trong khi await Task.Delay() không chặn UI Thread, giúp ứng dụng luôn phản hồi tốt.

5.2 Từ ADO.NET đến EF Core

Vì sao cần kết nối cơ sở dữ liệu

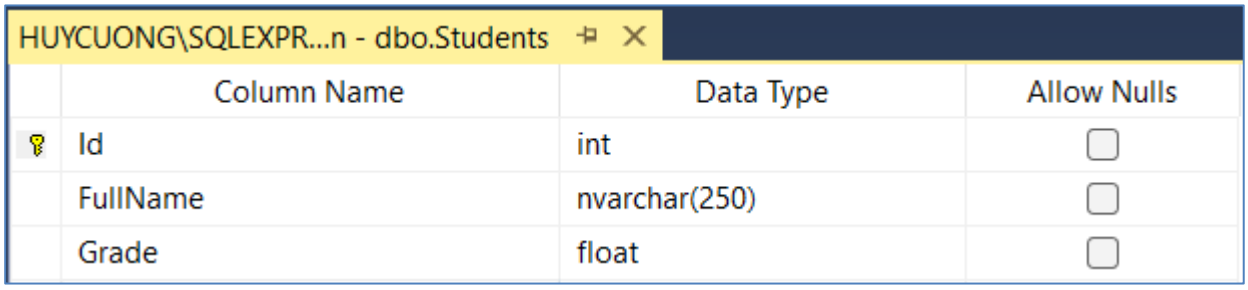
Ở các bài trước, dữ liệu được lưu trong List<T> nên chỉ tồn tại trong bộ nhớ RAM. Khi đóng ứng dụng, toàn bộ dữ liệu sẽ bị mất.

Trong thực tế, dữ liệu cần được lưu trữ lâu dài, cho phép truy cập lại sau khi khởi động ứng dụng và có thể được nhiều người dùng cùng sử dụng. Vì vậy, ứng dụng cần kết nối với cơ sở dữ liệu (Database).

Để làm việc với cơ sở dữ liệu, ứng dụng phải có cơ chế kết nối, thêm, sửa, xóa và truy vấn dữ liệu. Trong .NET, hai công nghệ phổ biến là ADO.NET và Entity Framework Core (EF Core). Trong giáo trình này, chúng ta sẽ sử dụng Entity Framework Core, một thư viện ORM giúp thao tác với cơ sở dữ liệu thông qua các đối tượng C#.

Hệ quản trị cơ sở dữ liệu sử dụng là SQL Server, một hệ quản trị cơ sở dữ liệu quan hệ của Microsoft, tích hợp tốt với Visual Studio và .NET. Trong SQL Server, dữ liệu được lưu trong các bảng (Table), gồm các cột (Column) và các dòng (Row), mỗi dòng biểu diễn một bản ghi.

Ví dụ, một bảng Students có thể có cấu trúc:



Cấu trúc bảng này tương ứng gần như trực tiếp với một class trong C# — mỗi cột ứng với một property, mỗi dòng dữ liệu ứng với một object cụ thể được tạo ra từ class đó. Sự tương đồng này chính là nền tảng cho khái niệm ORM.

Để giao tiếp với SQL Server (gửi dữ liệu vào, truy vấn dữ liệu ra), người ta dùng ngôn ngữ truy vấn SQL (Structured Query Language) — một ngôn ngữ chuyên biệt cho việc thao tác dữ liệu dạng bảng, với các câu lệnh quen thuộc như SELECT (truy vấn), INSERT (thêm), UPDATE (sửa), DELETE (xóa). Câu lệnh SQL cơ bản nhất, dùng để lấy toàn bộ dữ liệu từ bảng Students, có dạng:

```
SELECT * FROM Students;
```

Để cài đặt và quản lý SQL Server một cách trực quan, thường dùng kèm công cụ SQL Server Management Studio (SSMS) — phần mềm miễn phí của Microsoft, cho phép xem cấu trúc bảng, chạy thử câu lệnh SQL, và quan sát trực tiếp dữ liệu dưới dạng bảng, tương tự giao diện Excel.

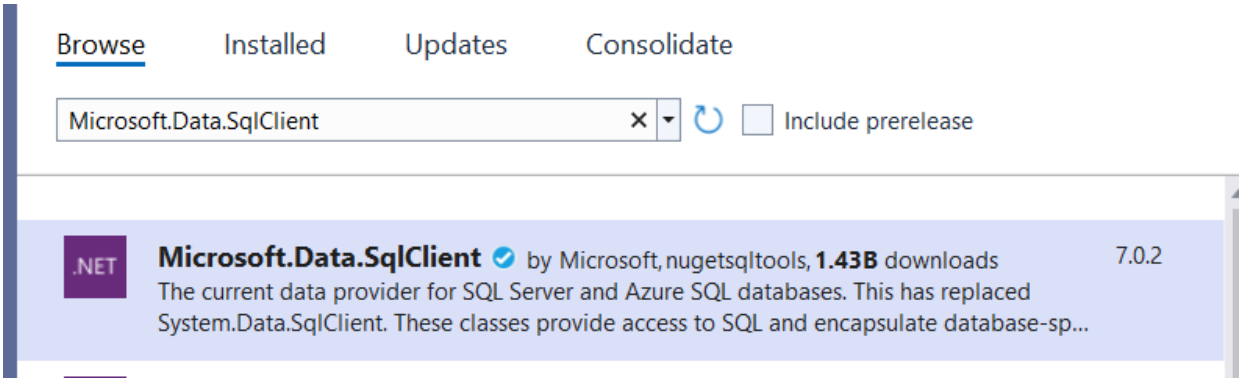
Phần tiếp theo sẽ trình bày hai cách để ứng dụng C# giao tiếp với SQL Server: viết tay câu lệnh SQL thông qua ADO.NET, và cách tiếp cận hiện đại hơn, gần với tư duy hướng đối tượng hơn, thông qua EF Core.

5.2.1 ADO.NET — cách kết nối CSDL truyền thống

.NET cung cấp sẵn một bộ thư viện nền tảng để giao tiếp trực tiếp với cơ sở dữ liệu, gọi là ADO.NET (ActiveX Data Objects .NET). Đây là lớp công nghệ thấp nhất, gần với cơ sở dữ liệu nhất mà .NET cung cấp — mọi công nghệ truy cập dữ liệu hiện đại hơn, kể cả EF Core, đều được xây dựng dựa trên nền tảng này ở phía dưới cùng.

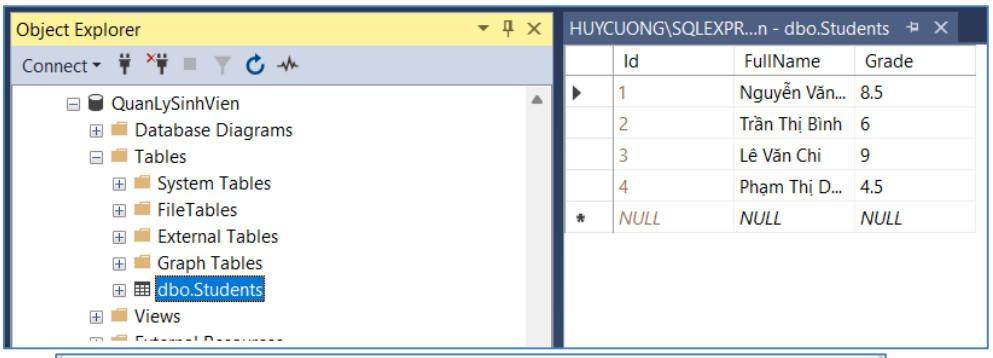
Làm việc với ADO.NET đòi hỏi thực hiện đầy đủ các bước thủ công: mở kết nối đến cơ sở dữ liệu, viết câu lệnh SQL bằng tay, thực thi câu lệnh đó, rồi tự đọc từng dòng

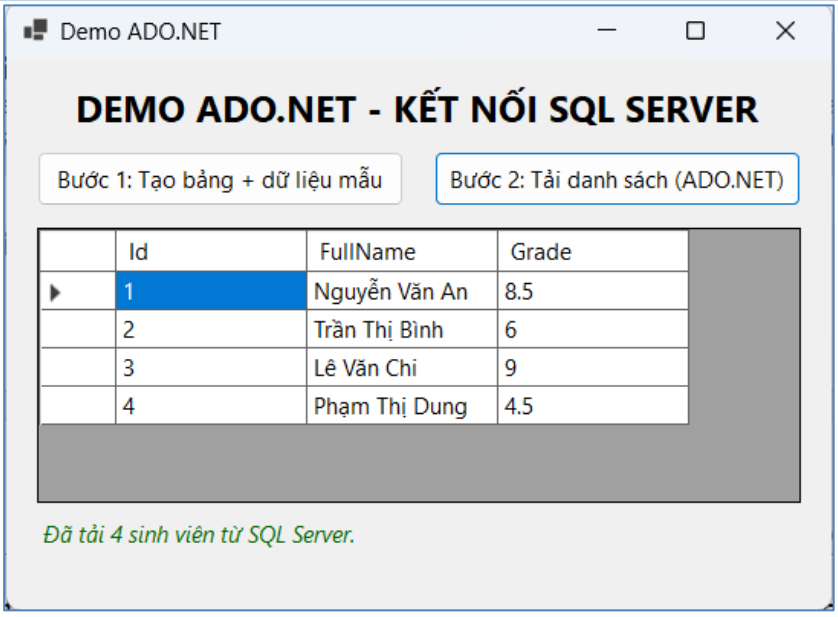
kết quả trả về và gán vào đúng thuộc tính của object tương ứng. Ba thành phần cốt lõi thường gặp khi làm việc với ADO.NET cho SQL Server là SqlConnection, SqlCommand, và SqlDataReader.



SqlConnection — mở kết nối đến SQL Server

SqlConnection đại diện cho một kết nối vật lý đến máy chủ SQL Server, được khởi tạo thông qua một chuỗi kết nối (connection string) — chứa thông tin về địa chỉ máy chủ, tên cơ sở dữ liệu, và cách xác thực:





```
public partial class Form1 : Form
{
    // Sửa lại Server, Database cho đúng máy của bạn trước khi chạy thử
    private readonly string connectionString =
        "Server=HUYCUONG\\SQLEXPRESS;" +
        "Database=QuanLySinhVien;" +
        "Trusted_Connection=True;TrustServerCertificate=True;";

    public Form1()
    {
        InitializeComponent();
    }

    private void Form1_Load(object sender, EventArgs e)
    {
        lblTrangThai.Text = "Bấm nút để tải danh sách sinh viên từ SQL Server (ADO.NET).";
    }

    // ===== Đọc dữ liệu bằng ADO.NET thuần (SqlConnection, SqlCommand, SqlDataReader)
=====
    private void btnTaiDuLieu_Click(object sender, EventArgs e)
    {
        try
        {
            List<Student> danhSach = LayDanhSachSinhVien();

            dgvSinhVien.DataSource = danhSach;
            lblTrangThai.Text = $"Đã tải {danhSach.Count} sinh viên từ SQL Server.";
            lblTrangThai.ForeColor = System.Drawing.Color.DarkGreen;
        }
        catch (Exception ex)
        {
            lblTrangThai.Text = "Lỗi kết nối: " + ex.Message;
            lblTrangThai.ForeColor = System.Drawing.Color.Red;
        }
    }

    private List<Student> LayDanhSachSinhVien()
    {
        List<Student> danhSach = new List<Student>();

        using (SqlConnection conn = new SqlConnection(connectionString))
        {
            conn.Open();

            SqlCommand cmd = new SqlCommand("SELECT Id, FullName, Grade FROM Students",
conn);
            SqlDataReader reader = cmd.ExecuteReader();

            while (reader.Read())
            {
                Student sv = new Student
                {
                    Id = Convert.ToInt32(reader["Id"]),
                    FullName = reader["FullName"].ToString(),
                    Grade = Convert.ToDouble(reader["Grade"])
                };
                danhSach.Add(sv);
            }
        } // using tự động đóng kết nối ở đây
```

```
        return danhSach;
    }

    // ===== Tạo sẵn CSDL mẫu (chạy 1 lần để có dữ liệu thử nghiệm) =====
    private void btnTaoDuLieuMau_Click(object sender, EventArgs e)
    {
        try
        {
            using (SqlConnection conn = new SqlConnection(connectionString))
            {
                conn.Open();

                string sqlTaoBang = @"
                    IF NOT EXISTS (SELECT * FROM sysobjects WHERE name='Students' AND
xtype='U')
                    CREATE TABLE Students (
                        Id INT IDENTITY(1,1) PRIMARY KEY,
                        FullName NVARCHAR(250),
                        Grade FLOAT
                    )";

                new SqlCommand(sqlTaoBang, conn).ExecuteNonQuery();

                string sqlXoaDuLieuCu = "DELETE FROM Students";
                new SqlCommand(sqlXoaDuLieuCu, conn).ExecuteNonQuery();

                string sqlThemDuLieu = @"
                    INSERT INTO Students (FullName, Grade) VALUES
                    (N'Nguyễn Văn An', 8.5),
                    (N'Trần Thị Bình', 6.0),
                    (N'Lê Văn Chi', 9.0),
                    (N'Phạm Thị Dung', 4.5)";

                new SqlCommand(sqlThemDuLieu, conn).ExecuteNonQuery();
            }

            lblTrangThai.Text = "Đã tạo bảng Students và thêm dữ liệu mẫu thành công!";
            lblTrangThai.ForeColor = System.Drawing.Color.DarkGreen;
        }
        catch (Exception ex)
        {
            lblTrangThai.Text = "Lỗi: " + ex.Message;
            lblTrangThai.ForeColor = System.Drawing.Color.Red;
        }
    }
}
```

SqlCommand — chứa câu lệnh SQL cần thực thi

`SqlCommand` đóng gói một câu lệnh SQL (viết tay dưới dạng chuỗi văn bản) cùng với kết nối sẽ dùng để thực thi câu lệnh đó:

SqlDataReader — đọc kết quả trả về

`SqlDataReader` cho phép đọc kết quả trả về từ câu lệnh `SELECT`, theo từng dòng một, thông qua method `Read()` (trả về `true` nếu còn dòng dữ liệu để đọc, `false` nếu đã hết):

Với mỗi dòng đọc được, phải tự tay truy xuất đúng tên cột (`reader["FullName"]`, `reader["Grade"]`), tự ép kiểu dữ liệu cho đúng (`ToString()`, `Convert.ToDouble()`), rồi tự gán vào từng thuộc tính của object `Student` — toàn bộ quá trình ánh xạ từ dữ liệu bảng sang object C# đều do người viết code thực hiện thủ công, không có bất kỳ sự tự động hóa nào.

5.2.2 Hạn chế của ADO.NET

Cách làm bằng ADO.NET, dù hoàn toàn khả thi và vẫn được dùng trong một số tình huống đặc thù, bộc lộ một số hạn chế đáng kể khi áp dụng vào những ứng dụng có quy mô lớn hơn một vài câu truy vấn đơn giản.

Câu lệnh SQL không được kiểm tra lúc biên dịch

Câu lệnh SQL viết tay dưới dạng chuỗi văn bản thuần túy không được trình biên dịch C# kiểm tra cú pháp hay kiểm tra kiểu dữ liệu. Một lỗi gõ sai tên cột, sai tên bảng, hay sai cú pháp SQL sẽ không bị phát hiện ngay khi biên dịch như các lỗi C# thông thường — chỉ khi chương trình thực sự chạy đến dòng đó, lỗi mới lộ ra:

```
// Gõ nhầm "FullNme" thay vì "FullName" – biên dịch vẫn thành công bình thường!
SqlCommand cmd = new SqlCommand("SELECT Id, FullNme, Grade FROM Students", conn);
```

Lỗi gõ nhầm tên cột này hoàn toàn "qua mặt" được trình biên dịch — vì đối với C#, đây chỉ là một chuỗi văn bản (`string`) bình thường, không khác gì bất kỳ chuỗi nào khác. Chỉ đến khi ứng dụng thực sự chạy và gửi câu lệnh này xuống SQL Server, lỗi `Invalid column name 'FullNme'` mới xuất hiện — một dạng lỗi runtime đáng lẽ có thể được phát hiện sớm hơn rất nhiều nếu công cụ biên dịch có khả năng "hiểu" được cấu trúc của cơ sở dữ liệu.

Ánh xạ dữ liệu hoàn toàn thủ công

Với mỗi cột trong kết quả trả về, phải tự tay truy xuất đúng tên cột, tự ép kiểu dữ liệu cho đúng, rồi tự gán vào từng property của object tương ứng:

```
Student sv = new Student
 {
     Id = Convert.ToInt32(reader["Id"]),
     FullName = reader["FullName"].ToString(),
     Grade = Convert.ToDouble(reader["Grade"])
 };
```

Với một class chỉ có 3 property như Student, việc này còn chấp nhận được. Nhưng hãy hình dung một bảng có 15-20 cột — đoạn code ánh xạ thủ công như trên sẽ kéo dài tương ứng, lặp đi lặp lại một khuôn mẫu gần như giống hệt nhau cho từng cột, vừa tốn công viết, vừa dễ phát sinh lỗi do gõ sai tên cột hoặc nhầm thứ tự gán.

Một vấn đề tinh vi hơn liên quan trực tiếp đến class Student vừa được cập nhật ở phần trước: thuộc tính FullName được khai báo với từ khóa required, nghĩa là về mặt thiết kế, một object Student không bao giờ nên tồn tại mà thiếu FullName. Nhưng với cách ánh xạ thủ công qua object initializer như trên, trình biên dịch hoàn toàn không biết dữ liệu lấy từ reader["FullName"] có thực sự hợp lệ (không null) hay không — toàn bộ trách nhiệm đảm bảo tính đúng đắn của dữ liệu đổ dồn lên người viết code, ADO.NET không có cơ chế nào hỗ trợ kiểm tra điều này.

Phải tự quản lý vòng đời kết nối

Kết nối đến cơ sở dữ liệu là một tài nguyên hệ thống có giới hạn — SQL Server chỉ cho phép một số lượng kết nối đồng thời nhất định. Nếu quên đóng kết nối sau khi dùng xong, kết nối đó sẽ tiếp tục "chiếm chỗ" dù không còn dùng đến, dần dần dẫn đến tình trạng cạn kiệt kết nối khả dụng. Việc dùng từ khóa using là cách giảm thiểu rủi ro này, nhưng vẫn đòi hỏi người viết code phải chủ động nhớ áp dụng đúng cách ở mọi nơi có mở kết nối — không có cơ chế nào tự động nhắc nhở hay ép buộc.

Không tận dụng được LINQ đã học

Toàn bộ logic truy vấn nằm gói gọn trong một chuỗi SQL tách biệt hoàn toàn khỏi thế giới C# — không thể viết Where, OrderBy quen thuộc của LINQ trực tiếp lên dữ liệu cơ sở dữ liệu như đã làm với List<T>. Muốn lọc sinh viên đạt điểm từ 5 trở lên và sắp xếp giảm dần, phải tự viết lại bằng cú pháp SQL hoàn toàn riêng biệt:

```
SqlCommand cmd = new SqlCommand(
    "SELECT Id, FullName, Grade FROM Students WHERE Grade >= 5 ORDER BY Grade DESC",
    conn
);
```

So với cách viết LINQ quen thuộc `danhSach.Where(sv => sv.Grade >= 5).OrderByDescending(sv => sv.Grade)`, đoạn SQL trên đòi hỏi một cú pháp và tư duy hoàn toàn khác — kiến thức LINQ không thể tái sử dụng trực tiếp khi làm việc với dữ liệu từ cơ sở dữ liệu theo cách ADO.NET thuần túy này.

Bốn hạn chế trên — thiếu kiểm tra lúc biên dịch, ánh xạ thủ công, tự quản lý kết nối, và không tận dụng được LINQ — chính là động lực trực tiếp dẫn đến sự ra đời của các công nghệ ORM hiện đại hơn, trong đó EF Core là đại diện sẽ được trình bày ngay trong mục tiếp theo.

5.3 Entity Framework Core

EF Core (Entity Framework Core) là một thư viện ORM (Object-Relational Mapping) chính thức của Microsoft dành cho .NET — công nghệ ánh xạ tự động giữa các bảng trong cơ sở dữ liệu quan hệ và các class trong code hướng đối tượng, EF Core hỗ trợ làm việc với nhiều hệ quản trị cơ sở dữ liệu khác nhau (SQL Server, MySQL, PostgreSQL, SQLite...), thông qua các gói Provider riêng cho từng loại — môn học này tập trung vào SQL Server, nên sẽ dùng provider tương ứng.EF Core không phải một phần mặc định có sẵn trong .NET SDK — đây là một thư viện riêng, cần được cài đặt qua NuGet vào từng project cụ thể trước khi sử dụng được.

Các gói NuGet cần cài đặt

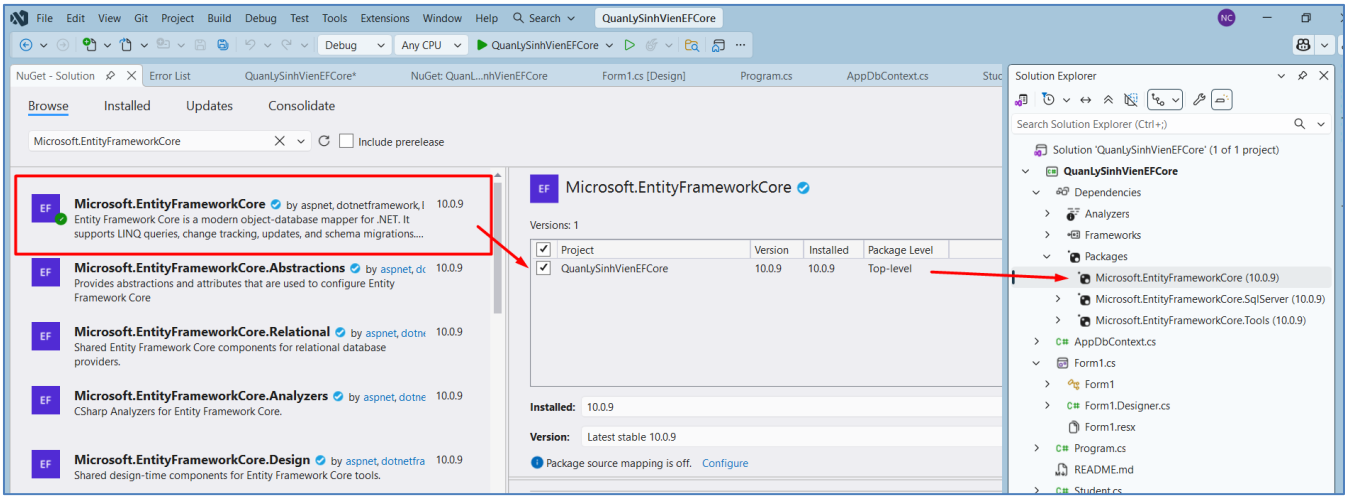
Để làm việc với EF Core và SQL Server trong project WinForms, cần cài đặt tối thiểu **các gói NuGet** sau:

Gói NuGet	**Vai trò**
Microsoft.EntityFrameworkCore.SqlServer	Provider giúp EF Core giao tiếp được với SQL Server cụ thể
Microsoft.EntityFrameworkCore.Tools	Cung cấp các câu lệnh dùng trong Migration
Microsoft.EntityFrameworkCore	Thư viện lõi — chứa `DbContext`, `DbSet<T>` (thường được cài tự động kèm theo provider)

Cách cài đặt qua giao diện Visual Studio

Chuột phải vào tên project trong Solution Explorer → chọn Manage NuGet Packages... → chuyển sang tab Browse → gõ tìm lần lượt từng tên gói ở trên → chọn đúng gói (chú ý chọn đúng tên, vì NuGet có rất nhiều gói tên gần giống nhau) → nhấn Install.

Lưu ý quan trọng: nên cài cùng một phiên bản cho cả hai gói, và phiên bản đó nên khớp với phiên bản .NET đang dùng cho project (ví dụ project dùng .NET 10 thì nên chọn các gói EF Core phiên bản tương ứng mới nhất khả dụng) — tránh tình trạng xung đột phiên bản gây lỗi khó hiểu khi build.



Cách cài đặt qua dòng lệnh (cách thay thế)

Với những ai quen thao tác dòng lệnh, có thể mở **Package Manager Console** (View → Other Windows → Package Manager Console) và gõ:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
```

NuGet có cơ chế "dependency" (phụ thuộc) tự động — khi một package A khai báo rằng nó cần package B để hoạt động, thì khi cài A, NuGet sẽ tự động cài kèm B theo, không cần gõ lệnh cài B riêng.

Cụ thể ở đây:

Microsoft.EntityFrameworkCore.SqlServer (provider cho SQL Server) được Microsoft khai báo là phụ thuộc vào Microsoft.EntityFrameworkCore (package lõi chứa DbContext, DbSet<T>) — vì muốn làm provider cho SQL Server thì bắt buộc phải có sẵn các thành phần lõi đó trước.

Vì vậy, khi gõ Install-Package Microsoft.EntityFrameworkCore.SqlServer, NuGet tự nhận ra "package này cần Microsoft.EntityFrameworkCore nữa", và tự động tải cả hai về cùng lúc — dù chỉ gõ một lệnh.

Hoặc dùng `dotnet` **CLI** (mở Terminal, đứng trong thư mục project):

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer

dotnet add package Microsoft.EntityFrameworkCore.Tools
```

5.3.1 DbContext và DbSet

DbContext là gì

DbContext là class trung tâm của EF Core, đóng vai trò cầu nối giữa ứng dụng C# và cơ sở dữ liệu thật. Có thể hình dung DbContext như một "phiên làm việc" với cơ sở dữ liệu — nó biết cách kết nối tới đúng máy chủ nào, biết những Entity (class) nào sẽ được ánh xạ thành bảng, và chịu trách nhiệm theo dõi mọi thay đổi dữ liệu (thêm, sửa, xóa) trước khi thực sự ghi chúng xuống cơ sở dữ liệu.

Để sử dụng, cần tự định nghĩa một class kế thừa từ DbContext — đây là nơi khai báo những bảng nào ứng dụng sẽ làm việc cùng, và cấu hình thông tin kết nối:

```
public class AppDbContext : DbContext
 {
     public DbSet<Student> Students { get; set; }

     protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
     {
         // Sửa lại Server cho đúng máy của bạn trước khi chạy thử
         optionsBuilder.UseSqlServer(
             "Server=HUYCUONG\\SQLEXPRESS;" +
             "Database=QuanLySinhVien;" +
             "Trusted_Connection=True;TrustServerCertificate=True;"
         );
     }
 }
```

Phương thức `OnConfiguring` được override lại để chỉ định chuỗi kết nối — thông tin về máy chủ SQL Server và tên cơ sở dữ liệu sẽ làm việc cùng, giống hệt connection string đã thấy ở ADO.NET, chỉ là giờ được khai báo một lần duy nhất ngay trong **`DbContext`**, thay vì phải lặp lại ở mọi nơi có thao tác CSDL như trước.

DbSet<T> là gì

DbSet<T> đại diện cho một bảng cụ thể trong cơ sở dữ liệu, với T là Entity (class) tương ứng với bảng đó. Trong ví dụ trên, DbSet<Student> Students nghĩa là: "có một bảng tên Students trong cơ sở dữ liệu, mỗi dòng dữ liệu trong bảng đó tương ứng với một object Student".

Đây chính là điểm khởi đầu cho mọi thao tác CRUD (Create, Read, Update, Delete) sẽ thực hiện — tất cả đều thông qua thuộc tính Students này:

```
List<Student> danhSach = await context.Students.ToListAsync();
```

`DbSet<T>` có thể xem như một dạng "danh sách đặc biệt" — về cách sử dụng, khá giống `List<T>` (cũng `.Add()`, cũng dùng được LINQ `Where`, `OrderBy`), nhưng khác ở bản chất: mọi thao tác trên `DbSet<T>` đều **liên kết trực tiếp với một bảng thật trong cơ sở dữ liệu**, chứ không chỉ tồn tại trong bộ nhớ tạm thời như `List<T>` thuần túy.

Mỗi DbContext có thể có nhiều DbSet

Một ứng dụng thực tế thường làm việc với nhiều bảng khác nhau — `DbContext` không giới hạn chỉ một `DbSet`, có thể khai báo nhiều thuộc tính `DbSet<T>` cho từng Entity tương ứng:

```
public class AppDbContext : DbContext
{
    public DbSet<Student> Students { get; set; }
    public DbSet<Teacher> Teachers { get; set; }
    public DbSet<Course> Courses { get; set; }
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
   {
      optionsBuilder.UseSqlServer("...");
    }
}
```

Mỗi `DbSet<T>` ở đây sẽ tương ứng với một bảng riêng biệt trong cùng một cơ sở dữ liệu — `Students`, `Teachers`, `Courses` — tất cả đều được quản lý chung bởi một `AppDbContext` duy nhất.

Cách sử dụng DbContext trong code

Mỗi khi cần thao tác với cơ sở dữ liệu, tạo một object AppDbContext mới — thường đặt trong khối using để đảm bảo tài nguyên kết nối được giải phóng đúng cách ngay sau khi dùng xong, đúng nguyên tắc đã quen thuộc từ SqlConnection ở ADO.NET:

```
using (var context = new AppDbContext())
{
    List<Student> danhSach = await context.Students.ToListAsync();
    // Sau khi ra khỏi khối using, context tự động được giải phóng
}
```

So với việc phải tự viết `SqlConnection`, `SqlCommand`, `SqlDataReader` như ADO.NET việc khởi tạo và sử dụng **`DbContext`** đơn giản hơn nhiều — toàn bộ chi tiết kết nối đã được cấu hình sẵn một lần trong `OnConfiguring`, lập trình viên chỉ cần tạo object và gọi đến đúng **`DbSet<T>`** tương ứng với bảng cần thao tác.

5.3.2 Code First và Database First

Khi làm việc với một ORM như EF Core, có hai hướng tiếp cận khác nhau, tùy vào việc cơ sở dữ liệu đã tồn tại sẵn hay chưa.

Code First

Code First là mô hình trong đó Entity (class C#) được viết trước, sau đó dùng Migration để công cụ EF Core tự động sinh ra cấu trúc cơ sở dữ liệu tương ứng. Đây là mô hình phù hợp khi bắt đầu một dự án hoàn toàn mới, chưa có sẵn cơ sở dữ liệu nào — lập trình viên chỉ cần tập trung thiết kế các class theo đúng tư duy hướng đối tượng đã quen thuộc, còn việc tạo bảng, cột, khóa chính trong CSDL sẽ do EF Core tự động đảm nhiệm thông qua Migration.

Entity (class C#) → Migration → CSDL được tự động tạo ra

Ưu điểm lớn nhất của Code First: toàn bộ "nguồn sự thật" (source of truth) về cấu trúc dữ liệu nằm trong code, dễ dàng theo dõi lịch sử thay đổi qua các Migration, và đặc biệt phù hợp khi làm việc nhóm với hệ thống quản lý phiên bản (Git) — mọi thay đổi cấu trúc CSDL đều được ghi lại dưới dạng code, có thể xem lại, đối chiếu như bất kỳ thay đổi code nào khác.

Database First

Database First là mô hình ngược lại — **cơ sở dữ liệu đã tồn tại sẵn từ trước** (có thể do người khác thiết kế, hoặc kế thừa từ một hệ thống cũ), và EF Core được dùng để

tự động sinh ra các Entity (class C#) tương ứng với cấu trúc bảng đã có sẵn đó, thông qua một công cụ gọi là **Scaffolding** (lệnh `Scaffold-DbContext`).

> CSDL đã có sẵn → Scaffolding → Entity (class C#) được tự động tạo ra

Mô hình này phù hợp khi làm việc với một cơ sở dữ liệu doanh nghiệp đã vận hành từ lâu, cấu trúc bảng phức tạp, có nhiều ràng buộc đã được DBA (Database Administrator) thiết kế kỹ lưỡng từ trước — viết lại toàn bộ Entity bằng tay trong trường hợp này vừa tốn công, vừa dễ sai sót so với việc để công cụ tự sinh ra dựa trên cấu trúc bảng thật.

Tiêu chí	**Code First**	**Database First**
Bắt đầu từ đâu	Class C# (Entity)	Cơ sở dữ liệu có sẵn
Công cụ chính	Migration (`Add-Migration`, `Update-Database`)	Scaffolding (`Scaffold-DbContext`)
Phù hợp khi	Dự án mới, CSDL chưa tồn tại	CSDL đã có sẵn, cần tạo Entity tương ứng
Quản lý thay đổi cấu trúc	Qua lịch sử Migration trong code	Phải chạy lại Scaffolding khi CSDL đổi

Môn học này tập trung vào **Code First**, vì đây là cách tiếp cận tự nhiên hơn để học EF Core — tận dụng trực tiếp tư duy hướng đối tượng đã xây dựng, và phù hợp với bối cảnh đang xây dựng một ứng dụng hoàn toàn mới, chưa có CSDL có sẵn từ trước.

5.3.3 Định nghĩa Entity (Model)

Entity là một class C# thông thường, được thiết kế để **ánh xạ tương ứng với một bảng** trong cơ sở dữ liệu — mỗi property của Entity tương ứng với một cột trong bảng đó. Đây chính xác là kiểu class đã quen thuộc (Hướng đối tượng) — không có gì khác biệt về bản chất, chỉ là giờ class đó được "gắn" thêm một vai trò mới: làm khuôn mẫu cho dữ liệu lưu trữ lâu dài trong CSDL. Xuyên suốt phần thực hành, một Entity duy nhất — `Student` — sẽ được dùng để minh họa trọn vẹn cả bốn thao tác CRUD.

```
public class Student
 {
     public int Id { get; set; }
     public required string FullName { get; set; }
     public double Grade { get; set; }

     public override string ToString()
     {
         return $"{Id} - {FullName} - {Grade}";
     }
 }
```

Quy ước (convention) EF Core tự nhận diện

EF Core áp dụng một số quy ước mặc định để tự suy luận cấu trúc bảng tương ứng, mà không cần khai báo gì thêm:

- Property tên Id (hoặc TênClass + Id, ví dụ StudentId) sẽ tự động được EF Core hiểu là khóa chính (Primary Key) của bảng, và mặc định tự tăng giá trị (Identity) mỗi khi thêm bản ghi mới — không cần tự gán giá trị cho Id khi tạo object mới.
- Property kiểu string được khai báo required sẽ được EF Core ánh xạ thành cột NOT NULL — đảm bảo cột đó luôn phải có giá trị, không được để trống trong CSDL.
- Property kiểu string không có required sẽ được ánh xạ thành cột cho phép NULL.
- Tên thuộc tính DbSet (sẽ khai báo, ví dụ Students) sẽ trở thành tên bảng thực tế trong CSDL — không nhất thiết phải trùng tên class Student.

Ý nghĩa từng property trong Entity Student

Property	Kiểu dữ liệu C#	Vai trò	Cột tương ứng trong CSDL
`Id`	`int`	Khóa chính, tự tăng	`INT, PRIMARY KEY, IDENTITY`
`FullName`	`string` (required)	Họ tên sinh viên	`NVARCHAR, NOT NULL`
`Grade`	`double`	Điểm số	`FLOAT`

5.3.4 Viết DbContext — Connection String

Sau khi Entity `Student` đã sẵn sàng, bước tiếp theo là tạo ra **DbContext** — class đóng vai trò "cầu nối" giữa Entity đó và một cơ sở dữ liệu SQL Server thật.

Bước 1 — Tạo file DbContext mới

Trong Solution Explorer, chuột phải vào project → **Add** → **Class...** → đặt tên `AppDbContext.cs` → **Add**. Tên file này không bắt buộc phải là `AppDbContext` — có thể đặt tên khác tùy ý — nhưng nên chọn một cái tên phản ánh đúng vai trò của nó (đại diện cho toàn bộ "phiên làm việc" với CSDL của ứng dụng), để dễ nhận biết khi project có nhiều file khác nhau.

Bước 2 — Cho class kế thừa từ DbContext

Đây là bước quan trọng nhất về mặt kỹ thuật: nhờ kế thừa (`: DbContext`) — `AppDbContext` ngay lập tức thừa hưởng toàn bộ năng lực vốn có của `DbContext` gốc trong EF Core: khả năng mở/đóng kết nối, theo dõi thay đổi dữ liệu, và dịch các thao tác LINQ thành câu lệnh SQL. Tại thời điểm này, class đã hợp lệ về mặt cú pháp, nhưng chưa biết phải kết nối tới CSDL nào, và chưa quản lý bảng nào cả — hai bước tiếp theo sẽ bổ sung đầy đủ hai phần còn thiếu đó.

Bước 3 — Khai báo DbSet cho Entity Student

Dòng `DbSet<Student> Students` chính là lời tuyên bố: "trong cơ sở dữ liệu sẽ làm việc cùng, có một bảng tên `Students`, và mỗi dòng dữ liệu trong bảng đó tương ứng với một object `Student`". Đây cũng là điểm khởi đầu cho mọi thao tác CRUD sẽ thực hiện— toàn bộ các lệnh thêm, đọc, sửa, xóa sau này đều sẽ thông qua đúng thuộc tính `Students` vừa khai báo ở đây.

Bước 4 — Override OnConfiguring để khai báo Connection String

Phương thức `OnConfiguring` vốn đã tồn tại sẵn trong class `DbContext` gốc dưới dạng `virtual` (tương tự `GioiThieu()` ở ví dụ `Person`) — ở đây chỉ cần `override` lại để cung cấp đúng thông tin kết nối cho riêng `AppDbContext` của ứng dụng. Phương thức `UseSqlServer()` báo cho EF Core biết loại CSDL sẽ làm việc cùng là SQL Server (vì EF Core còn hỗ trợ nhiều loại CSDL khác như MySQL, SQLite), kèm theo chuỗi kết nối cụ thể.

Bước 5 — Điều chỉnh Connection String cho đúng máy thực hành

Chuỗi kết nối ở Bước 4 chỉ là một ví dụ mẫu — trước khi chạy thử, cần kiểm tra và điều chỉnh lại cho khớp với cấu hình SQL Server thực tế đang dùng:

Nếu SQL Server cài dưới dạng instance riêng (ví dụ bản Express thường có tên SQLEXPRESS), đổi Server=localhost thành Server=localhost\SQLEXPRESS.

Tên Database=QuanLySinhVien có thể đổi tùy ý theo sở thích đặt tên — không cần tạo trước cơ sở dữ liệu này, vì Migration sẽ tự động tạo nếu nó chưa tồn tại.

Nếu SQL Server được cấu hình dùng tài khoản riêng (SQL Authentication) thay vì xác thực Windows, thay Trusted_Connection=True bằng cặp User Id=...;Password=... tương ứng.

Kết quả — toàn bộ class AppDbContext hoàn chỉnh

```
    public class AppDbContext : DbContext
    {
        public DbSet<Student> Students { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
        {
            // Sửa lại Server cho đúng máy của bạn trước khi chạy thử
            optionsBuilder.UseSqlServer(
                "Server=.;" +
                "Database=QuanLySinhVien;" +
                "Trusted_Connection=True;TrustServerCertificate=True;"
            );
        }
    }
```

5.3.5 Migration

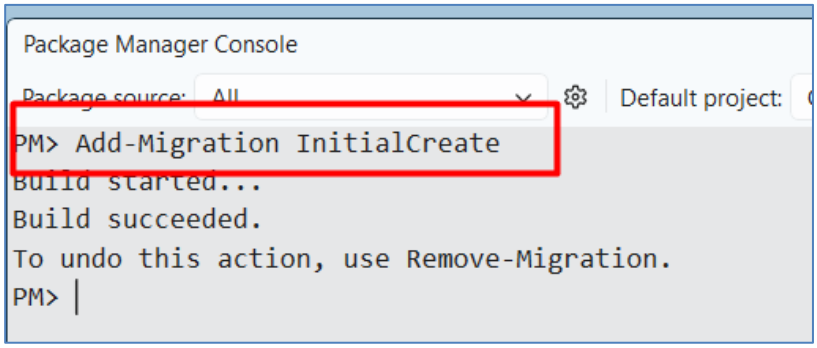
Đến đây, Entity (`Student.cs`) và DbContext (`AppDbContext.cs`) đã hoàn chỉnh trong code — nhưng cơ sở dữ liệu thật trên SQL Server **vẫn chưa hề được tạo ra**. Mọi thứ vừa viết cho đến lúc này chỉ tồn tại dưới dạng các class C# thuần túy, hoàn toàn chưa có liên hệ vật lý nào với SQL Server. **Migration** chính là cơ chế của EF Core giúp lấp đầy khoảng trống đó — tự động sinh ra cấu trúc cơ sở dữ liệu (bảng, cột, khóa chính) dựa trên Entity đã định nghĩa, đúng theo đúng tinh thần mô hình **Code First** đã trình bày.

Bước 1 — Mở Package Manager Console

Trong Visual Studio, vào menu **View → Other Windows → Package Manager Console**. Đây là nơi gõ các câu lệnh Migration, được cung cấp sẵn từ gói `Microsoft.EntityFrameworkCore.Tools` đã cài đặt— nếu cửa sổ này không gõ được lệnh hoặc báo lỗi "lệnh không tồn tại", cần kiểm tra lại đã cài đúng gói `Tools` hay chưa.

Bước 2 — Tạo Migration

Gõ lệnh "Add-Migration InitialCreate" vào Package Manager Console, rồi nhấn Enter:



Lệnh này **không** tác động trực tiếp đến cơ sở dữ liệu — nó chỉ thực hiện một việc: quét toàn bộ Entity và `DbContext` hiện có trong project, rồi sinh ra một thư mục mới tên `Migrations/` ngay trong project, chứa các file C# mô tả chi tiết cấu trúc bảng cần tạo (tên bảng, tên cột, kiểu dữ liệu tương ứng, khóa chính...). Tên `InitialCreate` chỉ là một cái tên gợi nhớ tự đặt theo ý muốn, đánh dấu đây là Migration đầu tiên của project — những lần thay đổi Entity sau này (ví dụ thêm một property mới) sẽ tạo ra các Migration tiếp theo, với một tên gọi khác phản ánh đúng thay đổi đó.

Bước 3 — Áp dụng Migration vào CSDL thật

Tiếp tục gõ lệnh thứ hai:" Update-Database"

Đây mới là lệnh thực sự **kết nối đến SQL Server** và thực thi các thay đổi đã được mô tả trong Migration vừa tạo ở Bước 2. Cụ thể, EF Core sẽ: kiểm tra xem database `QuanLySinhVien` (tên đã khai báo trong connection string) đã tồn tại hay chưa — nếu chưa, tự động tạo mới hoàn toàn; sau đó tạo bảng `Students` với đầy đủ các cột tương ứng đúng theo Entity `Student` đã định nghĩa.

Bước 4 — Kiểm tra kết quả

Mở **SQL Server Management Studio (SSMS)**, kết nối đến cùng máy chủ đã khai báo trong connection string. Nếu mọi bước trước đó thành công, sẽ thấy cấu trúc xuất hiện như sau:

Nếu thấy đúng cấu trúc này, nghĩa là toàn bộ quy trình Code First đã hoàn tất thành công — từ một class C# thuần túy, EF Core đã tự động "dịch" và tạo ra một cơ sở dữ liệu thật, sẵn sàng để thực hiện các thao tác CRUD.

Khi cần thay đổi Entity sau này

Một tình huống rất hay gặp trong thực tế: sau khi đã chạy Migration thành công, nếu cần chỉnh sửa lại Entity — ví dụ thêm property mới như Email vào class Student — cơ sở dữ liệu sẽ không tự động cập nhật theo, dù code đã thay đổi. Cần lặp lại đúng hai bước đã làm, chỉ khác tên Migration:

Add-Migration ThemCotEmail

Update-Database

EF Core đủ thông minh để so sánh cấu trúc hiện tại của CSDL với cấu trúc Entity mới, và Migration lần này sẽ chỉ chứa đúng phần thay đổi (thêm cột Email), không tạo lại toàn bộ bảng từ đầu — đây cũng là lý do Migration được xem là một "lịch sử thay đổi" có thể theo dõi được qua thời gian, mỗi lần điều chỉnh Entity tương ứng với một Migration mới được ghi lại đầy đủ trong thư mục Migrations/.

5.3.6 CRUD cơ bản trên bảng

Với cơ sở dữ liệu đã sẵn sàng, đây là lúc thực hiện bốn thao tác nền tảng nhất khi làm việc với dữ liệu — thường được gọi tắt là CRUD: Create (thêm), Read (đọc), Update (sửa), Delete (xóa). Cả bốn thao tác đều được viết dưới dạng method độc lập, áp dụng đúng cú pháp async/await đã học, và đều thao tác trên cùng một bảng Students duy nhất.

Bước 1 — Create: Viết method thêm sinh viên mới

```
public async Task ThemSinhVienAsync(string hoTen, double diem)
 {
     using (var context = new AppDbContext())
     {
         Student svMoi = new Student
         {
```

```
                FullName = hoTen,
                Grade = diem
            };

            context.Students.Add(svMoi);
            await context.SaveChangesAsync();
        }
    }
```

Quy trình của thao tác Create luôn gồm ba phần: tạo một object Entity mới (`new Student {...}`), gọi `context.Students.Add()` để **đánh dấu** object đó sẽ được thêm vào — lưu ý, ở bước này dữ liệu vẫn **chưa** thực sự được ghi xuống CSDL — và cuối cùng gọi `await context.SaveChangesAsync()`, đây mới chính là thời điểm EF Core thực sự dịch thao tác đó thành câu lệnh `INSERT` và gửi xuống SQL Server.

Bước 2 — Read: Viết method đọc dữ liệu

```
    public async Task<List<Student>> LayDanhSachAsync()
    {
        using (var context = new AppDbContext())
        {
            return await context.Students.ToListAsync();
        }
    }
```

Đây là thao tác đơn giản nhất trong bốn thao tác — chỉ cần gọi `ToListAsync()` trực tiếp trên `context.Students`, không cần thêm bất kỳ bước chuẩn bị nào khác. Có thể mở rộng thêm, kết hợp với LINQ to Entities, để lọc và sắp xếp ngay trên cơ sở dữ liệu trước khi kết quả được trả về:

```
    public async Task<List<Student>> LocSinhVienDatAsync()
    {
        using (var context = new AppDbContext())
        {
            return await context.Students
                .Where(sv => sv.Grade >= 5)
                .OrderByDescending(sv => sv.Grade)
                .ToListAsync();
        }
    }
```

Bước 3 — Update: Viết method sửa thông tin sinh viên

```
    public async Task SuaSinhVienAsync(int id, string hoTenMoi,
double diemMoi)
    {
```

```
        using (var context = new AppDbContext())
        {
            Student sv = await context.Students.FindAsync(id);

            if (sv != null)
            {
                sv.FullName = hoTenMoi;
                sv.Grade = diemMoi;
                await context.SaveChangesAsync();
            }
        }
    }
```

Thao tác Update bắt đầu bằng việc tìm đúng bản ghi cần sửa thông qua `FindAsync(id)` — method này tìm kiếm theo khóa chính, trả về `null` nếu không tìm thấy bản ghi nào khớp. Có một điểm khác biệt quan trọng so với Create: ở đây **không cần gọi lại `Add()`** — vì object `sv` vừa lấy ra đã được EF Core tự động "theo dõi" (tracking) ngay từ lúc `FindAsync()` trả về, nên chỉ cần gán lại giá trị mới cho từng property, rồi gọi `SaveChangesAsync()` để EF Core tự nhận diện những gì đã thay đổi và dịch thành câu lệnh `UPDATE` tương ứng.

Bước 4 — Delete: Viết method xóa sinh viên

```
        public async Task XoaSinhVienAsync(int id)
        {
            using (var context = new AppDbContext())
            {
                Student sv = await
context.Students.FindAsync(id);

                if (sv != null)
                {
                    context.Students.Remove(sv);
                    await context.SaveChangesAsync();
                }
            }
        }
```

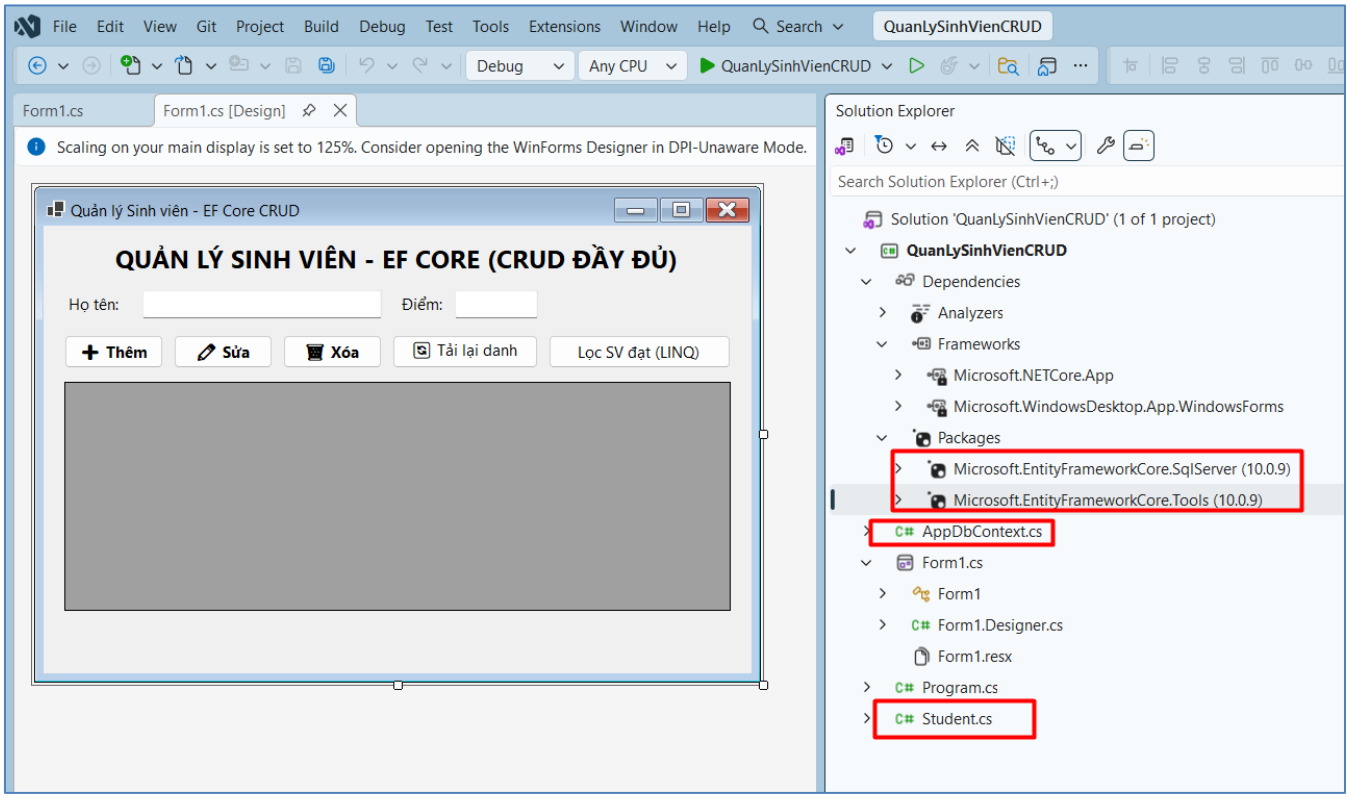
Cấu trúc gần như tương tự Update: tìm đúng bản ghi bằng `FindAsync()`, sau đó gọi `context.Students.Remove(sv)` để đánh dấu bản ghi đó sẽ bị xóa, và kết thúc bằng `SaveChangesAsync()` để EF Core thực thi câu lệnh `DELETE` thật xuống cơ sở dữ liệu.

Một quy luật dễ nhận ra xuyên suốt cả bốn thao tác: bất kỳ thao tác nào **làm thay đổi dữ liệu** thực sự (Create, Update, Delete) đều bắt buộc kết thúc bằng

`SaveChangesAsync()` — đây chính là thời điểm duy nhất EF Core thực sự giao tiếp với SQL Server để ghi thay đổi xuống cơ sở dữ liệu; còn riêng `Read`, vì bản chất chỉ đọc dữ liệu mà không làm thay đổi gì, nên không cần đến bước này.

5.3.7 Áp dụng vào ứng dụng windows forms

Đây là bước tổng hợp gộp toàn bộ bốn method CRUD đã viết vào một Form WinForms thực sự, có giao diện trực quan, để có thể tận mắt nhìn thấy và thao tác trực tiếp với một ứng dụng quản lý sinh viên kết nối CSDL thật.



Bước 1 — Thiết kế giao diện Form

Control	Name	Vai trò
TextBox	`txtHoTen`	Nhập họ tên
TextBox	`txtDiem`	Nhập điểm
Button	`btnThem`	Thực hiện Create
Button	`btnSua`	Thực hiện Update

Button	`btnXoa`	Thực hiện Delete
Button	`btnTaiLai`	Thực hiện Read (làm mới danh sách)
DataGridView	`dgvSinhVien`	Hiển thị danh sách sinh viên
Label	`lblTrangThai`	Hiển thị thông báo kết quả

Bước 2: Chuẩn bị công cụ và "khung sườn" EF Core, gồm 4 việc nhỏ theo đúng thứ tự bắt buộc:

Cài đặt EF Core

Tạo Entity (Student.cs)

Tạo DbContext (AppDbContext.cs)

Chạy Migration (Add-Migration + Update-Database)

Bước 3 — Viết CRUD và áp dụng vào Form WinForms

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private async void Form1_Load(object sender, EventArgs e)
    {
        lblTrangThai.Text = "Sẵn sàng. Hãy chạy Migration trước khi dùng (xem README.md).";
        await TaiDanhSach();
    }

    // ===== READ =====
    private async Task TaiDanhSach()
    {
        try
        {
            using (var context = new AppDbContext())
            {
                List<Student> danhSach = await context.Students.ToListAsync();
                dgvSinhVien.DataSource = null;
                dgvSinhVien.DataSource = danhSach;
                lblTrangThai.Text = $"Đã tải {danhSach.Count} sinh viên.";
                lblTrangThai.ForeColor = System.Drawing.Color.DarkGreen;
            }
        }
        catch (Exception ex)
        {
            lblTrangThai.Text = "Lỗi: " + ex.Message;
            lblTrangThai.ForeColor = System.Drawing.Color.Red;
```

```
        }
    }

    private async void btnTaiLai_Click(object sender, EventArgs e)
    {
        await TaiDanhSach();
    }

    // ===== Chọn dòng -> hiển thị lên ô nhập liệu =====
    private void dgvSinhVien_SelectionChanged(object sender, EventArgs e)
    {
        if (dgvSinhVien.CurrentRow?.DataBoundItem is Student sv)
        {
            txtHoTen.Text = sv.FullName;
            txtDiem.Text = sv.Grade.ToString();
        }
    }

    // ===== CREATE =====
    private async void btnThem_Click(object sender, EventArgs e)
    {
        if (string.IsNullOrWhiteSpace(txtHoTen.Text))
        {
            MessageBox.Show("Vui lòng nhập họ tên!");
            return;
        }

        if (!double.TryParse(txtDiem.Text, out double diem))
        {
            MessageBox.Show("Điểm không hợp lệ!");
            return;
        }

        try
        {
            using (var context = new AppDbContext())
            {
                context.Students.Add(new Student { FullName = txtHoTen.Text, Grade = diem });
                await context.SaveChangesAsync();
            }

            lblTrangThai.Text = "Đã thêm sinh viên mới!";
            lblTrangThai.ForeColor = System.Drawing.Color.DarkGreen;
            txtHoTen.Clear();
            txtDiem.Clear();
            await TaiDanhSach();
        }
        catch (Exception ex)
        {
            lblTrangThai.Text = "Lỗi: " + ex.Message;
            lblTrangThai.ForeColor = System.Drawing.Color.Red;
        }
    }

    // ===== UPDATE =====
    private async void btnSua_Click(object sender, EventArgs e)
    {
        if (dgvSinhVien.CurrentRow?.DataBoundItem is not Student svDangChon)
        {
            MessageBox.Show("Vui lòng chọn một dòng để sửa!");
            return;
        }

        if (string.IsNullOrWhiteSpace(txtHoTen.Text))
        {
```

```
            MessageBox.Show("Vui lòng nhập họ tên!");
            return;
        }

        if (!double.TryParse(txtDiem.Text, out double diemMoi))
        {
            MessageBox.Show("Điểm không hợp lệ!");
            return;
        }

        try
        {
            using (var context = new AppDbContext())
            {
                Student sv = await context.Students.FindAsync(svDangChon.Id);
                if (sv != null)
                {
                    sv.FullName = txtHoTen.Text;
                    sv.Grade = diemMoi;
                    await context.SaveChangesAsync();
                }
            }

            lblTrangThai.Text = "Đã cập nhật thông tin!";
            lblTrangThai.ForeColor = System.Drawing.Color.DarkGreen;
            await TaiDanhSach();
        }
        catch (Exception ex)
        {
            lblTrangThai.Text = "Lỗi: " + ex.Message;
            lblTrangThai.ForeColor = System.Drawing.Color.Red;
        }
    }

    // ===== DELETE =====
    private async void btnXoa_Click(object sender, EventArgs e)
    {
        if (dgvSinhVien.CurrentRow?.DataBoundItem is not Student svDangChon)
        {
            MessageBox.Show("Vui lòng chọn một dòng để xóa!");
            return;
        }

        DialogResult ketQua = MessageBox.Show(
            $"Bạn có chắc muốn xóa \"{svDangChon.FullName}\"?",
            "Xác nhận xóa",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question
        );

        if (ketQua != DialogResult.Yes) return;

        try
        {
            using (var context = new AppDbContext())
            {
                Student sv = await context.Students.FindAsync(svDangChon.Id);
                if (sv != null)
                {
                    context.Students.Remove(sv);
                    await context.SaveChangesAsync();
                }
            }

            lblTrangThai.Text = "Đã xóa sinh viên!";
```

```
            lblTrangThai.ForeColor = System.Drawing.Color.DarkGreen;
            txtHoTen.Clear();
            txtDiem.Clear();
            await TaiDanhSach();
        }
        catch (Exception ex)
        {
            lblTrangThai.Text = "Lỗi: " + ex.Message;
            lblTrangThai.ForeColor = System.Drawing.Color.Red;
        }
    }

    // ===== LINQ to Entities: Lọc sinh viên đạt =====
    private async void btnLocDat_Click(object sender, EventArgs e)
    {
        try
        {
            using (var context = new AppDbContext())
            {
                List<Student> ketQua = await context.Students
                    .Where(sv => sv.Grade >= 5)
                    .OrderByDescending(sv => sv.Grade)
                    .ToListAsync();

                dgvSinhVien.DataSource = null;
                dgvSinhVien.DataSource = ketQua;
                lblTrangThai.Text = $"Tìm thấy {ketQua.Count} sinh viên đạt (>=5).";
                lblTrangThai.ForeColor = System.Drawing.Color.DarkGreen;
            }
        }
        catch (Exception ex)
        {
            lblTrangThai.Text = "Lỗi: " + ex.Message;
            lblTrangThai.ForeColor = System.Drawing.Color.Red;
        }
    }
}
```

Chạy thử và kiểm tra (F5)

Sau khi viết đủ 5 phần trên, nhấn F5 chạy ứng dụng và kiểm tra lần lượt: thêm một sinh viên mới, xem có xuất hiện trên DataGridView không; chọn một dòng, sửa thông tin, bấm Sửa, xem dữ liệu có cập nhật đúng không; chọn một dòng, bấm Xóa, xác nhận, xem dòng đó có biến mất không. Nếu cả ba thao tác đều hoạt động đúng, có thể mở SSMS để xác nhận thêm: dữ liệu hiển thị trên Form khớp chính xác với dữ liệu thật đang nằm trong bảng Students trên SQL Server — minh chứng rõ ràng nhất rằng ứng dụng đã thực sự kết nối và làm việc với cơ sở dữ liệu thật, không chỉ là dữ liệu giả lập trong bộ nhớ như các bài học trước.

TÓM TẮT

Trong bài học này, ứng dụng được chuyển từ lưu dữ liệu tạm thời bằng List<T> sang lưu trữ lâu dài trên SQL Server, tạo nền tảng để xây dựng các ứng dụng quản lý thực tế.

Học viên đã tìm hiểu lập trình bất đồng bộ với async/await nhằm tránh hiện tượng UI Freeze khi làm việc với cơ sở dữ liệu. Đồng thời, bài học giới thiệu hai cách truy cập dữ liệu là ADO.NET và Entity Framework Core (EF Core), trong đó EF Core được sử dụng nhờ khả năng thao tác dữ liệu thông qua các đối tượng C# và LINQ.

Cuối cùng, học viên đã thực hành quy trình Code First với EF Core: tạo Entity, DbContext, thực hiện Migration và xây dựng đầy đủ các thao tác CRUD (Create, Read, Update, Delete) trong ứng dụng Windows Forms kết nối với SQL Server.

CÂU HỎI ÔN TẬP

Câu 1: Phân biệt lập trình đồng bộ và bất đồng bộ. Giải thích vì sao một ứng dụng WinForms có thể bị "đứng hình" (UI freeze) nếu thực hiện một thao tác tốn thời gian theo kiểu đồng bộ ngay trong sự kiện Click của Button.

Câu 2: Trình bày cú pháp async/await trong C#. Vì sao method xử lý sự kiện (event handler) trong WinForms phải dùng async void thay vì async Task như các method bất đồng bộ thông thường?

Câu 3: Trình bày các bước cơ bản để đọc dữ liệu bằng ADO.NET (`SqlConnection`, `SqlCommand`, `SqlDataReader`). Nêu ít nhất 3 hạn chế của cách làm này so với EF Core.

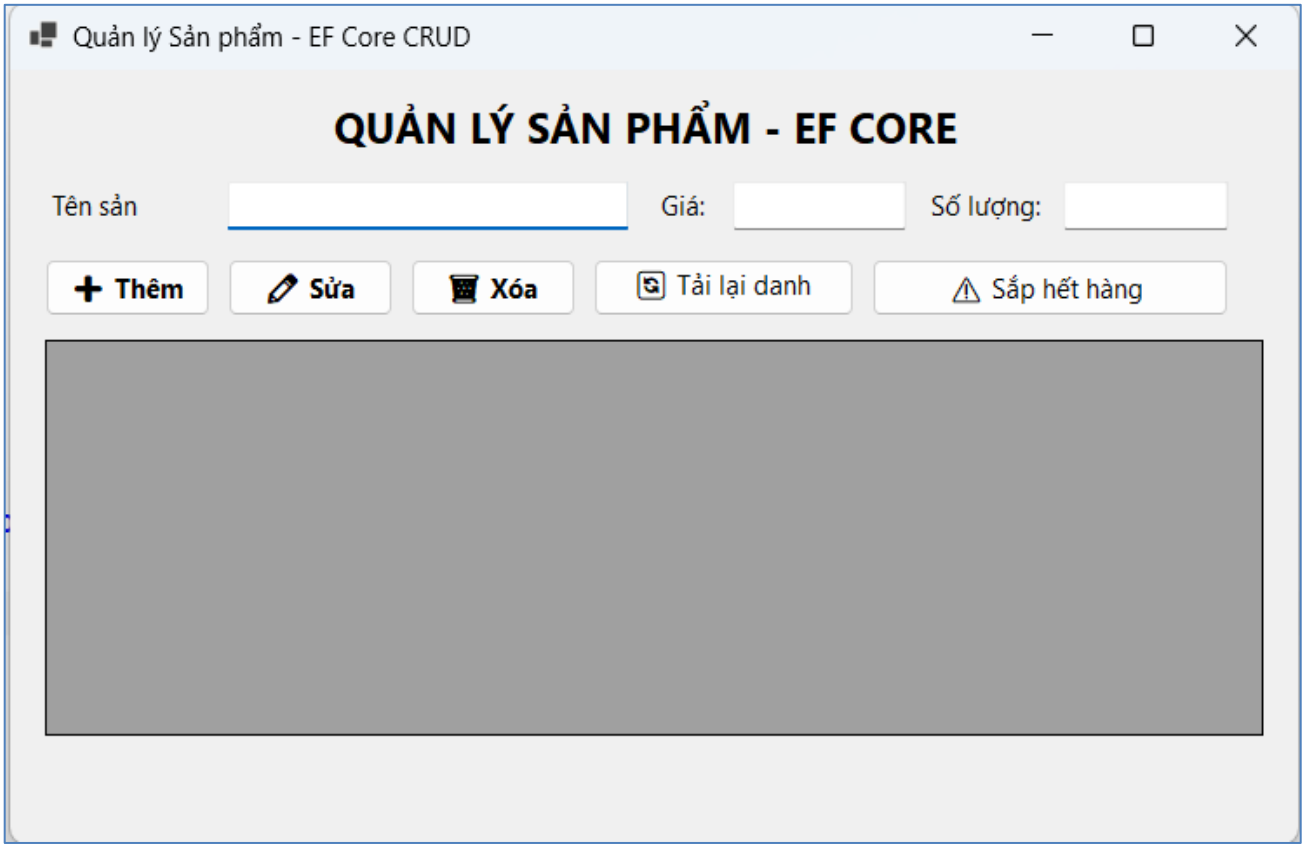
Câu 4: EF Core là gì? Giải thích khái niệm ORM (Object-Relational Mapping) và vai trò của nó trong việc kết nối ứng dụng C# với cơ sở dữ liệu quan hệ.

Câu 5: Phân biệt hai mô hình **Code First** và **Database First**. Vì sao môn học này lựa chọn Code First làm hướng tiếp cận chính?

Câu 6: Trình bày vai trò của `DbContext` và `DbSet<T>` trong EF Core. Cho ví dụ minh họa bằng một class `AppDbContext` đơn giản với một `DbSet`.

Câu 7: Migration là gì? Trình bày đúng thứ tự các bước cần thực hiện để chạy Migration lần đầu tiên cho một project EF Core mới (từ lúc chỉ có Entity cho đến khi có CSDL thật trên SQL Server).

Câu 8: Xây dựng ứng dụng WinForms quản lý sản phẩm trong kho, kết nối CSDL thật qua EF Core (Code First), đáp ứng đầy đủ:



- Entity Product (Id, Name bắt buộc, Price, Quantity)
- CRUD đầy đủ: Thêm, Sửa, Xóa, Xem danh sách
- Dùng LINQ to Entities: tìm sản phẩm sắp hết hàng (Quantity < 10)
- Validate dữ liệu đầu vào, dùng async/await cho mọi thao tác CSDL

BÀI 6: TRUY VẤN VÀ QUẢN LÝ DỮ LIỆU VỚI ENTITY FRAMEWORK CORE

Sau khi học xong bài này, học viên có thể:

- *Xây dựng và quản lý các mối quan hệ giữa các Entity.*
- *Truy vấn dữ liệu có quan hệ bằng Include và LINQ.*
- *Thực hiện các thao tác Transaction để đảm bảo tính nhất quán của dữ liệu.*
- *Đồng bộ dữ liệu giữa DataGridView và nguồn dữ liệu bằng **BindingSource**.*
- *Kiểm tra tính hợp lệ của dữ liệu trước khi lưu vào cơ sở dữ liệu.*

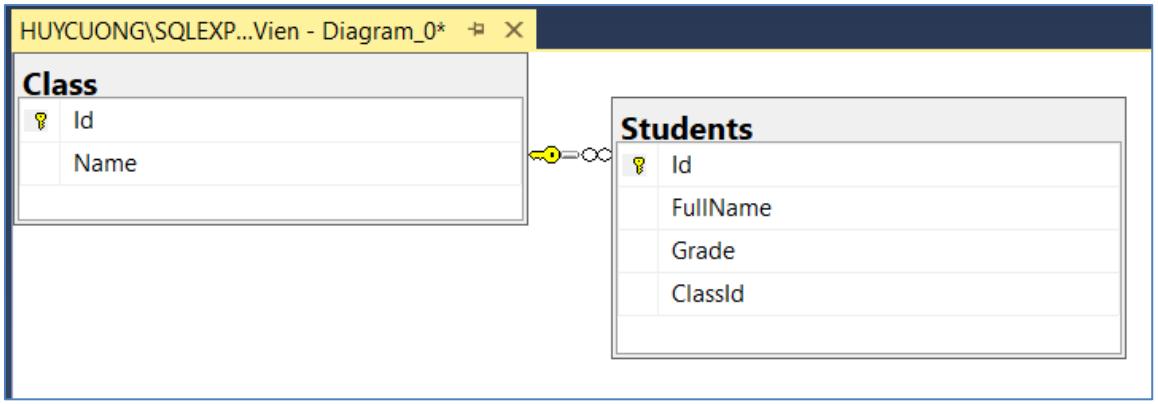
6.1 Quan hệ giữa các bảng trong EF Core

Entity Framework (EF) là một framework ánh xạ quan hệ đối tượng (ORM) dành cho ADO.NET, là 1 phần của .NET Framework. EF cho phép các nhà phát triển tương tác với cơ sở dữ liệu quan hệ theo phương pháp hướng đối tượng. Lợi ích lớn nhất của EF là giúp lập trình viên giảm thiểu việc lập trình mã nguồn cần thiết để truy cập và tương tác với cơ sở dữ liệu. EF được Microsoft hỗ trợ phát triển lâu dài và bền vững, bằng cách sử dụng Entity Framework, truy vấn LINQ, thì việc truy xuất và thao tác với cơ sở dữ liệu cũng như các đối tượng trở nên mạnh mẽ hơn.

6.1.1 Quan hệ 1-n (One-to-Many)

Quan hệ 1-n (One-to-Many) là loại quan hệ phổ biến nhất trong cơ sở dữ liệu quan hệ: một bản ghi ở bảng này có thể liên kết với nhiều bản ghi ở bảng kia, nhưng mỗi bản ghi ở bảng kia chỉ liên kết với đúng một bản ghi ở bảng đầu. Ví dụ tiêu biểu,

dùng xuyên suốt bài học này: **một lớp học có nhiều sinh viên, mỗi sinh viên chỉ thuộc về đúng một lớp**.



Cột `ClassId` trong bảng `Student` là **khóa ngoại (Foreign Key)** — chứa giá trị trỏ về khóa chính (`Id`) của bảng `Class`, thể hiện sinh viên đó thuộc lớp nào.

EF Core quản lý quan hệ giữa các Entity thông qua **Navigation Property** — một property không tương ứng với cột dữ liệu nào, mà đại diện cho mối liên kết đến Entity khác:

```
public class Student
 {
     public int Id { get; set; }
     public required string FullName { get; set; }
     public double Grade { get; set; }

     public override string ToString()
     {
         return $"{Id} - {FullName} - {Grade}";
     }
     public Class? Class { get; set; }
 }
 public class Class
 {
     public int Id { get; set; }
     public required string Name { get; set; }

     public List<Student> Students { get; set; }
 }
```

Bốn thành phần mới trong cặp Entity trên:

Thành phần	Vị trí	Vai trò
`ClassId`	**`Student`**	Khóa ngoại — tương ứng cột thật trong bảng `Students`
`Class? Class`	**`Student`**	Navigation Property phía "n" — truy cập object lớp mà sinh viên thuộc về
`List<Student> Students`	**`Class`**	Navigation Property phía "1" — truy cập toàn bộ sinh viên thuộc lớp
`Id`	**`Class`**	Khóa chính, đích đến của `ClassId`

EF Core tự nhận diện quy ước này (property **`ClassId`** đi kèm Navigation Property **`Class`** cùng gốc tên) mà không cần khai báo gì thêm — chỉ cần viết Entity đúng cấu trúc, Migration sẽ tự tạo khóa ngoại tương ứng

Bước 1 — Khai báo thêm DbSet cho Class trong DbContext

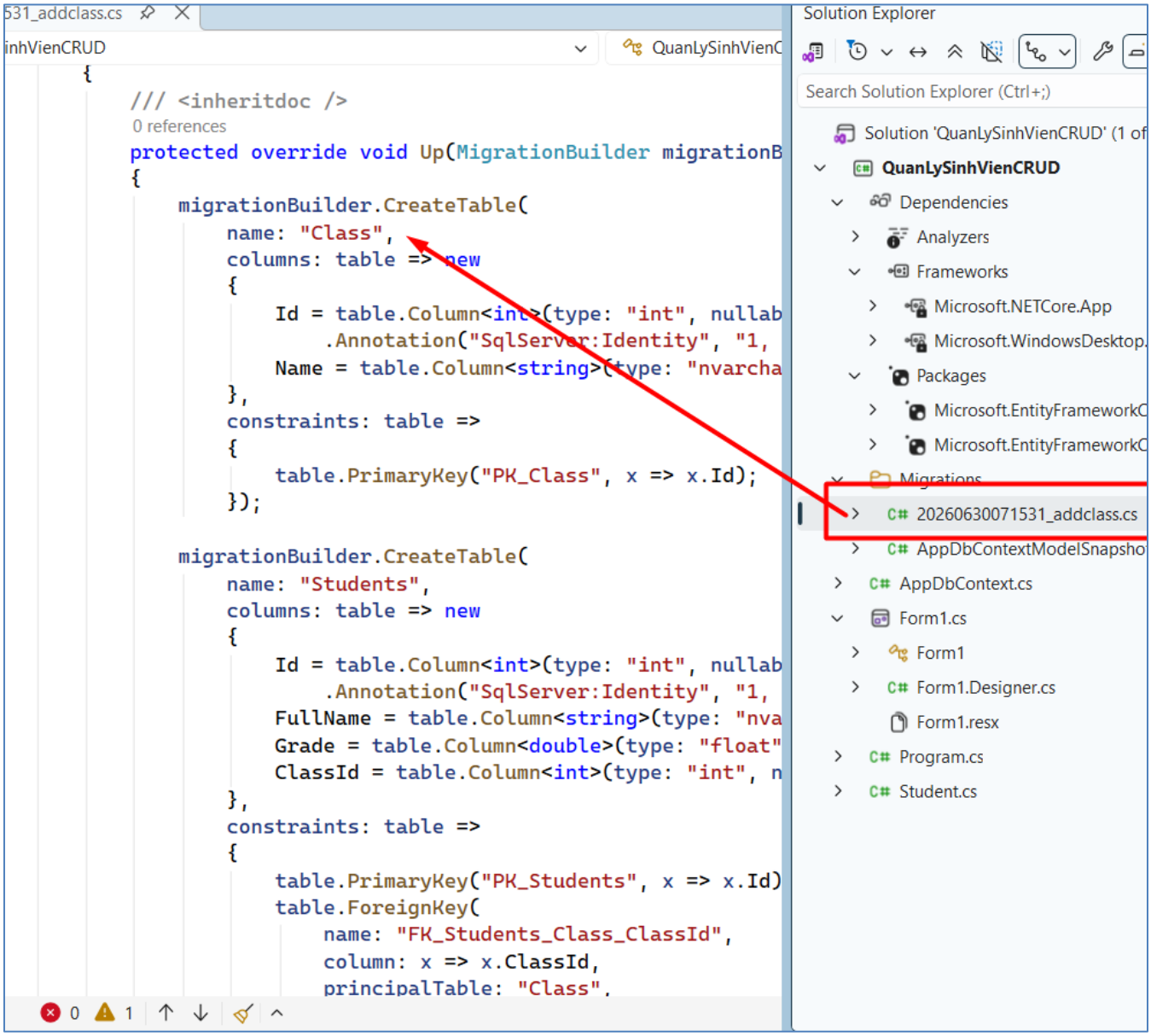
```
public class AppDbContext : DbContext
{
    6 references
    public DbSet<Student> Students { get; set; }
    0 references
    public DbSet<Class> Classes { get; set; }   // Thêm dòng này

    0 references
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        // Sửa lại Server cho đúng máy của bạn trước khi chạy thử
        optionsBuilder.UseSqlServer(
            "Server=localhost;Database=QuanLySinhVien;Trusted_Connection=True;TrustServerCertificate=True;"
        );
    }
}
```

Bước 2 — Tạo Migration mới, Áp dụng Migration vào CSDL thật

Vì Entity `Student` đã thay đổi (thêm `ClassId`, `Class`) và có thêm Entity `Class` hoàn toàn mới, EF Core sẽ quét và sinh ra Migration mô tả đầy đủ: tạo bảng `Classes`, thêm cột `ClassId` vào bảng `Students`, và tạo **ràng buộc khóa ngoại (Foreign Key Constraint)** liên kết hai bảng.

```
PM> add-migration addclass
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Acquiring an exclusive lock for migration application. See ht
Applying migration '20260630071531_addclass'.
```



Cột ClassId trong bảng Students giờ xuất hiện kèm ký hiệu FK (Foreign Key) trong SSMS — xác nhận EF Core đã tự động tạo đúng ràng buộc khóa ngoại liên kết hai bảng, đúng như đã khai báo bằng Navigation Property trong code.

Lưu ý quan trọng — dữ liệu cũ và khóa ngoại bắt buộc

Nếu bảng Students đã có sẵn dữ liệu (trước khi thêm quan hệ này), Update-Database có thể báo lỗi — vì cột ClassId mới thêm vào yêu cầu giá trị, nhưng các dòng dữ liệu cũ chưa có giá trị nào cho cột đó. Hai cách xử lý:

- Xóa toàn bộ dữ liệu cũ trong bảng Students trước khi Update-Database (phù hợp khi đang ở giai đoạn thực hành, dữ liệu chỉ là dữ liệu thử nghiệm).
- Cho phép ClassId nhận null tạm thời, bằng cách đổi kiểu khai báo trong Entity:

```
public int? ClassId { get; set; } // Thêm dấu ? để cho phép null
```

6.1.2 Quan hệ n-n (Many-to-Many)

Quan hệ n-n (Many-to-Many) là loại quan hệ mà nhiều bản ghi ở bảng này có thể liên kết với nhiều bản ghi ở bảng kia, và ngược lại. Ví dụ tiêu biểu: một sinh viên có thể đăng ký nhiều môn học, và một môn học có thể có nhiều sinh viên đăng ký — khác hẳn quan hệ 1-n, nơi mỗi sinh viên chỉ thuộc về đúng một lớp.

Khai báo quan hệ n-n trong EF Core

Khác với quan hệ 1-n (cần khóa ngoại trực tiếp trong Entity), EF Core hiện đại hỗ trợ khai báo n-n theo cách rất gọn — chỉ cần khai báo **Navigation Property dạng danh sách ở cả hai phía**, không cần tự định nghĩa thêm Entity trung gian:

```
public class Student
 {
     public int Id { get; set; }
     public required string FullName { get; set; }
     public double Grade { get; set; }
     public List<Course> Courses { get; set; } = new List<Course>();
 }
 public class Course
 {
     public int Id { get; set; }
     public required string Name { get; set; }

     public List<Student> Students { get; set; } = new List<Student>();
 }
```

Chỉ với hai property dạng `List<T>` này — `Student.Courses` và `Course.Students` — EF Core tự nhận diện đây là quan hệ n-n, và khi chạy Migration, sẽ tự động sinh ra một

bảng trung gian (Junction Table) ẩn phía sau, thường có tên dạng `CourseStudent`, chỉ chứa hai khóa ngoại trỏ về `Course.Id` và `Student.Id` — không cần tự tay viết Entity cho bảng trung gian này.

6.2 Truy vấn dữ liệu có quan hệ

6.2.1 Include

Dù đã khai báo Navigation Property, một vấn đề quan trọng cần lưu ý: EF Core không tự động nạp dữ liệu bảng liên quan mỗi khi truy vấn, trừ khi được yêu cầu rõ ràng. Đây là một cơ chế tối ưu hiệu năng có chủ đích — nếu chỉ cần lấy danh sách sinh viên để hiển thị tên và điểm, việc tự động kéo theo toàn bộ thông tin lớp học của từng sinh viên (khi không thực sự cần) sẽ gây lãng phí tài nguyên không cần thiết.

Vì vậy, nếu chỉ viết:

```
  Student sv = await context.Students.FindAsync(1);
  Console.WriteLine(sv.Class.Name);   // Lỗi NullReferenceException!
sv.Class đang là null
```

Dòng sv.Class.Name sẽ gây lỗi, dù sinh viên này thực sự có lớp trong cơ sở dữ liệu — vì EF Core chỉ lấy đúng dữ liệu của bảng Students, không tự "đi" lấy thêm dữ liệu bảng Classes liên quan.

`Include()` là phương thức giải quyết đúng vấn đề này — yêu cầu EF Core nạp kèm dữ liệu của bảng liên quan ngay trong cùng một truy vấn:

```
Student sv = await context.Students
                  .Include(s => s.Class)
                  .FirstAsync(s => s.Id == 1);

 Console.WriteLine(sv.Class.Name);  // Hoạt động đúng – sv.Class đã
được nạp kèm
```

Tham số truyền vào `Include()` là một lambda expression, chỉ rõ Navigation Property nào cần nạp kèm — ở đây là `s => s.Class`, nghĩa là "nạp kèm Navigation Property `Class` của mỗi `Student`".

Áp dụng Include() khi đọc cả danh sách

```
List<Student> danhSach = await context.Students.Include(s =>
s.Class).ToListAsync();
 foreach (Student sv in danhSach)
```

```
{
    Console.WriteLine($"{sv.FullName} - Lớp: {sv.Class.Name}");
}
```

Nếu bỏ `.Include(s => s.Class)` ở đây, vòng lặp `foreach` phía dưới sẽ gây lỗi `NullReferenceException` ngay tại `sv.Class.Name`, vì `Class` của từng sinh viên vẫn đang là `null`.

ThenInclude() — nạp kèm nhiều cấp

Trong tình huống cần nạp kèm dữ liệu xuyên qua hai cấp quan hệ liên tiếp — ví dụ: lấy danh sách lớp học, kèm theo danh sách sinh viên của từng lớp, và với mỗi sinh viên đó, kèm theo luôn danh sách môn học mà sinh viên đó đã đăng ký — cần nối thêm ThenInclude() ngay sau Include():

```
List<Class> danhSachLop = await context.Classes
    .Include(c => c.Students)
    .ThenInclude(sv => sv.Courses).ToListAsync();
```

`Include(c => c.Students)` nạp kèm danh sách sinh viên của mỗi lớp (cấp 1); `ThenInclude(sv => sv.Courses)` nạp kèm tiếp danh sách môn học của từng sinh viên đó (cấp 2) — tạo thành một chuỗi nạp dữ liệu xuyên suốt nhiều bảng có quan hệ với nhau, chỉ trong một truy vấn duy nhất.

6.2.2 LINQ to Entities nâng cao — GroupBy, Join

Với dữ liệu đã có quan hệ giữa các bảng, LINQ to Entities phát huy thêm nhiều khả năng mạnh mẽ hơn — đặc biệt là **GroupBy** (nhóm dữ liệu) và **Join** (kết hợp dữ liệu từ nhiều bảng), hai phương thức rất hữu ích khi cần tổng hợp, thống kê dữ liệu thực tế.

GroupBy — nhóm dữ liệu theo một tiêu chí

GroupBy gom các bản ghi có chung một giá trị nào đó lại thành từng nhóm, tương tự khái niệm "nhóm theo" quen thuộc trong Excel (Pivot Table) hay câu lệnh GROUP BY trong SQL.

Ví dụ: Đếm số lượng sinh viên trong mỗi lớp học.

```
var ketQua = await context.Students
    .GroupBy(sv => sv.ClassId)
    .Select(nhom => new
    {
```

```
            ClassId = nhom.Key,
            SoLuongSinhVien = nhom.Count()
        })
        .ToListAsync();
foreach (var item in ketQua)
{
    Console.WriteLine($"Lớp {item.ClassId}:{item.SoLuongSinhVien} sinh
viên");
}
```

Phân tích cú pháp: GroupBy(sv => sv.ClassId) gom toàn bộ sinh viên thành từng nhóm theo ClassId giống nhau; mỗi nhom trong Select là một tập hợp các sinh viên cùng lớp, với nhom.Key chính là giá trị ClassId của nhóm đó, và nhom.Count() đếm số phần tử trong nhóm. Kết quả trả về là một danh sách các object ẩn danh (new { ... }), mỗi object chứa ClassId và số lượng sinh viên tương ứng.

Có thể mở rộng thêm, tính cả điểm trung bình của mỗi lớp trong cùng một truy vấn:

```
var thongKe = await context.Students
                           .GroupBy(sv => sv.ClassId)
                           .Select(nhom => new
                           {
                               ClassId = nhom.Key,
                               SoLuong = nhom.Count(),
                               DiemTrungBinh = nhom.Average(sv => sv.Grade)
                           })
                           .ToListAsync();
```

Bổ sung: CountBy() — cách viết gọn hơn GroupBy() khi chỉ cần đếm

Với nhu cầu đơn giản — chỉ đếm số lượng sinh viên theo từng lớp, không cần thêm phép tính nào khác — CountBy() (có từ .NET 9) là lựa chọn gọn và hiệu quả hơn GroupBy(), vì nó không cần tạo ra các nhóm (group) trung gian rồi mới đếm, mà đếm trực tiếp:

```
var ketQua = await context.Students.CountBy(sv =>
sv.ClassId).ToListAsync();

foreach (var item in ketQua)
{
    Console.WriteLine($"Lớp {item.Key}: {item.Value} sinh viên");
}
```

Join — kết hợp dữ liệu từ nhiều bảng

`Join` cho phép kết hợp dữ liệu từ hai nguồn (hai `DbSet`) dựa trên một điều kiện khớp giữa chúng — tương tự `JOIN` trong SQL. Trong thực tế, khi đã có Navigation Property kết hợp với `Include()`, nhu cầu dùng `Join` trực tiếp giảm đi nhiều — vì EF Core đã tự động xử lý việc kết hợp dữ liệu thông qua quan hệ đã khai báo. Tuy vậy, `Join` vẫn hữu ích trong một số tình huống cần kết hợp dữ liệu theo một tiêu chí **không phải** là quan hệ khóa ngoại đã định nghĩa sẵn.

```
var ketQua = await context.Students
                .Join(
                    context.Classes,
                    sv => sv.ClassId,
                    lop => lop.Id,
                    (sv, lop) => new
                    {
                        TenSinhVien = sv.FullName,
                        TenLop = lop.Name
                    }
                ).ToListAsync();
```

Bốn tham số truyền vào `Join` lần lượt là: nguồn dữ liệu thứ hai cần kết hợp (`context.Classes`), khóa để so khớp từ nguồn đầu tiên (`sv => sv.ClassId`), khóa để so khớp từ nguồn thứ hai (`lop => lop.Id`), và cách kết hợp kết quả từ cả hai khi khớp nhau (`(sv, lop) => new {...}`). Kết quả của `Join` cũng là tập hợp các object ẩn danh, kết hợp dữ liệu từ cả hai bảng theo đúng điều kiện đã chỉ định.

6.3 Transaction — Đảm bảo tính toàn vẹn dữ liệu

6.3.1 Vấn đề: nhiều thao tác phải cùng thành công hoặc cùng thất bại

Hãy hình dung một tình huống nghiệp vụ cụ thể: khi xóa một Class (lớp học), nghiệp vụ yêu cầu đồng thời cập nhật lại toàn bộ sinh viên thuộc lớp đó về trạng thái "chưa có lớp" (ClassId = null), rồi mới thực sự xóa bản ghi Class. Đây là hai thao tác CSDL riêng biệt, nhưng về mặt nghiệp vụ, chúng phải được xem là một khối thống nhất: hoặc cả hai cùng thành công, hoặc nếu có bất kỳ lỗi nào xảy ra ở giữa chừng (mất kết nối mạng, lỗi ràng buộc dữ liệu...), cả hai đều phải được hủy bỏ hoàn toàn, đưa dữ liệu trở lại đúng trạng thái ban đầu trước khi thao tác bắt đầu.

Nếu không đảm bảo được tính chất này, có thể xảy ra tình huống nguy hiểm: sinh viên đã được cập nhật thành "chưa có lớp", nhưng việc xóa Class lại thất bại giữa chừng — kết quả là dữ liệu rơi vào trạng thái không nhất quán, vừa không đúng với trạng thái cũ, vừa không đúng với trạng thái mới mong muốn.

Transaction (giao dịch) là cơ chế của cơ sở dữ liệu giải quyết chính xác vấn đề này — đảm bảo một nhóm thao tác được xem là một đơn vị không thể chia nhỏ: hoặc toàn bộ các thao tác trong đó cùng được ghi nhận (gọi là Commit), hoặc nếu có lỗi xảy ra, toàn bộ thao tác đã thực hiện trước đó trong giao dịch sẽ được hủy bỏ và khôi phục lại (gọi là Rollback), như chưa từng có gì xảy ra.

6.3.2 SaveChangesAsync()

Một điều quan trọng cần biết trước khi đi sâu hơn: **mỗi lần gọi `SaveChangesAsync()`, EF Core đã tự động bọc toàn bộ thay đổi đang chờ xử lý (pending changes) trong một Transaction ngầm**, dù không hề khai báo Transaction một cách tường minh trong code.

```
using (var context = new AppDbContext())
{
     Student sv1 = new Student { FullName = "An", Grade = 8.5, ClassId = 1 };
     Student sv2 = new Student { FullName = "Bình", Grade = 7.0, ClassId = 1
};

     context.Students.Add(sv1);
     context.Students.Add(sv2);

     await context.SaveChangesAsync();// Transaction ngầm bao trùm CẢ HAI lần
Add() này
 }
```

Trong ví dụ trên, dù có hai lần gọi `Add()` riêng biệt, cả hai chỉ thực sự được ghi xuống CSDL khi `SaveChangesAsync()` được gọi — và bản thân lệnh `SaveChangesAsync()` này tự đảm bảo: nếu việc thêm `sv2` gặp lỗi (ví dụ vi phạm ràng buộc dữ liệu nào đó), thì việc thêm `sv1` cũng sẽ **tự động bị hủy theo**, không để xảy ra tình huống chỉ thêm được một trong hai. Đây chính là Transaction ngầm mà EF Core tự quản lý, không cần can thiệp thêm.

6.3.3 Transaction khi cần nhiều SaveChangesAsync

Transaction ngầm trên chỉ bao trùm được **trong phạm vi một lần gọi `SaveChangesAsync()`**. Nhưng quay lại tình huống đầu mục — cập nhật `ClassId` của sinh viên rồi mới xóa `Class` — nếu viết tự nhiên thành hai bước riêng biệt, mỗi bước gọi `SaveChangesAsync()` của chính nó, Transaction ngầm sẽ **không** còn bao trùm được cả hai bước đó cùng lúc nữa:

```
  using (var context = new AppDbContext())
  {
      // Bước 1 – cập nhật sinh viên
      List<Student> dsSinhVien = await context.Students
          .Where(sv => sv.ClassId == idLopCanXoa)
          .ToListAsync();

      foreach (Student sv in dsSinhVien)
      {
          sv.ClassId = null;
      }
      await context.SaveChangesAsync();   // Transaction ngầm #1 – chỉ riêng bước này

      // Bước 2 – xóa lớp
      Class lop = await context.Classes.FindAsync(idLopCanXoa);
      context.Classes.Remove(lop);
      await context.SaveChangesAsync();   // Transaction ngầm #2 – tách biệt với bước
trên
  }
```

Nếu Bước 2 gặp lỗi, Bước 1 đã SaveChangesAsync() thành công từ trước đó sẽ không tự động được hủy — dữ liệu rơi đúng vào tình trạng không nhất quán.

Để gộp nhiều lần SaveChangesAsync() vào chung một Transaction thực sự, cần khai báo Transaction một cách tường minh, thông qua **context.Database.BeginTransactionAsync()**:

```
using (var context = new AppDbContext())
using (var transaction = await context.Database.BeginTransactionAsync())
{
    try
    {
        List<Student> dsSinhVien = await context.Students
            .Where(sv => sv.ClassId == idLopCanXoa)
            .ToListAsync();

        foreach (Student sv in dsSinhVien)
        {
            sv.ClassId = null;
        }
        await context.SaveChangesAsync();

        Class lop = await context.Classes.FindAsync(idLopCanXoa);
        context.Classes.Remove(lop);
        await context.SaveChangesAsync();

        await transaction.CommitAsync();   // Cả hai bước cùng được xác nhận
```

```
    }
    catch (Exception)
    {
        await transaction.RollbackAsync();   // Có lỗi – hủy bỏ TOÀN BỘ, kể cả bước đã
SaveChangesAsync trước đó
        throw;
    }
}
```

Cấu trúc `try-catch` ở đây đóng vai trò quyết định: nếu mọi thao tác bên trong khối `try` đều thành công, `transaction.CommitAsync()` xác nhận toàn bộ thay đổi là vĩnh viễn; nếu có bất kỳ ngoại lệ nào xảy ra ở bất kỳ đâu trong khối đó, khối `catch` sẽ gọi `transaction.RollbackAsync()`, hủy bỏ **toàn bộ** thay đổi đã thực hiện trong cả hai lần `SaveChangesAsync()` trước đó — đưa cơ sở dữ liệu trở về đúng trạng thái như trước khi Transaction bắt đầu.

6.4 BindingSource

6.4.1 Vấn đề: DataGridView không tự cập nhật khi List thay đổi

Mỗi khi cần cập nhật lại nội dung hiển thị trên `DataGridView` sau một thao tác CRUD, phải lặp lại đúng khuôn mẫu sau:

```
dgvSinhVien.DataSource = null;
dgvSinhVien.DataSource = ketQua;
```

Cách làm này hoạt động nhưng có nhược điểm là phải **lặp lại sau mỗi thao tác CRUD**, khiến mã nguồn dài và khó bảo trì. Để giải quyết vấn đề này, Windows Forms cung cấp **BindingSource**, một lớp trung gian giúp tự động đồng bộ dữ liệu giữa nguồn dữ liệu và các Control hiển thị như **DataGridView**.

6.4.2 BindingSource

BindingSource là một Control trung gian, đóng vai trò "cầu nối" giữa nguồn dữ liệu (`List<T>`) và Control hiển thị (`DataGridView`, `ComboBox`...). Thay vì gán `DataSource` trực tiếp lên `DataGridView`, dữ liệu được gán vào `BindingSource` trước, rồi `DataGridView` lấy dữ liệu thông qua `BindingSource` đó:

List<T> → BindingSource → DataGridView

Lợi ích: khi dữ liệu trong `BindingSource` thay đổi, nó tự động **thông báo** cho mọi Control đang gắn với nó (gọi là cơ chế **data binding hai chiều**) biết để tự cập nhật lại hiển thị — không cần phải tự tay gọi `DataSource = null` rồi gán lại như trước.

Thêm BindingSource vào Form

Từ Toolbox, nhóm **Data**, kéo `BindingSource` vào Form — control này không hiển thị trực tiếp trên giao diện (nằm ở khu vực "tray" phía dưới Designer, giống `ContextMenuStrip` hay `Timer`).



Gán dữ liệu vào BindingSource, rồi gán BindingSource cho DataGridView

```
    private List<Student> danhSachSinhVien;
    private BindingSource bindingSource = new BindingSource();

    private async void Form1_Load(object sender, EventArgs e)
    {
        using (var context = new AppDbContext())
        {
            danhSachSinhVien = await context.Students.ToListAsync();
        }

        bindingSource.DataSource = danhSachSinhVien;
        dgvSinhVien.DataSource = bindingSource;   // DataGridView
lấy dữ liệu qua BindingSource
    }
```

Thêm/xóa phần tử qua BindingSource — DataGridView tự cập nhật ngay

```
private void btnThem_Click(object sender, EventArgs e)

{

  Student svMoi = new Student { FullName = txtHoTen.Text,

                                Grade = double.Parse(txtDiem.Text) };

  bindingSource.Add(svMoi);
```

```
    // Thêm qua BindingSource - không cần gán lại DataSource
}
```

6.4.3 Áp dụng BindingSource vào CRUD

Kết hợp BindingSource với EF Core trong một luồng CRUD hoàn chỉnh — lưu ý điểm quan trọng: `BindingSource` chỉ giúp đồng bộ hiển thị trên giao diện ngay lập tức, **không thay thế** cho việc gọi `SaveChangesAsync()` để ghi dữ liệu xuống CSDL thật; cả hai vẫn cần được thực hiện song song.

```
private async void btnThem_Click(object sender, EventArgs e)
 {
     if (string.IsNullOrWhiteSpace(txtHoTen.Text)) return;
     if (!double.TryParse(txtDiem.Text, out double diem)) return;

     Student svMoi = new Student { FullName = txtHoTen.Text, Grade = diem
};

     using (var context = new AppDbContext())
     {
         context.Students.Add(svMoi);
         await context.SaveChangesAsync();   // Ghi xuống CSDL thật
     }

     bindingSource.Add(svMoi);   // Cập nhật ngay lập tức trên giao diện
 }
```

Lấy bản ghi đang được chọn qua BindingSource

Thay vì lấy qua `dgvSinhVien.CurrentRow?.DataBoundItem` như cách đã quen, có thể lấy trực tiếp qua thuộc tính `Current` của **`BindingSource`** — về bản chất cho cùng kết quả, nhưng đi qua đúng "cầu nối" trung tâm đã thiết lập:

```
private async void btnXoa_Click(object sender, EventArgs e)
{
    if (bindingSource.Current is not Student svDangChon) return;

    using (var context = new AppDbContext())
    {
        Student sv = await context.Students.FindAsync(svDangChon.Id);
        if (sv != null)
        {
            context.Students.Remove(sv);
            await context.SaveChangesAsync();
        }
    }
```

```
    bindingSource.Remove(svDangChon);    // Cập nhật ngay trên giao diện
}
```

Việc chuyển sang dùng `BindingSource` không bắt buộc phải áp dụng cho mọi tình huống, nhưng là một thực hành tốt nên làm quen, vì đây là cách tiếp cận chuẩn mực hơn trong WinForms — đặc biệt hữu ích khi ứng dụng có quy mô lớn dần, với nhiều Control cùng hiển thị một nguồn dữ liệu, hoặc cần đồng bộ thay đổi giữa nhiều nơi trên giao diện cùng lúc.

6.5 Validation dữ liệu trước khi lưu

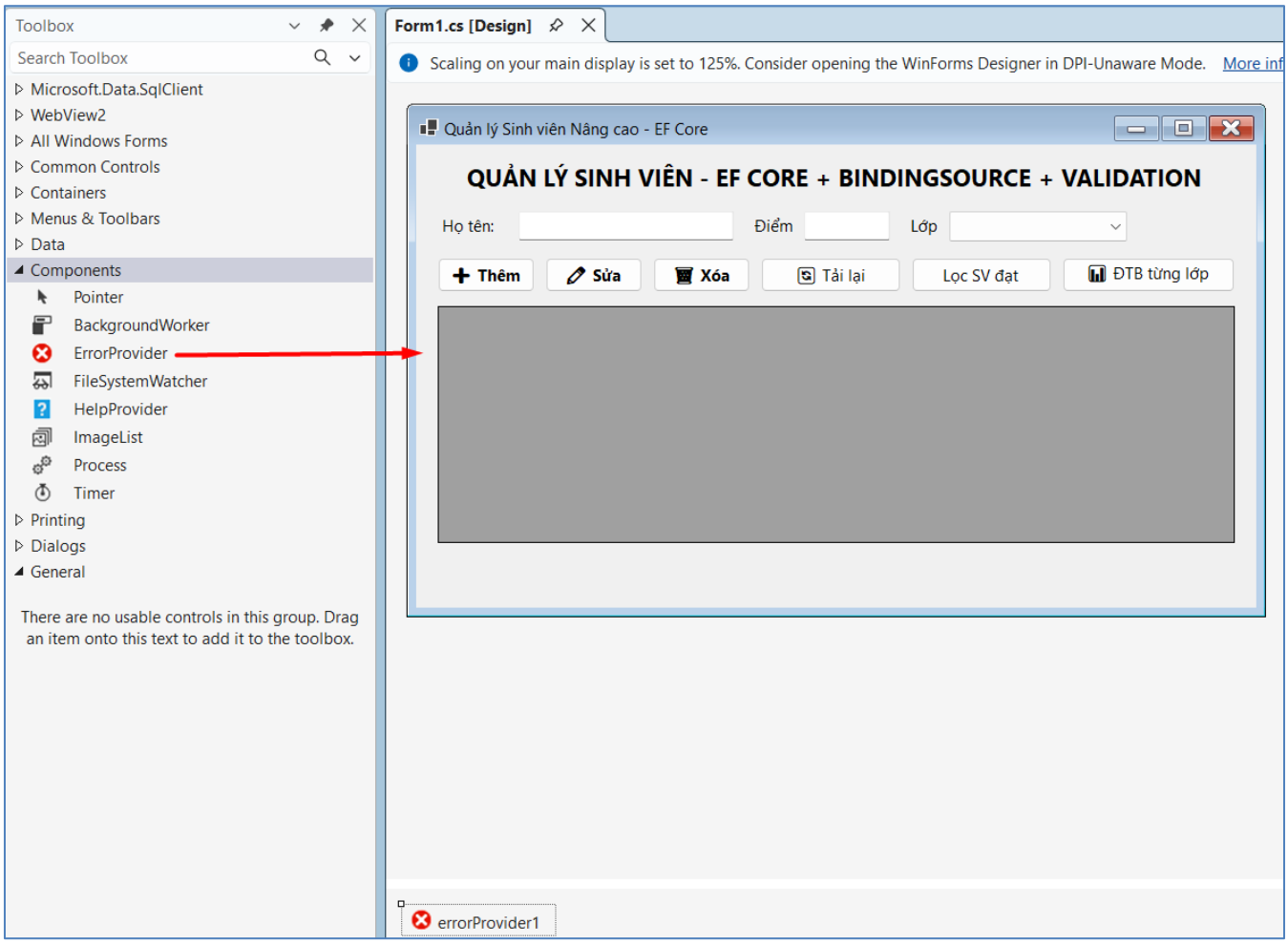
6.5.1 ErrorProvider

Xuyên suốt các bài học trước, cách validate dữ liệu vẫn dừng ở mức cơ bản: dùng MessageBox.Show() để thông báo lỗi, sau đó return để dừng xử lý. Cách này tuy hiệu quả, nhưng có một hạn chế về trải nghiệm người dùng — hộp thoại che mất giao diện chính, và người dùng phải đóng nó lại trước khi biết chính xác ô nào đang bị lỗi.

ErrorProvider là Control giải quyết đúng vấn đề này — hiển thị một biểu tượng cảnh báo nhỏ (thường là dấu chấm than đỏ) ngay cạnh Control đang có lỗi, kèm theo dòng chữ giải thích khi rê chuột vào biểu tượng đó — tương tự cách các form đăng ký trên web thường báo lỗi ngay dưới từng ô nhập liệu.

Thêm ErrorProvider vào Form

Từ Toolbox, nhóm Components, kéo ErrorProvider vào Form — control này không hiển thị trực tiếp, nằm ở khu vực "tray" phía dưới Designer, giống BindingSource đã học.



Hiển thị lỗi trên một Control, Xóa lỗi khi dữ liệu đã hợp lệ

Method `SetError()` nhận hai tham số: Control cần đánh dấu lỗi, và nội dung thông báo lỗi sẽ hiện ra khi rê chuột vào biểu tượng cảnh báo bên cạnh Control đó.

Truyền vào chuỗi rỗng (`""`) là cách để "tắt" biểu tượng lỗi đã hiển thị trước đó — quy ước này cần nhớ kỹ, vì không có method riêng tên `ClearError()`.

```
if (string.IsNullOrWhiteSpace(txtHoTen.Text))
 {
     errorProvider1.SetError(txtHoTen, "Họ tên không được để trống!");
     e.Cancel = true;
 }
 else
 {
     errorProvider1.SetError(txtHoTen, ""); // Chuỗi rỗng = không còn lỗi
 }
```

6.5.2 Sự kiện Validating

Thay vì chỉ validate khi người dùng bấm nút "Lưu", một trải nghiệm tốt hơn là validate ngay khi người dùng rời khỏi một ô nhập liệu — giúp phát hiện lỗi sớm, trước khi họ kịp điền hết toàn bộ form rồi mới biết có chỗ sai.

Sự kiện Validating của một Control được kích hoạt đúng vào thời điểm Control đó sắp mất tiêu điểm (trước khi người dùng chuyển sang ô khác bằng Tab hoặc click) — đây là cơ hội để kiểm tra dữ liệu vừa nhập và quyết định có cho phép rời khỏi ô đó hay không.

```
private void txtHoTen_Validating(object sender, CancelEventArgs e)
{
    if (string.IsNullOrWhiteSpace(txtHoTen.Text))
    {
        errorProvider1.SetError(txtHoTen, "Họ tên không được để trống!");
        e.Cancel = true;
    }
    else
    {
        errorProvider1.SetError(txtHoTen, ""); // Chuỗi rỗng = không còn
lỗi
    }
}
```

Tham số `e` (kiểu `CancelEventArgs`) có thuộc tính `Cancel` — gán `e.Cancel = true` sẽ **giữ tiêu điểm lại** ở Control đang validate, không cho người dùng chuyển sang Control khác cho đến khi dữ liệu hợp lệ. Đây là một cơ chế khá mạnh, cần cân nhắc kỹ khi sử dụng — dùng quá nghiêm ngặt có thể gây khó chịu nếu người dùng chỉ đang tạm thời để trống ô đó.

Validate điểm số — kết hợp kiểm tra định dạng và khoảng giá trị

```
private void txtDiem_Validating(object sender, CancelEventArgs e)
 {
     if (!double.TryParse(txtDiem.Text, out double diem) || diem < 0 ||
diem > 10)
     {
         errorProvider1.SetError(txtDiem, "Điểm phải là số từ 0 đến 10!");
         e.Cancel = true;
     }
     else
     {
         errorProvider1.SetError(txtDiem, "");
     }
 }
```

6.5.3 Kết hợp validate với EF Core

Validate qua sự kiện Validating chỉ đảm bảo từng ô riêng lẻ hợp lệ ngay tại thời điểm người dùng rời khỏi nó. Trước khi thực sự gọi SaveChangesAsync() để ghi dữ liệu xuống CSDL, cần thêm một bước kiểm tra tổng thể toàn bộ Form, vì có thể tồn tại trường hợp người dùng chưa từng "rời khỏi" một ô nào đó (ví dụ bỏ trống và bấm thẳng nút Lưu mà không click qua ô đó), khiến Validating chưa từng được kích hoạt.

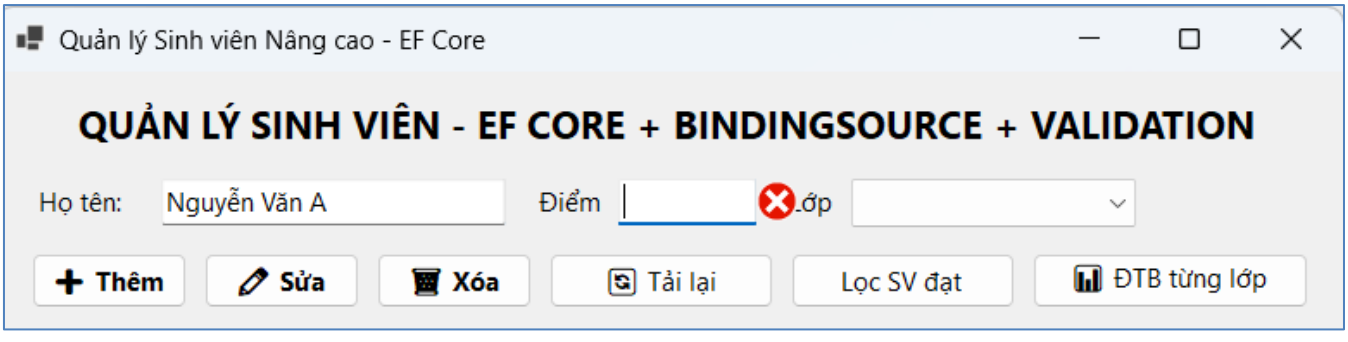
ValidateChildren() — validate toàn bộ Form trong một lệnh

```
private async void btnThem_Click(object sender, EventArgs e)
{
    if (!this.ValidateChildren())
    {
        MessageBox.Show("Vui lòng kiểm tra lại thông tin đã nhập!");
        return;
    }

    try
    {
        Student svMoi = new Student
        {
            FullName = txtHoTen.Text,
            Grade = double.Parse(txtDiem.Text),
            ClassId = cboLop.SelectedValue as int?
        };

        using (var context = new AppDbContext())
        {
            context.Students.Add(svMoi);
            await context.SaveChangesAsync();
        }

        bindingSource.Add(svMoi);
        lblTrangThai.Text = "Đã thêm sinh viên mới!";
        lblTrangThai.ForeColor = System.Drawing.Color.DarkGreen;
        XoaTrang();
    }
    catch (Exception ex)
    {
        HienThiLoi(ex.Message);
    }
}
```



this.ValidateChildren() chủ động kích hoạt sự kiện Validating trên toàn bộ Control con của Form, bất kể Control đó đã từng được người dùng tương tác hay chưa — trả về false nếu có bất kỳ Control nào đang ở trạng thái lỗi (đã e.Cancel = true trong Validating), và true nếu mọi Control đều hợp lệ. Đây là bước kiểm tra "chốt chặn cuối cùng", đảm bảo dữ liệu không hợp lệ không bao giờ lọt được đến SaveChangesAsync().

TÓM TẮT

Trong bài này, học viên làm quen với các kỹ thuật nâng cao khi làm việc với cơ sở dữ liệu thông qua Entity Framework Core, bao gồm:

- *Khai báo quan hệ giữa các bảng: quan hệ 1-n và n-n thông qua Navigation Property và khóa ngoại*
- *Truy vấn dữ liệu có quan hệ với Include(), ThenInclude() và LINQ to Entities nâng cao (GroupBy, CountBy, AggregateBy)*
- *Đảm bảo tính toàn vẹn dữ liệu với Transaction tường minh (BeginTransactionAsync, CommitAsync, RollbackAsync)*
- *Liên kết dữ liệu với giao diện thông qua BindingSource*
- *Kiểm tra dữ liệu đầu vào với ErrorProvider, sự kiện Validating, và ValidateChildren() trước khi lưu xuống cơ sở dữ liệu*

CÂU HỎI ÔN TẬP

Câu 1: Phân biệt quan hệ 1-n và n-n trong cơ sở dữ liệu quan hệ. Cho một ví dụ thực tế cho mỗi loại quan hệ và mô tả cách khai báo tương ứng trong EF Core thông qua Navigation Property.?

Câu 2: Khóa ngoại (Foreign Key) là gì? Trong EF Core theo mô hình Code First, khóa ngoại được khai báo như thế nào trong Entity, và EF Core tự nhận diện quan hệ dựa trên quy ước đặt tên nào?

Câu 3: Giải thích tại sao Navigation Property có thể trả về `null` dù dữ liệu liên quan thực sự tồn tại trong cơ sở dữ liệu. Phương thức nào của EF Core giải quyết vấn đề này, và cú pháp sử dụng ra sao?

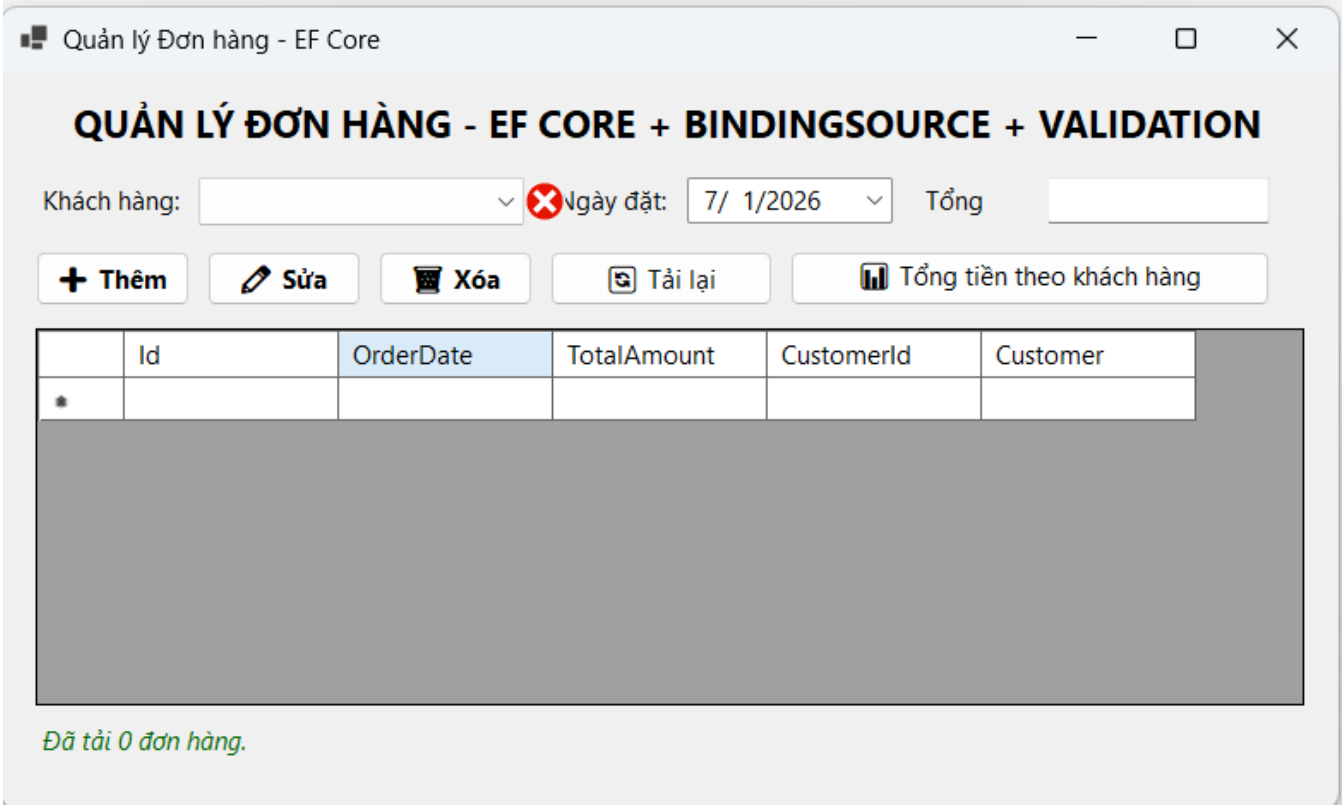
Câu 4: Trình bày sự khác biệt giữa `Include()` và `ThenInclude()`. Viết đoạn mã truy vấn danh sách lớp học, kèm theo danh sách sinh viên của từng lớp, và với mỗi sinh viên đó kèm theo danh sách môn học đã đăng ký?

Câu 5: Phân biệt `CountBy()`, `AggregateBy()` và `GroupBy()` trong LINQ to Entities. Nêu tình huống phù hợp để sử dụng từng phương thức, và phiên bản .NET nào mới hỗ trợ `CountBy()` và `AggregateBy()`.?

Câu 6: `BindingSource` là gì và giải quyết hạn chế nào của cách gán `DataSource` trực tiếp vào `DataGridView`? Trình bày cách thiết lập `BindingSource` và sử dụng `bindingSource.Add()`, `bindingSource.Remove()` trong luồng CRUD.

Câu 7: Trình bày cơ chế hoạt động của `ErrorProvider` kết hợp với sự kiện `Validating`. Giải thích vai trò của `e.Cancel = true` trong sự kiện `Validating` và lý do cần gọi thêm `ValidateChildren()` trước khi thực hiện `SaveChangesAsync()`.?

Câu 8: Xây dựng ứng dụng WinForms quản lý đơn hàng



Entity:

Customer (Id, FullName bắt buộc, Phone)

Order (Id, OrderDate, TotalAmount, CustomerId — quan hệ 1-n với Customer)

Yêu cầu:

1. CRUD đầy đủ cho Order, ComboBox chọn Customer
2. Include() khi hiển thị đơn hàng kèm tên khách hàng
3. BindingSource liên kết với DataGridView
4. ErrorProvider + Validating kiểm tra dữ liệu đầu vào
5. GroupBy thống kê tổng tiền theo từng khách hàng
6. Transaction khi xóa khách hàng kèm toàn bộ đơn hàng của họ

BÀI 7: MÔ HÌNH TỔ CHỨC DỰ ÁN

Sau khi học xong bài này, học viên có thể nắm được:

- Nhận diện vấn đề phát sinh khi toàn bộ code dồn vào Form.cs — vi phạm nguyên tắc đơn trách nhiệm, khó bảo trì và mở rộng
- Hiểu kiến trúc 3 lớp (3-Layer Architecture): vai trò và trách nhiệm của từng lớp Presentation, Business Logic và Data Access
- Tổ chức code theo đúng từng lớp trong cùng một project bằng cách phân chia thư mục và namespace hợp lý
- Tách BLL và DAL thành các Class Library độc lập trong một Solution nhiều project, liên kết qua Project Reference
- Kết hợp kiến trúc 3 lớp với Entity Framework Core để xây dựng ứng dụng hoàn chỉnh
- Lựa chọn cách tổ chức dự án phù hợp tùy theo quy mô và bối cảnh thực tế

7.1 Đặt vấn đề

Nhìn lại các ứng dụng đã xây dựng, có thể thấy **Form1.cs** có thể nhận ra một điểm chung: toàn bộ ứng dụng — từ việc lấy giá trị từ TextBox, gọi EF Core để đọc/ghi dữ liệu, kiểm tra dữ liệu hợp lệ, cho đến cập nhật lại DataGridView — đều nằm gộp chung trong một file duy nhất. Một file `Form1.cs` điển hình ở bài trước có thể dài đến vài trăm dòng, và nếu đọc kỹ, sẽ thấy nó đang đảm nhiệm đồng thời ít nhất bốn vai trò hoàn toàn khác nhau:

```
private async void btnThem_Click(object sender, EventArgs e)
 {
     // Vai trò 1 – Giao diện: lấy dữ liệu từ Control
```

```
      string hoTen = txtHoTen.Text;
      double diem = double.Parse(txtDiem.Text);

      // Vai trò 2 – Kiểm tra dữ liệu: validate nghiệp vụ
      if (string.IsNullOrWhiteSpace(hoTen)) return;
      if (diem < 0 || diem > 10) return;

      // Vai trò 3 – Truy cập CSDL: gọi EF Core trực tiếp
      using (var context = new AppDbContext())
      {
          context.Students.Add(new Student { FullName = hoTen, Grade = diem
});
          await context.SaveChangesAsync();
      }

      // Vai trò 4 – Giao diện: cập nhật lại DataGridView
      await TaiDanhSach();
  }
```

Chỉ trong một method xử lý sự kiện dài chưa đến 15 dòng, Form đang vừa đọc dữ liệu từ giao diện, vừa kiểm tra dữ liệu, vừa gọi EF Core, vừa cập nhật lại giao diện — bốn công việc hoàn toàn khác nhau về bản chất, bị ép vào cùng một chỗ chỉ vì chúng "xảy ra vào cùng một lúc".

Cách tổ chức này không gây vấn đề gì khi ứng dụng còn nhỏ và chỉ có một người viết — nhưng khi ứng dụng bắt đầu lớn lên, hoặc cần nhiều người cùng tham gia phát triển, những hậu quả cụ thể sẽ bắt đầu xuất hiện như:

Khó bảo trì: khi cần thay đổi cách lưu dữ liệu (ví dụ từ SQL Server sang MySQL, hoặc từ EF Core sang một thư viện khác), phải lần lượt tìm và sửa từng method trong Form — không có một nơi tập trung nào chứa toàn bộ logic truy cập CSDL để sửa một lần.

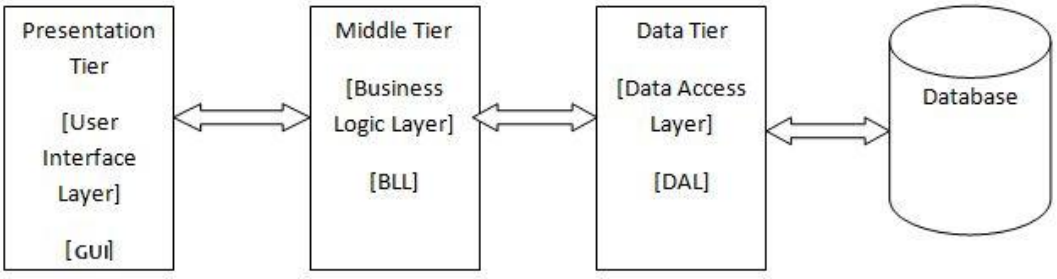
Khó mở rộng: nếu cần thêm một Form khác cũng có chức năng thêm sinh viên (ví dụ một Form nhập nhanh, không cần đầy đủ các trường như Form chính), logic thêm sinh viên không thể tái sử dụng — phải viết lại từ đầu cho Form mới, dẫn đến code trùng lặp ở nhiều nơi.

Dễ phát sinh lỗi không mong muốn: khi sửa phần validate dữ liệu, rất dễ vô tình ảnh hưởng đến phần giao diện ngay bên cạnh trong cùng một method — vì ranh giới giữa các trách nhiệm không được phân chia rõ ràng, chỉ là các dòng code nằm kế tiếp nhau.

Tất cả những vấn đề trên bắt nguồn từ một nguyên nhân chung: Form đang vi phạm nguyên tắc "một class chỉ nên có một lý do để thay đổi" (Single Responsibility Principle) — một trong những nguyên tắc nền tảng của thiết kế phần mềm tốt. Khi một class phải thay đổi vì nhiều lý do khác nhau (thay đổi giao diện, thay đổi nghiệp vụ, thay đổi cách truy cập CSDL), nó trở thành điểm tập trung rủi ro của toàn bộ ứng dụng.

7.2 Kiến trúc 3 tầng (3-Layer Architecture)

Kiến trúc 3 tầng (3-Layer Architecture) là một cách tổ chức code ứng dụng thành ba tầng riêng biệt, mỗi tầng chịu trách nhiệm đúng một loại công việc và chỉ giao tiếp với tầng liền kề — không tầng nào được "nhảy cóc" qua tầng khác để làm việc trực tiếp:



Quy tắc giao tiếp giữa các tầng chỉ theo một chiều từ trên xuống:

- Presentation gọi xuống Business Logic — không bao giờ gọi thẳng xuống Data Access
- Business Logic gọi xuống Data Access — không biết gì về giao diện phía trên
- Data Access chỉ làm việc với CSDL — không biết gì về nghiệp vụ hay giao diện

Nguyên tắc này đảm bảo mỗi tầng có thể thay đổi độc lập mà không kéo theo thay đổi ở tầng khác: muốn đổi từ SQL Server sang MySQL, chỉ sửa Data Access Layer; muốn đổi giao diện từ WinForms sang một công nghệ khác, chỉ sửa Presentation Layer — Business Logic hoàn toàn không bị ảnh hưởng.

7.2.1 Presentation Layer — tầng giao diện

Đây là tầng duy nhất người dùng nhìn thấy và tương tác trực tiếp — các Form, Control, sự kiện Click, hiển thị dữ liệu lên DataGridView. Presentation Layer **không** chứa bất kỳ logic nghiệp vụ hay câu lệnh truy vấn CSDL nào — nó chỉ biết cách hiển thị dữ liệu nhận được từ tầng dưới, và chuyển tiếp yêu cầu của người dùng xuống tầng Business Logic:

```
// Presentation Layer – Form chỉ gọi Service, không biết EF Core tồn tại
private async void btnThem_Click(object sender, EventArgs e)
{
    string hoTen = txtHoTen.Text;
    double diem = double.Parse(txtDiem.Text);
    await _studentService.ThemSinhVienAsync(hoTen, diem);// Gọi BLL
    await TaiDanhSach();
}
```

7.2.2 Business Logic Layer — tầng nghiệp vụ

Đây là "trái tim" của ứng dụng — nơi chứa toàn bộ logic nghiệp vụ: validate dữ liệu theo đúng quy tắc nghiệp vụ (khác với validate giao diện đơn thuần ở Presentation), tính toán, xử lý các quy tắc đặc thù của từng ứng dụng. Business Logic Layer nhận yêu cầu từ Presentation, xử lý nghiệp vụ, rồi gọi xuống Data Access để thực hiện các thao tác CSDL:

```
// Business Logic Layer – Service xử lý nghiệp vụ và validate
public class StudentService
{
    private readonly StudentRepository _repository;

    public StudentService(StudentRepository repository)
    {
        _repository = repository;
    }

    public async Task ThemSinhVienAsync(string hoTen, double diem)
    {
        // Validate nghiệp vụ – logic này không nên nằm trong Form
        if (string.IsNullOrWhiteSpace(hoTen))
            throw new ArgumentException("Họ tên không được để trống!");

        if (diem < 0 || diem > 10)
            throw new ArgumentException("Điểm phải từ 0 đến 10!");

        Student svMoi = new Student { FullName = hoTen, Grade = diem };
        await _repository.ThemAsync(svMoi);   // Gọi xuống DAL
    }
```

```
}
```

7.2.3 Data Access Layer — tầng truy cập dữ liệu

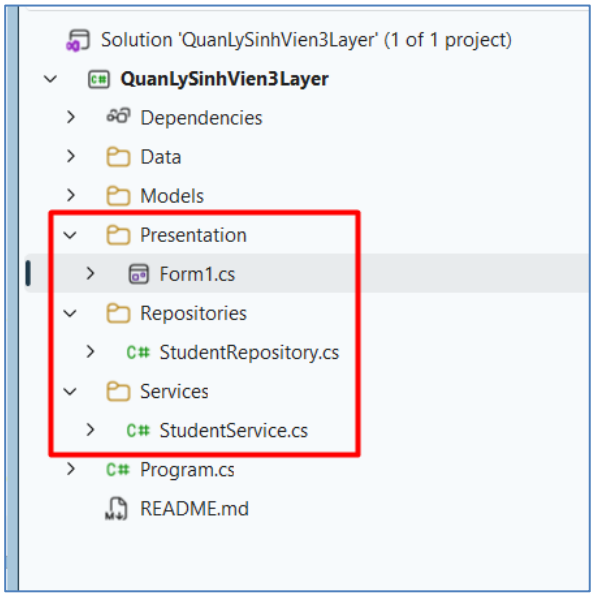
Đây là tầng thấp nhất, chịu trách nhiệm duy nhất: giao tiếp với cơ sở dữ liệu thông qua EF Core. Data Access Layer không biết gì về nghiệp vụ hay giao diện — nó chỉ nhận yêu cầu từ Business Logic và thực thi đúng thao tác CSDL tương ứng:

```
// Data Access Layer - Repository chỉ làm việc với EF Core
 public class StudentRepository
 {
     public async Task ThemAsync(Student sv)
     {
         using (var context = new AppDbContext())
         {
             context.Students.Add(sv);
             await context.SaveChangesAsync();
         }
     }

     public async Task<List<Student>> LayDanhSachAsync()
     {
         using (var context = new AppDbContext())
         {
             return await context.Students.ToListAsync();
         }
     }
 }
```

7.2.4 Cách tổ chức thư mục trong project

Kiến trúc 3 tầng không chỉ là cách tổ chức logic trong code — nó còn được thể hiện rõ qua cấu trúc thư mục của project, giúp bất kỳ ai mở project lên đều nhận ra ngay cấu trúc tổng thể:



Khi cấu trúc thư mục phản ánh đúng kiến trúc 3 tầng như trên, việc tìm kiếm và chỉnh sửa code trở nên rất trực quan: muốn sửa giao diện thì vào `Presentation/`, muốn sửa logic nghiệp vụ thì vào `Services/`, muốn sửa câu truy vấn thì vào `Repositories/` — không cần phải lần lượt đọc qua một file dài hàng trăm dòng để tìm đúng đoạn cần sửa.

7.3 Tách thành nhiều Project

Cách tổ chức 3 tầng trong một project duy nhất đã học giải quyết được vấn đề về tổ chức code — mỗi tầng nằm trong thư mục riêng, mỗi class có trách nhiệm rõ ràng. Tuy nhiên, đây vẫn là một ranh giới logic (chỉ phân chia bằng thư mục và namespace), không phải ranh giới vật lý — về mặt kỹ thuật, một lập trình viên vẫn có thể từ Form1.cs gọi thẳng vào StudentRepository mà Visual Studio hoàn toàn không ngăn cản, vì tất cả đều nằm trong cùng một project. Khi ứng dụng phát triển lên quy mô lớn hơn, hoặc khi có thêm các tình huống thực tế xuất hiện, ranh giới logic đó bắt đầu bộc lộ những hạn chế mà chỉ việc tách thành project riêng biệt mới giải quyết được.

Tái sử dụng logic nghiệp vụ cho nhiều ứng dụng

Hình dung hệ thống quản lý sinh viên đã xây dựng cần mở rộng: ngoài ứng dụng WinForms dành cho nhân viên phòng đào tạo, trường còn muốn có thêm một ứng dụng web (ASP.NET Core) để sinh viên tự tra cứu điểm qua trình duyệt. Toàn bộ logic

nghiệp vụ — cách tính điểm trung bình, quy tắc validate, các thao tác CRUD với cơ sở dữ liệu — đều giống hệt nhau cho cả hai ứng dụng, chỉ có giao diện là khác.

Nếu BLL và DAL nằm trong cùng project WinForms, chúng không thể được tái sử dụng cho ứng dụng web — buộc phải viết lại toàn bộ logic đó một lần nữa cho project ASP.NET Core, dẫn đến hai nơi chứa cùng một logic, và khi cần thay đổi (ví dụ quy tắc xếp loại điểm thay đổi), phải sửa ở hai nơi thay vì một.

Khi BLL và DAL được tách thành Class Library riêng (file .dll độc lập), cả WinForms lẫn ASP.NET Core đều có thể tham chiếu (reference) cùng một thư viện đó:

```
QuanLySinhVien.BLL.dll  ⟵ WinForms app tham chiếu
                        ⟵ ASP.NET Core app tham chiếu
                        ⟵ App mobile (MAUI) tham chiếu (tương lai)
```

Thay đổi logic nghiệp vụ một lần trong BLL, toàn bộ ứng dụng tham chiếu đến nó đều được cập nhật theo — đây là lợi ích cốt lõi mà cách tổ chức 1 project không thể đạt được.

Tường ngăn cứng giữa các tầng

Khi BLL và DAL là hai project riêng biệt, Visual Studio tự động thiết lập một **tường ngăn kỹ thuật** không thể vượt qua: project UI chỉ có thể gọi đến những gì được public trong BLL (vì UI chỉ reference đến BLL, không reference đến DAL); project BLL không thể gọi ngược lên UI vì không có reference đó. Đây không còn là quy ước "nhớ mà làm theo" nữa — trình biên dịch sẽ tự báo lỗi nếu vi phạm.

Làm việc nhóm hiệu quả hơn

Trong môi trường làm việc nhóm, khi BLL và DAL là các project độc lập, các thành viên có thể phân công rõ ràng và làm việc song song mà không ảnh hưởng lẫn nhau:

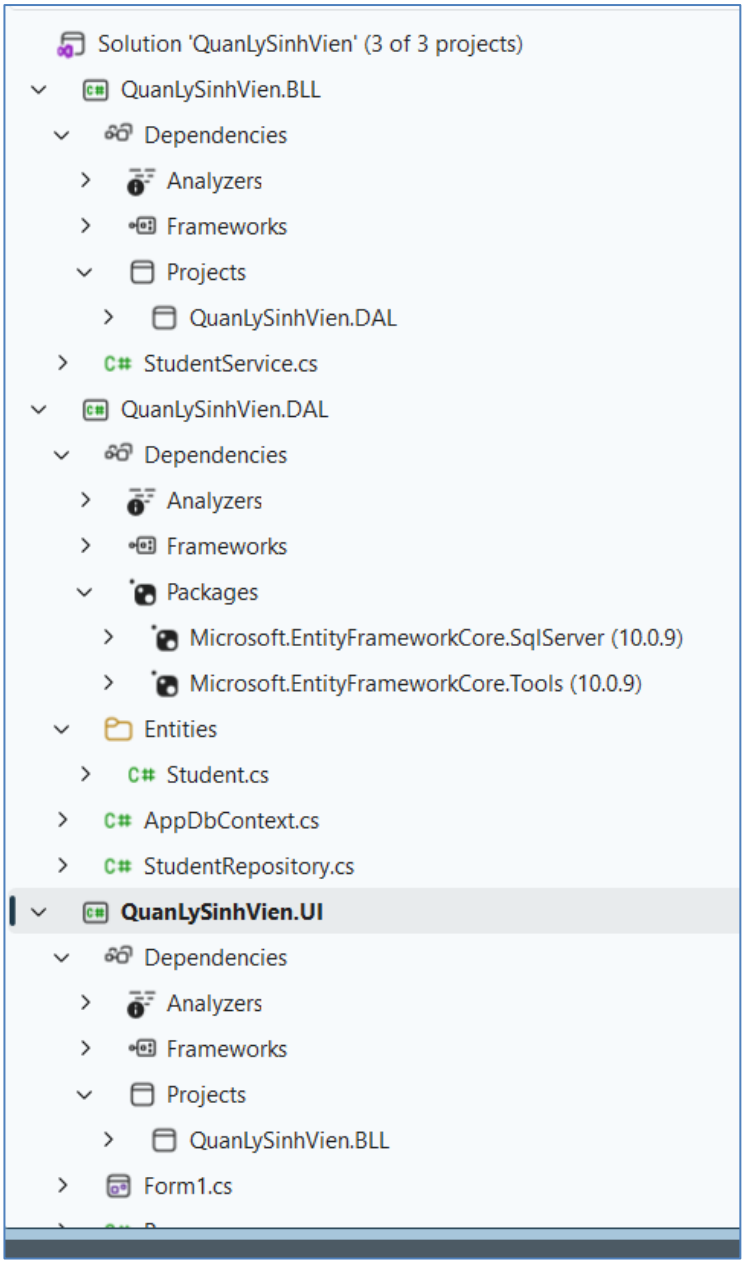
Nhóm phụ trách UI chỉ cần biết BLL cung cấp những method gì (qua tên method và tham số), không cần quan tâm BLL được cài đặt bên trong như thế nào

Nhóm phụ trách BLL và DAL có thể thay đổi cách cài đặt bên trong (ví dụ tối ưu câu truy vấn) mà không làm ảnh hưởng đến code UI, miễn là tên và chữ ký method không thay đổi

Đây cũng là lý do trong các công ty phần mềm, các dự án thực tế hầu như luôn được tổ chức theo Solution nhiều project — không phải vì bắt buộc, mà vì đây là cách duy nhất đảm bảo được những yêu cầu trên ở quy mô thực sự.

7.3.1 Cách tổ chức thư mục trong project

Một Solution tổ chức theo kiến trúc 3 tầng với 3 project riêng biệt có cấu trúc như sau:



Cách tạo và liên kết project trong Visual Studio

Bước 1 — Tạo Solution và project DAL

Mở Visual Studio, chọn Create a new project → Class Library → đặt tên QuanLySinhVien.DAL. Ở bước Additional information, đặt tên Solution là QuanLySinhVien.

Bước 2 — Thêm project BLL vào Solution

Trong Solution Explorer, chuột phải vào tên Solution (không phải project) → Add → New Project → chọn Class Library → đặt tên QuanLySinhVien.BLL.

Bước 3 — Thêm project UI vào Solution

Tương tự Bước 2, chuột phải vào Solution → Add → New Project → chọn Windows Forms App → đặt tên QuanLySinhVien.UI.

Bước 4 — Liên kết BLL tham chiếu đến DAL

Trong Solution Explorer, chuột phải vào project QuanLySinhVien.BLL → Add → Project Reference... → tick chọn QuanLySinhVien.DAL → OK.

Bước 5 — Liên kết UI tham chiếu đến BLL

Tương tự Bước 4, chuột phải vào project QuanLySinhVien.UI → Add → Project Reference... → tick chọn QuanLySinhVien.BLL → OK.

Lưu ý: UI không được tick chọn thêm QuanLySinhVien.DAL — đây chính là "tường ngăn cứng" đã đề cập.

Bước 6 — Cài đặt NuGet package đúng chỗ

Package EF Core chỉ cần cài vào project DAL (nơi duy nhất thực sự dùng EF Core), không cài vào BLL hay UI:

Chuột phải vào project QuanLySinhVien.DAL → Manage NuGet Packages → cài Microsoft.EntityFrameworkCore.SqlServer và Microsoft.EntityFrameworkCore.Tools

Bước 7 — Đặt UI làm Startup Project

Chuột phải vào project QuanLySinhVien.UI → Set as Startup Project — đây là project sẽ chạy khi nhấn F5.

Trong thực tế, DAL thường được tổ chức theo mẫu Repository Pattern và Unit of Work để tăng khả năng kiểm thử và thay thế. Đây là hướng nâng cao học viên có thể tìm hiểu thêm sau khi nắm vững kiến trúc 3 lớp cơ bản.

TÓM TẮT

Trong bài này, học viên tìm hiểu các cách tổ chức dự án WinForms từ đơn giản đến chuyên nghiệp, bao gồm:

Nhận diện vấn đề khi toàn bộ code dồn vào Form.cs: vi phạm nguyên tắc đơn trách nhiệm, khó bảo trì và mở rộng

Kiến trúc 3 tầng (3-Layer Architecture): phân chia rõ ràng thành Presentation Layer (giao diện), Business Logic Layer (nghiệp vụ), và Data Access Layer (truy cập dữ liệu) trong cùng một project

Tổ chức Solution nhiều project: tách BLL và DAL thành các Class Library độc lập, tạo tường ngăn vật lý giữa các tầng, cho phép tái sử dụng logic nghiệp vụ cho nhiều ứng dụng khác nhau

Lựa chọn cách tổ chức phù hợp tùy theo quy mô và bối cảnh dự án.

CÂU HỎI ÔN TẬP

Câu 1: Trình bày cấu trúc của kiến trúc 3 tầng (3-Layer Architecture). Mỗi tầng chịu trách nhiệm cho những công việc gì, và quy tắc giao tiếp giữa các tầng là gì?

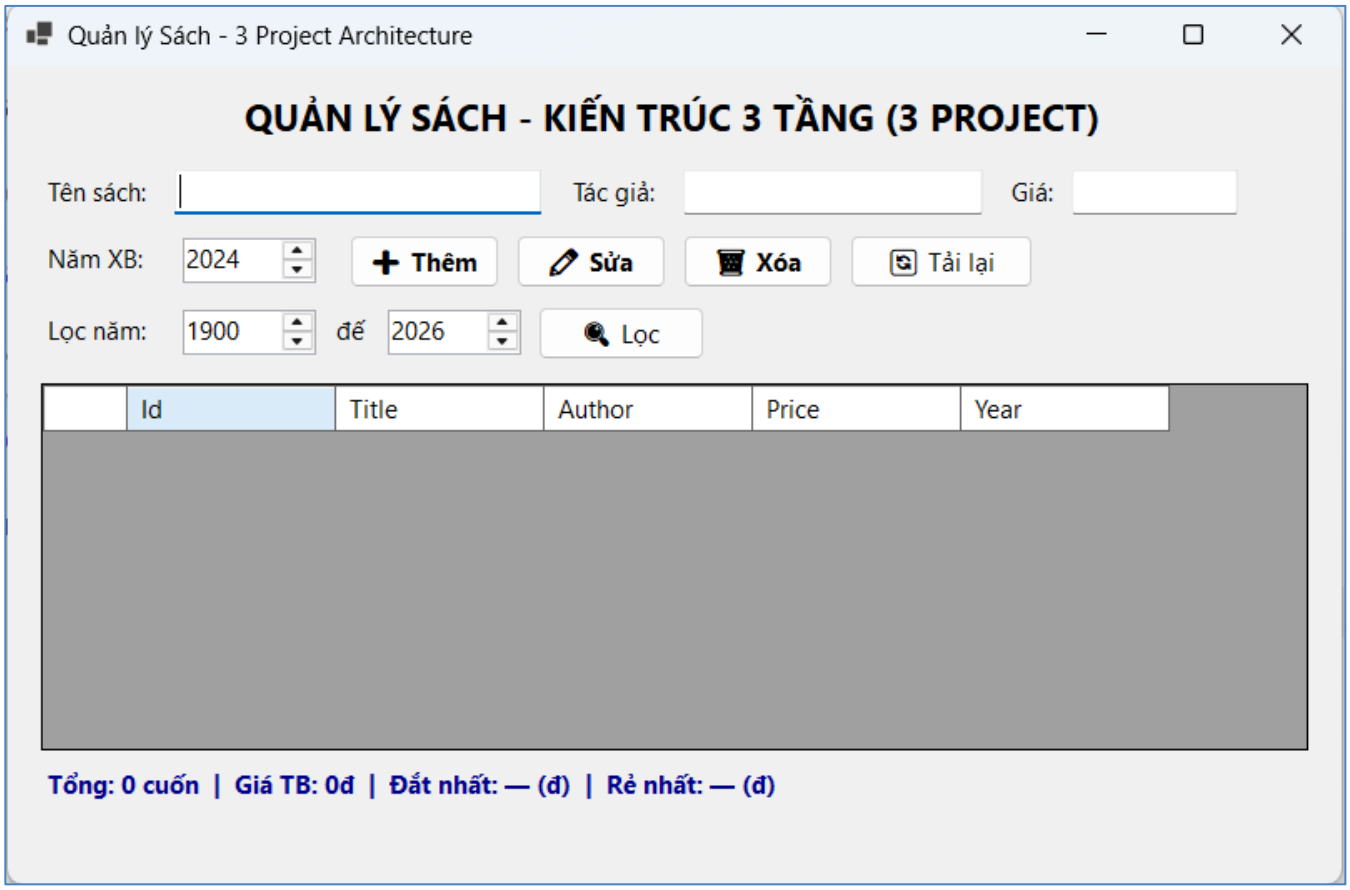
Câu 2: Trong kiến trúc 3 tầng tổ chức thành 3 project riêng biệt, hãy giải thích tại sao package EF Core chỉ cần cài đặt vào project DAL, không cần cài vào BLL hay UI. Điều gì xảy ra nếu cố tình thêm Project Reference từ UI trực tiếp đến DAL?

Câu 3: Cho tình huống sau: một công ty có ứng dụng WinForms quản lý nhân sự, và nay muốn bổ sung thêm một ứng dụng web (ASP.NET Core) để nhân viên tự tra cứu thông tin qua trình duyệt. Giải thích cách kiến trúc 3 project giúp tái sử dụng code trong tình huống này, và cách tổ chức 1 project không thể đáp ứng được.

Câu 4: Trong một `StudentService` (BLL), method `ThemSinhVienAsync` thực hiện validate dữ liệu trước khi gọi Repository. Giải thích sự khác biệt giữa validate nghiệp vụ (nằm

trong Service) và validate giao diện (nằm trong Form). Cho ví dụ minh họa cho mỗi loại.

Câu 5: Xây dựng một Solution gồm 3 project theo kiến trúc 3 tầng để quản lý danh sách sách trong thư viện



Yêu cầu Entity (`Book`):

- Id (int, khóa chính tự tăng)
- Title (string, bắt buộc — tên sách)
- Author (string, bắt buộc — tác giả)
- Price (double — giá bán)
- Year (int — năm xuất bản)

Yêu cầu từng tầng:

QuanLySach.DAL (Class Library)

- Entity Book
- AppDbContext với DbSet<Book>

- BookRepository với đầy đủ 5 method: LayDanhSachAsync, TimTheoIdAsync, ThemAsync, SuaAsync, XoaAsync

QuanLySach.BLL (Class Library, reference DAL)

- BookService với validate nghiệp vụ:

  Title và Author không được để trống
  Price phải lớn hơn 0
  Year phải từ 1000 đến năm hiện tại

- Method thống kê: TinhGiaTrungBinh(List<Book>) và LocSachTheoNam(int namBatDau, int namKetThuc)

QuanLySach.UI (WinForms App, reference BLL)

- Form CRUD đầy đủ (Thêm/Sửa/Xóa/Tải lại)
- ComboBox hoặc ô nhập lọc sách theo khoảng năm xuất bản
- Label hiển thị giá trung bình của danh sách đang hiển thị

Form chỉ được import namespace từ BLL và DAL.Entities — không được dùng AppDbContext hay BookRepository trực tiếp.

BÀI 8: TÍCH HỢP THƯ VIỆN

Sau khi học xong bài này, học viên có thể:

- *Hiểu khái niệm NuGet Package Manager và các tiêu chí lựa chọn thư viện phù hợp cho dự án (license, lượt tải, tương thích phiên bản .NET)*
- *Xuất dữ liệu ra file Excel (.xlsx) có định dạng chuyên nghiệp bằng ClosedXML*
- *Tạo báo cáo PDF với bố cục header, bảng dữ liệu và footer số trang bằng QuestPDF*
- *Vẽ biểu đồ thống kê (cột, tròn) trực tiếp trong WinForms và lưu ra file ảnh bằng ScottPlot*
- *Tạo mã QR từ dữ liệu ứng dụng, hiển thị trên PictureBox và lưu ra file ảnh bằng QRCoder*

8.1 Tổng quan về NuGet và thư viện bên ngoài

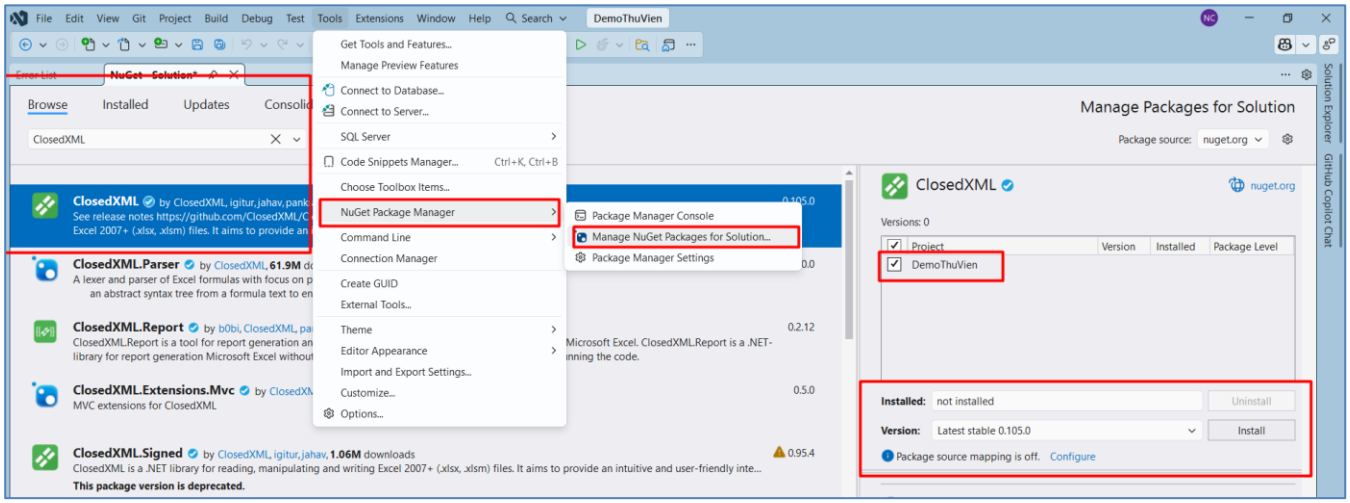
Trong quá trình xây dựng ứng dụng, có rất nhiều tính năng mà hầu như ứng dụng nào cũng cần — xuất dữ liệu ra file Excel, tạo báo cáo PDF, vẽ biểu đồ, gửi email, đọc mã QR... Thay vì tự viết từng chức năng đó từ đầu (một công việc tốn rất nhiều thời gian và dễ phát sinh lỗi), cộng đồng .NET đã xây dựng sẵn hàng chục nghìn thư viện dùng lại được, chia sẻ công khai qua một hệ thống quản lý gói gọi là NuGet.

NuGet là trình quản lý gói (package manager) chính thức của .NET — nơi các lập trình viên và tổ chức công bố các thư viện dưới dạng package (gói), và bất kỳ project .NET nào cũng có thể tải về và sử dụng ngay. Tính đến nay, NuGet Gallery chứa hơn 300.000 package khác nhau, từ các thư viện nhỏ giải quyết một vấn đề cụ thể, đến các framework lớn như Entity Framework Core — vốn đã được dùng xuyên suốt từ Bài 5 đến nay — cũng là một NuGet package.

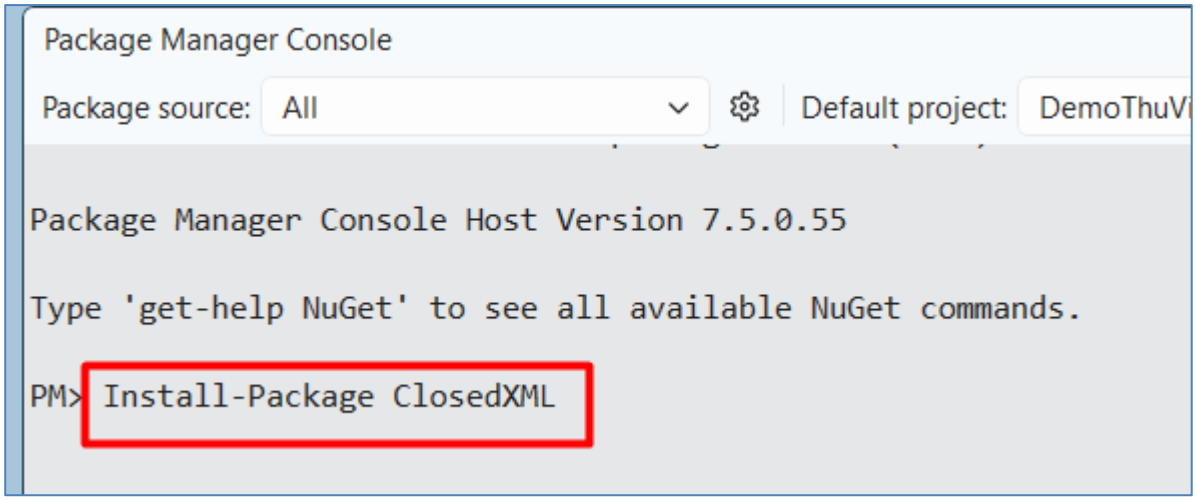
Cài đặt package qua Visual Studio

Có hai cách cài đặt NuGet package vào một project, tùy sở thích:

Cách 1 — Giao diện NuGet Package Manager: chuột phải vào project trong Solution Explorer → Manage NuGet Packages → tab Browse → gõ tên package vào ô tìm kiếm → chọn đúng package → nhấn Install.



Cách 2 — Package Manager Console: mở View → Other Windows → Package Manager Console, gõ lệnh: Install-Package ClosedXML



Sau khi cài đặt, Visual Studio tự động thêm dòng `PackageReference` vào file `.csproj` của project, và tải về các file `.dll` cần thiết — từ đó có thể dùng ngay trong code bằng cách thêm đúng `using` tương ứng.

Các tiêu chí chọn thư viện

Với hàng trăm nghìn package trên NuGet, việc chọn đúng thư viện phù hợp cho dự án đòi hỏi đánh giá dựa trên một số tiêu chí thực tế:

- License (giấy phép sử dụng): một số thư viện hoàn toàn miễn phí cho mọi mục đích (MIT, Apache 2.0), một số miễn phí cho mục đích phi thương mại nhưng yêu cầu mua license khi dùng trong sản phẩm thương mại. Cần đọc kỹ license trước khi tích hợp vào dự án thực tế.
- Lượt tải và mức độ phổ biến: một package có hàng triệu lượt tải mỗi tuần thường đồng nghĩa với việc đã được kiểm chứng bởi cộng đồng rộng lớn, ít lỗi tiềm ẩn hơn, và dễ tìm tài liệu hay câu trả lời khi gặp vấn đề.
- Tần suất cập nhật: package được cập nhật thường xuyên cho thấy tác giả vẫn đang tích cực bảo trì, vá lỗi bảo mật và hỗ trợ các phiên bản .NET mới. Ngược lại, một package không được cập nhật trong vài năm có thể không tương thích với .NET 8/10 hoặc chứa các lỗ hổng đã biết.
- Tương thích với phiên bản .NET đang dùng: luôn kiểm tra mục Target Frameworks trên trang NuGet của package — cần đảm bảo package hỗ trợ đúng phiên bản .NET đang dùng trong project.

Bảng phân loại một số thư viện sẽ sử dụng trong bài học này:

Nhu cầu	**Thư viện**	**License**
Xuất dữ liệu ra file Excel (.xlsx)	**ClosedXML**	MIT — miễn phí hoàn toàn
Tạo báo cáo PDF	**QuestPDF**	MIT — miễn phí cho phi thương mại
Vẽ biểu đồ thống kê trong Form	**ScottPlot**	MIT — miễn phí hoàn toàn
Tạo mã QR	**QRCoder**	MIT — miễn phí hoàn toàn

8.2 Xuất dữ liệu ra Excel — ClosedXML

ClosedXML là thư viện .NET mã nguồn mở giúp tạo, đọc và chỉnh sửa file Excel (.xlsx) mà không cần cài đặt Microsoft Office trên máy. Đây là điểm quan trọng trong môi

trường server hoặc máy tính không có bản quyền Office — ClosedXML làm việc trực tiếp với định dạng file, hoàn toàn độc lập với Office.

So với các thư viện Excel khác trong hệ sinh thái .NET, ClosedXML được đánh giá là lựa chọn phù hợp nhất cho người mới học nhờ cú pháp trực quan, gần với cách tư duy về Excel: worksheet là một lưới ô (cell), mỗi ô được truy cập bằng tọa độ hàng/cột, gán giá trị vào ô giống như gán giá trị cho một biến bình thường.

Cài đặt

Mở Package Manager Console, gõ: Install-Package ClosedXML

Hoặc vào Manage NuGet Packages → Browse → tìm ClosedXML → Install.

Sau khi cài xong, thêm using vào đầu file: using ClosedXML.Excel;

Cấu trúc cơ bản của một file Excel trong ClosedXML

Ba khái niệm cốt lõi cần nắm trước khi viết code:

- XLWorkbook — đại diện cho toàn bộ file Excel (một file .xlsx)
- IXLWorksheet — một sheet bên trong workbook (giống một tab trong Excel)
- Cell(hàng, cột) — một ô cụ thể, truy cập bằng số hàng và số cột (đều bắt đầu từ 1)

8.3 TẠO PDF — QuestPDF

QuestPDF là thư viện .NET hiện đại dùng để tạo file PDF, được thiết kế theo phong cách Fluent API — mô tả cấu trúc tài liệu bằng chuỗi các lời gọi method nối tiếp nhau, đọc như đang mô tả bố cục bằng ngôn ngữ tự nhiên thay vì tọa độ pixel cứng nhắc. So với các thư viện PDF cũ hơn như iTextSharp (đòi hỏi tự tính toán tọa độ từng phần tử), QuestPDF cho phép định nghĩa bố cục theo kiểu "phần tử này nằm trong phần tử kia" — gần với cách tư duy khi thiết kế bố cục hơn.QuestPDF hoàn toàn miễn phí cho dự án mã nguồn mở và phi thương mại (Community License), hỗ trợ đầy đủ cho .NET 10.

Cài đặt

Mở **Package Manager Console**, gõ: Install-Package QuestPDF

Sau khi cài xong, thêm `using` vào đầu file và khai báo license Community ngay trong `Form1` (hoặc `Program.cs`):

Cấu trúc cơ bản của một tài liệu PDF trong QuestPDF

Một tài liệu QuestPDF được xây dựng từ ngoài vào trong theo đúng cấu trúc phân cấp:

Document

└── Page (khổ giấy, lề)

├── Header (phần đầu trang)

├── Content (nội dung chính)

└── Footer (phần cuối trang, số trang)

Các thành phần bố cục hay dùng

`Column` — xếp các phần tử theo chiều dọc, từ trên xuống:

`Row` — xếp các phần tử theo chiều ngang, từ trái sang phải:

`Table` — tạo bảng dữ liệu với header và nhiều dòng:

8.4 VẼ BIỂU ĐỒ — ScottPlot

ScottPlot là thư viện vẽ biểu đồ mã nguồn mở, miễn phí hoàn toàn (MIT License), hỗ trợ đầy đủ WinForms trên.NET 10. ScottPlot cung cấp các loại biểu đồ phổ biến (cột, đường, tròn, scatter...) với khả năng tương tác trực tiếp trong Form (zoom, pan bằng chuột) mà không cần thêm bất kỳ cấu hình nào.

Cài đặt: Mở **Package Manager Console**, gõ: Install-Package ScottPlot.WinForms

Sau khi cài xong, **FormsPlot** — control biểu đồ của ScottPlot — sẽ tự động xuất hiện trong Toolbox. Kéo nó vào Form giống như kéo bất kỳ Control nào khác (Button, DataGridView...), đặt tên (ví dụ `formsPlot1`), căn chỉnh kích thước mong muốn. Không cần thêm `using` thủ công — Visual Studio tự xử lý.

Cấu trúc cơ bản

Mọi thao tác vẽ biểu đồ đều thông qua đối tượng `Plot` của `formsPlot1`, và kết thúc bằng `formsPlot1.Refresh()` để cập nhật hiển thị:

```
var plt = formsPlot1.Plot;
plt.Clear();               // Xóa biểu đồ cũ (nếu có)

// ... thêm dữ liệu vào biểu đồ ...

formsPlot1.Refresh();      // Vẽ lại
```

Một điểm nổi bật của ScottPlot so với các thư viện biểu đồ khác: biểu đồ hiển thị trong FormsPlot có thể tương tác ngay mà không cần viết thêm một dòng code nào:

- Cuộn chuột — zoom vào/ra tại vị trí con trỏ
- Kéo chuột trái — di chuyển (pan) biểu đồ
- Kéo chuột phải — zoom theo trục
- Double click — reset về view mặc định

Đây là tính năng rất hữu ích khi cần khám phá dữ liệu có nhiều điểm dữ liệu — người dùng có thể tự zoom vào vùng quan tâm mà không cần thêm nút hay slider nào trên giao diện.

8.5 MÃ QR — QRCoder

QRCoder là thư viện .NET mã nguồn mở, miễn phí hoàn toàn (MIT License), chuyên tạo mã QR — loại mã vạch 2 chiều có khả năng lưu trữ văn bản, URL, thông tin liên hệ, hay bất kỳ chuỗi dữ liệu nào, và có thể được quét bằng camera điện thoại để truy xuất nội dung đó ngay lập tức. Trong các ứng dụng quản lý, mã QR thường được dùng cho thẻ sinh viên (quét để xem thông tin), hóa đơn (quét để tra cứu đơn hàng), hay bảng kiểm diện (quét để điểm danh nhanh).QRCoder không phụ thuộc vào bất kỳ thư viện bên ngoài nào, nhẹ, và chỉ cần 3 dòng code để tạo ra một mã QR hoàn chỉnh.

Cài đặt: Mở **Package Manager Console**, gõ: Install-Package QRCoder

Sau khi cài xong, thêm `using` vào đầu file: using QRCoder;

Quy trình tạo mã QR trong QRCoder luôn gồm đúng 3 bước:

```
// Bước 1 – Tạo dữ liệu QR từ chuỗi văn bản
using QRCodeGenerator generator = new QRCodeGenerator();
QRCodeData data = generator.CreateQrCode(
    "Nội dung cần mã hóa",
    QRCodeGenerator.ECCLevel.Q      // Mức độ sửa lỗi
);

// Bước 2 – Chuyển dữ liệu QR thành ảnh PNG
```

```
using PngByteQRCode qrCode = new PngByteQRCode(data);
byte[] bytes = qrCode.GetGraphic(10);   // 10 = kích thước mỗi ô (pixel)

// Bước 3 – Chuyển mảng byte thành Image để hiển thị
using MemoryStream ms = new MemoryStream(bytes);
Image anhQR = Image.FromStream(ms);
```

Mã QR về bản chất chỉ là một chuỗi ký tự được mã hóa thành hình ảnh — nội dung bên trong hoàn toàn tùy ý, phần mềm quét sẽ quyết định xử lý chuỗi đó như thế nào dựa trên định dạng nhận ra được.

TÓM TẮT

Trong bài này, học viên tìm hiểu cách tích hợp các thư viện bên ngoài (NuGet package) vào ứng dụng WinForms nhằm bổ sung những tính năng phổ biến trong thực tế mà .NET không cung cấp sẵn, bao gồm:

- *Xuất dữ liệu ra file Excel (.xlsx) bằng ClosedXML: tạo workbook, định dạng tiêu đề, tô màu xen kẽ theo hàng, thêm công thức tính toán, AutoFilter và Freeze Panes — tất cả không phụ thuộc vào việc máy tính có cài Microsoft Office hay không*
- *Tạo báo cáo PDF bằng QuestPDF: mô tả bố cục tài liệu theo phong cách Fluent API (Page → Header / Content / Footer), xây dựng bảng dữ liệu nhiều cột với định dạng màu sắc, thêm thông tin thống kê và số trang tự động*
- *Vẽ biểu đồ thống kê trong Form bằng ScottPlot: biểu đồ cột so sánh giá trị, biểu đồ tròn phân bố tỉ lệ, lưu biểu đồ ra file ảnh — thay thế cho Chart control mặc định vốn không còn được hỗ trợ trên .NET 8 trở lên*
- *Tạo mã QR bằng QRCoder: mã hóa bất kỳ chuỗi văn bản nào thành mã QR chỉ với 3 bước, hiển thị trực tiếp trong PictureBox và lưu ra file ảnh để in hoặc chia sẻ*

CÂU HỎI ÔN TẬP

Câu 1: NuGet là gì? Trình bày các tiêu chí cần xem xét khi lựa chọn một NuGet package để tích hợp vào dự án thực tế (license, lượt tải, tần suất cập nhật, tương thích phiên bản .NET).

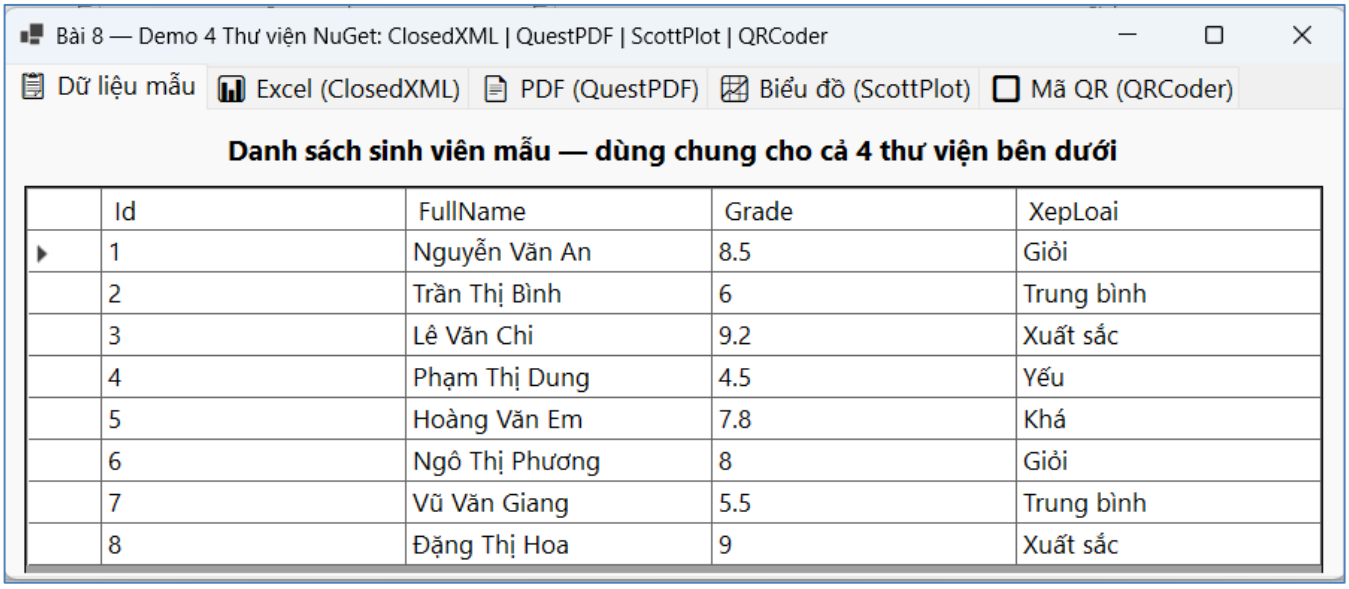
Câu 2: Trình bày các bước cơ bản để xuất một danh sách đối tượng C# (`List<T>`) ra file Excel sử dụng ClosedXML. Giải thích ý nghĩa của các thành phần `XLWorkbook`,

`IXLWorksheet`, và `Cell(hàng, cột)`. Viết đoạn mã tạo một file Excel đơn giản gồm 3 cột (STT, Tên sản phẩm, Giá) với hàng tiêu đề in đậm nền xanh.

Câu 3: QuestPDF tổ chức cấu trúc tài liệu PDF theo mô hình nào? Trình bày vai trò của `Header`, `Content`, và `Footer` trong một trang PDF. Giải thích sự khác biệt giữa `Column`, `Row`, và `Table` trong QuestPDF, và nêu tình huống phù hợp để sử dụng từng thành phần.

Câu 4: Trình bày quy trình 3 bước để tạo mã QR bằng QRCoder. Giải thích ý nghĩa của tham số `ECCLevel` (Error Correction Level) và sự khác biệt giữa các mức `L`, `M`, `Q`, `H`. Trong tình huống cần in mã QR lên thẻ sinh viên, nên chọn mức ECCLevel nào và vì sao?

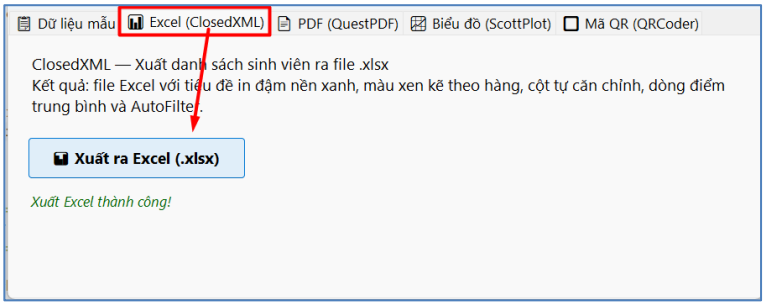
Câu 5: Xây dựng chương trình lấy thông tin danh sách sinh viên

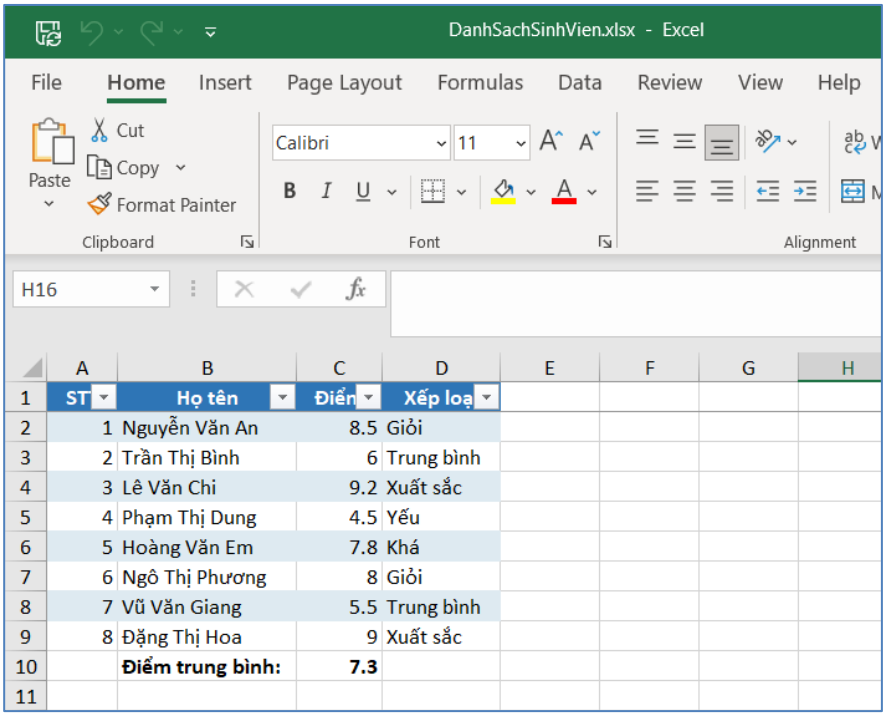


	Id	FullName	Grade	XepLoai
▶	1	Nguyễn Văn An	8.5	Giỏi
	2	Trần Thị Bình	6	Trung bình
	3	Lê Văn Chi	9.2	Xuất sắc
	4	Phạm Thị Dung	4.5	Yếu
	5	Hoàng Văn Em	7.8	Khá
	6	Ngô Thị Phương	8	Giỏi
	7	Vũ Văn Giang	5.5	Trung bình
	8	Đặng Thị Hoa	9	Xuất sắc

Project demo đầy đủ cả 4 thư viện — giao diện TabControl 5 tab, không cần SQL Server (dữ liệu mẫu hardcode sẵn):

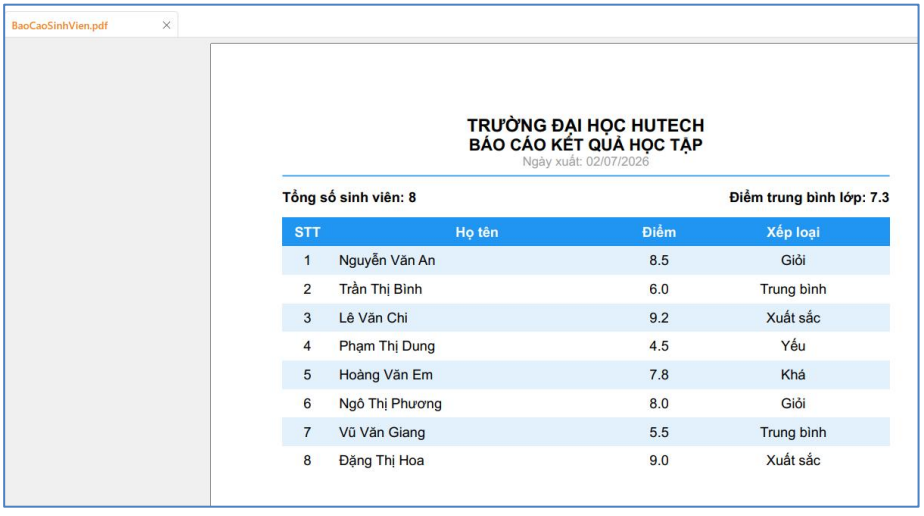
- Tab Excel: xuất .xlsx định dạng màu, AutoFilter, Freeze Panes, công thức AVERAGE



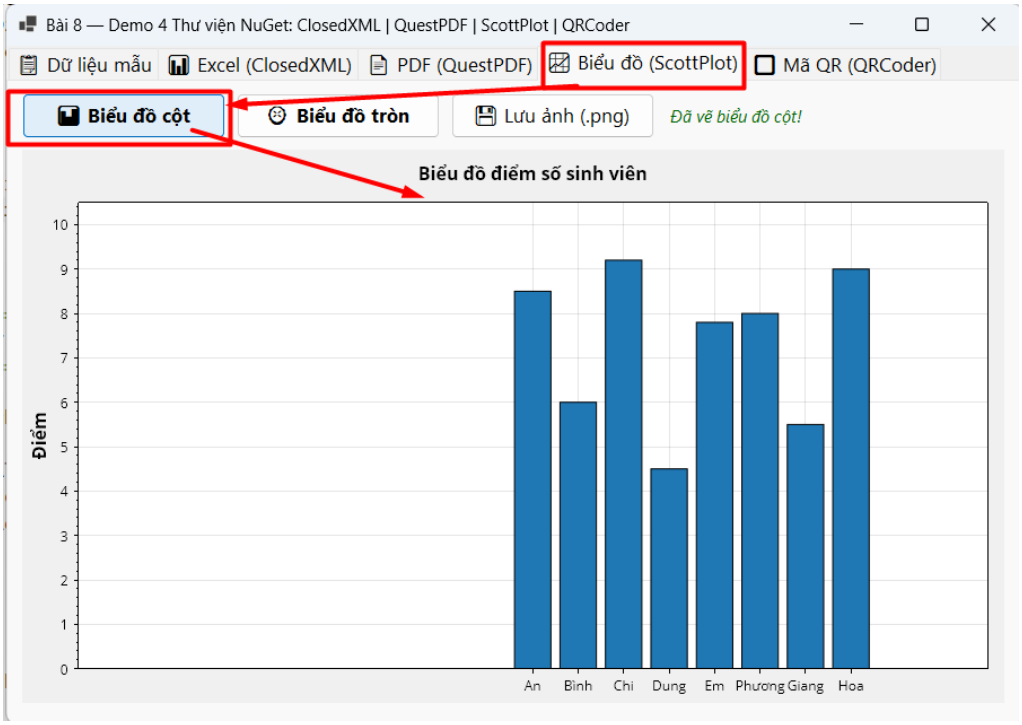


	A	B	C	D	E	F	G	H
1	STT	Họ tên	Điểm	Xếp loại				
2	1	Nguyễn Văn An	8.5	Giỏi				
3	2	Trần Thị Bình	6	Trung bình				
4	3	Lê Văn Chi	9.2	Xuất sắc				
5	4	Phạm Thị Dung	4.5	Yếu				
6	5	Hoàng Văn Em	7.8	Khá				
7	6	Ngô Thị Phương	8	Giỏi				
8	7	Vũ Văn Giang	5.5	Trung bình				
9	8	Đặng Thị Hoa	9	Xuất sắc				
10		**Điểm trung bình:**	**7.3**					
11								

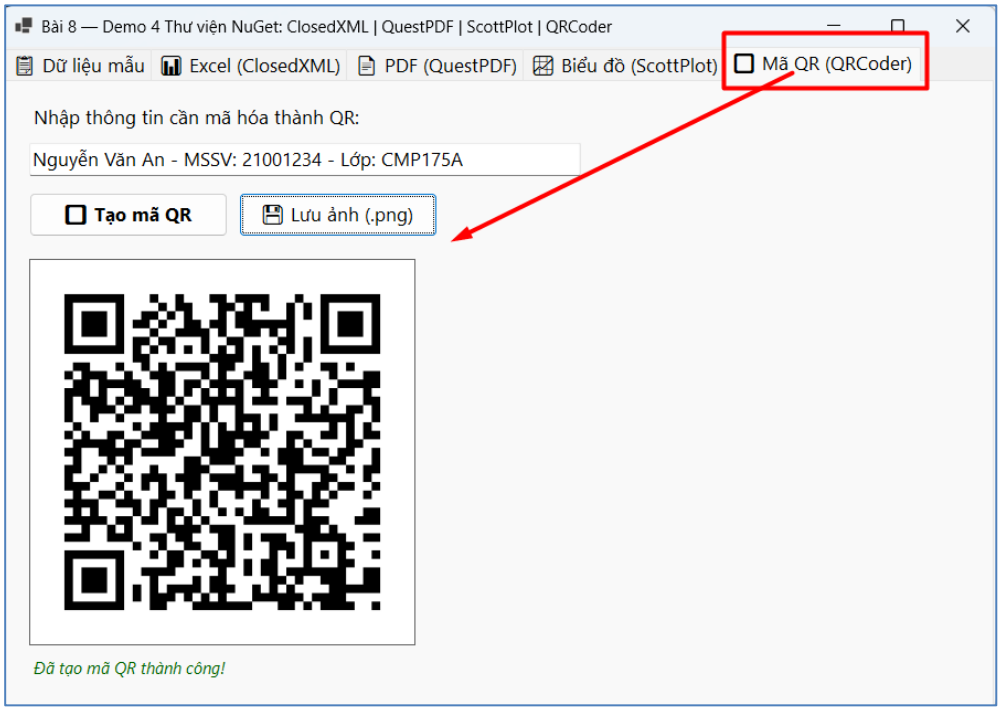
- Tab PDF: báo cáo .pdf với header, 3 ô thống kê, bảng màu xen kẽ, footer số trang



- Tab Biểu đồ: biểu đồ cột + biểu đồ tròn, zoom/pan bằng chuột, lưu .png



- Tab Mã QR: nhập text → tạo QR hiển thị ngay → lưu .png



TÀI LIỆU THAM KHẢO

1. Mark J. Price, C# 14 and .NET 10 — Modern Cross-Platform Development Fundamentals, 10th Edition, Packt Publishing, 2025.
2. Mark J. Price, Apps and Services with .NET 10, Packt Publishing, 2025.
3. Joseph Albahari, C# 12 in a Nutshell, O'Reilly Media, 2024.
4. Andrew Troelsen, Philip Japikse, Pro C# 10 with .NET 6, Apress, 2022.
5. Microsoft, C# Documentation, https://learn.microsoft.com/en-us/dotnet/csharp/, 2025.
6. Microsoft, Entity Framework Core Documentation, https://learn.microsoft.com/en-us/ef/core/, 2025.
7. Microsoft, Windows Forms Documentation for .NET, https://learn.microsoft.com/en-us/dotnet/desktop/winforms/, 2025.
8. ClosedXML Documentation, https://docs.closedxml.io, 2025.
9. QuestPDF Documentation, https://www.questpdf.com/documentation, 2025.
10. ScottPlot Documentation, https://scottplot.net/cookbook, 2025.
11. NuGet Gallery, https://www.nuget.org, 2025.